Py00

Python 程式設計

王頌文

主題

- Python 簡介
- 建置 Anaconda 開發環境
- Visual Studio Code 編輯器
- 程式設計的邏輯思維

程式語言(1/2)

- 如同英文有詞彙與文法規則，用以溝通及解決問題一般。
- 程式語言(programming language)則是用來與電腦溝通，透過人們為其設計的專屬語法，指揮電腦進行特定工作並解決問題。
  - 隨著人工智慧、加密貨幣、元宇宙等新興科技如火如荼發展，如何與電腦對話逐漸成為現代人必備的技能之一。
  - 許多國家將「與電腦對話的能力」視為重要競爭力，積極向下紮根。
  - 對於白領工作者而言，透過程式語言工具為自己加值或是斜槓，更是近幾年的趨勢。

程式語言(2/2)

- 維基百科(Wikipedia)列出高達七百多種知名的程式語言（包括當前使用與以前使用過），著實讓人眼花撩亂。
- 由於程式語言的基本概念大同小異，融會貫通一種後，要上手其他程式語言就會輕鬆許多。
  - 若還沒有頭緒，Python 這個通用型(general-purpose)程式語言是個不錯的選項。
  - 由於免費、開源、語法簡潔、有許多官方與協力廠商開發的套件等優勢，Python 在近幾年飛速成長且逐漸攻佔各領域的應用，更是受到全球教育以及開發人員的青睞。

Python 簡介(1/3)

- 1989.12 吉多·范羅蘇姆(Guido van Rossum)創建 Python，這是一種直譯式、通用型且支援物件導向的程式語言。
  - 范羅蘇姆在荷蘭出生，1982 年從阿姆斯特丹大學取得數學和計算機科學碩士學位。
  - 據說當初是為了打發聖誕節的空閒時間而編寫一個以 ABC 語言作基礎的腳本語言，並以電視劇《蒙提·派森的飛行馬戲團》（Monty Python's Flying Circus）為該語言命名。

Guido van Rossum

Python 簡介(2/3)

- 1989.12 吉多·范羅蘇姆(Guido van Rossum)創建 Python，這是一種直譯式、通用型且支援物件導向的程式語言。
  - 隨後更開放讓世界各地的開發者都可以參與 Python 的設計，並於 1991 年推出第一個 Python 編譯器後，越來越受到全球開發人員的關注與喜愛，在學術及教育領域也掀起風潮。

Guido van Rossum

Python 簡介(3/3)

- Python 2.0 於 2000.10 月發布，而另一個分支 Python 3.0 於 2008.12 月發布，有趣的是 3.0 在設計時沒有考慮向下相容，除了當時不可避免地引發許多論戰外，也造成開發者在挑選上的各種糾結。
  - 然而，隨著 Python 軟體基金會於 2020.1 月正式終止對 2.7 版本的支援，也就是說往後將不再對 Python 2.7 進行安全更新、修補臭蟲(bug)或執行其它改善，Python 用戶得以專注在 3.x 版本的開發及應用。

Python 特色(1/2)

- Python 之所以受到許多程式設計師的喜愛，當然有其獨到之處：
  - 簡單易學：Python 的設計哲學是優雅、明確及簡單，其語法簡化而不複雜的並強調自然語言，更貼近使用者的習慣，對新手來說相當友善。
  - 免費、開源且跨平台。
  - 成熟且具支持性的社群：經過三十數年的養成，Python 的社群已經成長茁壯，能支援橫跨初階入門到專家級別的開發人員，而活躍的社群也意味著來自不同地方的開發人員能獲得即時性的支持。

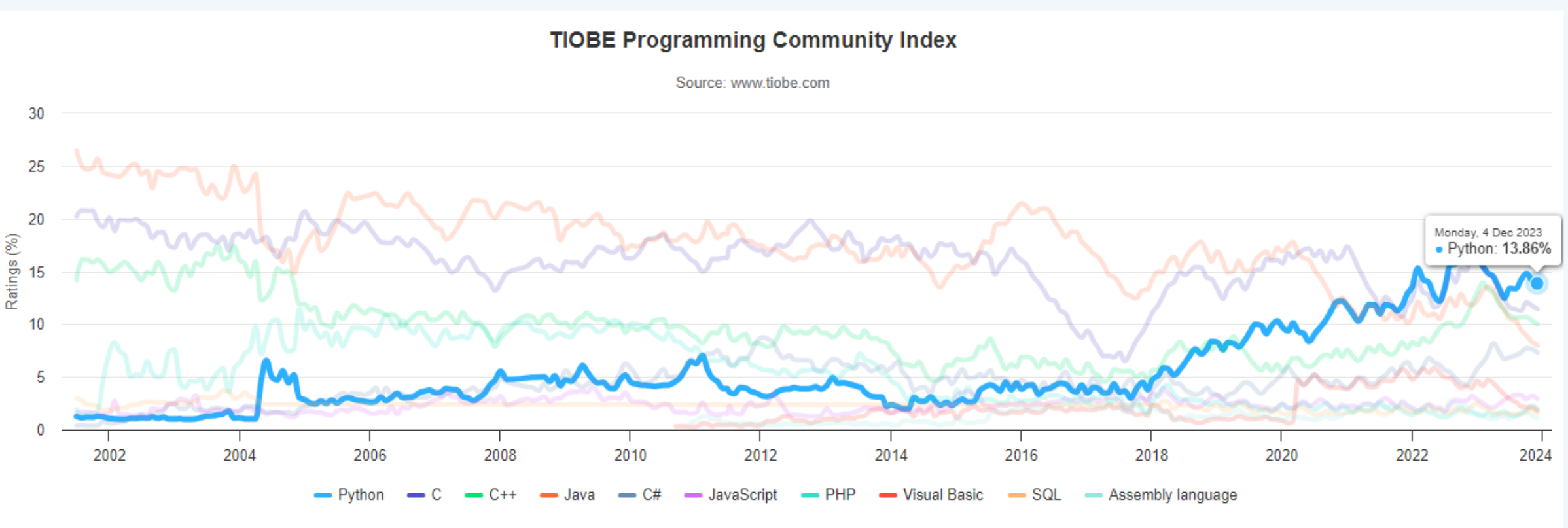
Python 特色(2/2)

- Python 之所以受到許多程式設計師的喜愛，當然有其獨到之處：
  - 豐富函式庫與套件：Python 有個應用領域的函式庫與套件，可大幅節省開發初期耗費的時間與精力，其豐富的套件可到 PyPI (Python Package Index)搜尋並下載安裝。
  - 數據分析、機器學習與深度學習：針對時下相當火紅的應用領域，Python 都是最受歡迎的工具之一，每天有著成千上百的相關專案以 Python 函式庫為基礎開發出來，在學術界與企業都受到青睞。

TIOBE Index

- TIOBE 排行是根據網路上的程式開發人員、課程和第三方廠商的數量，並利用 Google、Bing 等搜尋引擎及 Wikipedia、Amazon、YouTube 統計資料所得的排名。
  - 主要是反映程式語言的受歡迎程度，而非開發難易度或好壞。
  - 每月更新

Dec 2023	Dec 2022	Change	Programming Language	Ratings	Change
1	**1**		**Python**	**13.86%**	**-2.80%**
2	2		C	11.44%	-5.12%
3	3		C++	10.01%	-1.92%
4	4		Java	7.99%	-3.83%
5	5		C#	7.30%	+2.38%
6	7	↑	JavaScript	2.90%	-0.30%
7	10	↑	PHP	2.01%	+0.39%
8	6	↓	Visual Basic	1.82%	-2.12%
9	8	↓	SQL	1.61%	-0.61%
10	9	↓	Assembly language	1.11%	-0.76%
11	21	⇈	Scratch	1.08%	+0.41%
12	26	⇈	Fortran	1.07%	+0.64%

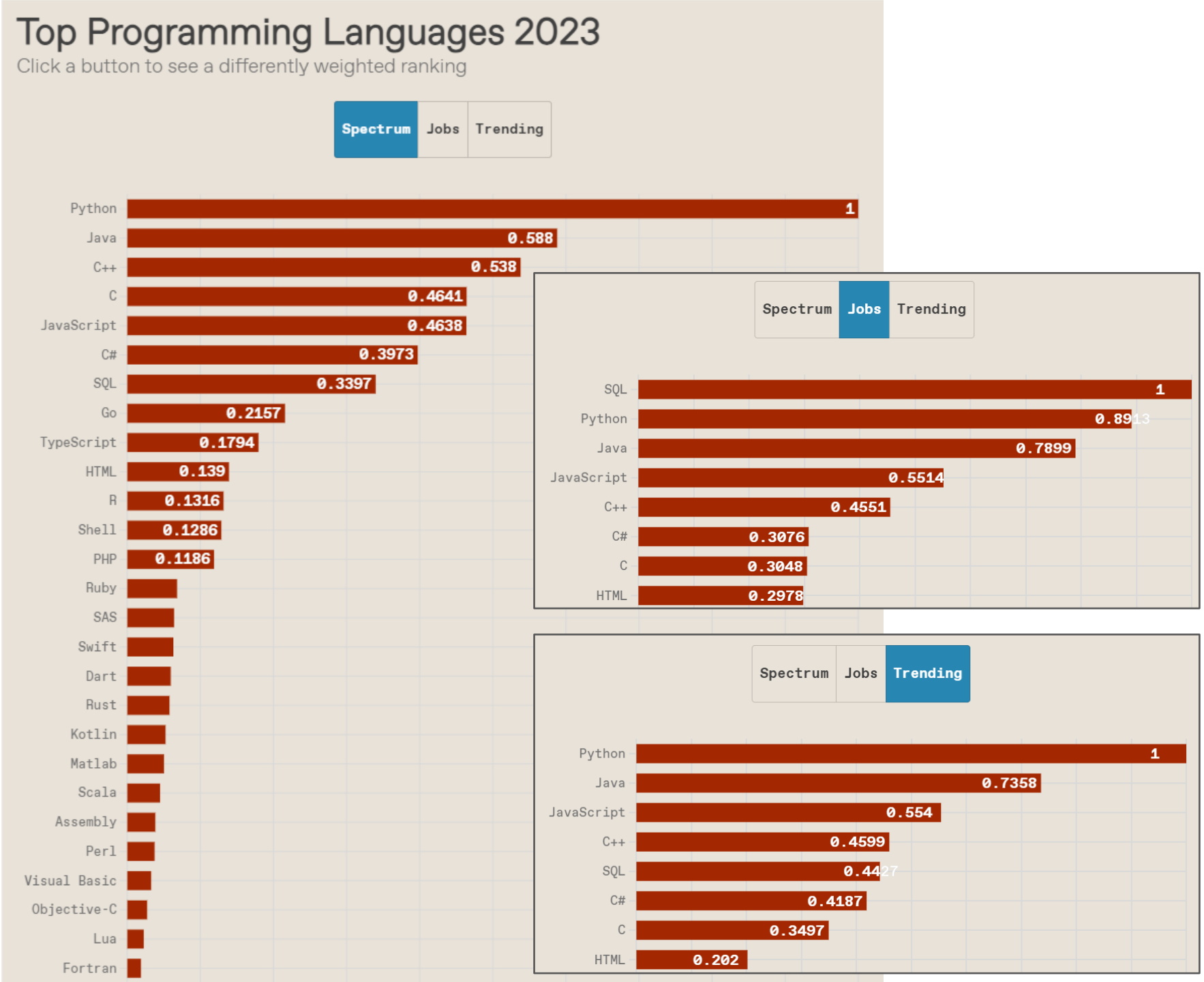

TIOBE Programming Community Index
Source: www.tiobe.com
Ratings (%)
30
25
20
15
10
5
0
2002
2004
2006
2008
2010
2012
2014
2016
2018
2020
2022
2024
Monday, 4 Dec 2023
Python: 13.86%
Python
C
C++
Java
C#
JavaScript
PHP
Visual Basic
SQL
Assembly language

- PYPL: PopularitY of Programming Language
  - 反映近一年透過 Google 搜尋該語言教學的頻率，同樣是每月更新一次。

Worldwide, Dec 2023 :

Rank	Change	Language	Share	1-year trend
1		Python	28.09 %	+0.1 %
2		Java	15.81 %	-0.9 %
3		JavaScript	8.93 %	-0.5 %
4	↑	C/C++	6.8 %	+0.1 %
5	↓	C#	6.64 %	-0.3 %
6		PHP	4.6 %	-0.6 %
7		R	4.53 %	+0.5 %
8		TypeScript	2.81 %	+0.0 %
9	↑	Swift	2.8 %	+0.7 %
10	↓	Objective-C	2.33 %	+0.2 %
11	↑↑	Rust	2.13 %	+0.3 %
12	↓	Go	2.03 %	+0.1 %
13	↓	Kotlin	1.76 %	-0.0 %
14		Matlab	1.65 %	+0.0 %
15	↑↑↑	Ada	1.04 %	+0.2 %

IEEE Spectrum ranking



Dark mode

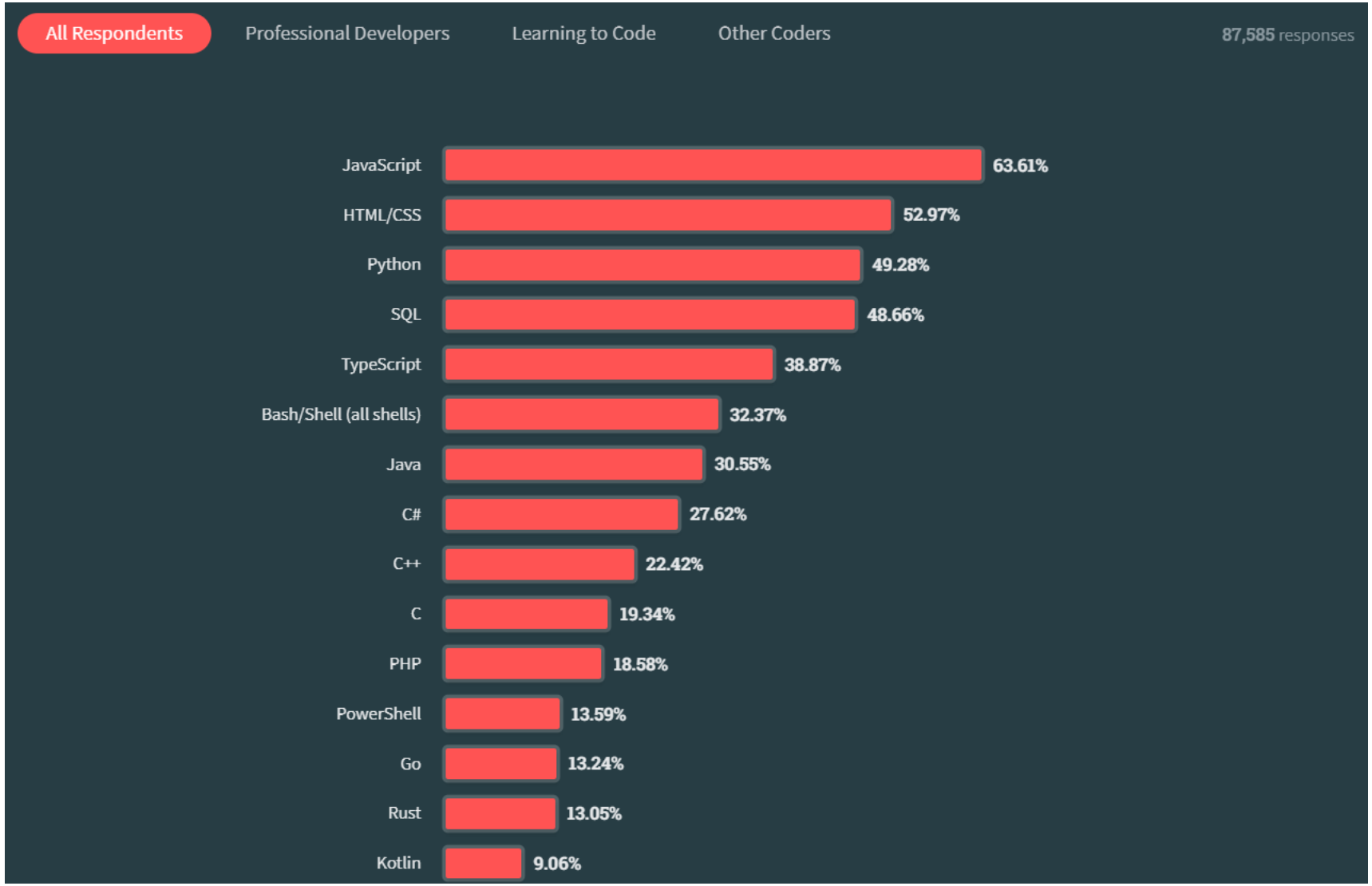
2023 Developer Survey

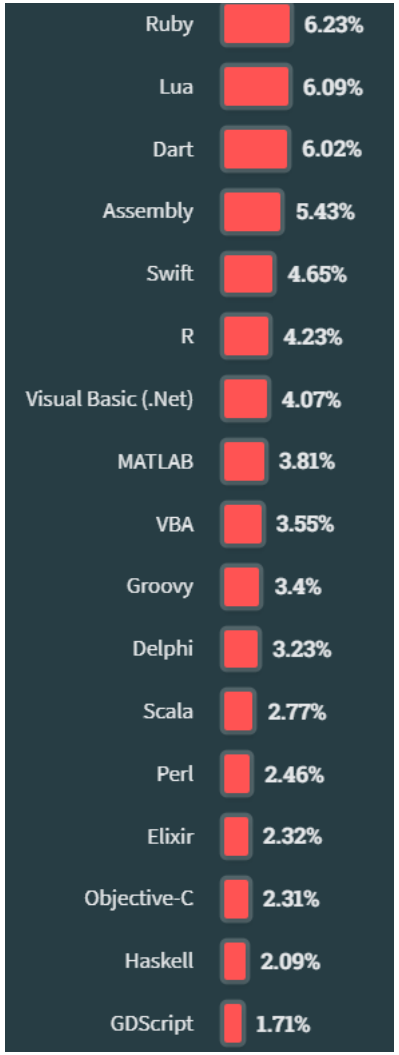
In May 2023 over 90,000 developers responded to our annual survey about how they learn and level up, which tools they're using, and which ones they want.

Read the overview →

Methodology →

Programming, scripting, and markup languages





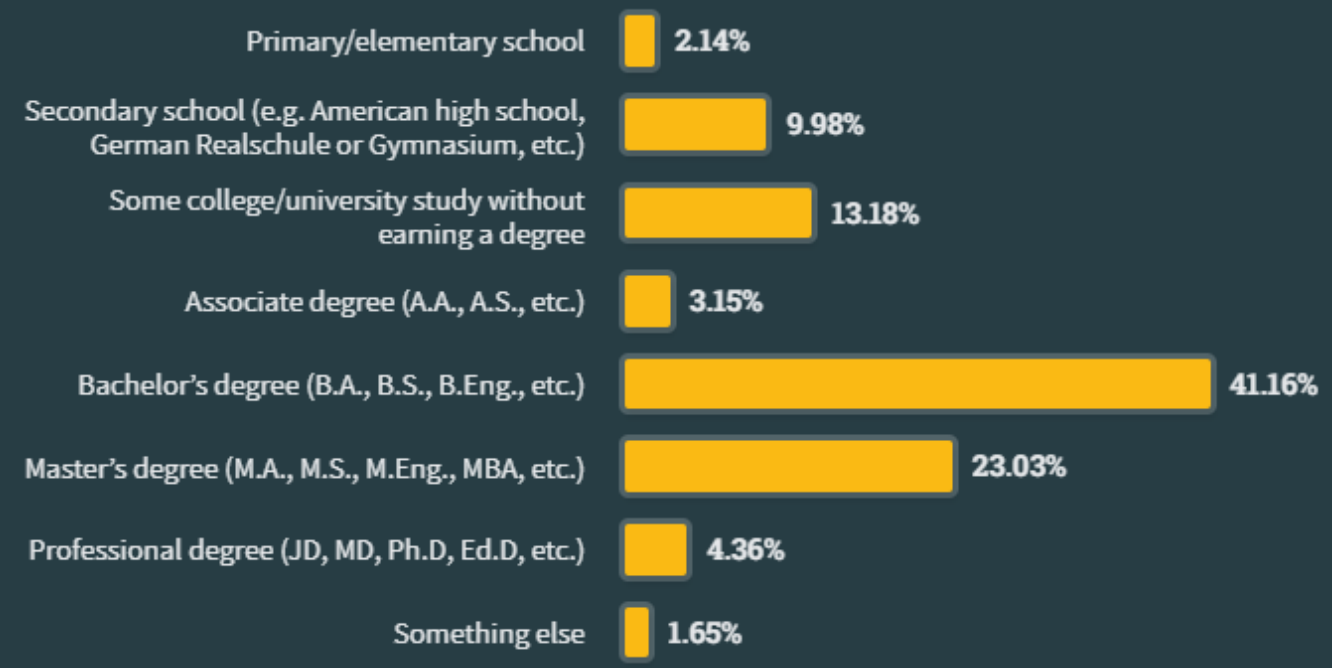
Educational attainment

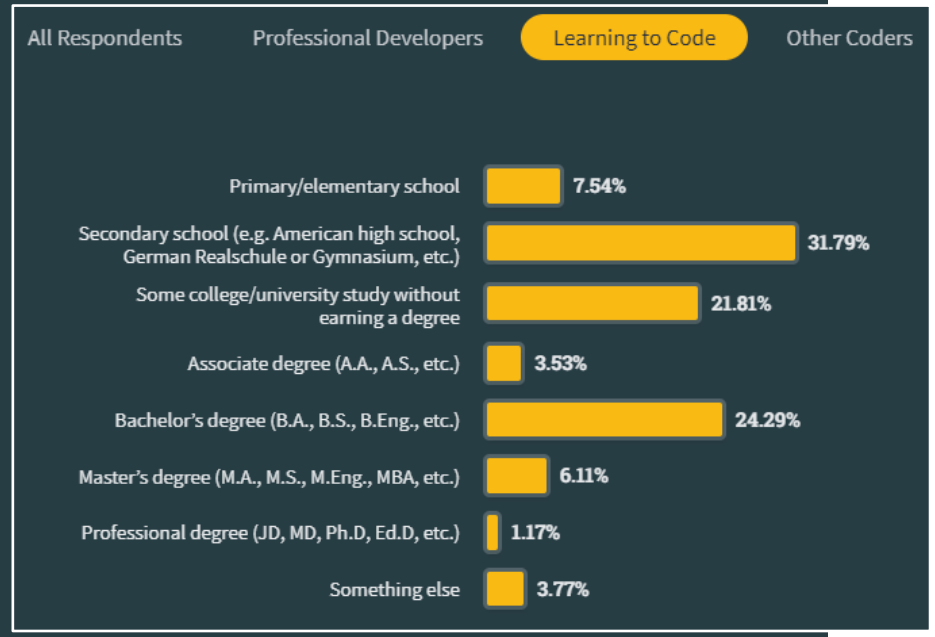
Most professional developers have attained a Bachelor's degree (47%) with a quarter attaining a Master's degree (26%).

For the developers who are learning to code, more than half are between 18-24 years old, so it makes sense that they are more likely to not have a Bachelor's degree. They are likely still in school.

All Respondents | Professional Developers | Learning to Code | Other Coders

89,184 responses





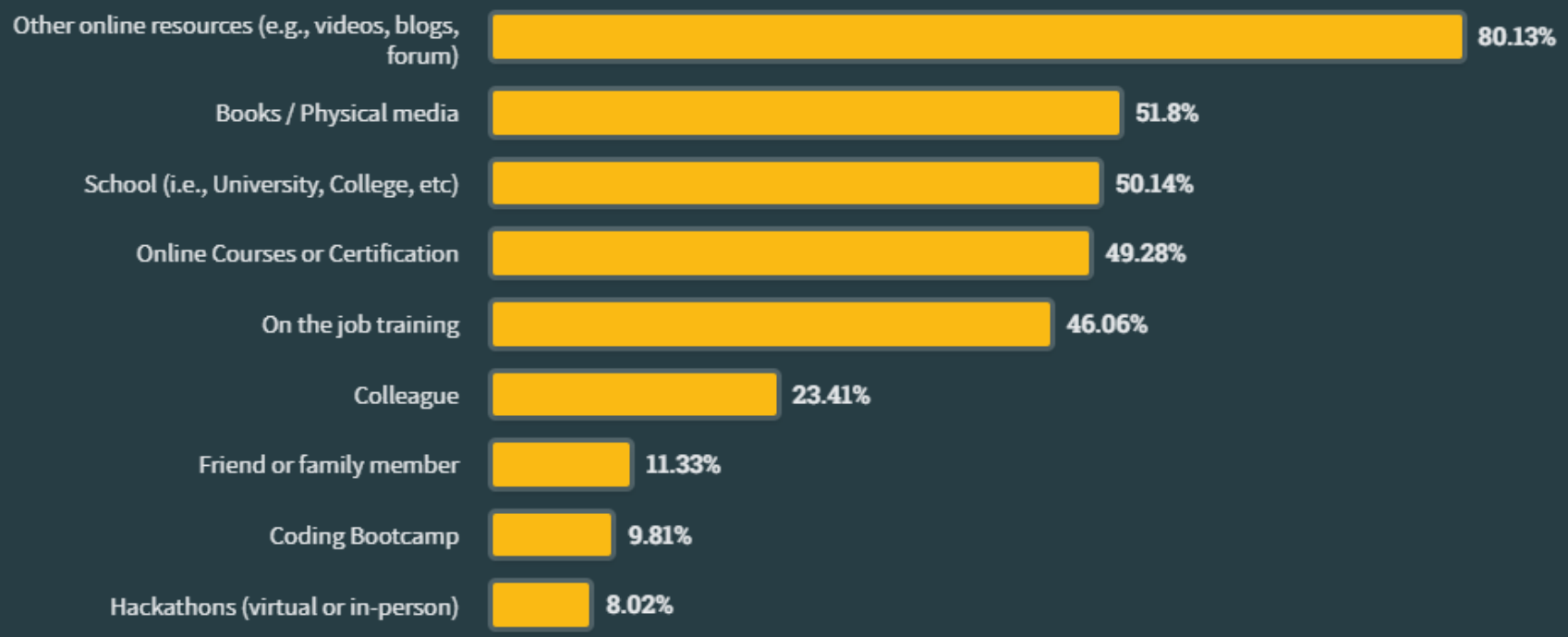
Learning how to code

Learning to code from online resources increased from 70% to 80% since the 2022 survey.

Respondents 18 and under are those most frequently selecting online resources (e.g., videos, blogs, forums) to learn from. Respondents 25 - 34 were the top age cohort to have learned from online courses or certifications (52%) but still learn more from traditional school (55%).

All Respondents | Learning to Code | By Age

87,663 responses



主題

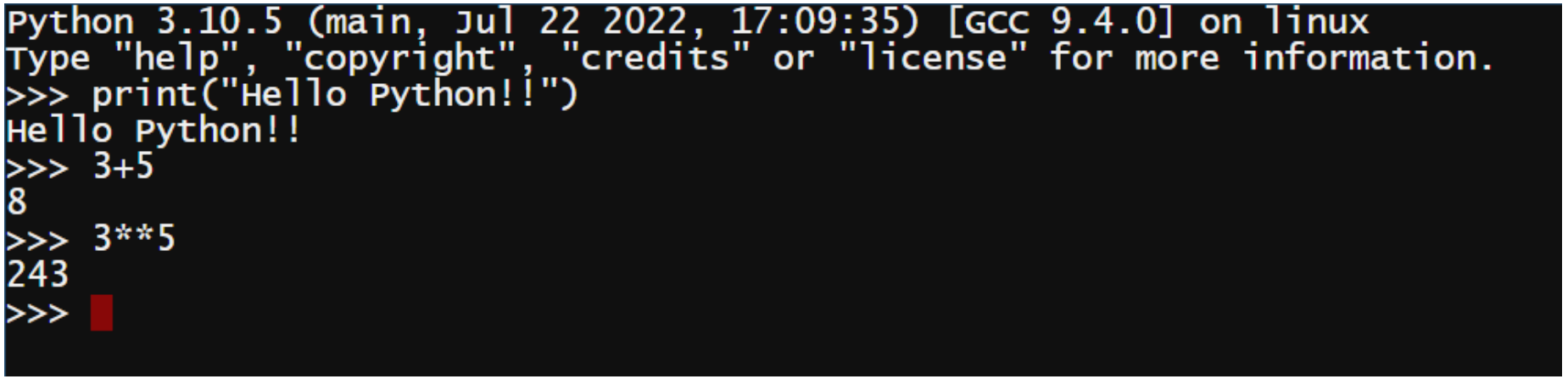
- Python 簡介
- 建置 Anaconda 開發環境
- Visual Studio Code 編輯器
- 程式設計的邏輯思維

整合式開發環境(IDE)

- 編寫程式需透過撰寫、編譯 / 直譯、除錯、執行等過程，早期負責這幾個過程的軟體都各自獨立，操作上並不方便，因此目前多數的高階程式語言採用整合式開發環境進行統整。
  - 所謂整合式開發環境(Integrated Develop Environment, IDE)是輔助程式設計人員開發軟體的工具軟體，通常會統整程式碼編輯器、編譯 / 直譯器、除錯輔助、圖形化介面等功能在一起。

Python IDE (1/2)

- The Python Wiki 網站列舉三十餘種 Python IDE，大多是開源且免費使用，同時也能滿足各種開發平臺。
  - 事實上，Python 官網有一個相當陽春的 IDLE 編輯器可撰寫及執行 Python 程式，可是難以滿足程式設計者的需求。



Python IDE (2/2)

- 常見的 Python IDE
  - Python 台灣使用者群組
  - The10 Best IDEs for Python Programmers
- 線上執行 Python
  - repl.it - Online REPL, Compiler & IDE
- 純文字 IDE
  - Notepad++, Sublime Text 3, VIM, …

Python IDE: Anaconda

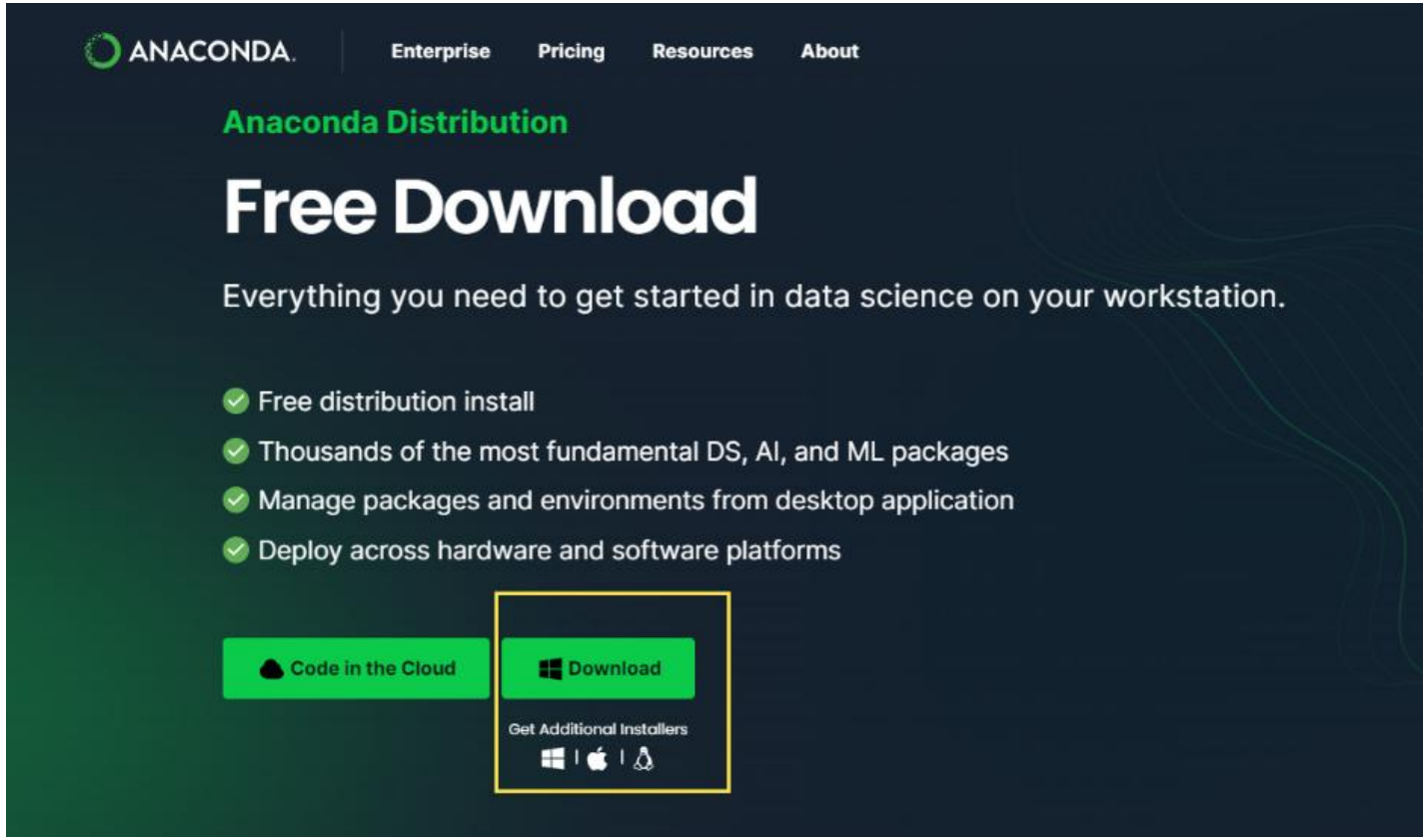
- 在眾多圖形介面的 Python IDE 中，可採用 Anaconda 為第一個學習環境，雖然這並非全球最多開發者使用的 IDE，但卻是一個讓初學者最容易搭建的 Python 學習環境。
- 特點：
  - 免費、開源且跨平台。
  - 內含 Spyder 單機與 Jupyter Notebook 網頁式編寫環境。
  - 內建眾多流行的科學、工程、數據分析套件，但也使其安裝檔超過 1GB。

主題

- Python 簡介
- 建置 Anaconda 開發環境
  - 安裝與使用 Anaconda
  - Spyder 編輯器
  - Jupyter 網頁式編輯器
- Visual Studio Code 編輯器
- 程式設計的邏輯思維

下載最新版 Anaconda

- https://www.anaconda.com/download



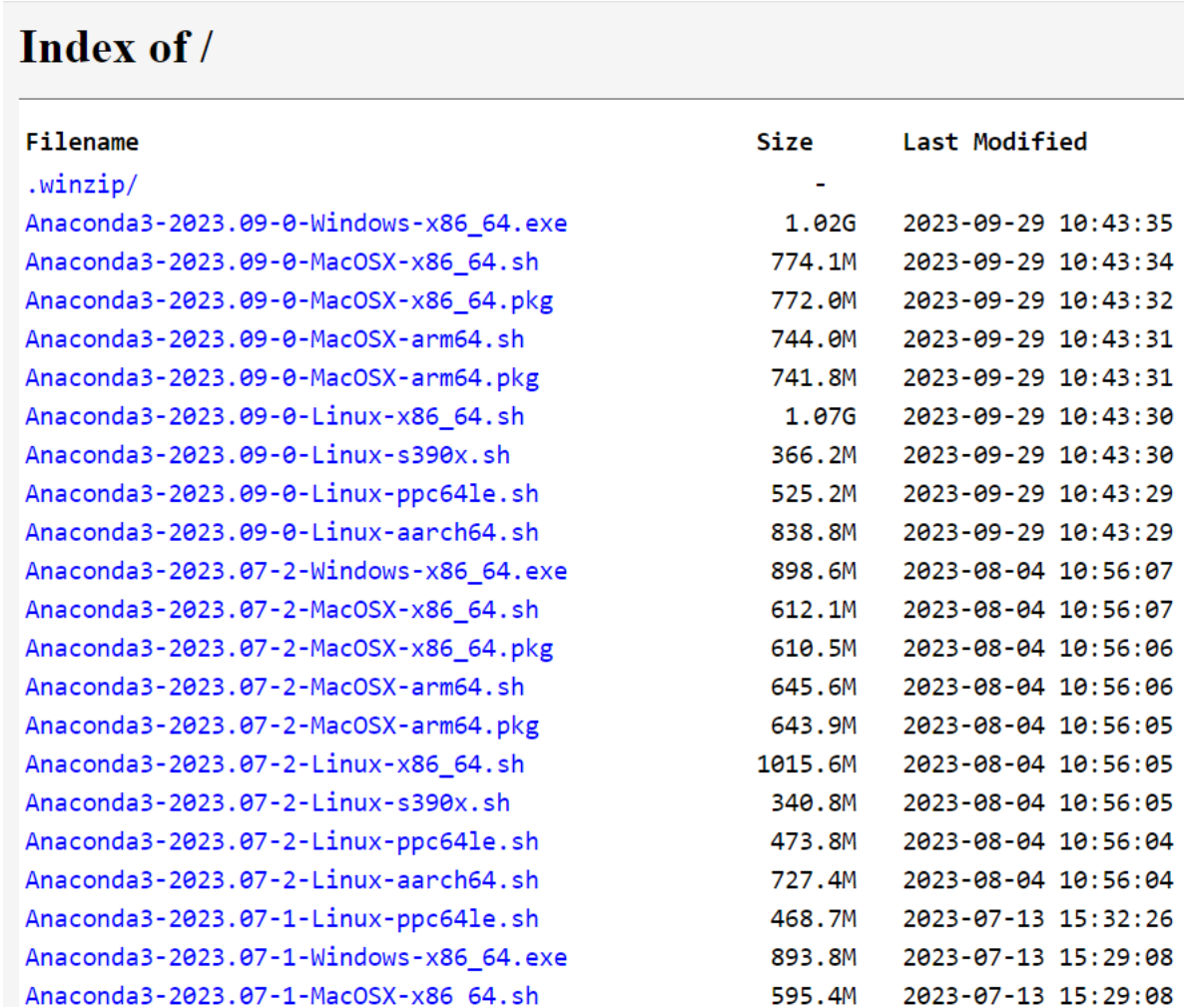
下載較舊的 Anaconda (1/2)

- 倘若考慮到穩定性、與其它套件的相容性（譬如知名的深度學習框架 TensorFlow 2 與 PyTorch 目前可支援 Python 3.8-3.11）等，可安裝較舊一點的 Python 版本。

Anaconda3 版本	基礎 Python 版本	Anaconda3 版本	基礎 Python 版本
2023.09-0	3.11	**2021.05**	3.8
2023.03-1	3.10	**2020.02**	3.7
2022.10	3.9	**5.2.0**	3.6

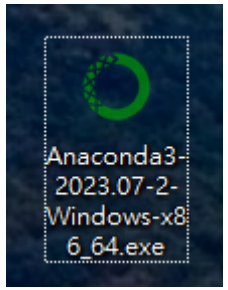
下載較舊的 Anaconda (2/2)

- https://repo.anaconda.com/archive/



Index of /

Filename	Size	Last Modified
.winzip/	-	
Anaconda3-2023.09-0-Windows-x86_64.exe	1.02G	2023-09-29 10:43:35
Anaconda3-2023.09-0-MacOSX-x86_64.sh	774.1M	2023-09-29 10:43:34
Anaconda3-2023.09-0-MacOSX-x86_64.pkg	772.0M	2023-09-29 10:43:32
Anaconda3-2023.09-0-MacOSX-arm64.sh	744.0M	2023-09-29 10:43:31
Anaconda3-2023.09-0-MacOSX-arm64.pkg	741.8M	2023-09-29 10:43:31
Anaconda3-2023.09-0-Linux-x86_64.sh	1.07G	2023-09-29 10:43:30
Anaconda3-2023.09-0-Linux-s390x.sh	366.2M	2023-09-29 10:43:30
Anaconda3-2023.09-0-Linux-ppc64le.sh	525.2M	2023-09-29 10:43:29
Anaconda3-2023.09-0-Linux-aarch64.sh	838.8M	2023-09-29 10:43:29
Anaconda3-2023.07-2-Windows-x86_64.exe	898.6M	2023-08-04 10:56:07
Anaconda3-2023.07-2-MacOSX-x86_64.sh	612.1M	2023-08-04 10:56:07
Anaconda3-2023.07-2-MacOSX-x86_64.pkg	610.5M	2023-08-04 10:56:06
Anaconda3-2023.07-2-MacOSX-arm64.sh	645.6M	2023-08-04 10:56:06
Anaconda3-2023.07-2-MacOSX-arm64.pkg	643.9M	2023-08-04 10:56:05
Anaconda3-2023.07-2-Linux-x86_64.sh	1015.6M	2023-08-04 10:56:05
Anaconda3-2023.07-2-Linux-s390x.sh	340.8M	2023-08-04 10:56:05
Anaconda3-2023.07-2-Linux-ppc64le.sh	473.8M	2023-08-04 10:56:04
Anaconda3-2023.07-2-Linux-aarch64.sh	727.4M	2023-08-04 10:56:04
Anaconda3-2023.07-1-Linux-ppc64le.sh	468.7M	2023-07-13 15:32:26
Anaconda3-2023.07-1-Windows-x86_64.exe	893.8M	2023-07-13 15:29:08
Anaconda3-2023.07-1-MacOSX-x86_64.sh	595.4M	2023-07-13 15:29:08



Anaconda3 2023.07-2 (64-bit) Setup

Welcome to Anaconda3 2023.07-2 (64-bit) Setup

Setup will guide you through the installation of Anaconda3 2023.07-2 (64-bit).

It is recommended that you close all other applications before starting Setup. This will make it possible to update relevant system files without having to reboot your computer.

Click Next to continue.

Next > Cancel

Anaconda3 2023.07-2 (64-bit) Setup

License Agreement

Please review the license terms before installing Anaconda3 2023.07-2 (64-bit).

Press Page Down to see the rest of the agreement.

End User License Agreement - Anaconda Distribution

Copyright 2015-2023, Anaconda, Inc.

All rights reserved under the 3-clause BSD License:

This End User License Agreement (the "Agreement") is a legal agreement between you and Anaconda, Inc. ("Anaconda") and governs your use of Anaconda Distribution (which was formerly known as Anaconda Individual Edition).

If you accept the terms of the agreement, click I Agree to continue. You must accept the agreement to install Anaconda3 2023.07-2 (64-bit).

Anaconda, Inc. < Back I Agree Cancel

Anaconda3 2023.07-2 (64-bit) Setup

Select Installation Type

Please select the type of installation you would like to perform for Anaconda3 2023.07-2 (64-bit).

Install for:

(•) Just Me (recommended)

( ) All Users (requires admin privileges)

Anaconda, Inc. < Back Next > Cancel

Anaconda3 2023.07-2 (64-bit) Setup

Choose Install Location

Choose the folder in which to install Anaconda3 2023.07-2 (64-bit).

Setup will install Anaconda3 2023.07-2 (64-bit) in the following folder. To install in a different folder, click Browse and select another folder. Click Next to continue.

可改成
C:\anaconda3

Destination Folder

C:\Users\Ian\anaconda3 Browse...

Space required: 4.9 GB

Space available: 670.6 GB

Anaconda, Inc. < Back Next > Cancel

Anaconda3 2023.07-2 (64-bit) Setup

Advanced Installation Options

Customize how Anaconda3 integrates with Windows

[x] Create start menu shortcuts (supported packages only).

[ ] Add Anaconda3 to my PATH environment variable

NOT recommended. This can lead to conflicts with other applications. Instead, use the Commmand Prompt and Powershell menus added to the Windows Start Menu.

[x] Register Anaconda3 as my default Python 3.11

Recommended. Allows other programs, such as VSCode, PyCharm, etc. to automatically detect Anaconda3 as the primary Python 3.11 on the system.

[ ] Clear the package cache upon completion

Recommended. Recovers some disk space without harming functionality.

Anaconda, Inc. < Back Install Cancel

Anaconda3 2023.07-2 (64-bit) Setup

Installation Complete

Setup was completed successfully.

Completed

Show details

Anaconda, Inc. < Back Next > Cancel

Anaconda3 2023.07-2 (64-bit) Setup

Anaconda3 2023.07-2 (64-bit)

Code with Anaconda in the Cloud

Everything you love about Anaconda now available from the cloud with Anaconda's fully-loaded Jupyter notebook.

No setup required. Get started for free.

Code in the cloud with Anaconda Notebooks

ANACONDA. jupyter

< Back Next > Cancel

Anaconda3 2023.07-2 (64-bit) Setup

Completing Anaconda3 2023.07-2 (64-bit) Setup

Thank you for installing Anaconda Distribution.

Here are some helpful resources to get you started. We recommend you bookmark the 'Getting Started with Anaconda Distribution' link so you can refer back to it later.

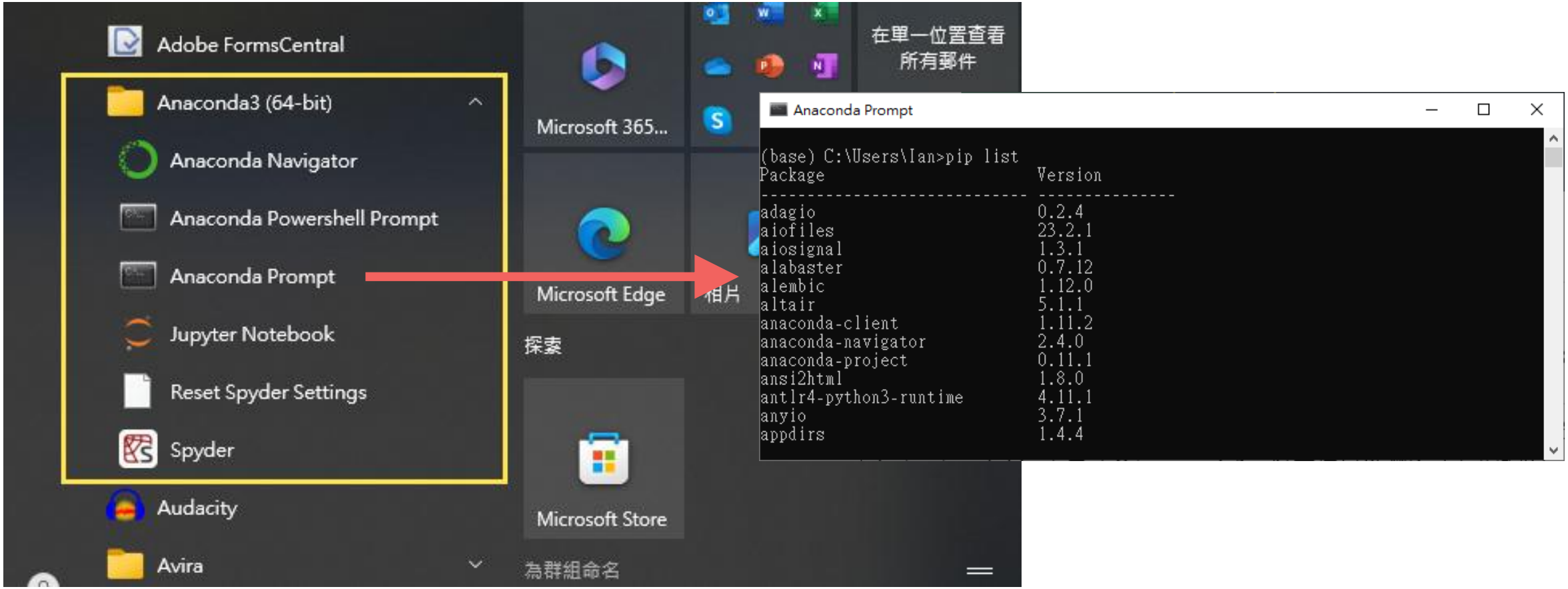
[x] Launch Anaconda Navigator

[x] Getting Started with Anaconda Distribution

< Back Finish Cancel

安裝完成後的操作項目(1/2)

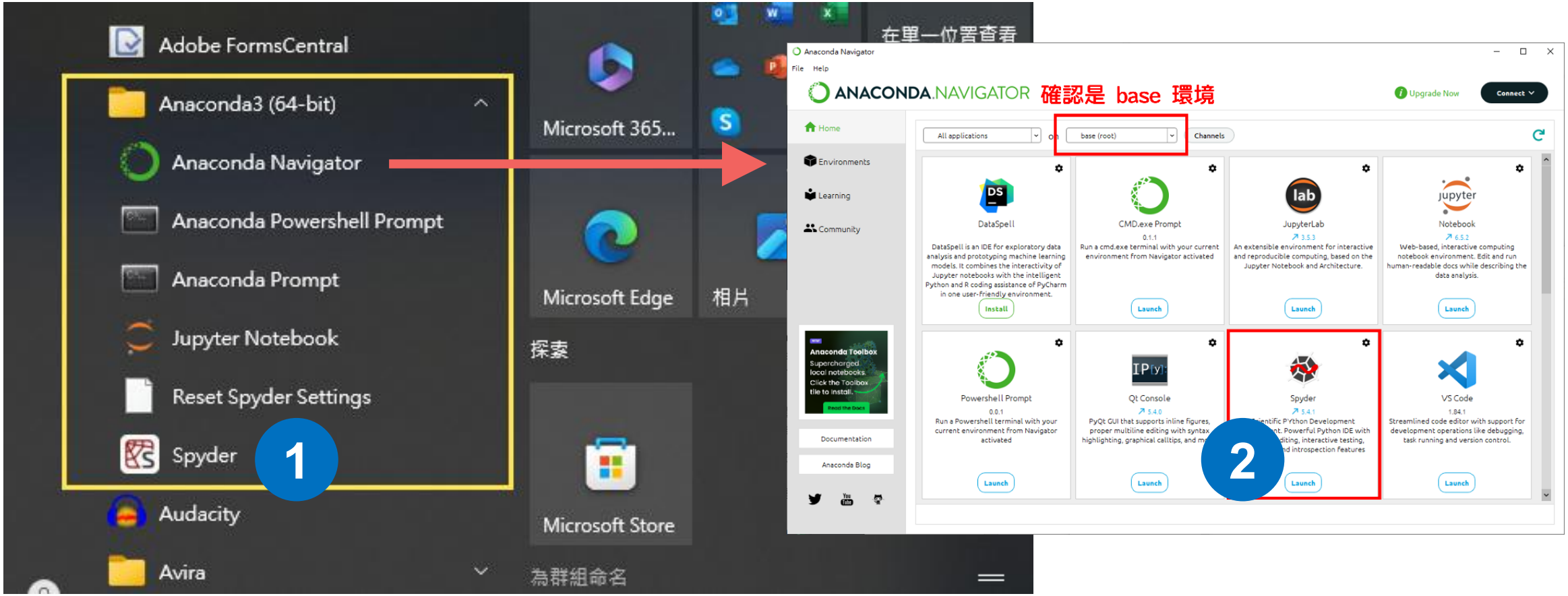
安裝完成後的操作項目(2/2)

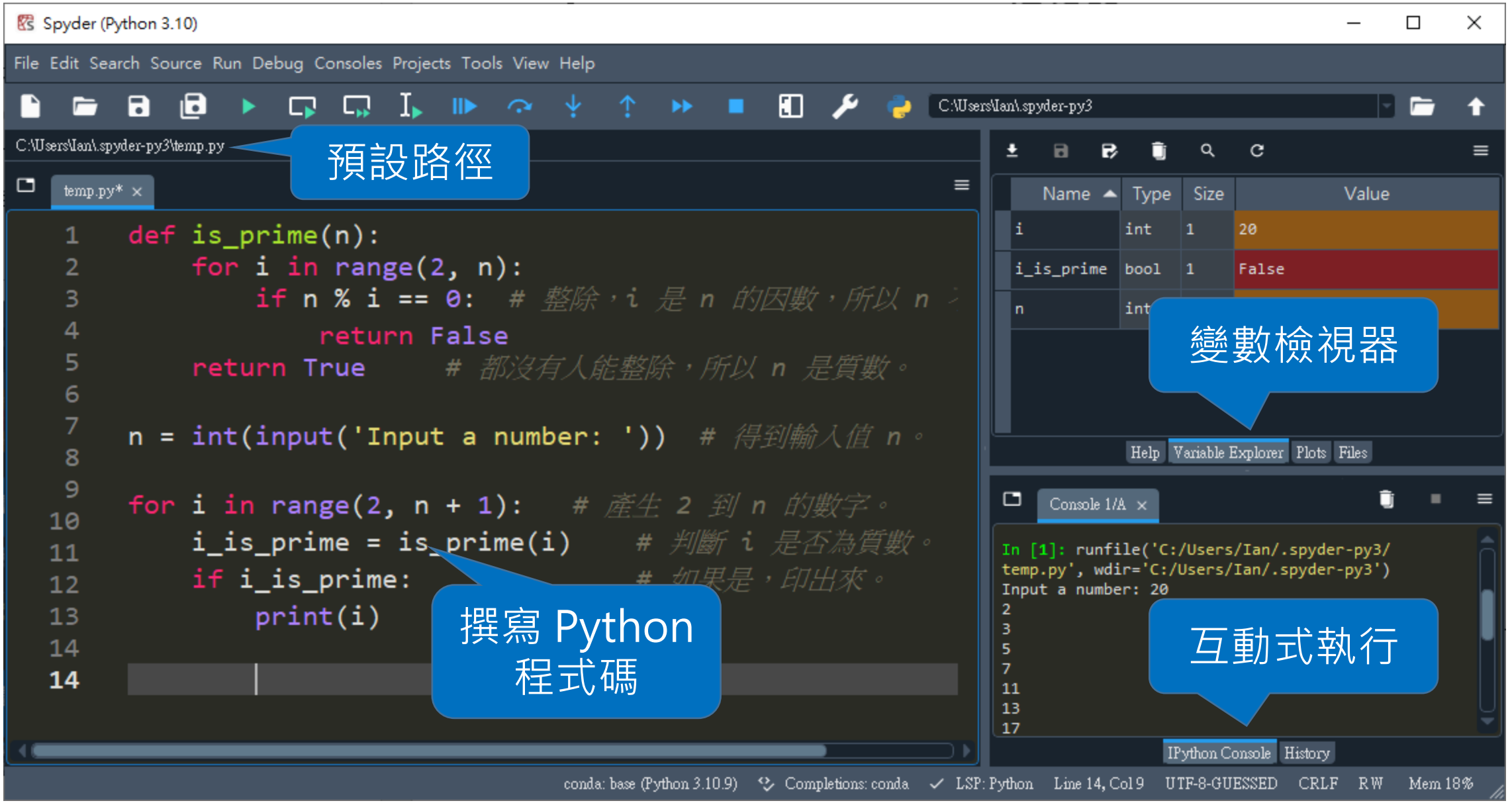


主題

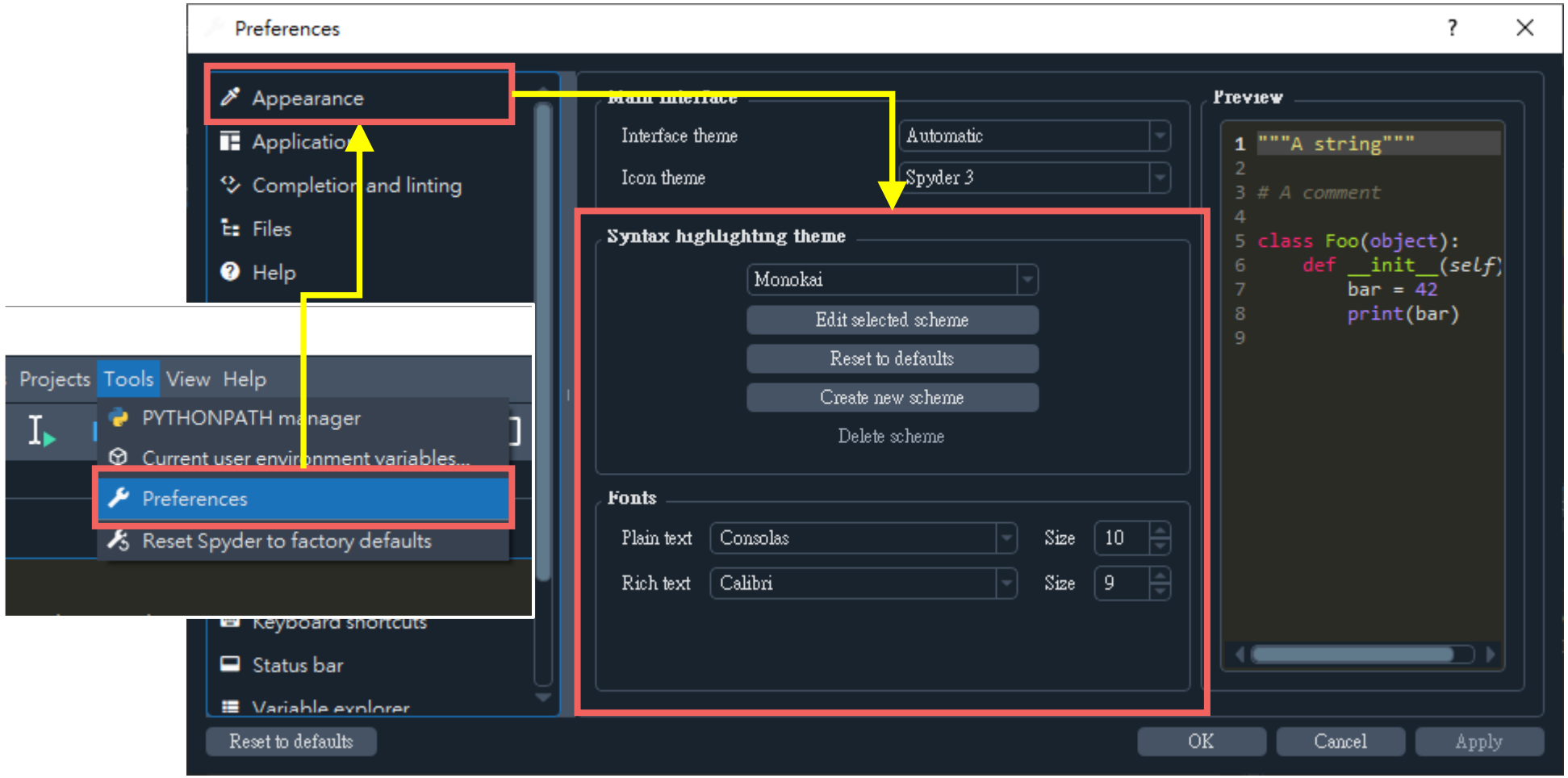
- Python 簡介
- 建置 Anaconda 開發環境
  - 安裝與使用 Anaconda
  - Spyder 編輯器
  - Jupyter 網頁式編輯器
- Visual Studio Code 編輯器
- 程式設計的邏輯思維

啟動 Spyder


Adobe FormsCentral
Anaconda3 (64-bit)
Anaconda Navigator
Anaconda Powershell Prompt
Anaconda Prompt
Jupyter Notebook
Reset Spyder Settings
Spyder
1
Audacity
Avira
Microsoft 365...
Microsoft Edge
相片
探索
Microsoft Store
為群組命名
Anaconda Navigator
File Help
ANACONDA.NAVIGATOR
確認是 base 環境
Upgrade Now
Connect
Home
Environments
Learning
Community
All applications
on
base (root)
Channels
DataSpell
CMD.exe Prompt
JupyterLab
Notebook
Launch
Install
Powershell Prompt
Qt Console
Spyder
VS Code
2
Documentation
Anaconda Blog


Spyder (Python 3.10)
File Edit Search Source Run Debug Consoles Projects Tools View Help
C:\Users\Ian\.spyder-py3
C:\Users\Ian\.spyder-py3\temp.py
預設路徑
temp.py*
def is_prime(n):
for i in range(2, n):
if n % i == 0: # 整除，i 是 n 的因數，所以 n
return False
return True # 都沒有人能整除，所以 n 是質數。
n = int(input('Input a number: ')) # 得到輸入值 n。
for i in range(2, n + 1): # 產生 2 到 n 的數字。
i_is_prime = is_prime(i) # 判斷 i 是否為質數。
if i_is_prime: # 如果是，印出來。
print(i)
撰寫 Python
程式碼
Name Type Size Value
i int 1 20
i_is_prime bool 1 False
n int
變數檢視器
Help Variable Explorer Plots Files
Console 1/A
In [1]: runfile('C:/Users/Ian/.spyder-py3/
temp.py', wdir='C:/Users/Ian/.spyder-py3')
Input a number: 20
2
3
5
7
11
13
17
互動式執行
IPython Console History
conda: base (Python 3.10.9) Completions: conda LSP: Python Line 14, Col 9 UTF-8-GUESSED CRLF RW Mem 18%

語法亮度


Preferences
Appearance
Application
Completion and linting
Files
Help
Interface theme
Automatic
Icon theme
Spyder 3
Syntax highlighting theme
Monokai
Edit selected scheme
Reset to defaults
Create new scheme
Delete scheme
Fonts
Plain text
Consolas
Size
10
Rich text
Calibri
Size
9
Preview
"""A string"""
# A comment
class Foo(object):
def __init__(self)
bar = 42
print(bar)
Projects
Tools
View
Help
PYTHONPATH manager
Current user environment variables...
Preferences
Reset Spyder to factory defaults
Status bar
Reset to defaults
OK
Cancel
Apply

Spyder 的操作(1/2)

- **放大或縮小文字**：對於編輯區及命令視窗區的文字，可透過「Ctrl + 滑鼠滾輪向上或向下」或者「Ctrl 配合 +/–」來放大或縮小，對程式開發很方便。

- **自動補齊(auto-complete)**：在編寫程式時輸入部分文字後按下「Tab 鍵」，會列出所有可能符合的項目，使用者可用「↑/↓」鍵移動項目，並搭配「Enter 鍵」完成輸入。

Spyder 的操作(2/2)

- **多行註解**：先用滑鼠選取要註解的多行程式碼，按下「Ctrl + 1」即能註解或取消註解，這比 Python 的多行註解（成對引號）要方便。
- Spyder 的除錯工具

主題

- Python 簡介
- 建置 Anaconda 開發環境
  - 安裝與使用 Anaconda
  - Spyder 編輯器
  - Jupyter 網頁式編輯器
- Visual Studio Code 編輯器
- 程式設計的邏輯思維

Jupyter Notebook (1/3)

- 基於網頁的互動式計算環境，能在編寫程式時利用其直譯式的特性，達到高互動執行結果。

Jupyter Notebook (2/3)

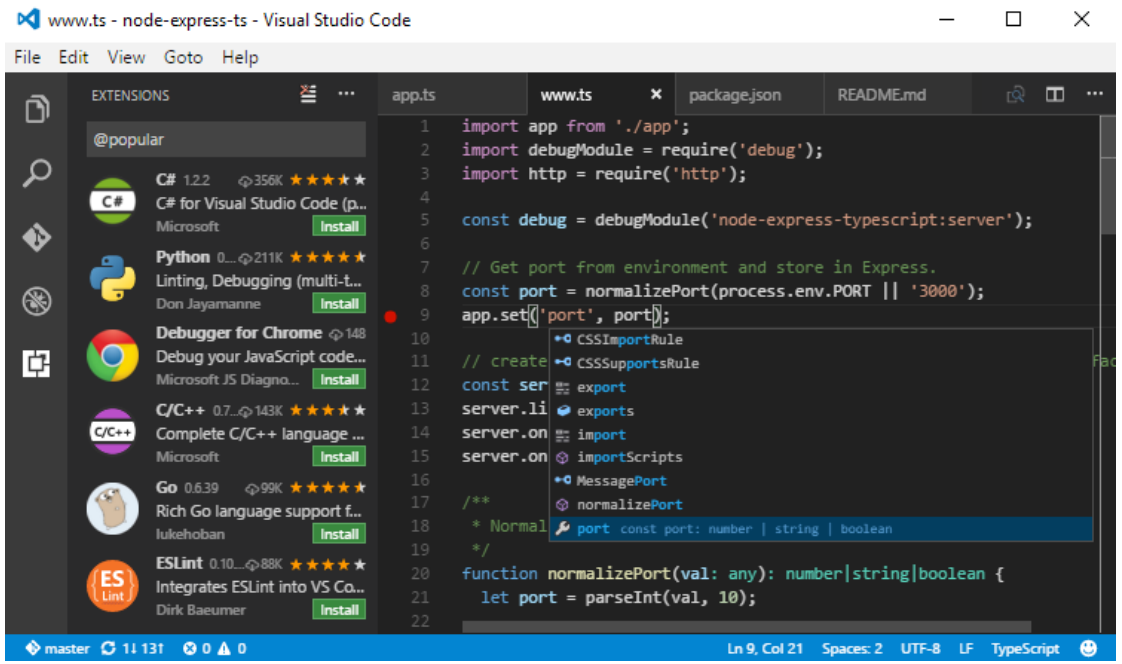
- Jupyter Notebook 主要包含兩個組成：
  - 網頁應用(web application)：基於網頁瀏覽器的互動創作及應用工具，包括可以計算、文件創作及豐富的多媒體輸出。
  - 筆記本文件(notebook documents)：顯示所有前述網頁應用當中的內容，包括計算的輸入/輸出、文件說明/解釋、數學運算及運算式、圖片及所有豐富多媒體內容。

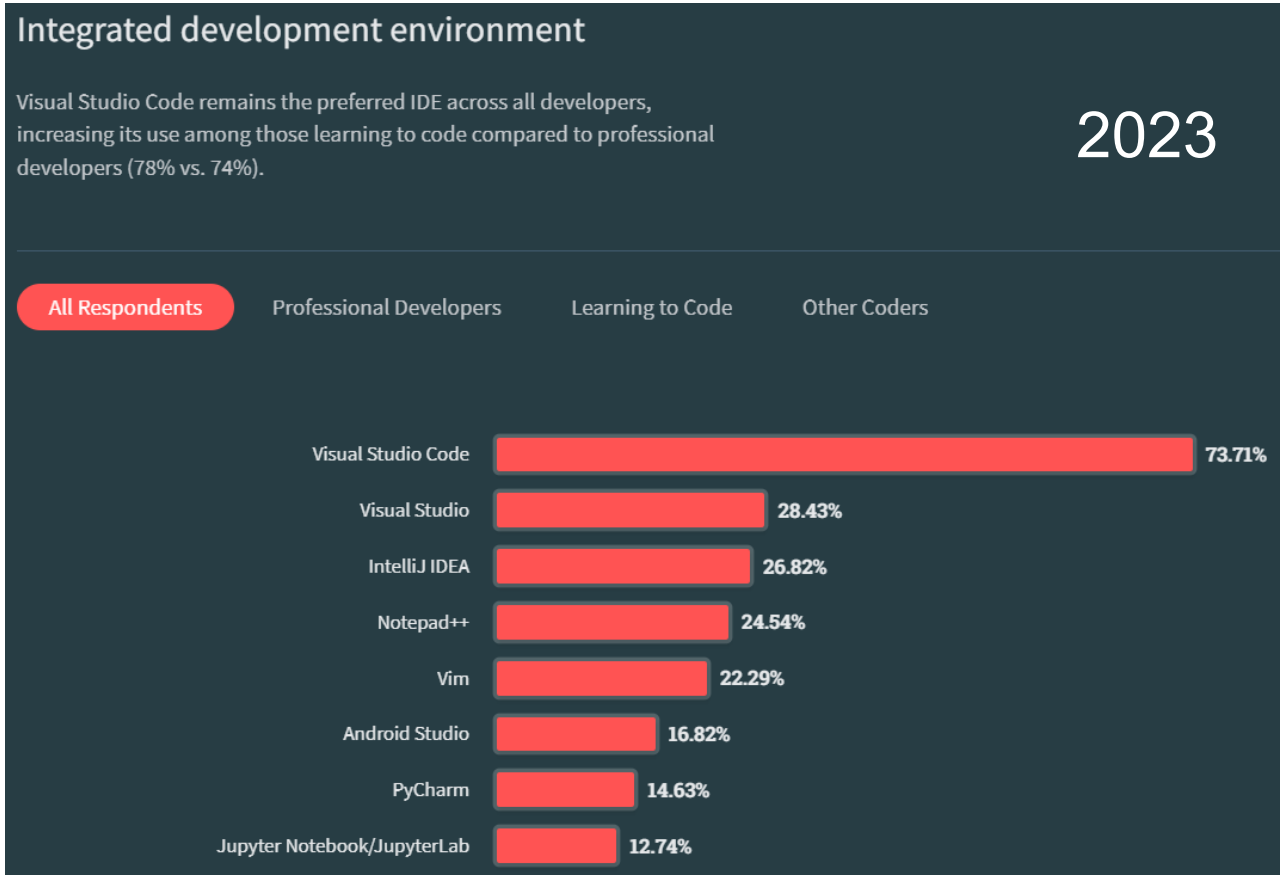
Jupyter Notebook (3/3)

主題

- Python 簡介
- 建置 Anaconda 開發環境
- Visual Studio Code 編輯器
  - 安裝與使用 VS Code
- 程式設計的邏輯思維

Visual Studio Code (VS Code)





https://survey.stackoverflow.co/2023/

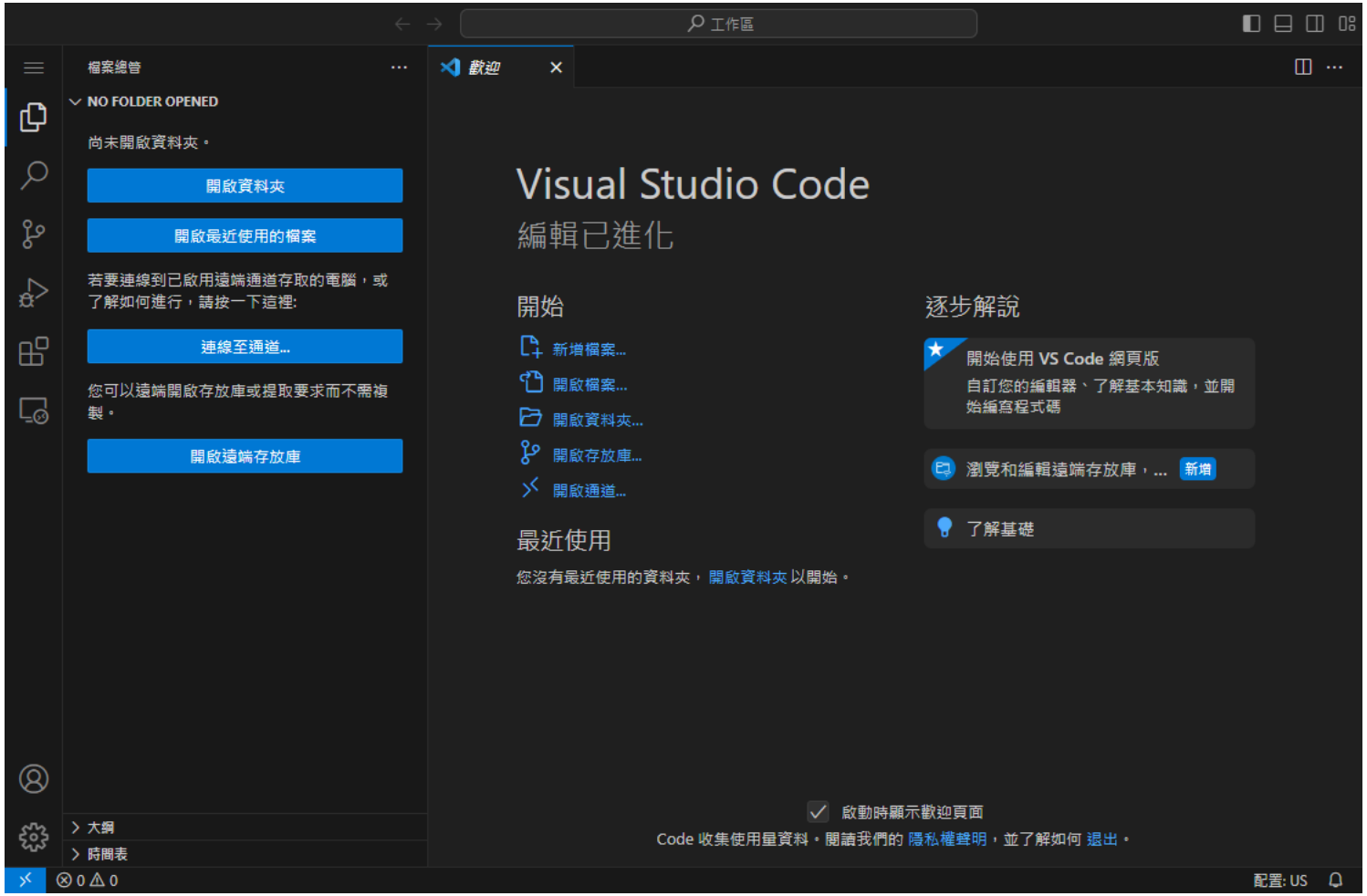
VS Code 特點

- 在操作方面，VS Code 提供語法突顯、程式碼自動補全(IntelliSense)、語法檢查等許多方便的功能，且其出色的除錯(debug)功能也大幅地減少開發時間。
- 有豐富的免費擴充插件(extension)，如語言支援、程式碼格式化工具、Git 整合等，可依個人喜好及需求安裝擴充，便於更加高效地進行開發工作。
- 許多程式語言，包括 Python、C\C++、Java、PHP 等，甚至是 Markdown 文件皆有支援，也有對應的擴充程式可增加開發效率。

主題

- Python 簡介
- 建置 Anaconda 開發環境
- Visual Studio Code 編輯器
  - 安裝與使用 VS Code
- 程式設計的邏輯思維

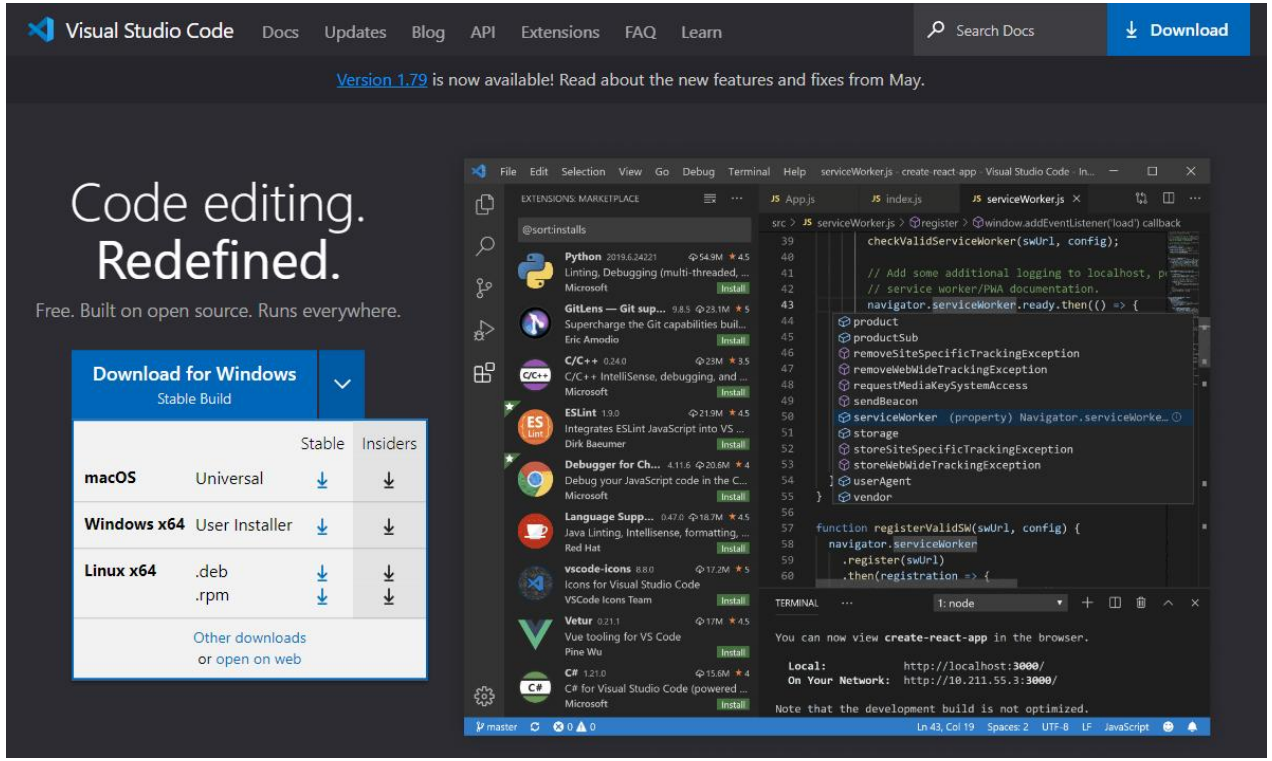
線上 VS Code


工作區
檔案總管
NO FOLDER OPENED
尚未開啟資料夾。
開啟資料夾
開啟最近使用的檔案
若要連線到已啟用遠端通道存取的電腦，或了解如何進行，請按一下這裡:
連線至通道...
您可以遠端開啟存放庫或提取要求而不需複製。
開啟遠端存放庫
大綱
時間表
歡迎
Visual Studio Code
編輯已進化
開始
新增檔案...
開啟檔案...
開啟資料夾...
開啟存放庫...
開啟通道...
最近使用
您沒有最近使用的資料夾，開啟資料夾以開始。
逐步解說
開始使用 VS Code 網頁版
自訂您的編輯器、了解基本知識，並開始編寫程式碼
瀏覽和編輯遠端存放庫，...
新增
了解基礎
啟動時顯示歡迎頁面
Code 收集使用量資料。閱讀我們的隱私權聲明，並了解如何退出。
⊗0 ⚠0
配置: US

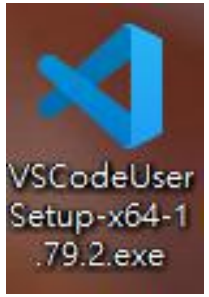
安裝 VS Code (1/7)

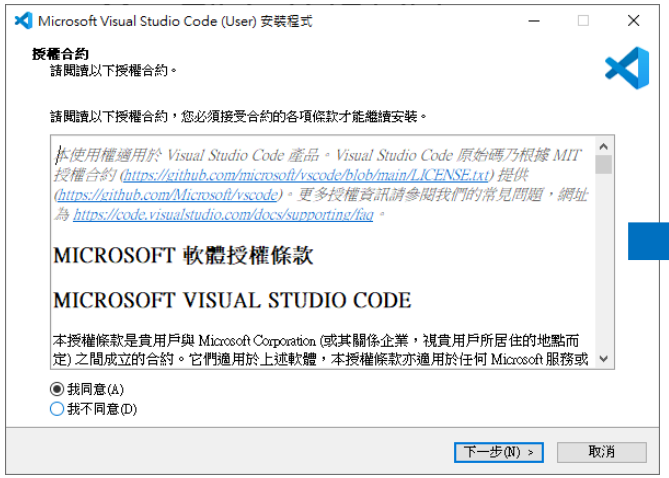
- 到官網下載 VS Code 安裝檔
  - VSCodeUserSetup-x64-1.79.2.exe (≈ 90 MB)

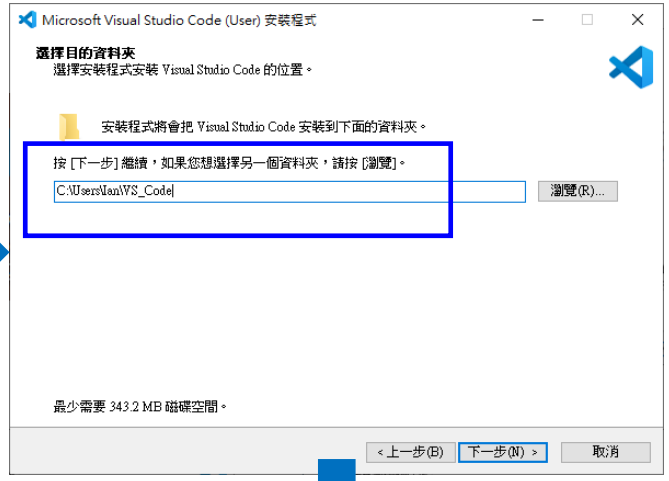
https://code.visualstudio.com/

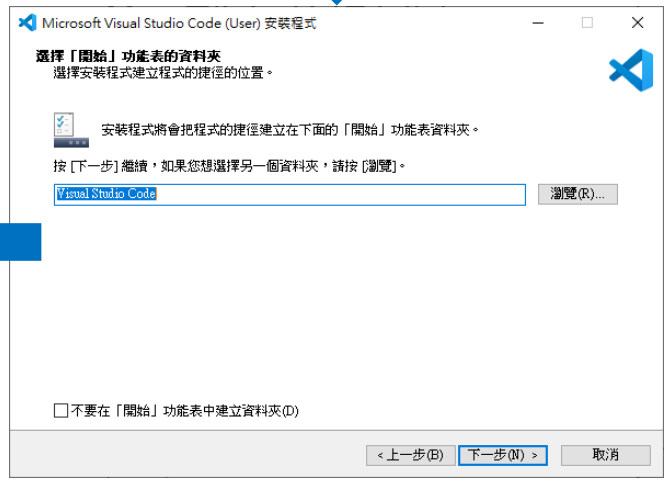


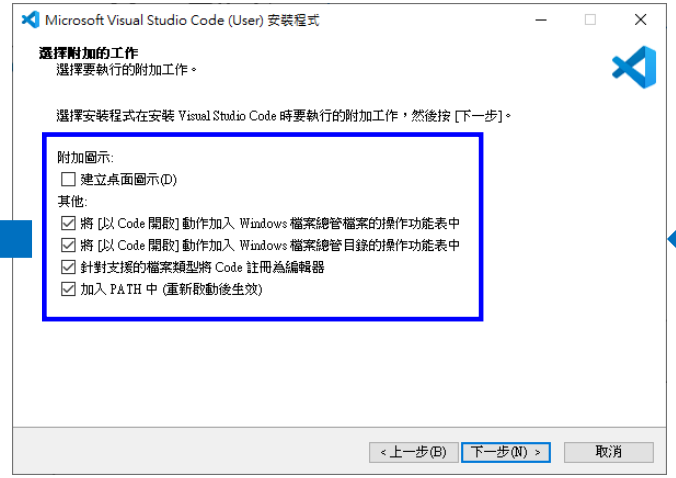
安裝 VS Code (2/7)

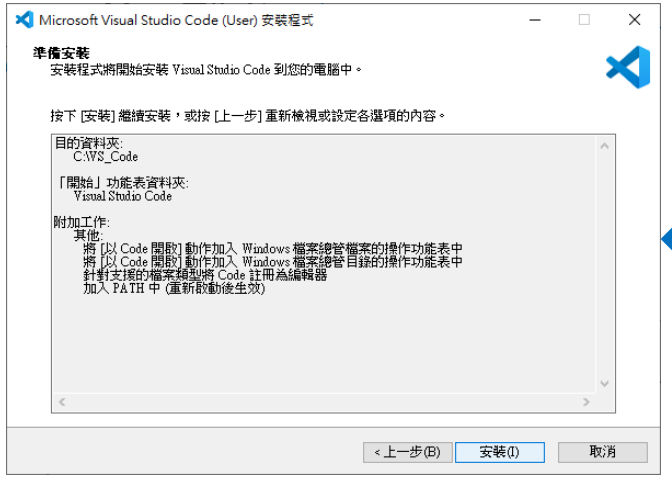

VSCodeUser
Setup-x64-1
.79.2.exe


Microsoft Visual Studio Code (User) 安裝程式
授權合約
請閱讀以下授權合約。
請閱讀以下授權合約，您必須接受合約的各項條款才能繼續安裝。
MICROSOFT 軟體授權條款
MICROSOFT VISUAL STUDIO CODE
本授權條款是貴用戶與 Microsoft Corporation (或其關係企業，視貴用戶所居住的地點而定) 之間成立的合約。它們適用於上述軟體，本授權條款亦適用於任何 Microsoft 服務或
我同意(A)
我不同意(D)
下一步(N) >
取消


Microsoft Visual Studio Code (User) 安裝程式
選擇目的資料夾
選擇安裝程式安裝 Visual Studio Code 的位置。
安裝程式將會把 Visual Studio Code 安裝到下面的資料夾。
按 [下一步] 繼續，如果您想選擇另一個資料夾，請按 [瀏覽]。
C:\Users\Ian\VS_Code
瀏覽(R)...
最少需要 343.2 MB 磁碟空間。
< 上一步(B)
下一步(N) >
取消


Microsoft Visual Studio Code (User) 安裝程式
選擇「開始」功能表的資料夾
選擇安裝程式建立程式的捷徑的位置。
安裝程式將會把程式的捷徑建立在下面的「開始」功能表資料夾。
按 [下一步] 繼續，如果您想選擇另一個資料夾，請按 [瀏覽]。
Visual Studio Code
瀏覽(R)...
不要在「開始」功能表中建立資料夾(D)
< 上一步(B)
下一步(N) >
取消

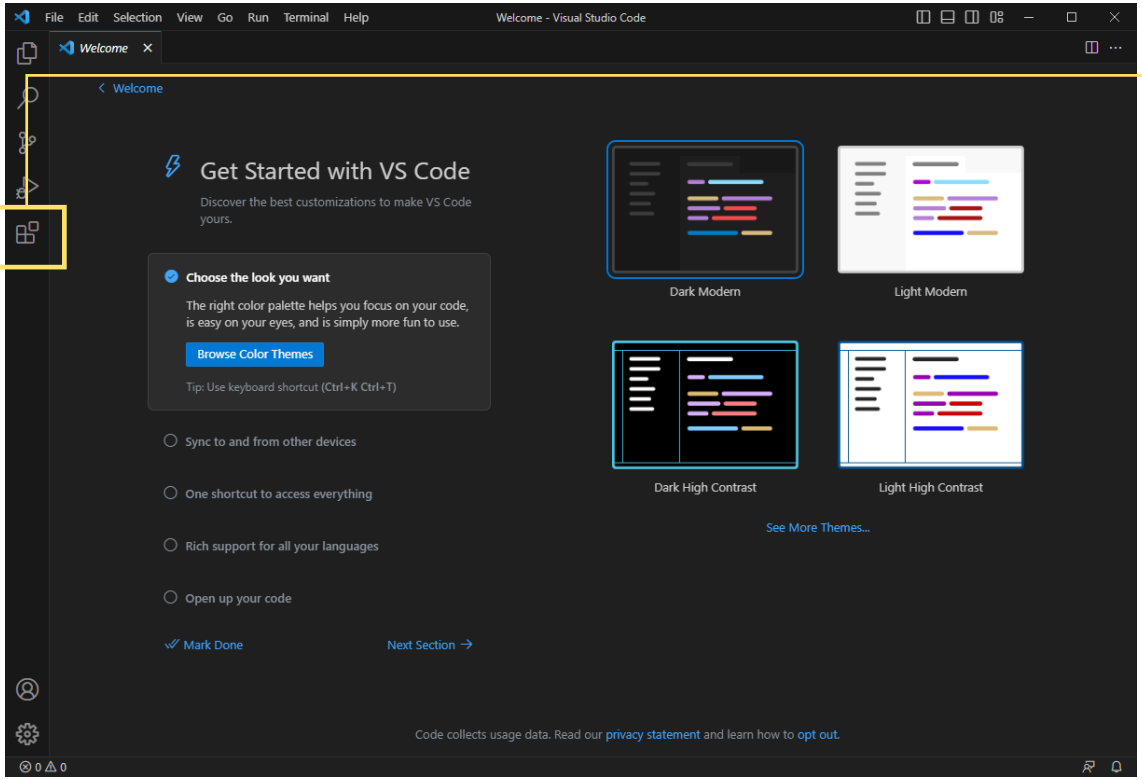

Microsoft Visual Studio Code (User) 安裝程式
選擇附加的工作
選擇要執行的附加工作。
選擇安裝程式在安裝 Visual Studio Code 時要執行的附加工作，然後按 [下一步]。
附加圖示:
建立桌面圖示(D)
其他:
將 [以 Code 開啟] 動作加入 Windows 檔案總管檔案的操作功能表中
將 [以 Code 開啟] 動作加入 Windows 檔案總管目錄的操作功能表中
針對支援的檔案類型將 Code 註冊為編輯器
加入 PATH 中 (重新啟動後生效)
< 上一步(B)
下一步(N) >
取消

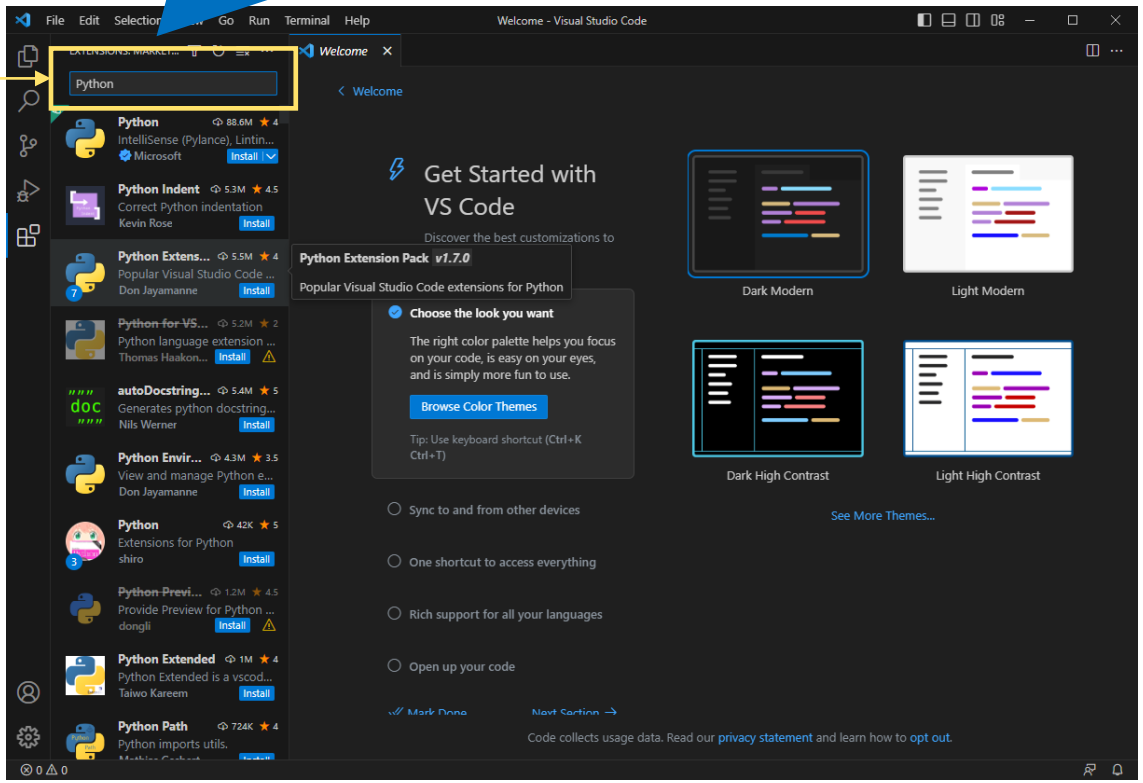

Microsoft Visual Studio Code (User) 安裝程式
準備安裝
安裝程式將開始安裝 Visual Studio Code 到您的電腦中。
按下 [安裝] 繼續安裝，或按 [上一步] 重新檢視或設定各選項的內容。
目的資料夾:
C:\VS_Code
「開始」功能表資料夾:
Visual Studio Code
附加工作:
其他:
將 [以 Code 開啟] 動作加入 Windows 檔案總管檔案的操作功能表中
將 [以 Code 開啟] 動作加入 Windows 檔案總管目錄的操作功能表中
針對支援的檔案類型將 Code 註冊為編輯器
加入 PATH 中 (重新啟動後生效)
< 上一步(B)
安裝(I)
取消

安裝 VS Code (3/7)

- 安裝編寫 Python 的擴充功能

- Python
- Code Runner (能讓執行程式更方便)
- Chinese (Traditional) Language Pack for Visual Studio Code (翻譯為繁中)

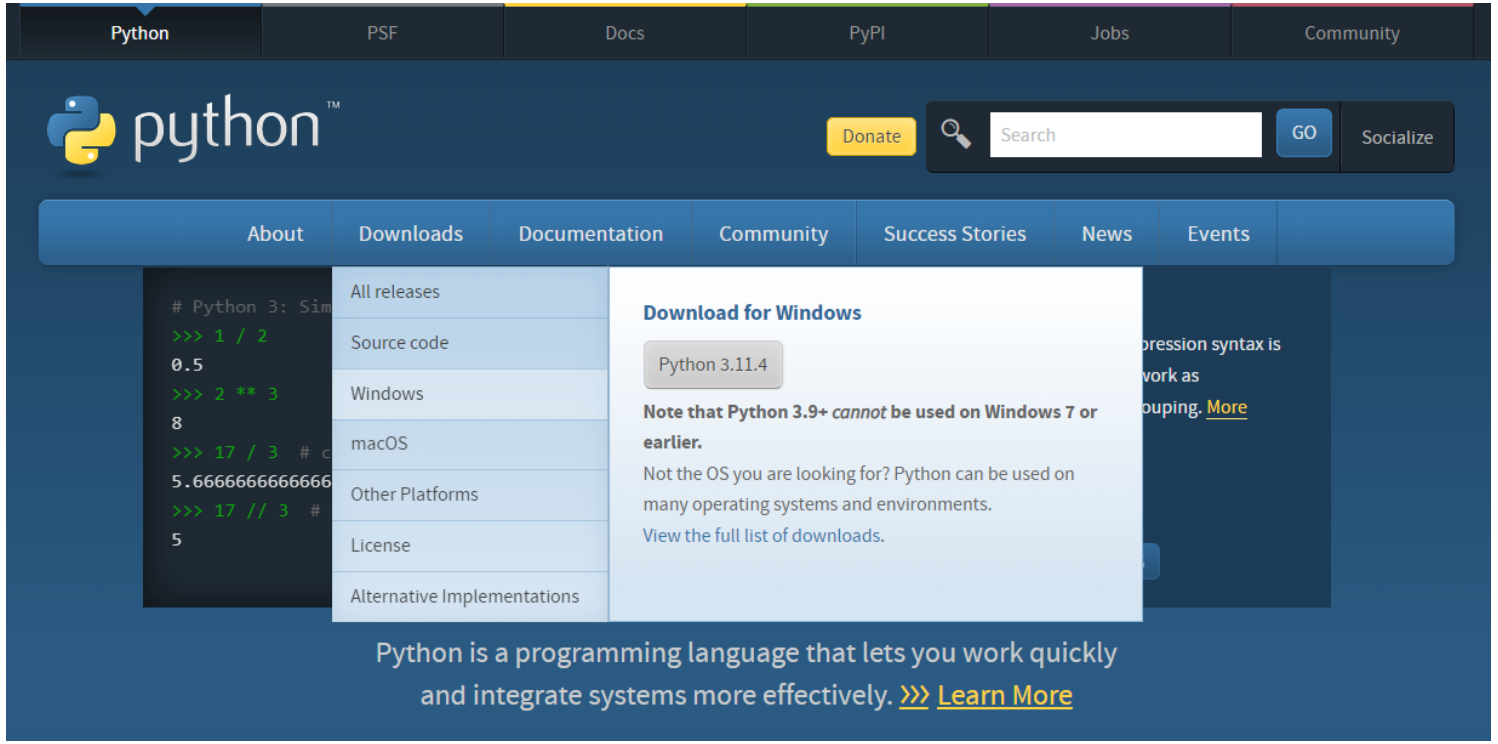




安裝 VS Code (4/7)

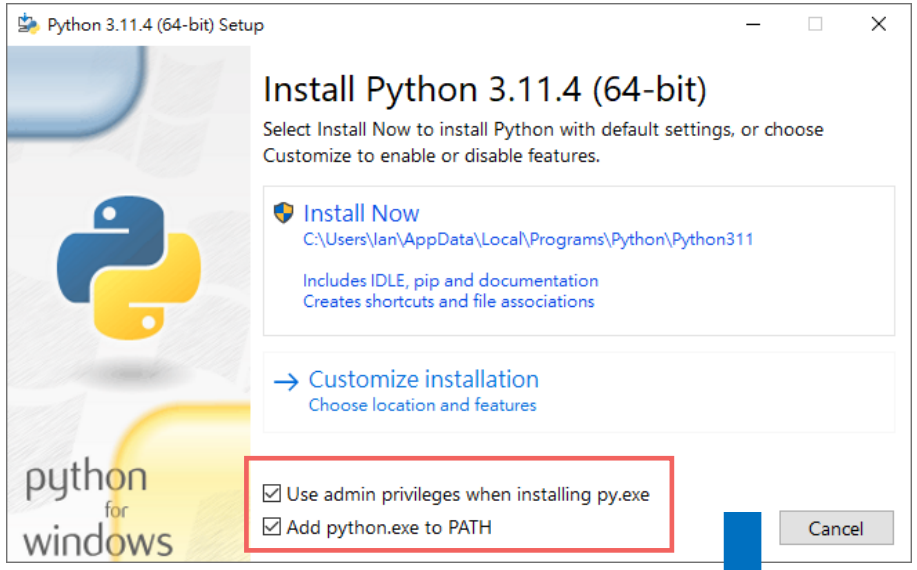
- 到官網下載 Python 安裝檔
  - python-3.11.4-amd64.exe (≈ 24 MB)

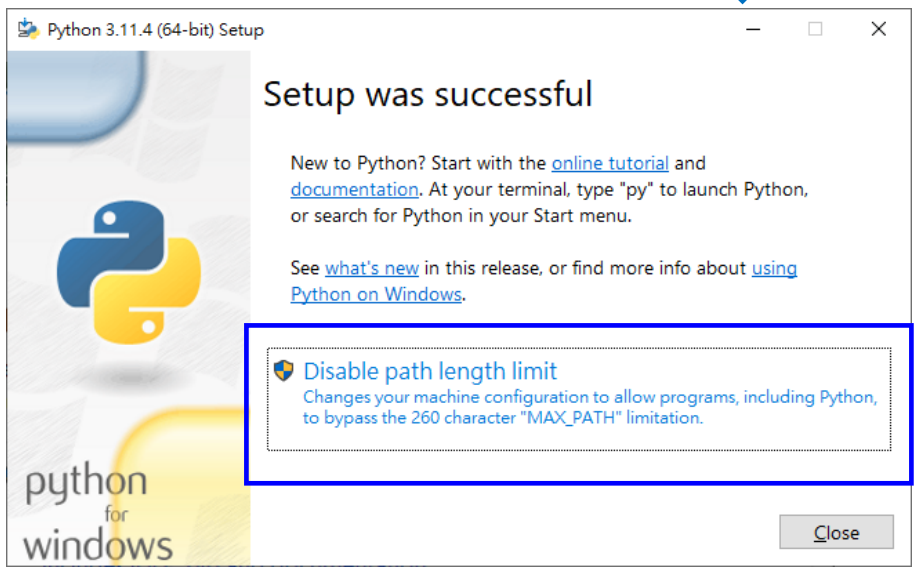
https://www.python.org/

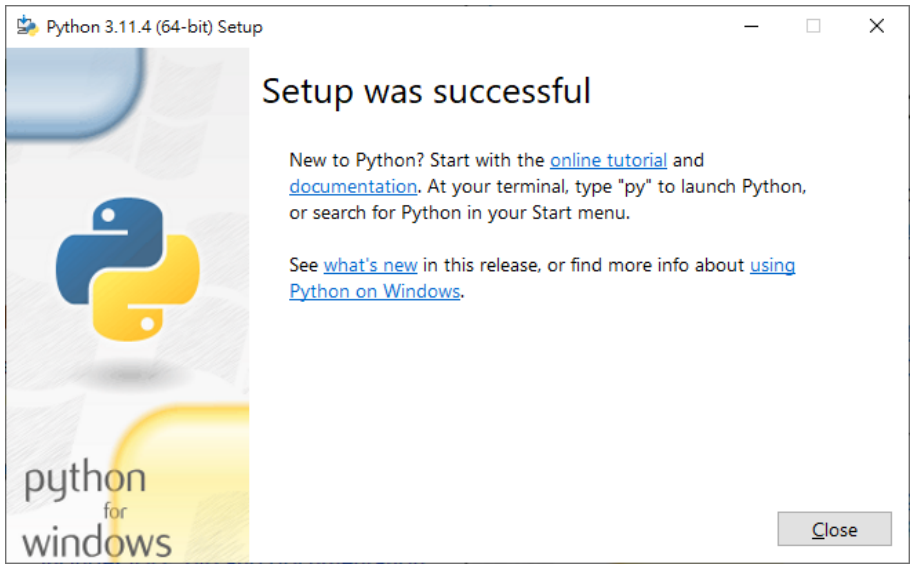


安裝 VS Code (5/7)

python-3.11.4-amd64.exe

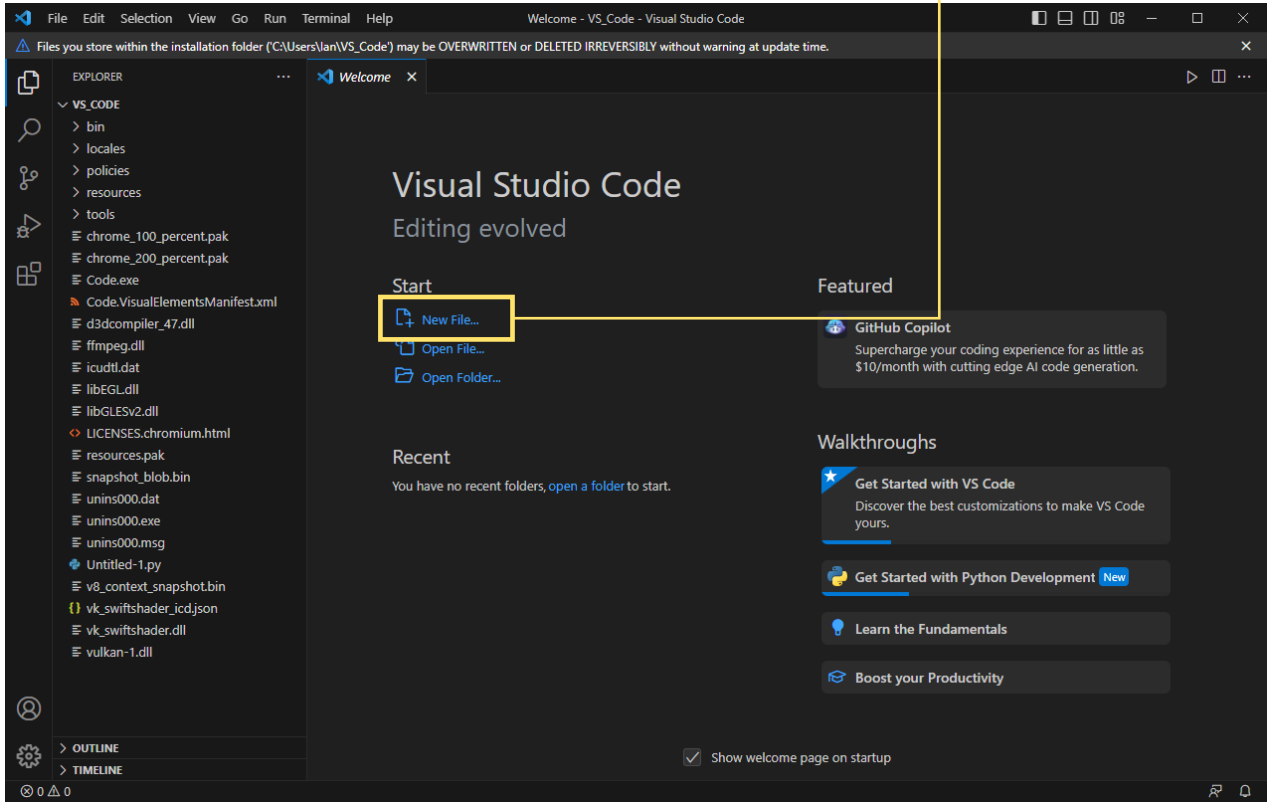

Python 3.11.4 (64-bit) Setup
Install Python 3.11.4 (64-bit)
Select Install Now to install Python with default settings, or choose
Customize to enable or disable features.
Install Now
C:\Users\Ian\AppData\Local\Programs\Python\Python311
Includes IDLE, pip and documentation
Creates shortcuts and file associations
Customize installation
Choose location and features
python for windows
Use admin privileges when installing py.exe
Add python.exe to PATH
Cancel

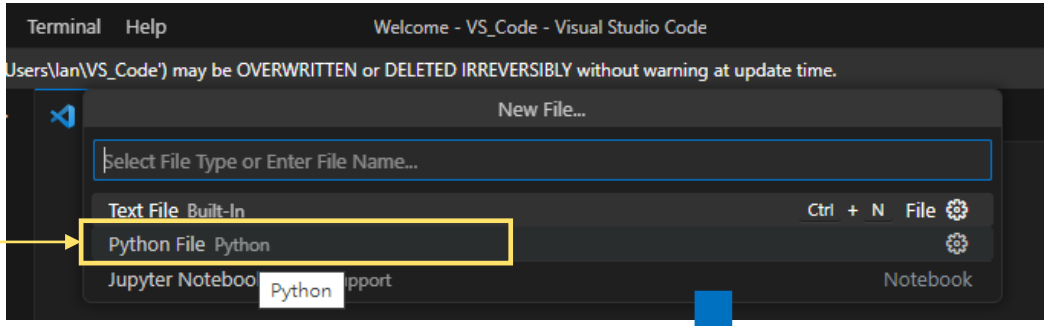

Python 3.11.4 (64-bit) Setup
Setup was successful
New to Python? Start with the online tutorial and
documentation. At your terminal, type "py" to launch Python,
or search for Python in your Start menu.
See what's new in this release, or find more info about using
Python on Windows.
Disable path length limit
Changes your machine configuration to allow programs, including Python,
to bypass the 260 character "MAX_PATH" limitation.
python for windows
Close

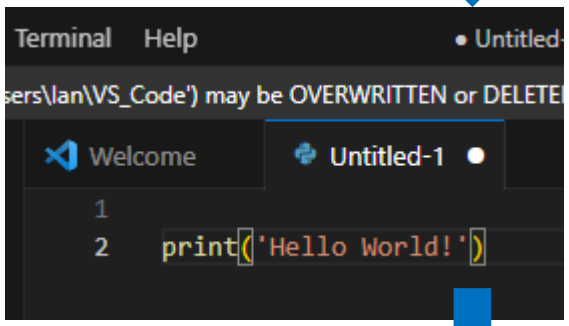

Python 3.11.4 (64-bit) Setup
Setup was successful
New to Python? Start with the online tutorial and
documentation. At your terminal, type "py" to launch Python,
or search for Python in your Start menu.
See what's new in this release, or find more info about using
Python on Windows.
python for windows
Close

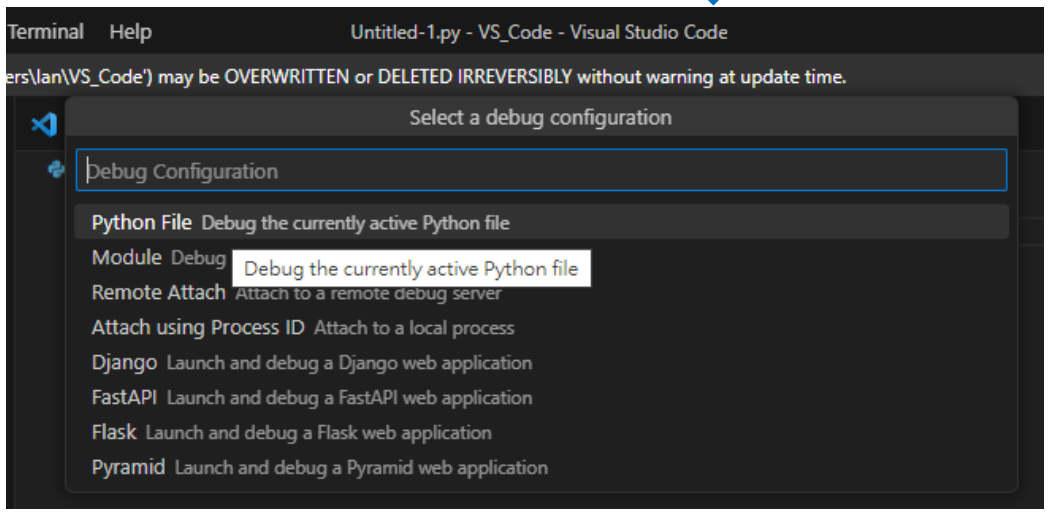
安裝 VS Code (6/7)

- 第一支 Python 程式



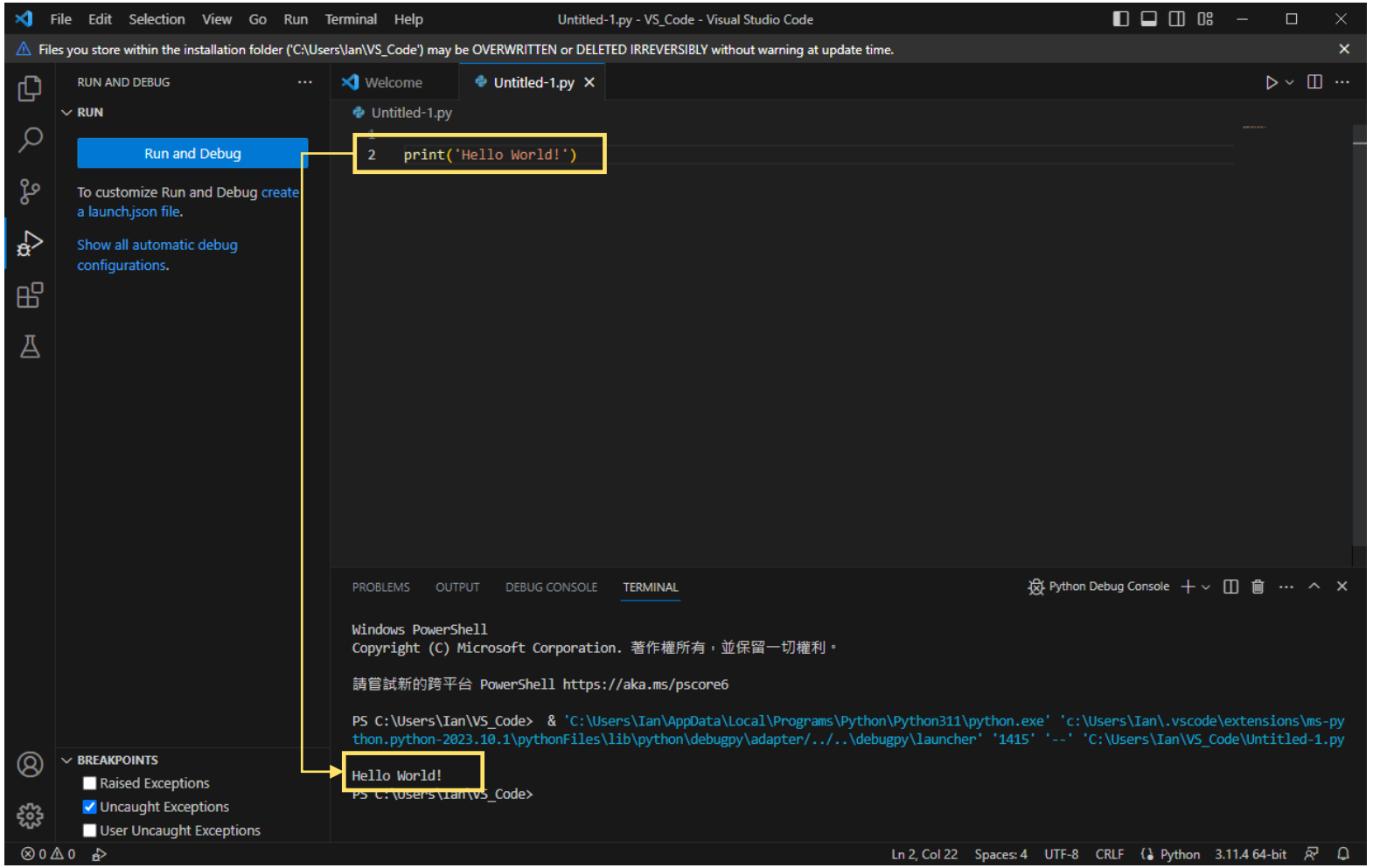






安裝 VS Code (7/7)

- 第一支 Python 程式



Documents [系統管理員]
檔案總管
DOCUMENTS
ArcGIS
My Videos
Python Scripts
R
SnagIt
xgen
.Rhistory
desktop.ini
LotusInstall.log
pymel.log
Untitled-1.py
大綱
時間表
歡迎
Untitled-1.py
Untitled-1.py
1 print("Hello")
問題
輸出
偵錯主控台
終端機
Python Debug Console
PS C:\Users\user\Documents> & 'C:\Users\user\AppData\Local\Programs\Python\Python312\python.exe' 'c:\Users\user\.vscode\extensions\ms-python.python-2023.20.0\pythonFiles\
s\user\Do
Hello
PS C:\Users\user\Documents>
第 1 行，第 15 欄
空格: 4
UTF-8
CRLF
Python
3.12.0 64-bit
快速功能鍵的顯示列表
快速功能鍵
程式撰寫區域
執行結果
狀態列

電腦技能基金會(CSF)

- 相比起 Anaconda，VS Code 的安裝方式的確有些繁瑣，對初學者而言並不友善。

- 隨著 TQC+ 各項程式語言認證陸續推出，也越來越多人使用 VS Code 應考，為了讓編輯環境的設定更便利，電腦技能基金會製作一個擴充插件組合包「CSF Extension」打包多個相依插件，以便於簡化安裝程序。
  - 安裝與設定方式可參考 https://csfrd.gitbook.io/csfextension-tao-jian-jie-shao/vscode。

主題

- Python 簡介
- 建置 Anaconda 開發環境
- Visual Studio Code 編輯器
- 程式設計的邏輯思維
  - 程式設計的錯誤類型
  - 生成式 AI 與大型語言模型的影響

運算思維(Computational Thinking)

- 在各種新興科技的推波助瀾下，程式設計受到全球高度重視，不僅是個人視其為職涯加值的工具，許多政府也將其視為打造國家競爭力的利器。

- 因應這股全球的程式浪潮，教育部在 108 課綱裡規定國、高中生必修「資訊科技」課程，讓學生能儘早接觸程式。
  - 以運算思維為核心，期待學生們能學會拆解問題，找出背後運作的規律與邏輯，進而透過程式設計，重複執行演算方式的指令流程。

運算思維(1/5)

- 運算思維可以被當成是解決問題的過程，它以結構化形式分解問題並利用觀察、創造力、解決問題等不同能力來設計解決方案，而這樣能重複且逐步執行的簡單步驟，正是電腦解決問題的方式。

- 運算思維訓練大腦進行分析並找到問題的合理解決方案，而程式設計則提供一種學習運算並嘗試自己想法的有趣方式，可說運算思維就是程式設計的邏輯思維。

運算思維(2/5)

- 根據 BBC 的說法，運算思維有底下四個階段：
  - 拆解(decomposition)：將一個任務或問題拆分成數個步驟或部分。
  - 模式識別(pattern recognition)：在問題間與問題內尋找規律或模式。
  - 抽象(abstraction)：專注在重要資訊而忽略無關的細節。
  - 演算法(algorithms)：設計出能解決類似問題且能重複執行的指令流程。

運算思維(3/5)

- 在面對一時間難以解決的複雜問題，藉由運算思維模式先
  - 將問題分解成一系列更容易處理的小問題（拆解），
  - 再單獨探查每個小問題，考慮以往如何處理類似問題（模式識別），
  - 並且只關注重要細節，而忽略無關訊息（抽象），
  - 最後就是設計出解決每個小問題的簡單步驟或規則（演算法）。

- 明確整個思維流程後，再透過程式設計實現這些簡單的步驟或規則，以最佳方式解決複雜問題。

運算思維(4/5)

- 時下許多學生，即使在經過多次習題練習之後，對於稍微變化的題型卻只能束手無策，因為他們大多已養成「面對問題先學會對應解法」的既定思維。

- 在訓練運算思維的過程中，能逐步培養學生用不同角度及既有資源解決問題的能力，而雖說程式設計並非磨練問題解決能力的唯一做法，卻是一條經濟、快速、靈活且符合當下趨勢的途徑。

運算思維(5/5)

- 蘋果公司執行長庫克(Tim Cook)認為程式語言能夠激發人們的創造力，且學習過程中也能夠發揮個人的批判思考能力。
  - 因此，無論未來是否會成為工程師，具備運算思維對於學業與生活，絕對是百利而無一害。

Source: 蘋果庫克力推程式教育：從小學就該教！

主題

- Python 簡介
- 建置 Anaconda 開發環境
- Visual Studio Code 編輯器
- 程式設計的邏輯思維
  - 程式設計的錯誤類型
  - 生成式 AI 與大型語言模型的影響

除錯(debug)

- 想要以程式實現運算思維的每一個階段，最終準確地操控電腦完成任務，邏輯思考的能力必不可少。
  - 從解析問題開始，到設計演算法流程，並能以正確語法語意表達每個步驟，都必須有清晰的思路。
  - 在遇到程式無法順利執行，或是得到非預期中的結果時，能反覆檢視程式碼並進行除錯。

程式設計的錯誤類型

- 除錯可說是程式設計人員的日常，尤其對火候不夠的新手來說，單單除錯就常常會佔掉絕大部分時間，更別說整個除錯過程是相當無聊且漫長。

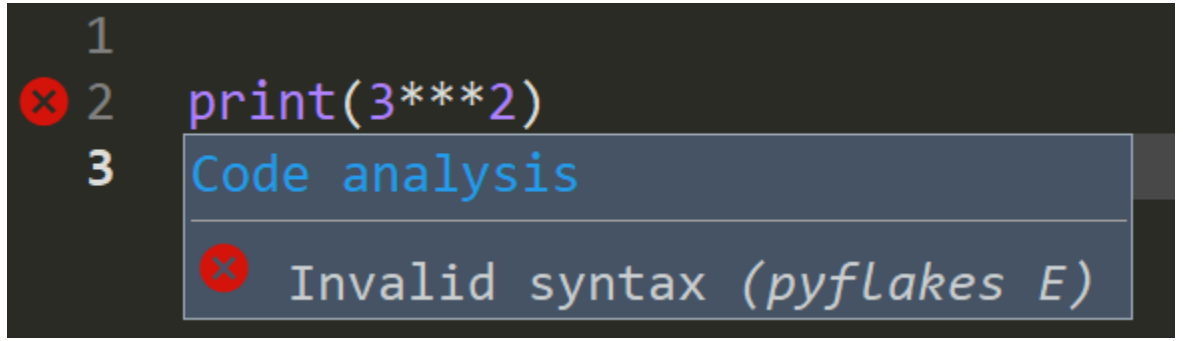
- 常見的程式設計錯誤有以下幾種類型：
  - 語法錯誤(syntax error)
  - 執行期間錯誤(runtime error)
  - 邏輯錯誤(logic error)

語法錯誤(1/2)

- 這是初學者最容易發生的錯誤，幸好也是最容易處理的錯誤。
  - 任何一種程式語言都有其專屬的語法（如縮排、關鍵字、語法結構等），必須嚴格遵守，一旦誤用就會引發錯誤。
  - 得力於各程式語言 IDE 的支持，多數語法錯誤皆能被標示出來，甚至還能自動更正。
  - 若是執行一段有語法錯誤的程式碼，也會得到「SyntaxError」的相關錯誤訊息，此時只要依照提示修正即可。

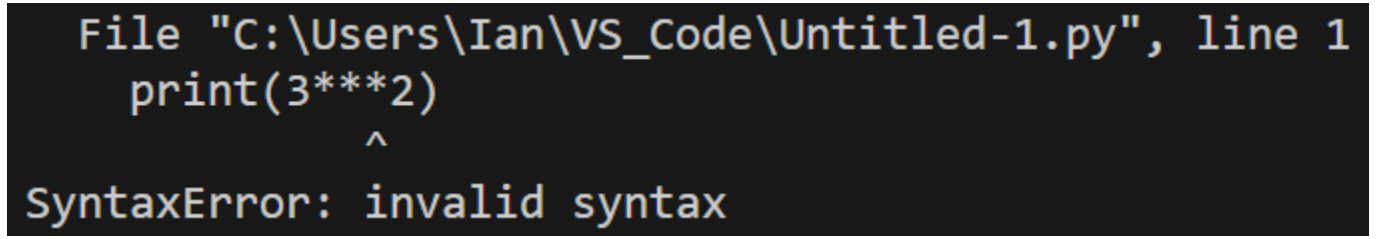
語法錯誤(2/2)

Spyder 編寫時的錯誤標示



VS Code 編寫時的錯誤標示

VS Code 執行時顯示的錯誤訊息



執行期間錯誤(1/2)

- 這是指程式在執行期間引發的錯誤，其肇因往往不是語法問題，而是某些看似正確卻無法執行的程式碼。

```
1    a = 1        # 設定變數 a 為 1
2    b = 0        # 設定變數 b 為 1
3    print(a/b)   # 將 a/b 的結果輸出到螢幕
```

執行這段程式碼
得到的錯誤訊息

```
  File "C:\Users\Ian\VS_Code\Untitled-1.py", line 3, in <module>
    print(a/b)  # 將 a/b 的結果輸出到螢幕
ZeroDivisionError: division by zero
```

執行期間錯誤(2/2)

- 此時，可直接根據顯示的錯誤訊息做修正，也可在 Stack Overflow 網站上找到各式各樣的解決方案。

```
1    a = 1        # 設定變數 a 為 1
2    b = 0        # 設定變數 b 為 1
3    print(a/b)   # 將 a/b 的結果輸出到螢幕
```

執行這段程式碼
得到的錯誤訊息

```
  File "C:\Users\Ian\VS_Code\Untitled-1.py", line 3, in <module>
    print(a/b)  # 將 a/b 的結果輸出到螢幕
ZeroDivisionError: division by zero
```

邏輯錯誤(1/2)

- 這是最麻煩的錯誤類型，不管是在編寫過程還是執行程式時皆不會主動產生錯誤訊息，而是需要透過不斷測試才能發掘出錯誤。

```
1    num = int(input("整數 x ="))
2    print("x 的平方 =", num*2)
```

輸入 0 或 2 皆得到正確答案

```
整數 x =0
x 的平方 = 0
```

```
整數 x =2
x 的平方 = 4
```

輸入 3 才發現答案錯誤

```
整數 x =3
x 的平方 = 6
```

邏輯錯誤(2/2)

- 需要由執行結果不符合預期來判斷是否有邏輯錯誤，而經驗不足時不僅僅難以發現有邏輯錯誤。因此，這類型的錯誤常導致開發及測試人員耗費大量時間與精力。

```
1    num = int(input("整數 x ="))
2    print("x 的平方 =", num*2)
```

輸入 0 或 2 皆得到正確答案

```
整數 x =0
x 的平方 = 0
```

```
整數 x =2
x 的平方 = 4
```

輸入 3 才發現答案錯誤

```
整數 x =3
x 的平方 = 6
```

主題

- Python 簡介
- 建置 Anaconda 開發環境
- Visual Studio Code 編輯器
- 程式設計的邏輯思維
  - 程式設計的錯誤類型
  - 生成式 AI 與大型語言模型的影響

生成式 AI (1/2)

- 人工智慧(AI)的發展如日中天，它在各領域的應用（如人臉識別、語音辨識、自動駕駛等）已逐漸融入我們生活中。
  - 生成式 AI (Generative AI)是人工智慧的一個分支，可以創造出文章、音樂、圖像等新內容。
  - 例如 OpenAI 推出的 DALL-E 與 Midjourney，能用簡單的文字描述和風格設定快速地生成對應圖像。

生成式 AI (2/2)

- 生成式 AI 的出現，讓人們能以更快、更好、更低成本的方式進行創作，這股革命的浪潮正在各個角落悄然展開，即將帶來前所未有的衝擊。
  - 諮詢機構麥肯錫(McKinsey & Company)在 2023.6 發布《生成式AI的經濟潛力》研究報告，指出生成式 AI 帶來的價值增長，約 75% 集中在客戶服務、市場行銷、產品銷售、軟體工程和開發等四個領域，這也意味著這四項業務受生成式 AI 影響最大。
  - 同時，報告也指出生成式 AI 可能對高薪的腦力工作者產生最大的影響。相較之下，以往自動化的影響主要集中在中低收入族群。

ChatGPT (1/2)

- 2022.11.30 OpenAI 發表的語言生成模型 ChatGPT 神速爆紅，不到一週突破百萬用戶，兩個月後的月活躍用戶數更突破1億。
  - ChatGPT支援多國語言與友善的人機介面，能透過文字與人類以自然對話方式進行互動。
  - 接著，2023.3.14 推出進階的 GPT-4 大型語言模型(Large Language Model, LLM)，隨後於 2023.11.6 發表更強大的 GPT-4 Turbo。

ChatGPT (2/2)

- GPT-4 除了更便宜的費用，支援高達 128k 的上下文內容（超過 2 萬個中文字）之外，能讀取的外部資料也更新截至 2023.4。

- 此外，由於訓練資料集中包含程式碼，加上強大的語義理解與程式碼生成能力，ChatGPT 也能生成各程式語言的程式碼，作為協助開發人員編寫程式的工具，能提高工作效率並擴展創意。

測試 ChatGPT

- 3n+1 猜想（考拉茲猜想）
  - 對一個正整數 x 進行一連串運算：
  - 如果 x 是奇數，則將其乘 3 再加 1
  - 而如果 x 是偶數，則將其除以 2，
  - 如此重複運算最終都能夠得到 1。

You

請撰寫一段Python程式，讓使用者輸入一個正整數x，驗證x是否符合「3n+1猜想」。如果符合猜想，則輸出x經過運算到1的數字序列。

ChatGPT

當然，雀特居披踢大師。這是我剛才為「3n+1猜想」撰寫的Python程式碼：

python — Copy code

```python
def collatz_conjecture(x):
    if x <= 0:
        return "請輸入一個正整數"

    sequence = [x]
    while x != 1:
        if x % 2 == 0:
            x = x // 2
        else:
            x = 3 * x + 1
        sequence.append(x)

    return sequence

# 示範使用函式
# 此處為了展示，我們先假設用戶輸入的數字是6
example_input = 6
collatz_conjecture(example_input)
```

這個函式會先檢查輸入的數字是否為正整數。如果是，它會開始應用「3n+1」的規則，直到數字變為1為止。同時，它會記錄下每一步的數字，最後將這個序列返回。

程式設計師飯碗不保!?

- GPT-4 生成的程式碼在結構、準確度的品質上已有相當水準，足可提昇開發人員的效率。
  - 使用 ChatGPT 輔助撰寫應用程式，雖然有流量限制、有時生成的速度過慢等缺點，其生成的程式碼確實能大幅縮短開發時間。
  - 正因為如此，一時間類似「程式設計師飯碗不保」的言論甚囂塵上，彷彿要顛覆整個程式設計行業一般，使得程式開發人員如坐針氈，而想從事相關行業也猶豫不決。

使用 ChatGPT (1/2)

- 從學習的角度來看，即便是有電子計算機，我們仍舊得學算術運算，背誦九九乘法表；而即使有科學計算軟體與程式套件能計算微分積分，微積分仍舊是理工商的大一必修課程。
  - 因此，即使 ChatGPT 能生成品質佳的程式碼，相信程式語言仍然是資訊相關科系的必修課，畢竟如前面所述，學寫程式重點在學運算思維與邏輯思考。
  - 在學習階段用 ChatGPT 輔助了解語法使用、除錯等地方，培養閱讀與分析程式碼的能力，萬萬不能照搬煮碗，不加思索就拿來使用。

使用 ChatGPT (2/2)

- 至於有一定程度的程式設計能力後，可以利用它來提升開發效率，但同時也要注意 ChatGPT 的侷限性。
  - 比如它有時會產生幻覺，生成虛構、不準確甚至是錯誤的程式碼，需要加以檢查和驗證。
  - 當用戶的語義不明時，它會自行猜測而非詢問清楚。
  - 它抵抗不懷好意的「提示工程」能力較差，也容易有資安風險。

本章結束

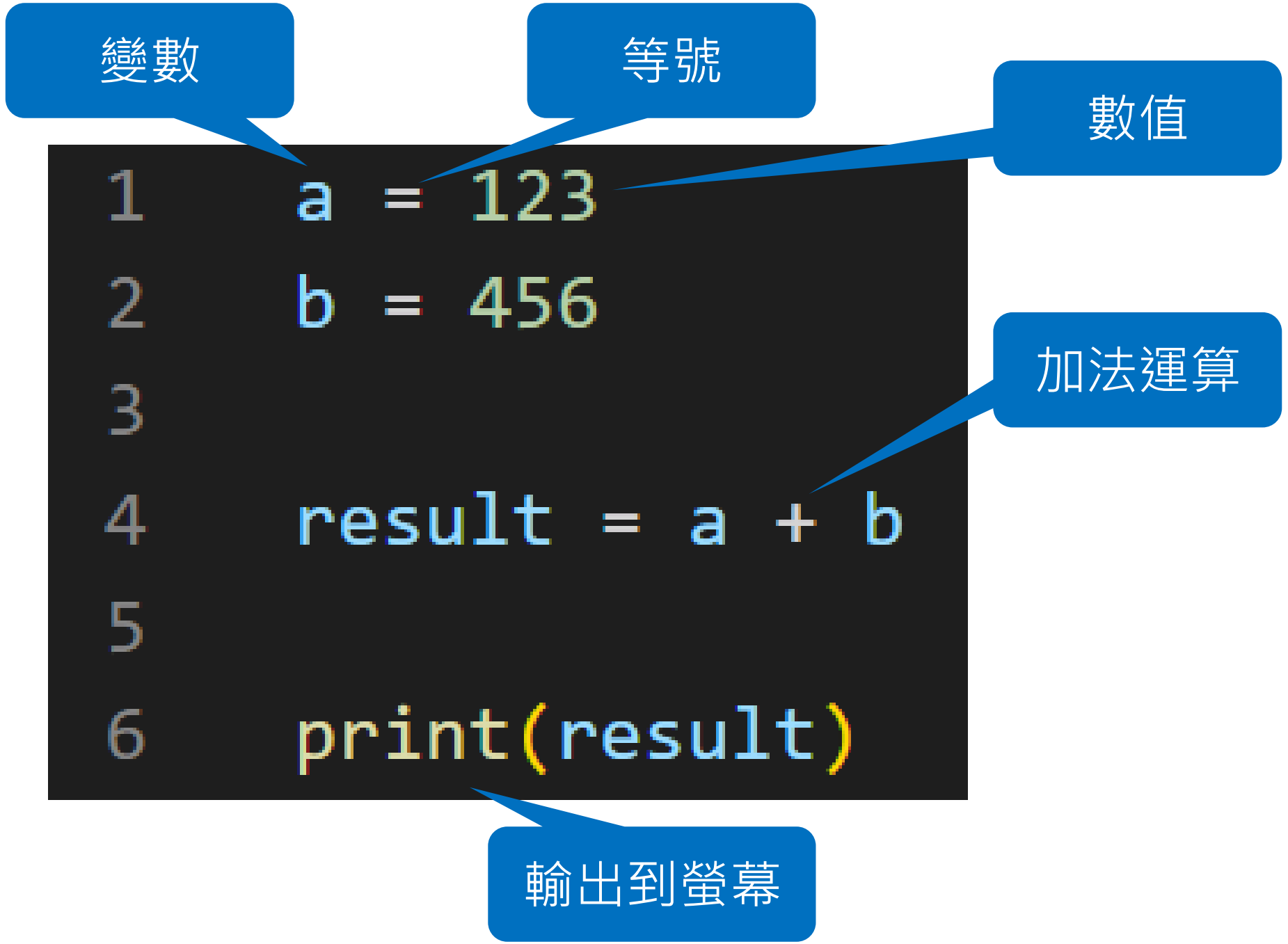
Py01

基本認識 (1)

王頌文

主題

- 變數與資料型別
  - 變數與命名規則
  - 數值、布林與字串資料型別
  - 資料型別轉換
  - 註解
- 輸出與輸入
- 運算式


變數
等號
數值
加法運算
1 a = 123
2 b = 456
3
4 result = a + b
5
6 print(result)
輸出到螢幕

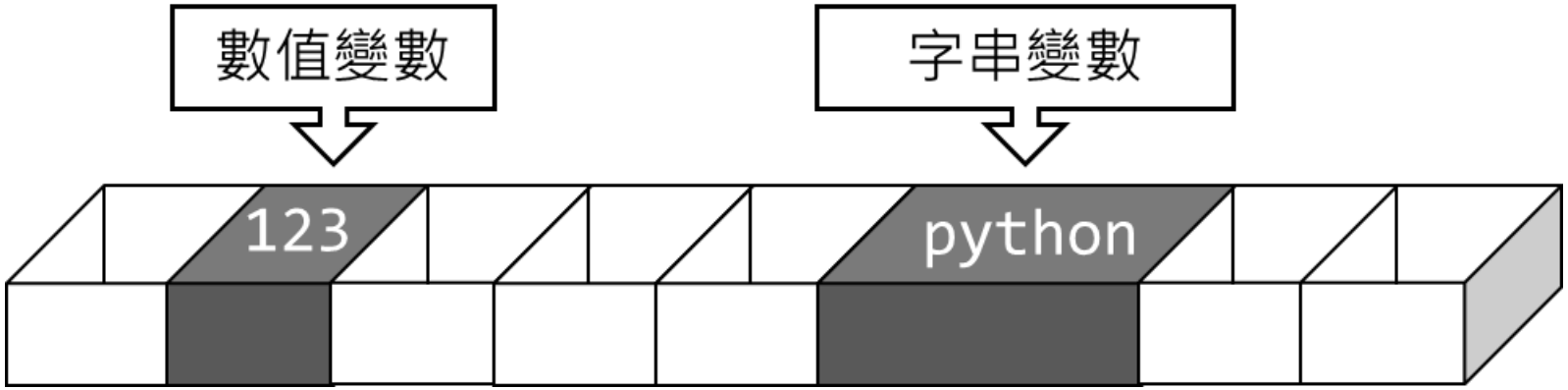
變數

儲存資料的地方

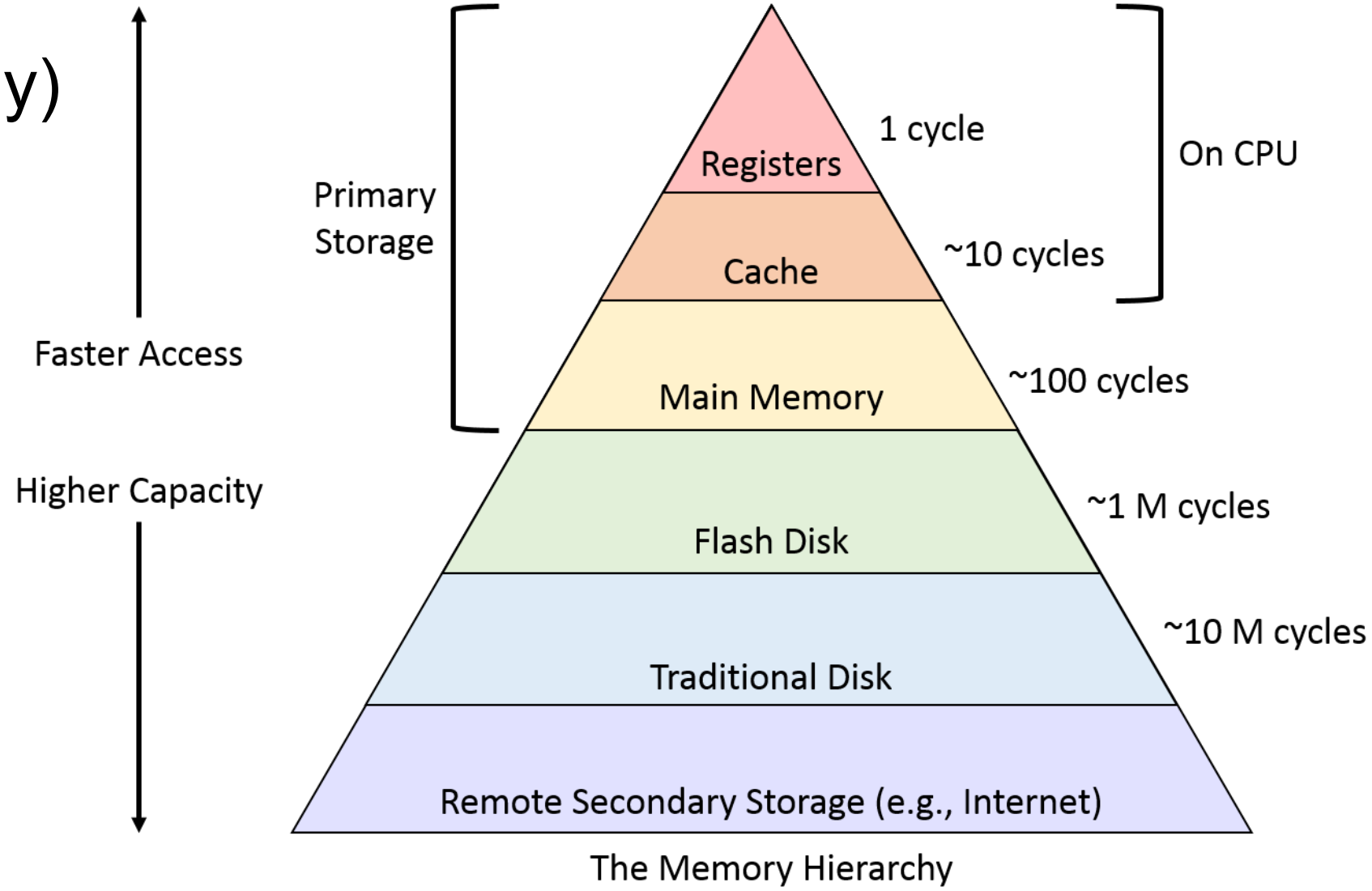
變數(variable)

- 當變數被使用時，系統會配置一塊記憶體空間給此變數。
  - 變數名稱 ➔ 此塊記憶體的位址
  - 資料型別 ➔ 此塊記憶體的大小

e.g., 0x7ffff34fff00



記憶體階層 (memory hierarchy)



Source: https://www.cs.swarthmore.edu/~kwebb/cs31/f18/memhierarchy/mem_hierarchy.html

主題

- 變數與資料型別
  - 變數與命名規則
  - 數值、布林與字串資料型別
  - 資料型別轉換
  - 註解
- 輸出與輸入
- 運算式

命名變數(1/2)

- 為變數命名必須遵守一定規則，程式才能正常運作。
- Python 的變數命名一般有四條規則：
  - ① 可以使用大小寫英文字母、數字及底線(_)
    - 一些特殊字元（如&、*等）不能使用
    - pokemon-hp ➔ ☹
  - ② 開頭不能為數字
    - 7_11 ➔ ☹
    - _11 ➔ ☺

命名變數(2/2)

- 為變數命名必須遵守一定規則，程式才能正常運作。
- Python 的變數命名一般有四條規則：
  - ③ 英文字母大小寫視為不同變數名稱
    - HP 與 hp 為不同變數
  - ④ 不能使用 Python 的保留字(reserved word)
    - 也稱為關鍵字(keyword)

Python 常見的保留字

and	def	global	or	False
as	del	if	pass	True
assert	elif	import	raise	None
async	else	in	return	
await	except	is	try	
break	finally	lambda	while	
class	for	nonlocal	with	
continue	from	not	yield	

這些變數名稱合法嗎？

pokemon_hp

true

7eleven

John&Mary

Angela baby

and

特別注意

- sum, max, min, len
  - 合法的變數名稱？

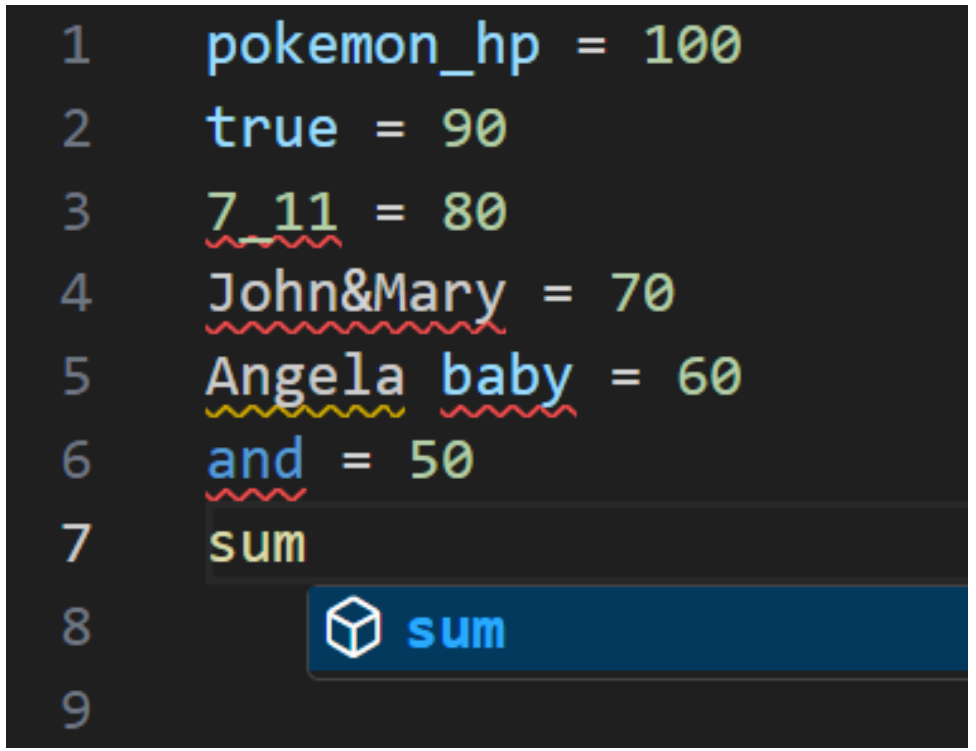
```
3 sum = 100
4 print(sum)
5
6 score = [90, 80, 95]
7 print(sum(score))
```

- 雖然合法，但**強烈不建議使用**
  - 因為和內建函式的名稱衝突

```
TypeError: 'int' object is not callable
```

善用編輯器功能

- 語法突顯(syntax highlighting) + 自動完成(auto-complete)



支援 Unicode

- Python 的變數命名一般有四條規則：
  - ① 可以使用大小寫英文字母、數字及底線(_)
    - 一些特殊字元（如&、*等）不能使用

- 上面這條是 Python 3.0 之前的版本所遵循
  - 官方在 2007 年設計 3.0 版本時增加對 Unicode（也稱為統一碼、萬國碼）的支援

PEP 3131 – Supporting Non-ASCII Identifiers

Author:	Martin von Löwis <martin at v.loewis.de>
Status:	Final
Type:	Standards Track
Created:	01-May-2007
Python-Version:	3.0

  - Unicode 不僅支援中文，更編碼世界上大部分的文字系統。

範例

- 仍然不建議使用中文為變數命名
  - 除了輸入不便、降低程式可攜性(portability)之外，也讓非中文語系的人不易閱讀。

```
2    寶可夢HP = 100
3    π = 3.14
4    θ = 80
5    ʘ_ʘ = 50
```

命名習慣(naming convention)

- 對電腦來說，變數名稱的好壞並不重要，真正有影響的是變數命名習慣，這將影響整個程式的可讀性及可維護性。
  - 畢竟變數是給人看的

- 命名習慣的優劣有兩點：
  - 可讀性：變數名稱要有意義，最好與儲存的資料性質有關。
  - 一致性：用相同的規則命名整個程式的變數。

常用的命名方式

- 駝峰式命名法(Camel Case)：第一個字母小寫，之後每一個單字的開頭為大寫，不能有空格。
  - e.g., studentName, stuName, pokemonHP

- 蛇形命名法(Snake Case)：單字皆為小寫，且單字間以底線 (_) 分離。
  - e.g., student_name, stu_name, pokemon_hp

Python 採用這個方式

常數的命名

- 有些變數儲存的值在程式執行期間不會變動，這類特殊的變數稱之為常數(constant)。
  - 比如圓周率 π、自然常數 e
  - 建議使用全部皆為大寫的字母來表示，以與一般變數有所區別。

```
1    PI = 3.1415926
2    E = 2.7182818
```

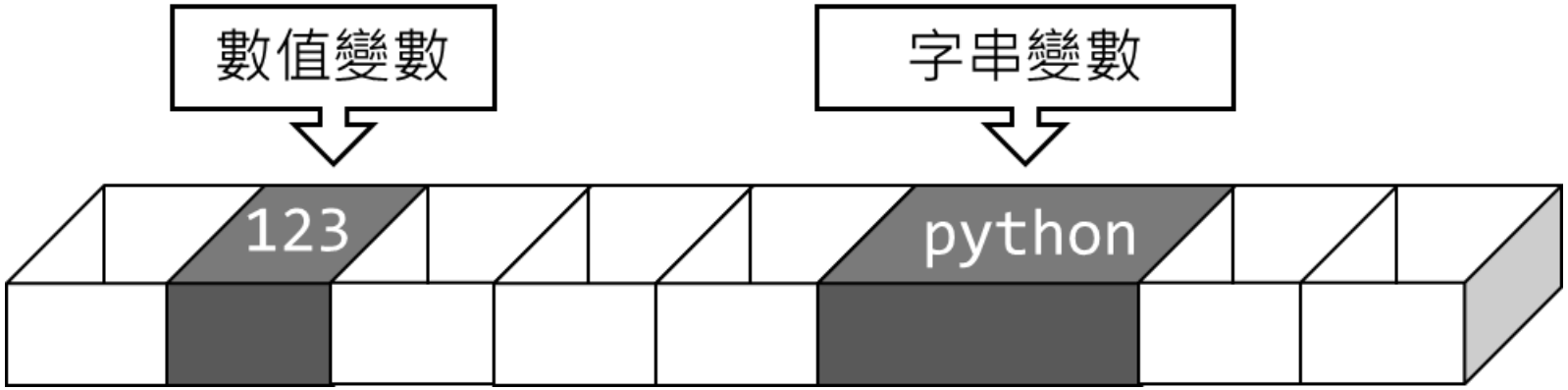
主題

- 變數與資料型別
  - 變數與命名規則
  - 數值、布林與字串資料型別
  - 資料型別轉換
  - 註解
- 輸出與輸入
- 運算式

Python 基本資料型別(1/4)

- 每個變數都有資料型別(data type)用來描述該變數儲存值的型別

- Python 的基本資料型別有三種：
  - 數值、布林、字串



查看資料型別

- 內建函式 type() 會取得變數的資料型別，語法為：
  - type(變數名稱或值)

```
1    print(type(2))     # 整數
2    print(type(2.0))   # 浮點數
3    print(type('2'))   # 字串
4    print(type(True)) # 布林值
```

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
```

Python 基本資料型別(2/4)

① 數值(numeric)

- 整數(integer)
  - 不含小數點的數值
  - e.g., 12, 0, -99
- 浮點數(float)
  - 包含小數點的數值
  - e.g., 12.21, 15.0

在數學上，整數與浮點數都是數字，並沒有太大差異，但是對電腦而言，兩者是不同的東西。

整數

- 囿於有限的記憶體空間，許多程式語言無法直接將一個超大的數儲存在變數內。
  - 以 C 語言為例，一個無號長整數(unsigned long)最大到 $2^{64}-1$，亦即 18446744073709551615。
  - 有時在科學運算過程中，真的會需要超大數的儲存及運算，這時常見做法是引入所謂大數運算(bignum arithmetic)。
  - Python 本身支援大數整數型別及運算，能夠對付任何長度的整數。

前提是記憶體要夠大 ☺

浮點數(1/5)

- Python 採用雙精度浮點數(double-precision floating-point)，使用 64 位元來儲存一個浮點數，能用來表示極大或是非常接近 0 的小數字。
  - 過大或過小的數字會直接以科學記號(scientific notation)表示
  - $123.456 \rightarrow 1.23456 \times 10^{2} \rightarrow 1.234560e+02$
  - $0.0123456 \rightarrow 1.23456 \times 10^{-2} \rightarrow 1.234560e-02$

浮點數(2/5)

- 兩個特別的數字，即無限大(inf)與未定義數(nan)，也是屬於浮點數的類型。

```
1    print(type(3.14))
2    print(1e+30000)
3    print(type(1e+30000))
4    print(0.00000000000000000000000000001)
5    print(type(0.00000000000000000000000000001))
```

```
<class 'float'>
inf
<class 'float'>
1e-28
<class 'float'>
```

浮點數(3/5)

- 不論顯示或儲存一個過大的浮點數，將產生溢位(overflow)問題，此時螢幕上會顯示錯誤訊息。

```
1    # ** 為次方運算，之後會介紹
2    print(123.456 ** 1000)
```

```
    print(123.456 ** 1000)
          ~~~~~~~~^^~~~~~
OverflowError: (34, 'Result too large')
```

浮點數(4/5)

- 從數學的角度，浮點數包含整數，按理整數是可以用浮點數表示。
  - 對電腦而言，兩者有相當大的差別。
  - 主要原因是有循環小數、無理數等小數點後有無窮多個數字的問題。
    - e.g., 1/3、$\log_{10}3$、π、e

- 有限儲存空間無法完整儲存這些擁有無窮小數的數值
  - 退而求其次採用 IEEE 756 雙精度浮點數格式來儲存其近似值

浮點數(5/5)

- 以 0.1 為例，若讓 Python 輸出完整的近似值，畫面上會看到 0.1000000000000000055511151231257827021181583404541015625 ☹

```
1    print(0.1 + 0.1)
2    print(0.1 + 0.1 + 0.1)
3    print(0.1 + 0.1 + 0.1 + 0.1)
```

```
0.2
0.30000000000000004
0.4
```

Python 基本資料型別(3/4)

② 布林(boolean)

- 真(True)、假(False)

大小寫有別

```
1    print(3 > 2)
2    print(-1 > 2)
```

```
True
False
```

Python 基本資料型別(4/4)

③ 字串(string)

- Python 字串資料型別是變數值以一對雙引號「"」或單引號「'」包起來，例如：
  - `str1 = "這是字串"`
  - `str2 = '這也是字串'`
- 如果字串要包含引號本身 (雙引號或單引號)，可使用另一種引號包住字串，例如：
  - `str3 = '胡適說過"要怎麼收穫，先那麼栽"'`

字串範例

- 有時候一串數字應該被當成字串來對待
  - 例如電話號碼 0912345678

```
1    str_ = '胡適說過"要怎麼收穫，先那麼栽"'
2
3    print(str_)
4    print(type(str_))
5    print(type('123 + 456'))
```

```
胡適說過"要怎麼收穫，先那麼栽"
<class 'str'>
<class 'str'>
```

跳脫字元(escape character)(1/2)

- 若字串含有特殊字元（如 Tab、換行等），要使用跳脫字元。
  - 以倒斜線字元「\」開頭，後面接一定格式的字元來代表特定意義。

跳脫字元	意義	跳脫字元	意義
\\	反斜線符號（\）	\n	換行符號（LF）
\'	單引號（'）	\r	返回符號（CR）
\"	雙引號（"）	\t	水平移位符號（Tab）
\a	響鈴符號（BEL）	\v	垂直跳位符號（VT）
\b	空格符號（BS）	\o	八進位編碼表示字元
\f	換頁符號（FF）	\x	十六進位編碼表示字元

跳脫字元(2/2)

跳脫字元	意義	跳脫字元	意義
\\	反斜線符號（\）	\n	換行符號（LF）
\'	單引號（'）	\r	返回符號（CR）
\"	雙引號（"）	\t	水平移位符號（Tab）
\a	響鈴符號（BEL）	\v	垂直跳位符號（VT）
\b	空格符號（BS）	\o	八進位編碼表示字元
\f	換頁符號（FF）	\x	十六進位編碼表示字元

```
1    print("你好\\歡迎光臨\\")
2    print()
3    print("恭喜\n大家發財")
4    print()
5    print("你好歡迎\r光臨")
6    print()
7    print("你好\t歡迎光臨")
8    print()
9    print("\x48\x45\x4C\x4C\x4F")
```

```
你好\歡迎光臨\

恭喜
大家發財

光臨歡迎

你好      歡迎光臨

HELLO
```

等號（=）

- 程式語言的等號並非數學上的兩邊相等，而是指派(assign)。
  - 將等號右邊的值指派給左邊的變數。
  - 有時也稱為賦值，閱讀時需要從等號右邊讀到左邊。
  - 因此「=」被稱為指派運算子(assign operator)，也稱為賦值。
    - 例如：a = 1 + 2
    - 意思是建立一個以 a 為名稱的變數，並指定 1 + 2 的運算結果為此變數值。

數學運算裡的等於符號，Python 裡要寫成兩個等號（==）。

範例

```
1    a = 1
2    a = a + 1
3
4    print(a)
```

```
1    a = 2
2    a = 4
3    a = 6
4
5    print(a + a + a)
```

Pythonic

- 具有 Python 特色(pythonic)的寫法

```
1    a = 123
2    b = 123
3    c = 123
4
5    name = "皮卡丘"
6    hp = 100
```

```
8    a = b = c = 123
9    name, hp = "皮卡丘", 100
```

宣告(declaration)

- 大多數程式語言在使用變數前要先宣告

C/C++

```
1  int a;
2  float b;
```

C#

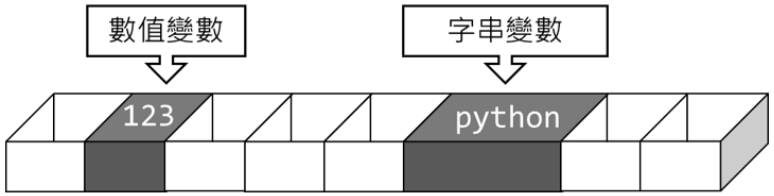
```
1  int a;
2  float b;
3  var c = "Hello";
```

JavaScript

```
1  var a = 123;
2  var b = '123';
```

變數(variable)

- 當變數被使用時，系統會配置一塊記憶體空間給此變數。
  - 變數名稱 ➔ 此塊記憶體的位址
  - 資料型別 ➔ 此塊記憶體的大小



Python 變數不需宣告就可使用

```
1    a = 3
2    print(a, type(a))
3    a = 3.14
4    print(a, type(a))
5    a = '3.14'
6    print(a, type(a))
```

```
3 <class 'int'>
3.14 <class 'float'>
3.14 <class 'str'>
```

斷行

C/C++

```
1  int a;
2  float b;
```

- Python 不需要斷行符號，而改用《Enter》鍵來標示。

```
1  a = 3
2  print(a, type(a))
3  a = 3.14
4  print(a, type(a))
5  a = '3.14'
6  print(a, type(a))
```

C#

```
1  int a;
2  float b;
3  var c = "Hello";
```

JavaScript

```
1  var a = 123;
2  var b = '123';
```

- 若是想將數列語句濃縮成一行，可以使用分號做區隔。

```
1  a = 1; b = 2; c = 3
```

刪除變數

- 如果變數不再使用，刪除掉該變數可以節省記憶體。

- 刪除變數 score 的語法為：
  - `del score`

這個實在少用

主題

- 變數與資料型別
  - 變數與命名規則
  - 數值、布林與字串資料型別
  - 資料型別轉換
  - 註解
- 輸出與輸入
- 運算式

為何要資料型別轉換？

2 + 3 = 5

2 + 3.0 = 5.0

2 + True = 3

2 + '3' = ?

資料型別轉換

- 如果系統無法自動進行資料型別轉換，就需要手動強制轉換。

- Python 強制資料型別轉換命令有：
  - `int()` ➔ 轉換為整數型別
  - `float()` ➔ 轉換為浮點數型別
  - `bool()` ➔ 轉換為布林型別
  - `str()` ➔ 轉換為字串型別

範例

```
1    print(float(2))     # 整數轉浮點數
2    print(float(12345))
3    print(float(10000000000000000000), end='\n\n')
4
5    print(int(2.3))     # 浮點數轉整數，進行無條件捨去
6    print(int(88.888))
7    print(int(1e+18))
```

```
2.0
12345.0
1e+18

2
88
1000000000000000000
```

試試看

```
1    # 如何修正下列錯誤
2
3    num = "12"
4    result = num + 34
5
6    print("結果為" + result)
```

解1：str(34)

解2：int(num)

解2：str(result)

```
TypeError: can only concatenate str (not "int") to str
```

轉換為布林型別

- 類似整數 0 轉換成布林值時會變成 False，稱為 Falsy value。
  - 其它常見型別的有浮點數 0.0、複數 0 + 0j、空字串""（或''）、None 等，其餘大多在轉換成布林值時會得到 True 的結果。

```
1    print(bool(0))
2    print(bool(3))
3    print(bool(3.14))
4    print(bool(-3.14))
```

```
False
True
True
True
```

轉換為字串型別

```
1    print(str(123))
2    print(str(3.14))
3    print(str(1111111111111111*9999999999999*10000000000000000))
4    print(str(1e32))
5    print(str(False))
```

```
123
3.14
111111111111099888888888889000000000000000
1e+32
False
```

有發現多一個 + 號嗎？

主題

- 變數與資料型別
  - 變數與命名規則
  - 數值、布林與字串資料型別
  - 資料型別轉換
  - 註解
- 輸出與輸入
- 運算式

註解(comment)

- 註解是在程式碼中「不會被執行的文字敘述」
  - 主要用來輔助說明程式碼的內容
  - 也是最通用的除錯(debug)手段

```
1    print(type(2))     # 整數
2    print(type(2.0))   # 浮點數
3    print(type('2'))   # 字串
4    print(type(True)) # 布林值
```

Python 的註解

- 每個程式語言用來表示註解的符號不盡相同
  - Python 使用一個井字號(#)做為單行註解，而多行註解則用成對的三個雙引號(""")或三個單引號(''')。

```
1    # 井字號後面可以註解一列
2    '''
3    這裡是多行註解，
4    程式不會執行這裡。
5    '''
6    print(3*5/2)   # 執行結果為浮點數
```

```
7.5
```

常用操作

- 快速註解多行
  - 先選取要註解的多行，再用組合鍵達成。
  - Spyder :《Ctrl + 1》

```
2   # a = 3
3   # print(type(a))
4   # a = 3.14
5   # print(type(a))
```

去除註解：再一次《Ctrl + 1》

  - VS code:《Ctrl + /》

```
1   # a = 3
2   # print(a, type(a))
3   # a = 3.14
4   # print(a, type(a))
5   # a = '3.14'
6   # print(a, type(a))
```

去除註解：再一次《Ctrl + /》

本章結束

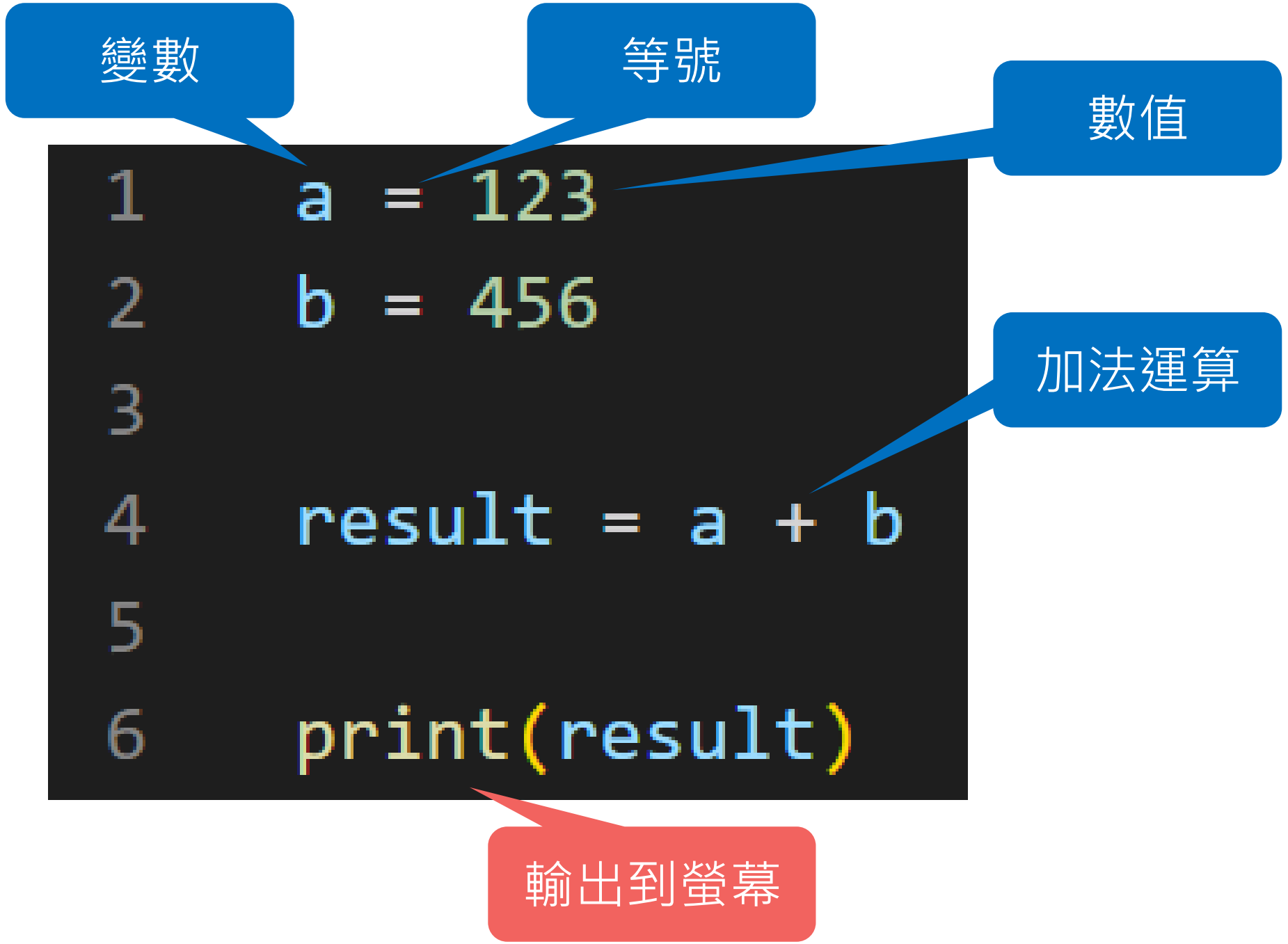
Py01

基本認識 (2)

王頌文

主題

- 變數與資料型別
- 輸出與輸入
  - 基本輸出
  - 格式化輸出
  - 由鍵盤輸入
- 運算式


變數
等號
數值
加法運算
1 a = 123
2 b = 456
3
4 result = a + b
5
6 print(result)
輸出到螢幕

主題

- 變數與資料型別
- 輸出與輸入
  - 基本輸出
  - 格式化輸出
  - 由鍵盤輸入
- 運算式

print() 的語法

- print() 能將指定項目輸出到螢幕，語法為：
  - print(項目1 [, 項目2, …, sep=分隔字元, end=結束字元])
    - 項目1, 項目2, … ：可以一次輸出多個項目，以逗號(,)分隔。
    - sep：分隔項目的字元，預設值為一個空白。
    - end：輸出完後自動加入的字元，預設值為換行字元(\n)。

```
1    name = '皮卡丘'
2    hp = 100
3
4    print(name, 'HP=', hp)
```

```
皮卡丘 HP= 100
```

更多範例(1/3)

```python
name = '皮卡丘'
hp = 100

print(name, 'HP=', hp, sep='|')
```

```
皮卡丘|HP=|100
```

```python
name = '皮卡丘'
hp = 100

print(name)
print('HP=', hp)

print(name, end=',')
print('HP=', hp, end='。')
```

```
皮卡丘
HP= 100
皮卡丘，HP= 100。
```

更多範例(2/3)

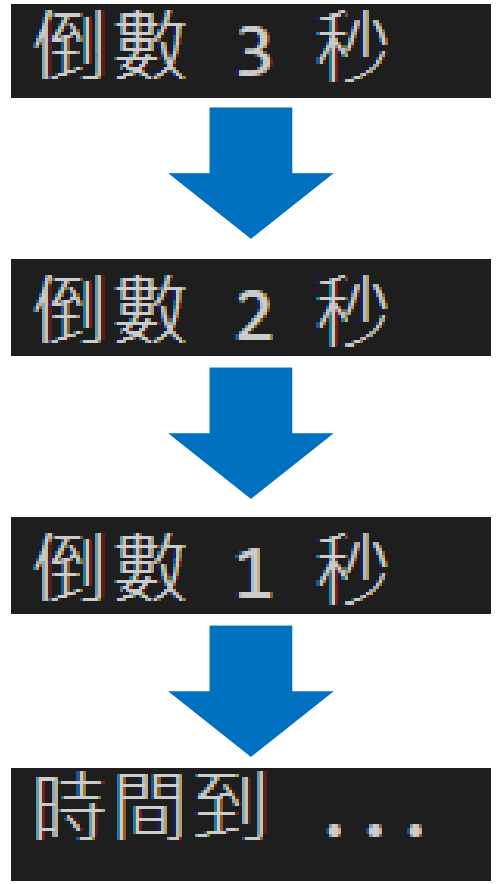
```python
name = '皮卡丘'
hp, atk = 100, 90

print(name, end='|')
print('HP=', hp, sep='', end='\t')
print('ATK=', atk, sep='')

print('小火龍|HP=80', 'ATK=85', sep='\t')
```

```
皮卡丘|HP=100    ATK=90
小火龍|HP=80     ATK=85
```

更多範例(3/3)

萬一倒數計時更多秒，這樣寫不僅相當繁瑣，程式碼也很“醜陋”。

```
1    import time       # 匯入時間模組
2
3    print('\r倒數 3 秒', end='')
4    time.sleep(1)     # 程式暫停 1 秒
5    print('\r倒數 2 秒', end='')
6    time.sleep(1)
7    print('\r倒數 1 秒', end='')
8    time.sleep(1)
9
10   print('\r時間到 ...', end='')
```



輸出彩色字體(1/2)

- Python 的 `print()` 可搭配「ANSI 跳脫序列」輸出彩色字體
  - 「`\033[`」代表 ANSI 跳脫序列的開頭，這之後可以增加多組數字。
  - 每個數字代表一個效果，不同數字間可用分號(;)隔開。
  - 一串效果數字的最後要加個 m 代表設定完成

```
1    print("小智的\033[33;40m皮卡丘\033[0m討厭進入精靈球")
```

```
小智的皮卡丘討厭進入精靈球
```

輸出彩色字體(2/2)

顏色名稱	字體顏色代碼	背景色代碼
黑	30	40
紅	31	41
綠	32	42
黃	33	43
藍	34	44
洋紅	35	45
青	36	46
白	37	47

```
1    print("小智的\033[33;40m皮卡丘\033[0m討厭進入精靈球")
```

小智的皮卡丘討厭進入精靈球

設定字體效果

代碼	效果	代碼	效果
0	重設	7	反白（前景色與背景色交換）
1	粗體	8	隱藏
3	斜體	9	刪除線
4	底線		

顏色名稱	字體顏色代碼	背景色代碼
黑	30	40
紅	31	41
綠	32	42
黃	33	43
藍	34	44
洋紅	35	45
青	36	46
白	37	47

```
1    print("皮卡丘為\033[4m電屬性\033[0m，絕招是\033[33;3m十萬伏特。")
```

皮卡丘為電屬性，絕招是十萬伏特。

更多 ANSI 跳脫序列的字體顏色與效果的設定可參考 https://en.wikipedia.org/wiki/ANSI_escape_code

主題

- 變數與資料型別
- 輸出與輸入
  - 基本輸出
  - 格式化輸出
  - 由鍵盤輸入
- 運算式

格式化輸出

- 除了前面的基本輸出外，`print()` 也可以進行格式化輸出。

- Python 進行格式化字串的方式有四種：
  ① 百分號(%)
  ② 利用 `format()`
  ③ f-string（也稱為 formatted string literals）
  ④ 樣板字串(template string)

參照 C 語言的格式化輸出

Python 3.6+ 才有支援，推薦使用。

主要在處理使用者的惡意輸入，不僅需要匯入 string 模組，也牽涉到物件導向的寫法，故不介紹。

① 百分號(1/4)

來自 C 語言

- 百分號(%)是 Python 最早的格式化字串方法，在百分符號之後指定格式化方式，語法為：
  - print(項目 %(參數列))
    - %s ➔ 字串
    - %d ➔ 整數
    - %f ➔ 浮點數

```
皮卡丘 的 HP=100
ATK=90.000000
```

```
1    name = '皮卡丘'
2    hp, atk = 100, 90
3
4    print("%s 的 HP=%d" % (name, hp)) # 格式化字串與整數
5    print("ATK=%f" % atk)             # 預設保留6位小數
```

① 百分號(2/4)

- 格式化輸出可精確控制輸出位置，讓輸出資料整齊排列，比如：
  - %5d，%5s：固定 5 個位置
    - 小於 5 位數 ➔ 在數字左方填入空白字元
    - 大於 5 位數 ➔ 全部輸出
  - %6.2f：固定 6 個位置（含小數點），小數後固定輸出 2 位數。
    - 小數、整數分開看

```
1    print("三位數=%5d" % 123)
2    print("浮點數=%.3f" % 12.3)
```

```
三位數=  123
浮點數=12.300
```

① 百分號(3/4)

- 格式化輸出可精確控制輸出位置，讓輸出資料整齊排列，比如：
  - %6.2f：固定 6 個位置（含小數點），小數後固定輸出 2 位數。
    - 小數 < 2 位數 ➔ 在數字右方填入「0」字元
    - 小數 ≥ 2 位數 ➔小數後第 3 位四捨五入
    - 整數 < 3 位數 ➔ 在數字左方填入空白字元
    - 整數 ≥ 3 位數 ➔ 全部輸出

```
5    PI = 3.1415926
6
7    print("圓周率=%2.3f" % PI)
8    print("圓周率=%.3f" % PI)
9    print("圓周率=%5.1f" % PI)
10   print("圓周率=%5.6f" % PI)
```

```
圓周率=3.142
圓周率=3.142
圓周率=  3.1
圓周率=3.141593
```

浮點數精確度問題

```
1    print("%.1f" % 0.14)
2    print("%.1f" % 0.15)
3    print("%.1f" % 0.16)
```

```
0.1
0.1
0.2
```

其他格式化輸出方式也會遇到同樣困擾

① 百分號(4/4)

- 用 +/- 控制靠左、右對齊
  - 預設為靠右對齊

```
1    print("%05d" % 123)
2
3    hp, atk = 95, 87
4    print("<%5d,%-5d>" % (hp, atk))
5
6    print("%10s" % "Pikachu")
```

```
00123
<   95,87   >
   Pikachu
```

② format() (1/2)

- 百分號的格式化輸出，有不適合輸出多個變數、可讀性低等缺點。因此，Python 設計了一個更好用的 format()。
  - 以「輸出字串.format()」方式進行格式化輸出
    - 在輸出字串內用一對大括號（{}）指定輸出目標在字串裡的位置
    - 接著在 format() 中放入欲輸出的變數、字串或數值

```
1    name, hp = '皮卡丘', 100
2
3    print("{} 的血量為 {}".format(name, hp))
4    print("{1} 的血量為 {0} {1}".format(name, hp))
```

```
皮卡丘 的血量為 100
100 的血量為 皮卡丘 100
```

② format() (2/2)

- 對於三個基本資料型別的格式化輸出，format() 與百分號的設定有些類似，但以「{:}」代替「%」，例如：
  - {:5d}：與百分號的設定一致。
  - {:5s}：與百分號的設定類似，但預設以靠左對齊的方式輸出字串。
  - {:6.2f}：與百分號的設定一致。

```
1   name, hp = '皮卡丘', 100
2
3   print("{:5s}的血量為{:5d}。\n".format(name, hp))
```

```
皮卡丘  的血量為  100。
```

format() 範例(1/3)

```
三位數=  123
浮點數=12.300

圓周率=3.142
圓周率=3.142
圓周率=  3.1
圓周率=3.141593
```

```
1    print("三位數=%5d" % 123)
2    print("浮點數=%.3f" % 12.3)
3    print()
4
5    PI = 3.1415926
6
7    print("圓周率=%2.3f" % PI)
8    print("圓周率=%.3f" % PI)
9    print("圓周率=%5.1f" % PI)
10   print("圓周率=%5.6f" % PI)
```

```
1    print("三位數={:5d}".format(123))
2    print("浮點數={:.3f}".format(12.3))
3    print()
4
5    PI = 3.1415926
6
7    print("圓周率={:2.3f}".format(PI))
8    print("圓周率={:.3f}".format(PI))
9    print("圓周率={:5.1f}".format(PI))
10   print("圓周率={:5.6f}".format(PI))
```

format() 範例(2/3)

```
1   print("{:0>5d}".format(123))
2   print("{:x<5d}".format(123))
3
4   print("{:>7d}".format(123)) # 靠右對齊
5   print("{:<7d}".format(123)) # 靠左對齊
6   print("{:-^7d}".format(123))# 置中對齊
7
8   print("{:,}".format(10000)) # 以逗點分隔
9   print("{:.2e}".format(10000))    # 科學記號表示
10  print("{:.2%}".format(0.25))     # 百分比格式
```

```
00123
123xx
    123
123
--123--
10,000
1.00e+04
25.00%
```

format() 範例(3/3)

```
1   print("{:4s}\t{:4s}\t{:4s}".format('Name', 'HP', 'ATK'))
2   print("{:4s}\t{:5d}\t{:5d}".format('皮卡丘', 35, 55))
3   print("{:4s}\t{:5d}\t{:5d}".format('六尾', 38, 41))
4   print("{:4s}\t{:5d}\t{:5d}".format('吉利蛋', 250, 5))
```

```
Name        HP          ATK
皮卡丘          35           55
六尾          38           41
吉利蛋         250            5
```

更多用法可參考 http://www.runoob.com/python/att-string-format.html

③ f-string (1/2)

- 儘管 format() 在使用上比百分號方便些，但當變數一多，要撰寫的程式碼仍然相當冗長。

- 底下介紹 f-string 的用法，Python 3.6+ 才有支援。
  - 在輸出字串前加上「f」即可進行格式化
  - 並在大括號內填入目標變數，是現在比較推薦的格式化方法。

```
1    name, hp = '皮卡丘', 100
2
3    print(f"{name} 的血量為 {hp}")
```

```
皮卡丘 的血量為 100
```

③ f-string (2/2)

- f-string 格式化字串的設定與 format() 類似

```
00123
123xx
    123
123
--123--
10,000
1.00e+04
25.00%
```

```python
print(f"{123:0>5d}")
print(f"{123:x<5d}")

print(f"{123:>7d}") # 靠右對齊
print(f"{123:<7d}") # 靠左對齊
print(f"{123:-^7d}")    # 置中對齊

print(f"{10000:,}") # 以逗點分隔
print(f"{10000:.2e}")   # 科學記號表示
print(f"{0.25:.2%}")    # 百分比格式
```

```python
print("{:0>5d}".format(123))
print("{:x<5d}".format(123))

print("{:>7d}".format(123)) # 靠右對齊
print("{:<7d}".format(123)) # 靠左對齊
print("{:-^7d}".format(123))# 置中對齊

print("{:,}".format(10000)) # 以逗點分隔
print("{:.2e}".format(10000))   # 科學記號表示
print("{:.2%}".format(0.25))    # 百分比格式
```

f-string 範例

```
Name      HP         ATK
皮卡丘        35          55
六尾          38          41
吉利蛋       250           5
```

```
1   print(f"{'Name':4s}\t{'HP':4s}\t{'ATK':4s}")
2   print(f"{'皮卡丘':4s}\t{35:5d}\t{55:5d}")
3   print(f"{'六尾':4s}\t{38:5d}\t{41:5d}")
4   print(f"{'吉利蛋':4s}\t{250:5d}\t{5:5d}")
```

```
1   print("{:4s}\t{:4s}\t{:4s}".format('Name', 'HP', 'ATK'))
2   print("{:4s}\t{:5d}\t{:5d}".format('皮卡丘', 35, 55))
3   print("{:4s}\t{:5d}\t{:5d}".format('六尾', 38, 41))
4   print("{:4s}\t{:5d}\t{:5d}".format('吉利蛋', 250, 5))
```

練習

- 請以 print() 三種格式化語法，分別輸出下列變數的內容：
  - name = '喬巴'
  - atk = 80
  - 萌度 = 95.5678

- 輸出結果為

```
喬巴，攻擊力=80，萌度=95.57。
```

主題

- 變數與資料型別
- 輸出與輸入
  - 基本輸出
  - 格式化輸出
  - 由鍵盤輸入
- 運算式

輸入資料

- print() 是輸出資料，而內建函式 input() 則是暫停程式等待使用者由「標準輸入」裝置輸入資料。
  - 如果沒有特別設定，標準輸入指的是鍵盤。
  - [變數 =] input([提示字串])

```
1    score = input("請輸入成績：")    # 等待輸入
2    print(score, '\n')
3
4    print(input())    # 等待輸入完成後立即輸出
```

```
請輸入成績：95
95

87
87
```

注意資料型別

- 這段程式碼是否有問題？

```
1    score = input("請輸入成績：")
2
3    print(score)
4    print(score + 10)
```

```
請輸入成績：87
87
Traceback (most recent call last):
  File "C:\Users\Ian\VS_Code\Ian_data\Untitled-1.py", line 4, in <module>
    print(score + 10)
TypeError: can only concatenate str (not "int") to str
```

修正方式

解1：int(input("請輸入成績："))

```
1    score = input("請輸入成績：")
2
3    print(score)
4    print(score + 10)
```

解2：int(score)

這有點問題 ...

```python
print(int(input("請輸入整數：")))        # 轉成整數
print(float(input("請輸入整數：")))      # 轉成浮點數
print(float(input("請輸入浮點數：")))    # 轉成浮點數
print(int(input("請輸入浮點數：")))      # 轉成整數
```

```
請輸入整數：3
3
請輸入整數：3
3.0
請輸入浮點數：3.14
3.14
請輸入浮點數：3.14
Traceback (most recent call last):
  File "C:\Users\Ian\VS_Code\Ian_data\Untitled-1.py", line 4, in <module>
    print(int(input("請輸入浮點數：")))     # 轉成整數
ValueError: invalid literal for int() with base 10: '3.14'
```

將使用者輸入看起來像浮點數的字串，強制轉換為整數型別 ☹

判斷小數點

- 熟悉程式設計的人可能會想多寫幾行程式碼來判斷輸入的是整數還是浮點數，畢竟這兩者的差別僅在於小數點的存在與否。
  - "87", "94.87"

```
1    inp = input("請輸入數字：")
2
3    if '.' in inp:
4        print('浮點數')
5    else:
6        print('整數')
```

```
請輸入數字：94.87
浮點數
```

字串處理之後才會學到☹，是否有比較簡單的處理方式？

內建函式 eval()

```
請輸入整數：3
3
請輸入浮點數：3.14
3.14
```

```
1    print(eval(input("請輸入整數：")))      # 轉成整數
2    print(eval(input("請輸入浮點數：")))    # 轉成浮點數
```

```
1    score = eval(input("請輸入整數："))
2    print(score + 10)
3
4    pi = eval(input("請輸入圓周率："))
5    print(3 * pi)
```

```
請輸入整數：50
60
請輸入圓周率：3.14
9.42
```

eval(運算式)

- eval() 真正用途是執行字串運算式(expression)，並回傳該運算式的運算結果。
  - 這個運算式可包含變數，也可由使用者直接輸入。

```
1    exp = input("請輸入數學運算式：")
2    print(type(exp))
3    print(eval(exp))
4
5    x = 10
6    print(eval("x + 5"))
```

```
請輸入數學運算式：2*3+5
<class 'str'>
11
15
```

簡化輸入

一次取得兩個輸入值

```
1   x, y = eval(input("請輸入兩個變數，並以逗點隔開："))
2
3   print(f"x 的型別為{type(x)}")
4   print(f"y 的型別為{type(y)}")
5   print(x + y)
```

```
請輸入兩個變數，並以逗點隔開：5, 3.14
x 的型別為<class 'int'>
y 的型別為<class 'float'>
8.14
```

注意

- `eval()` 會嘗試將一個字串轉換成可執行的程式碼並執行，但不能進行複雜的邏輯運算，比如賦值、迴圈等。

- 儘管方便使用，但也具有潛在的資安風險。
  - 使用者惡意輸入類似「系統指令」的字串，可能導致程式碼注入漏洞。
  - e.g., 使用 `eval("__import__('os').system('dir')")` 就能直接獲得當前資料夾的目錄與檔案資訊，因此使用時需小心謹慎。

練習：計算 BMI

- 請撰寫一程式，讓使用者輸入身高(cm)與體重(kg)，然後輸出其對應的 BMI 值（四捨五入至小數點後第二位）。
  - 範例輸入

```
請輸入身高(cm)、體重(kg)，輸入值以逗點隔開：176, 75
```

  - 範例輸出

```
BMI = 24.21
```

$$\text{BMI} = \frac{\text{體重}(kg)}{\text{身高}(m)^2}$$

本章結束

Py01

基本認識 (3)

王頌文

主題

- 變數與資料型別
- 輸出與輸入
- 運算式
  - 算術運算子
  - 複合指定運算子
  - 比較運算子
  - 邏輯運算子
  - 位元運算子

運算元(operand)
3 + 2
運算子(operator)

運算元？運算子？

x and y

運算元？運算子？

-100

運算子的種類

- 根據參與的運算元數量，可將運算子區分為
  - 一元運算子(unary operator)
    - 只需要一個運算元，比如負號、階乘符號、矩陣的轉置、開平方等。
  - 二元運算子(binary operator)
    - 像是加減乘除、指派（賦值）運算子「=」
    - 有些二元運算子左右兩邊的運算元是不能交換
  - 三元運算子(ternary operator)

數學上 …

- 一元運算
  - 若函數 $f: A \to A$ 且 A 是一個集合，則 f 是在 A 上的一元運算。
- 二元運算
  - e.g., 考慮函數 $f: A \times A \to A$
- 三元運算
  - 依此類推 …

比較運算子「>」是二元運算子，
但不是數學上的二元運算。

運算子的優先順序(operator precedence)

- 稍後會看到許多不同種類的運算子，而稍微複雜一點的運算往往使用到多個運算子，這時就要注意運算子的優先順序。
  - e.g., 統計上經常對資料進行標準化(standardization)得到所謂的 z 分數(z-score)，計算公式如下：

$$z = \frac{x - \mu}{\sigma}$$

平均數

標準差

假設 $\mu = 10, \sigma = 2$，當資料 $x = 8$ 時，會得到 z-score = –1，代表這筆資料比平均數要小一個標準差。

$$z = \frac{x - \mu}{\sigma} = \frac{8 - 10}{2} = -1$$

```
1    mu, sigma = 10, 2
2    x = 8
3
4    z = x - mu / sigma
5    print(z)
```

```
3.0
```

根據運算子
的優先順序

```
z = (x - (mu / sigma))
  = (8 - (10 / 2))
  = (8 - 5.0)
  = 3.0
```

假設 $\mu = 10, \sigma = 2$，當資料 $x = 8$ 時，會得到 z-score = –1，代表這筆資料比平均數要小一個標準差。

$$z = \frac{x - \mu}{\sigma} = \frac{8 - 10}{2} = -1$$

```
1    mu, sigma = 10, 2
2    x = 8
3
4    z = (x - mu) / sigma
5    print(z)
```

```
-1.0
```

Operator	Description
`(expressions...)`, `[expressions...]`, `{key: value...}`, `{expressions...}`	Binding or parenthesized expression, list display, dictionary display, set display
`x[index]`, `x[index:index]`, `x(arguments...)`, `x.attribute`	Subscription, slicing, call, attribute reference
`await x`	Await expression
`**`	Exponentiation [5]
`+x`, `-x`, `~x`	Positive, negative, bitwise NOT
`*`, `@`, `/`, `//`, `%`	Multiplication, matrix multiplication, division, floor division, remainder [6]
`+`, `-`	Addition and subtraction
`<<`, `>>`	Shifts
`&`	Bitwise AND
`^`	Bitwise XOR
`|`	Bitwise OR
`in`, `not in`, `is`, `is not`, `<`, `<=`, `>`, `>=`, `!=`, `==`	Comparisons, including membership tests and identity tests
`not x`	Boolean NOT
`and`	Boolean AND
`or`	Boolean OR
`if – else`	Conditional expression
`lambda`	Lambda expression
`:=`	Assignment expression

Source: Python–Operator precedence

只是加減乘除運算還好，
e.g., 1+2*3/4-5，但 …

建議還是用**括號**
標示運算順序

主題

- 變數與資料型別
- 輸出與輸入
- 運算式
  - 算術運算子
  - 複合指定運算子
  - 比較運算子
  - 邏輯運算子
  - 位元運算子

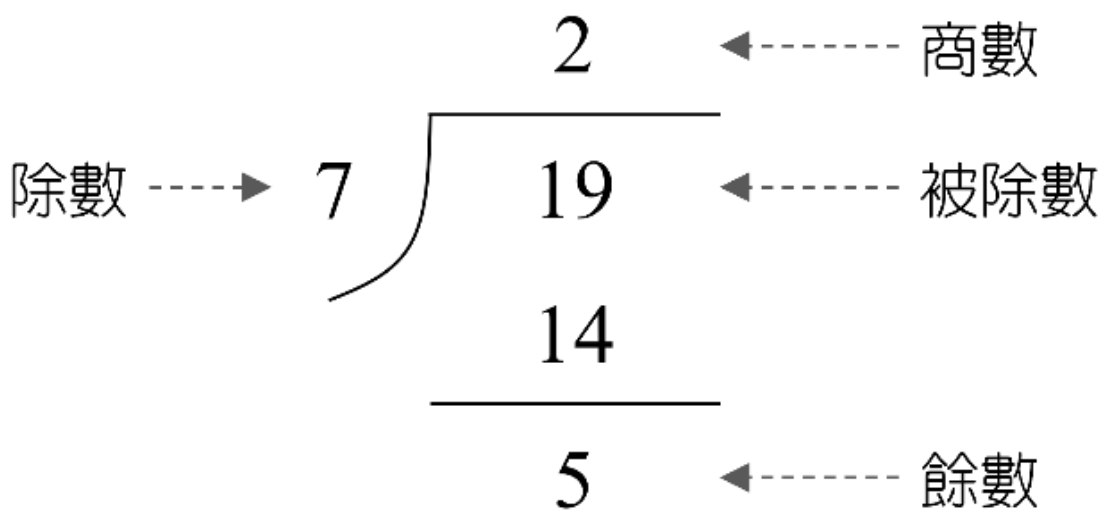
算術運算子(arithmetic operator)

- 算術運算子進行數值的數學計算，得到的結果是另一個數值。

運算子	說明	範例	範例結果
+	兩運算元相加	2 + 3	5
–	兩運算元相減	2 – 3	–1
*	兩運算元相乘	2 * 3	6
/	兩運算元相除	32 / 5	6.4
//	兩運算元相除的商數（無條件捨去）	32 // 5	6
%	兩運算元相除的餘數	32 % 5	2
**	兩運算元進行指數運算	2**3	8

除數不能為 0

三種除法



- 「//」是將相除後的浮點數，無條件捨去小數點的結果。
- 「%」稱為餘數運算子
  - 運算結果是經過除法後的餘數(remainder)
  - 有時也叫做模數(modulo)

範例：三種除法

```
print(6 / 3)      # 除法的結果為浮點數
print(3 / 2)
print(12.5 / 0.25)
print(1.27 / 3, end='\n\n')

print(10 // 3)    # 除法結果的商數，可能是整數或浮點數
print(10.0 // 3)
print(10 // 0.3, end='\n\n')

print(10 % 3)     # 除法結果的餘數，可能是整數或浮點數
print(10 % 3.0)
print(10.0 % 3)
print(10 % -3)
print(-10 % 3)
```

```
2.0
1.5
50.0
0.42333333333333334

3
3.0
33.0

1
1.0
1.0
-2
2
```

(10%-3) → -2 ?

- 10%-3 ?
  - 10/-3 → -3.33333
  - math.floor(-3.33333) → -4
  - (-4 × -3 + (-2)) % -3 → -2

  - e.g., -3%7 → (-1 × 7 + 4) % 7 → 4
    -5%2 → (-3 × 2 + 1) % 2 → 1

範例：不同資料型別做運算

```
1    print(3 + 0.2)
2    print(3e2 - 120)
3    print(3.0 * 2)
4    print(3 ** 2.0)
5    print(3 * True)      # True 先轉成 1 再運算
6    print(3 * False)     # False 先轉成 0 再運算
```

```
3.2
180.0
6.0
9.0
3
0
```

主題

- 變數與資料型別
- 輸出與輸入
- 運算式
  - 算術運算子
  - 複合指定運算子
  - 比較運算子
  - 邏輯運算子
  - 位元運算子

複合指定運算子(compound assignment operator)

- 同時做「運算」及「指派」兩件事
  - `i += 2`
  - `i = i + 2`

運算子	說明	範例（a = 5）	範例結果
+=	相加後再指定給原變數	a += 2	7
–=	相減後再指定給原變數	a –= 2	3
*=	相減後再指定給原變數	a *= 2	10
/=	相除後再指定給原變數	a /= 2	2.5
//=	相除後取得商，再指定給原變數	a //= 2	2
%=	相除後取得餘數，再指定給原變數	a %= 2	1
**=	指數運算後再指定給原變數	a **= 2	25

範例：求圓面積

- 已知圓周長為 314 公分，要求圓面積大小。
  - 這道題目可以依圓周長公式先求出半徑，再計算圓面積。

```
1    PI = 3.14
2    圓周長 = 314
3    圓周長 **= 2
4    圓面積 = 圓周長 / (4*PI)
5
6    print(f"圓面積 = {圓面積}")
```

```
圓面積 = 7850.0
```

練習：計算導數值

- 使用者輸入一個數 a，並給定一個函數 $f(x)$ 如下，請計算 $f(x)$ 在 $x = a$ 的導數(derivative)。
  - 輸入：一個數 a
  - 輸出：$f(x)$ 在 $x = a$ 的導數

$$f(x) = 3x^4 - 7x^2 + 5$$

  - 注意：請利用導數的極限定義來計算，勿直接將導數寫在程式碼裡。

主題

- 變數與資料型別
- 輸出與輸入
- 運算式
  - 算術運算子
  - 複合指定運算子
  - 比較運算子
  - 邏輯運算子
  - 位元運算子

比較運算子(comparison operator)

- 也稱為關係運算子(relational operator)，是進行大小比較關係的運算。
  - 其比較結果會得到一個布林值，代表這個大小關係是否成立。
  - 若比較結果正確，則得到 True；反之，比較結果錯誤會得到 False。

這種比較常見於邏輯判斷使用

比較運算子

運算子	說明	範例	範例結果
==	左右兩邊是否相等	3 == 2	False
!=	左右兩邊是否不相等	3 != 2	True
>	左邊是否大於右邊	3 > (2**2)	False
<	左邊是否小於右邊	3 < (2**2)	True
>=	左邊是否大於或等於右邊	3 >= 4	False
<=	左邊是否小於或等於右邊	3 <= 3	True

「>=」 → 大於或等於
「<=」 → 小於或等於

注意

- 浮點數的比較運算因為精確度的緣故，有時候會得到錯誤的比較結果，而這個問題常發生在等於判斷的時候。
  - 因此，宜盡量避免直接對浮點數進行等於判斷，若逼不得已也可自訂一個可容忍的誤差值。

```
1    print(0.1 + 0.2 == 0.3)
```

```
False
```

主題

- 變數與資料型別
- 輸出與輸入
- 運算式
  - 算術運算子
  - 複合指定運算子
  - 比較運算子
  - 邏輯運算子
  - 位元運算子

邏輯運算子(logical operator)

- Python 可以判斷的三種邏輯狀態有且(and)、或(or)、非(not)，邏輯運算的結果依下面的真值表(truth table)來決定。

x	y	x and y	x or y	not x
F	F	F	F	T
F	T	F	T	T
T	F	F	T	F
T	T	T	T	F

T → True
F → False

邏輯運算子

- 常見在一般程式語言使用的邏輯運算子符號為
  - 且(&&)、或(||)、非(~)
- Python 則以直覺的方式表達邏輯運算
  - 也可以使用「&」代替 and，「|」代替 or。

運算子	說明	範例	範例結果
and	當兩個運算元皆為 True 時，邏輯運算結果為 True；其餘情況皆得到 False。	(3>2) and (5>3)	True
or	當兩個運算元皆為 False 時，邏輯運算結果為 False；其餘情況皆得到 True。	(3<2) or (5<3)	False
not	得到與運算元相反的值。	not (5>3)	False

範例

```
1    x, y = 2, 5
2
3    print((3 > x) and (y < 5))
4    print((3 > x) & (y < 5))
5    print((3 < x) or (y <= 5))
6    print(not (3 > x))
```

```
False
False
True
False
```

```
1    a, b = 3, 1
2
3    print((1<=b) and (b<=a) and (a<10))
4    print(1 <= b <= a <= 10)    # Pythonic 寫法
```

```
True
True
```

範例：交換兩變數值

```
1   a, b = 99, 1    # 指派變數a和b的值
2   print(f"交換前: a = {a}, b = {b}")
3   tmp = a
4   a = b
5   b = tmp
6   print(f"交換後: a = {a}, b = {b}")
7   print("========================")
8
9   a, b = 99, 1    # 重新指派變數a和b的值
10  print(f"交換前: a = {a}, b = {b}")
11  a, b = b, a
12  print(f"交換後: a = {a}, b = {b}")
```

```
交換前: a = 99, b = 1
交換後: a = 1, b = 99
========================
交換前: a = 99, b = 1
交換後: a = 1, b = 99
```

練習：XOR

- 輸入兩個布林值 A, B，計算並輸出 A XOR B 的值。
  - 利用 `and`, `or`, `not`

x	y	x XOR y
T	T	F
T	F	T
F	T	T
F	F	F

主題

- 變數與資料型別
- 輸出與輸入
- 運算式
  - 算術運算子
  - 複合指定運算子
  - 比較運算子
  - 邏輯運算子
  - 位元運算子

位元運算子(bitwise operator)

- 基本上，電腦內的所有數值皆以二進位表示（即 0、1），而位元運算子即是針對每個數值的二進位數字進行運算。
  - 4 → 0100
  - 5 → 0101

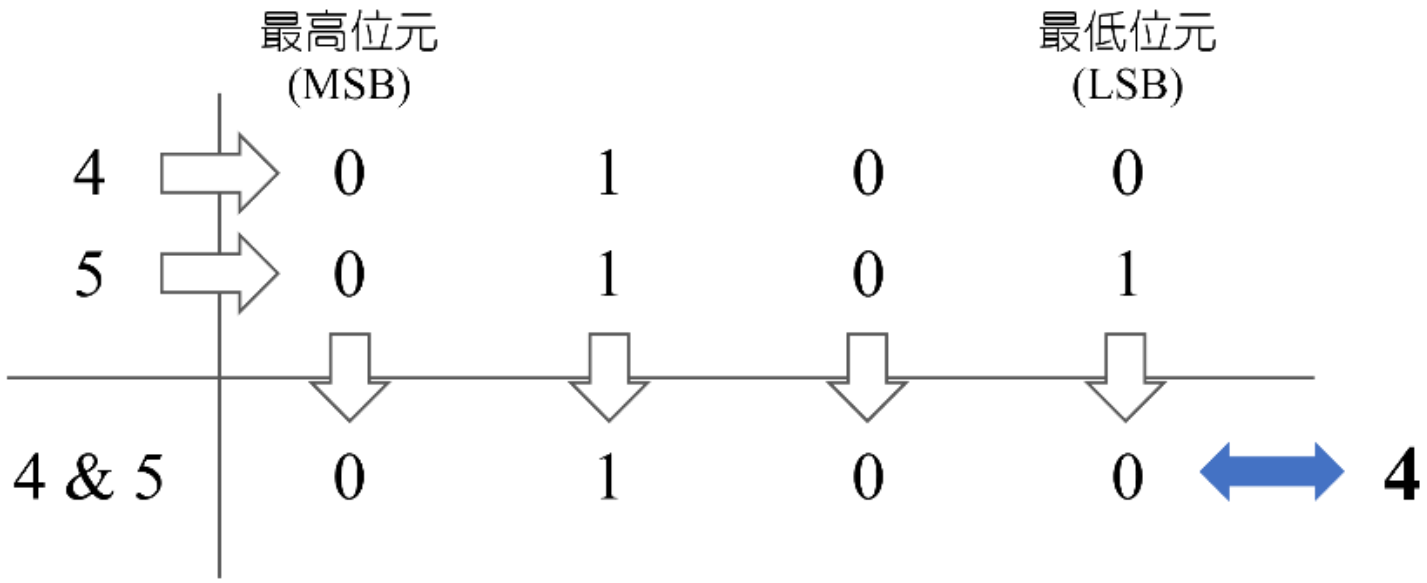
- C 語言裡，有一些運算子可以對記憶體裡的值進行位元(bit)操作
  - Python 承襲自 C 語言，也有與之對應的六個位元運算子。

位元運算子

運算子	說明	範例	範例結果
&	進行位元間的且（AND）運算	4 & 5	4
|	進行位元間的或（OR）運算	4 | 5	5
^	進行位元間的互斥或（XOR）運算	4 ^ 5	1
~	進行位元間的否（NOT）運算	~ 4	–5
>>	所有位元往右移動指定位數	5 >> 2	1
<<	所有位元往左移動指定位數	5 << 2	20

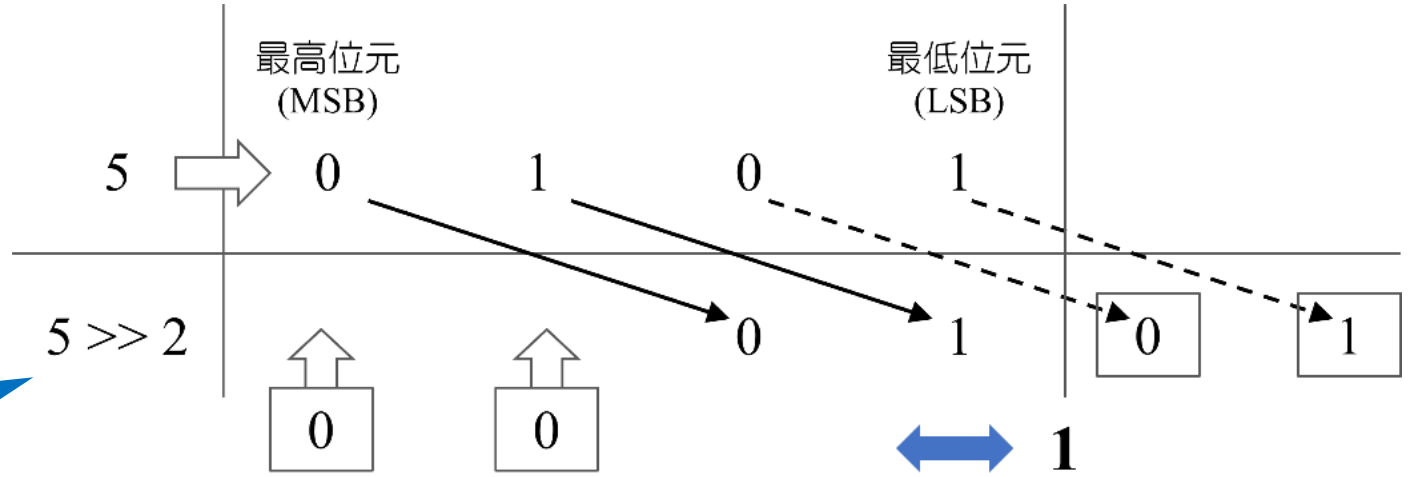
位移運算子(shift operator)

運算範例(1/2)

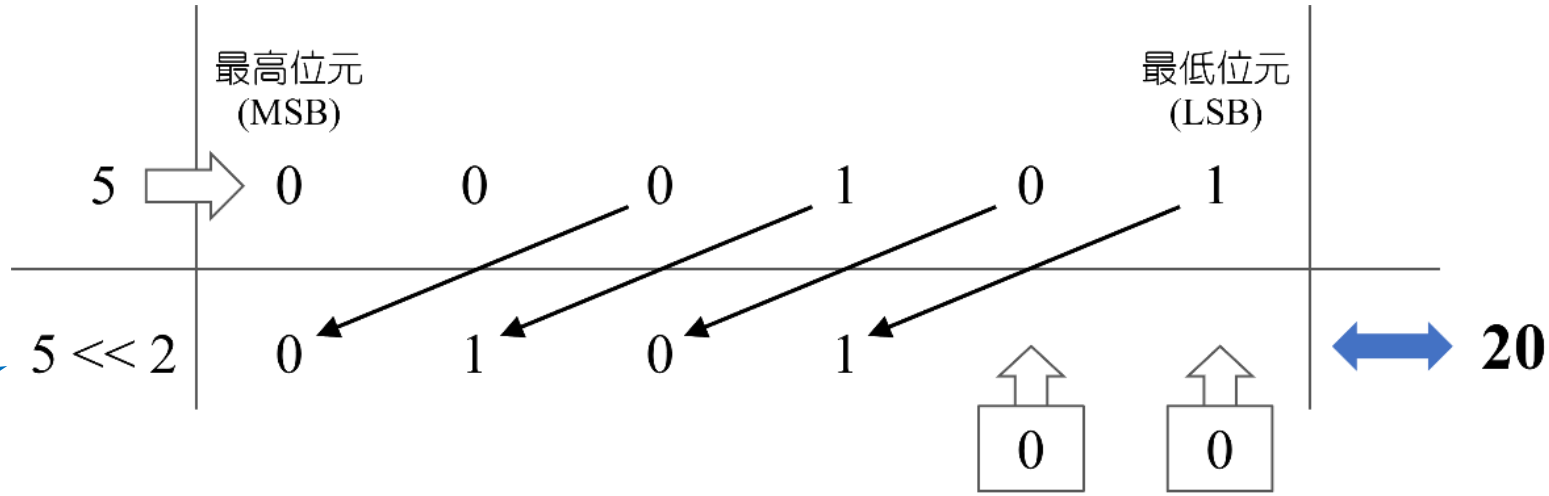


運算範例(2/2)

相當於 5 // 2



相當於 5 * 2



範例：奇偶數判斷

```
1    a = 5
2
3    print(f"整數為：{a}")
4    print(f"是奇數？{bool(a % 2)}")
5    print(f"是奇數？{bool(a & 1)}")
```

```
整數為：5
是奇數？True
是奇數？True
```

本章結束

Py02

選擇敘述與迴圈 (1)

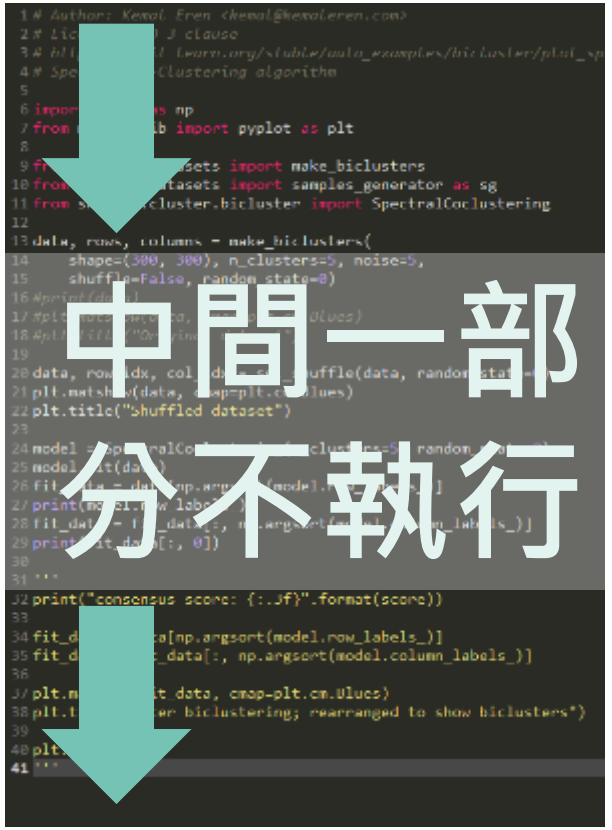
王頌文

主題

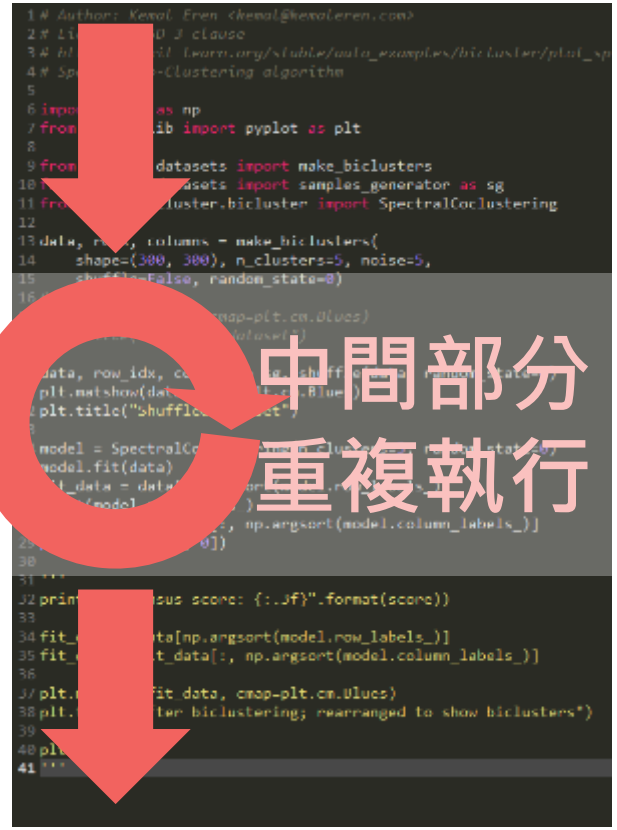
- 程式流程控制
- 單向條件式(`if ...`)
- 雙向條件式(`if ... else ...`)
- 多向條件式(`if ... elif ... else ...`)
- 巢狀條件式
- 結構模式比對(`match ... case ...`)

結構化程式設計(structured programming)

循序(sequence)



選擇(selection)



重複(repetition)

流程控制(flow control) (1/2)

- 這兩類皆屬於 Python 的流程控制命令：
  - ① 條件表達式(conditional statement)：根據比較或邏輯運算的**條件式**來決定程式執行的流程
    - `if ... else ...`

流程控制(2/2)

- 這兩類皆屬於 Python 的流程控制命令：
  - ② 迴圈：根據比較或邏輯運算**條件式**的結果以決定是否重複執行指定的程式碼。
    - `for ...`
    - `while ...`

主題

- 程式流程控制
- 單向條件式(`if ...`)
- 雙向條件式(`if ... else ...`)
- 多向條件式(`if ... elif ... else ...`)
- 巢狀條件式
- 結構模式比對(`match ... case ...`)

選擇敘述

- 我們在要做決策的當下，通常手邊會有一些輔助資訊或情報，作為未來行動的參考依據。
  - 一旦這些資訊滿足某個條件時，就能依既定的動作來進行。

條件判斷只有真、假兩種結果（即布林值），用以代表條件是否滿足。

- 電腦內的每一個運算步驟都要非常明確，不能模糊(ambiguous)。
  - 作為條件判斷的資訊通常是可以量化，或是有明確定義的具體敘述。
  - 在條件式中大多有比較運算子或關係運算子

範例

```
1    score = 85
2    print(score >= 60)
3
4    bmi = 24.21
5    print(18.5 <= bmi < 24)     # 健康體位
6
7    print(0.1 + 0.1 + 0.1 == 0.3)
8
9    a, b = 1, 3
10   print(a = b)
```

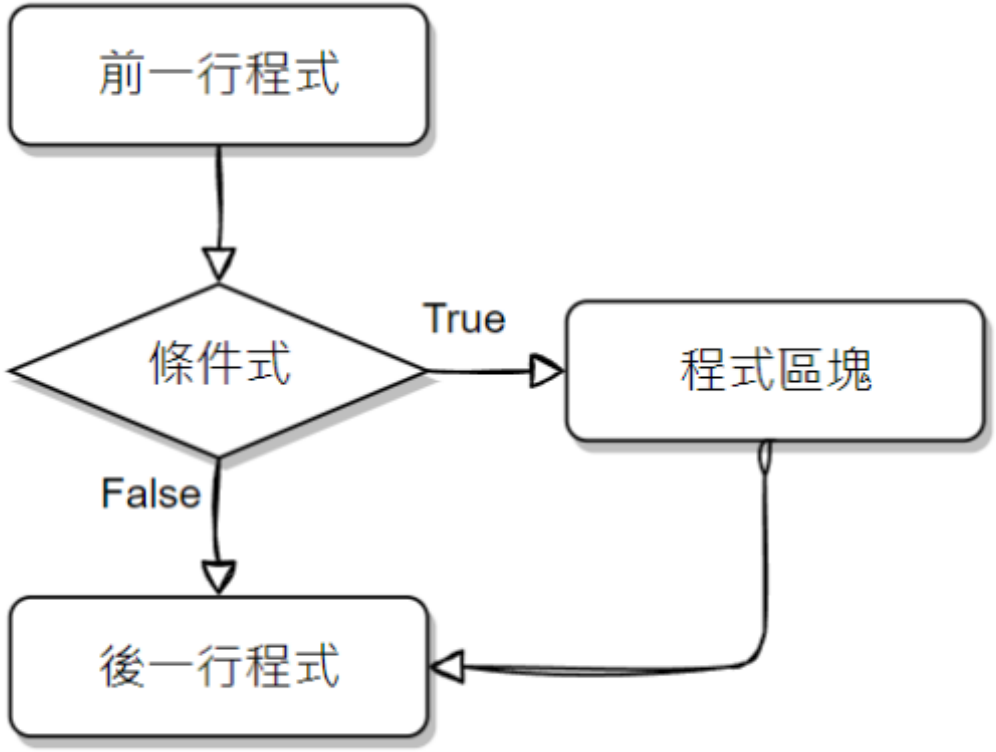
```
True
False
False
Traceback (most recent call last):
  File "C:\Users\user\VS_code\Ian_data\Untitled-1.py", line 10, in <module>
    print(a = b)
TypeError: 'a' is an invalid keyword argument for print()
```

單向條件式（if ...）

- 這是選擇敘述中最基本的型態，語法為：

```
if (條件式):
    程式區塊
```

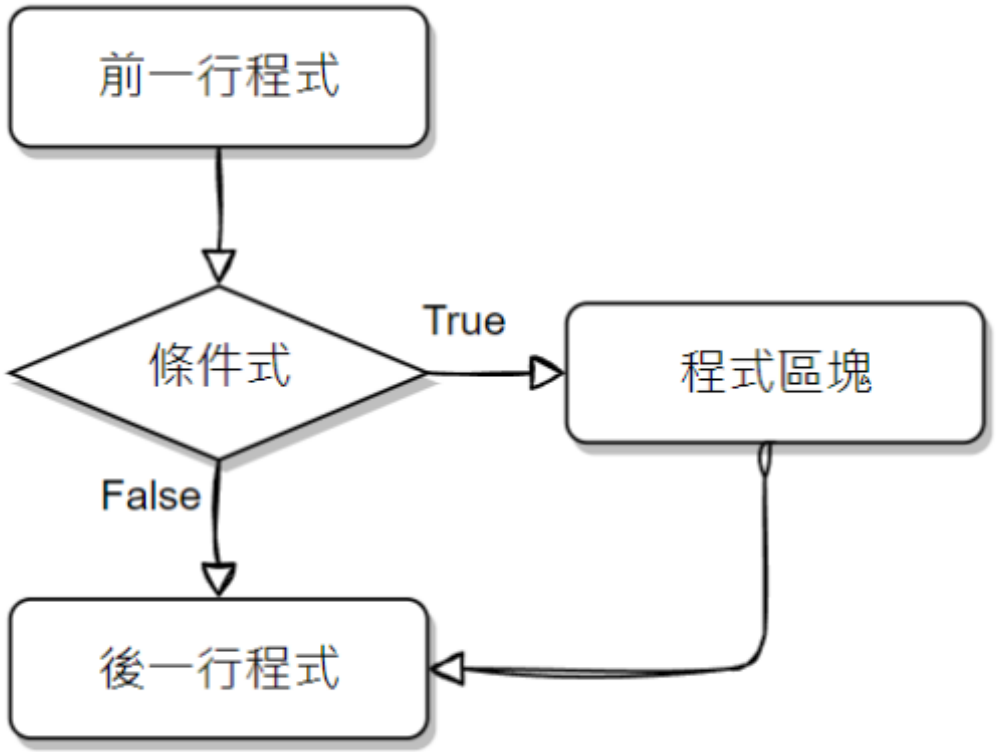
```
1    score = 85
2
3    if score >= 60:
4        print('PASS')
```



if ...

- 程式區塊內一般會有數行程式碼，若只有單行也可與 `if` 敘述合併成一行。

```
1    score = 85
2
3    if score >= 60:    print('PASS')
```



有時會看到這種寫法

```
1    n = int(input())
2
3    if n:
4        print(n, end='')
5
6    if n%2:
7        print("不是偶數")
```

往內縮？

```
23
23不是偶數
```

縮排(indentation)

- 比較底下 C 語言的程式碼與我們想像中的 Python 程式

```c
int score = 80;

if (score >= 60){
    printf("得分為%d", score);
    printf("恭喜及格！");
}
```

縮排

- 縮排是把程式碼依照結構作整理，使其有更高的可讀性。
  - 大部分程式語言(e.g., C\C++, Java)是以一對大括號「{}」來表示程式區塊

```
1  int score = 80;
2
3  if (score >= 60){
4      printf("得分為%d", score);
5      printf("恭喜及格！");
6  }
```

```
1  int score = 80;
2
3  if (score >= 60){
4  printf("得分為%d", score);
5  printf("恭喜及格！");
6  }
```

對電腦來說，是否縮排不影響執行結果，
畢竟縮排是方便我們閱讀而使用。

Python 的縮排(1/2)

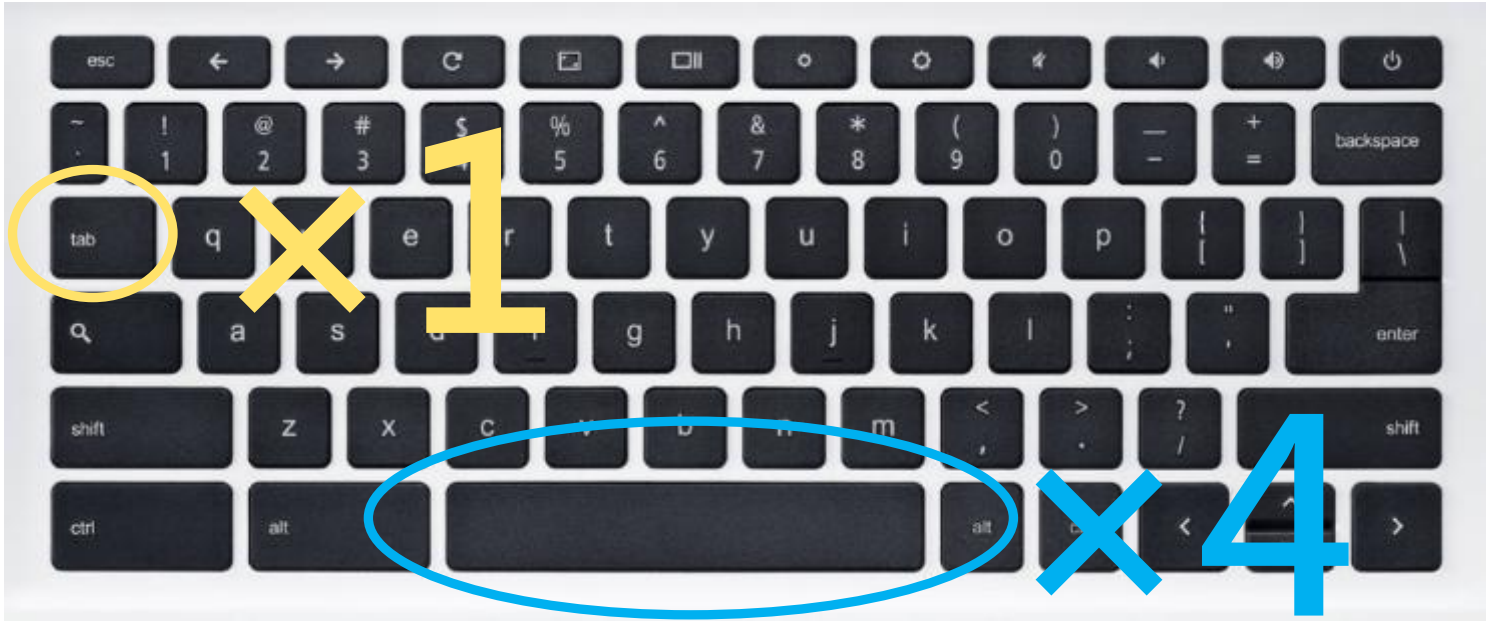
- Python 以冒號 + 縮排來表示程式區塊

```
1    score = 80
2
3    if (score >= 60):
4        print("得分為%d" % score)
5            print("恭喜及格！")
```

縮排

Python 的縮排(2/2)

- 縮排時留意兩個小地方：
  - 內縮數量：雖然根據《PEP8》的規定，使用四個空白作為內縮標準，但其實理論上想要用幾個都可以，只是大多預設為四個空白。
  - 內縮方式：



用哪一種有差嗎？

- 《Tab》*1
  - 優點：tab 鍵縮排的速度快、整齊、檔案小
  - 缺點：不同平台的編輯器將 Tab 字元視為不同長度，用不同編輯器打開，可能造成存取出錯、排版亂掉、檔案合併上的困難。
    - e.g., Unix 是 8 個字元，Windows、macOS 是 4 個字元
- 《Space》*4
  - 優點：不同平台的解讀有一致性
  - 缺點：鍛鍊手指肌肉

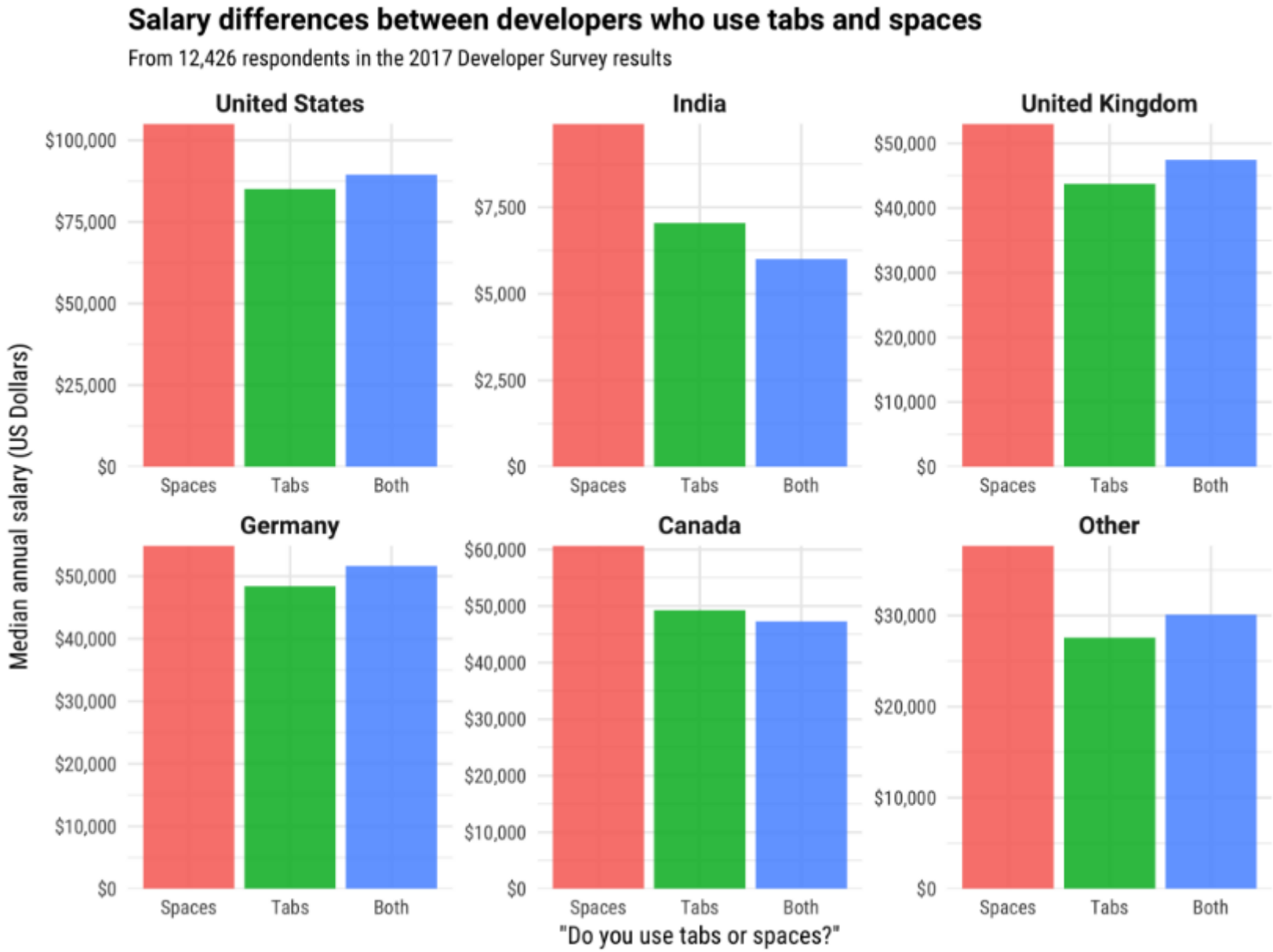
空白鍵 vs Tab鍵

- 彼此看不順眼的橋段，也曾被美劇「矽谷群瞎傳」(Silicon Valley)作為素材搬上螢幕。
  - 劇中男主角因為太介意曖昧對象使用空白鍵縮排，兩人因此鬧翻。

https://www.bnext.com.tw/article/44961/developers-use-spaces-make-money-use-tabs

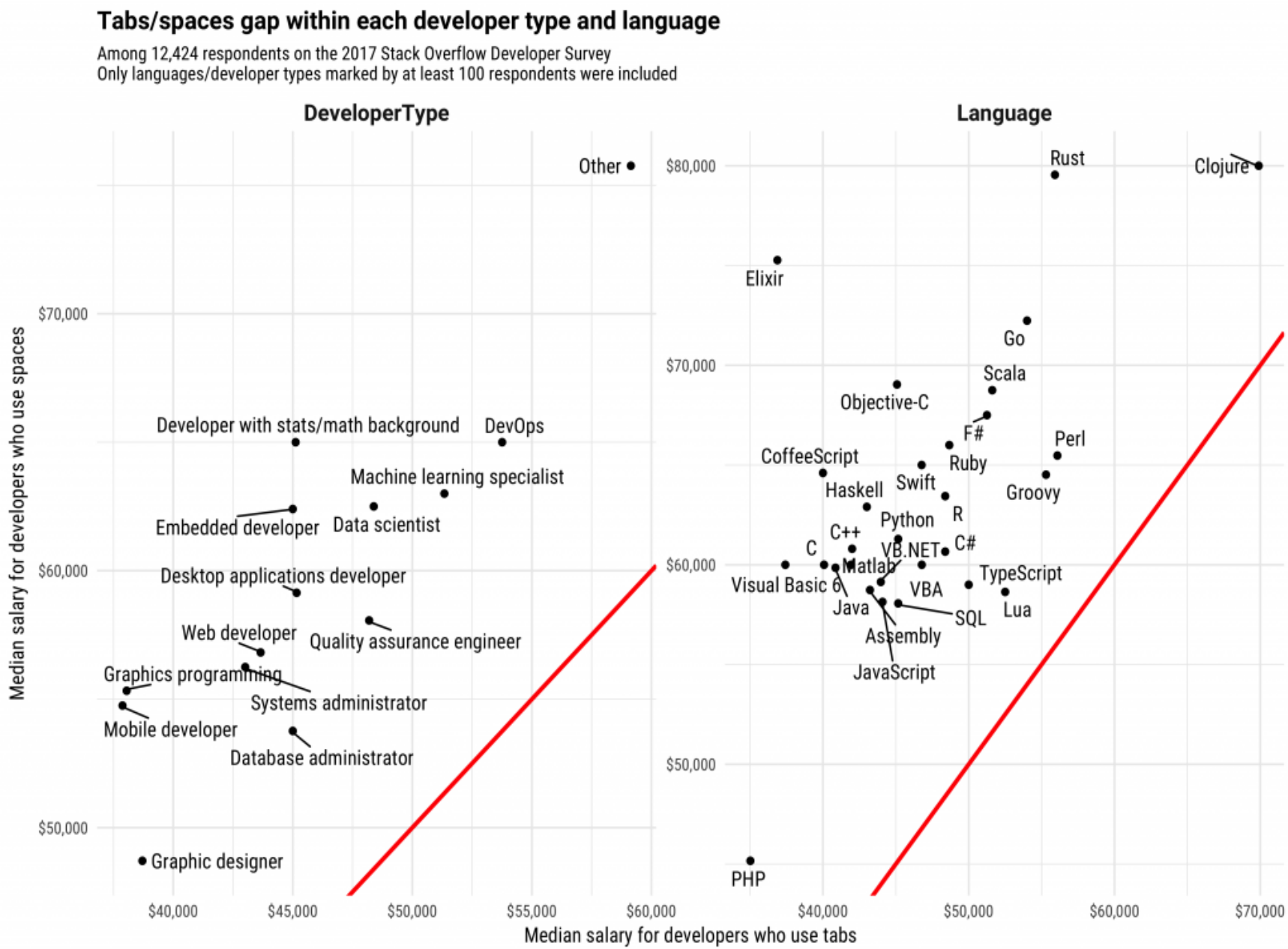
💬 無論年資長短，使用空白鍵縮排的開發者，薪水都較使用tab的開發者高。

圖片來源：Stack Overflow



💬 以不同國家來看，也發現使用tab和空白鍵縮排的使用習慣，和薪水有顯著差異。

圖片來源：Stack Overflow



💬 不論是哪種開發類型或程式語言，使用空白鍵的開發者平均薪資皆高於使用tab。

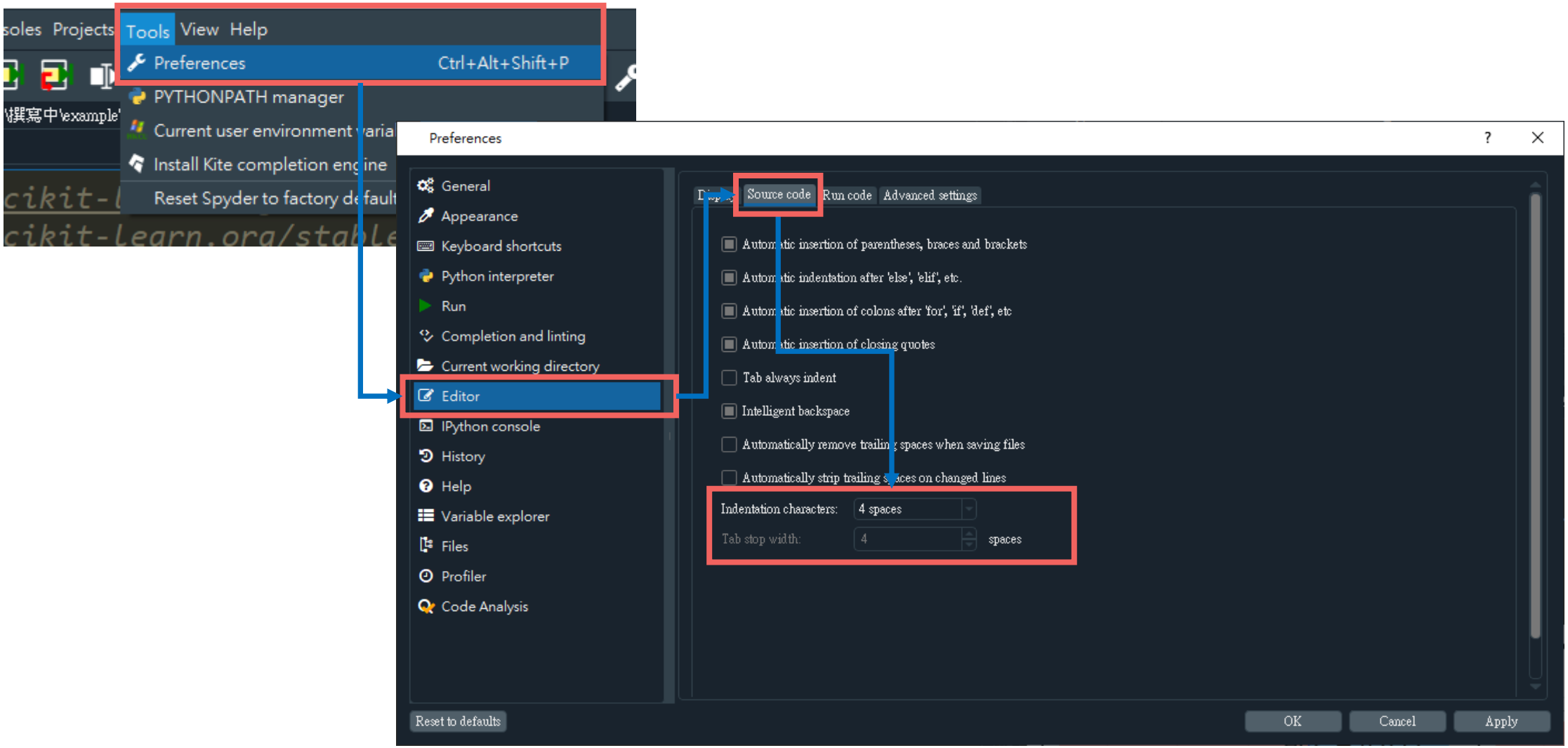
圖片來源：Stack Overflow

分析結果

- 整體而言，使用《Space》開發者的薪水較使用《Tab》的高出

8.6%

Soft-Tab


Tools View Help
Preferences Ctrl+Alt+Shift+P
PYTHONPATH manager
Current user environment
Install Kite completion engine
Reset Spyder to factory default
Preferences
General
Appearance
Keyboard shortcuts
Python interpreter
Run
Completion and linting
Current working directory
Editor
IPython console
History
Help
Variable explorer
Files
Profiler
Code Analysis
Source code
Run code
Advanced settings
Automatic insertion of parentheses, braces and brackets
Automatic indentation after 'else', 'elif', etc.
Automatic insertion of colons after 'for', 'if', 'def', etc
Automatic insertion of closing quotes
Tab always indent
Intelligent backspace
Automatically remove trailing spaces when saving files
Automatically strip trailing spaces on changed lines
Indentation characters: 4 spaces
Tab stop width: 4 spaces
Reset to defaults
OK
Cancel
Apply

主題

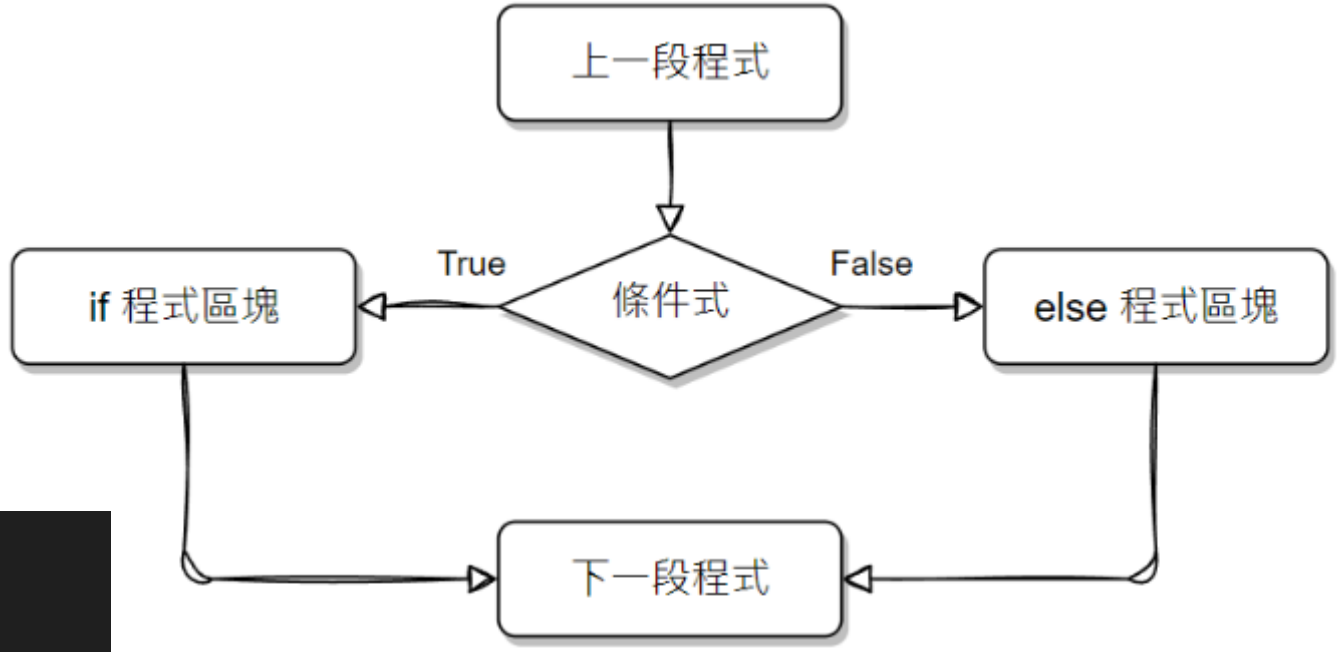
- 程式流程控制
- 單向條件式(`if ...`)
- 雙向條件式(`if ... else ...`)
- 多向條件式(`if ... elif ... else ...`)
- 巢狀條件式
- 結構模式比對(`match ... case ...`)

雙向條件式(if ... else ...)

- 總感覺只有 `if ...` 少了點甚麼？

```
1    score = 85
2
3    if score >= 60:
4        print('PASS')
```

雙向條件式



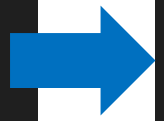

```
1    score = 85
2
3    if score >= 60:
4        print('PASS')
5    else:
6        print('Fail')
```

注意冒號

注意縮排

三元運算子(1/3)

```
3    score = 50
4
5    if score < 60:
6        score = 60
7    else:
8        score += 5
```



```
11   score = 50
12   score = 60 if (score < 60) else score+5
```

三元運算子的用法為：

```
變數 = 值1 if 條件式 else 值2
```

三元運算子(2/3)

```
1    pwd = input("請輸入密碼：")
2
3    if pwd == "123":
4        print("密碼正確")
5    else:
6        print("輸入錯誤！")
```

```
8    status = "密碼正確" if (pwd == "123") else "輸入錯誤！"
9    print(status)
```

三元運算子(3/3)

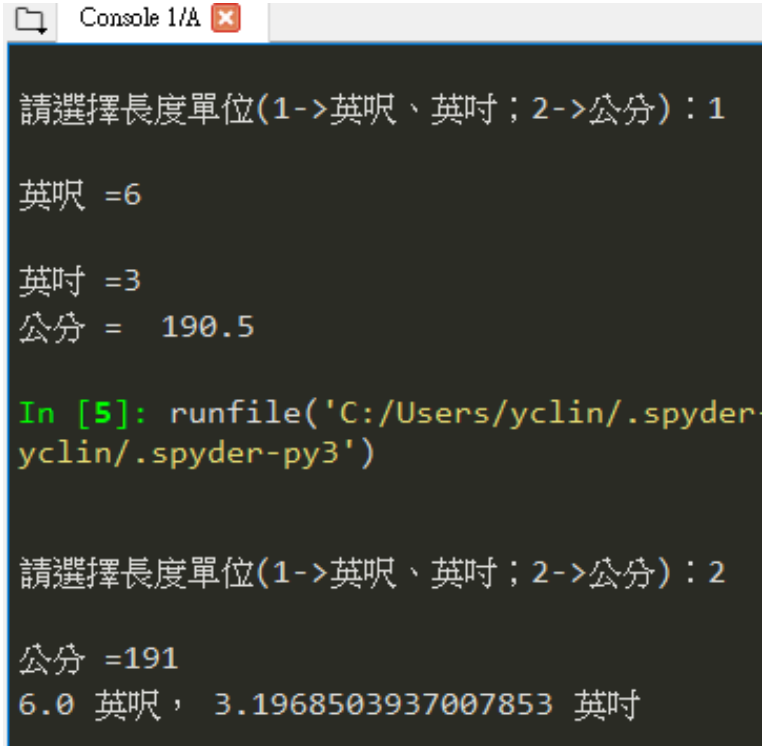
```
1    score = eval(input("請輸入成績："))
2
3    print(f"分數{score}, {'不及格' if score < 60 else '及格'}")
```

```
請輸入成績：80
分數80, 及格
```

```
請輸入成績：50
分數50, 不及格
```

練習：英制與公制互換

- 國外長度單位常使用英制單位，例如身高常使用英呎、英吋表示，請寫一程式進行長度單位的輸入與轉換。
  - 1 英呎 = 12 英吋，1 英吋 = 2.54 公分
  - 利用 `if ... else ...` 進行判斷



練習：計算導數值

- 使用者輸入一個數 a，並給定一個函數 $f(x)$ 如下，請計算 $f(x)$ 在 $x = a$ 的導數(derivative)。
  - 輸入：一個數 a
  - 輸出：$f(x)$ 在 $x = a$ 的導數

$$f(x) = \frac{3x^4 - 7x^2 + 5}{x}$$

  - 注意：請利用導數的極限定義來計算，勿直接將導數寫在程式碼裡。

主題

- 程式流程控制
- 單向條件式(`if ...`)
- 雙向條件式(`if ... else ...`)
- 多向條件式(`if ... elif ... else ...`)
- 巢狀條件式
- 結構模式比對(`match ... case ...`)

範例

```
1    score = eval(input("請輸入成績："))
2
3    if 0 <= score < 60:
4        print("不及格！")
5    else:
6        print("及格")
```

輸入 -1

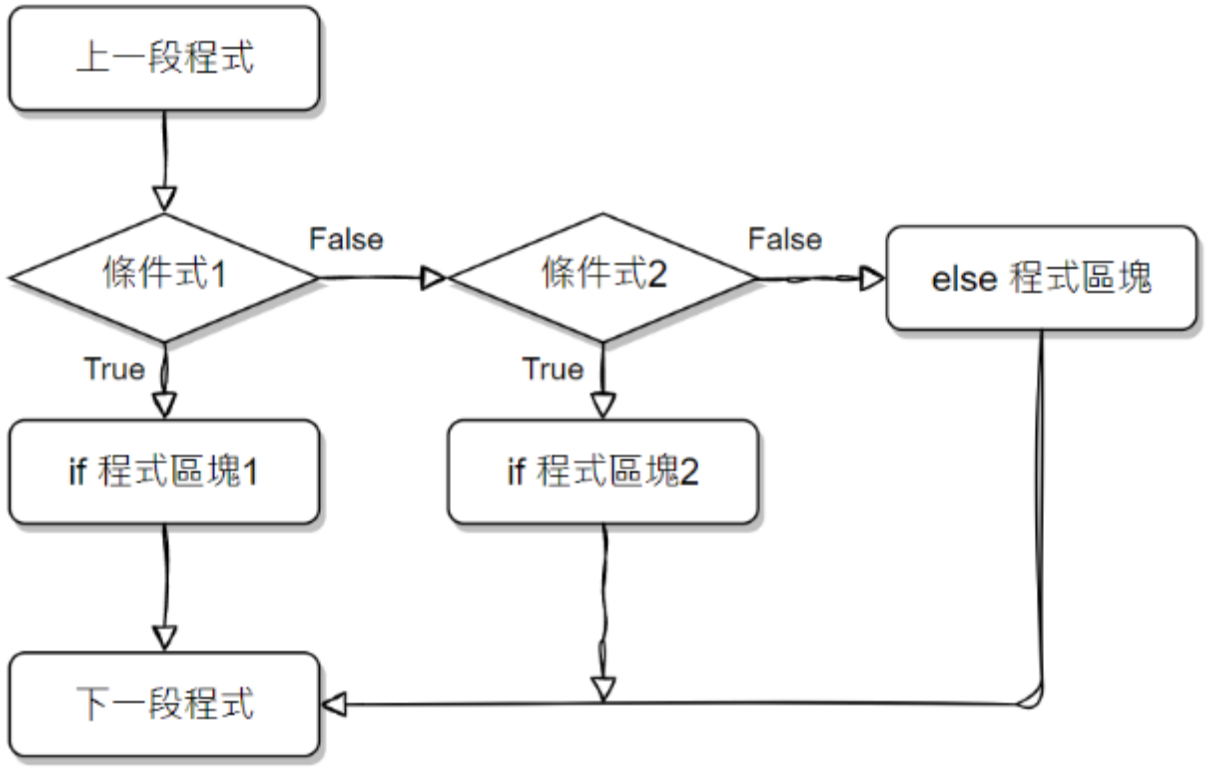
這有個小問題

修改一下

```
1    score = eval(input("請輸入成績："))
2
3    if 0 <= score < 60:
4        print("不及格！")
5    elif 60 <= score <=100:
6        print("及格")
7    else:
8        print("輸入錯誤！")
```

多向條件式(if ... elif ... else ...)

```
1   score = eval(input("請輸入成績："))
2
3   if 0 <= score < 60:
4       print("不及格！")
5   elif 60 <= score <=100:
6       print("及格")
7   else:
8       print("輸入錯誤！")
```



範例

- 輸入：考試成績（滿分100）
- 輸出：若成績在
  - 90 分以上顯示「優等」，
  - 80 - 89 分顯示「甲等」，
  - 60 - 79 分顯示「乙等」，
  - 0 - 59 分顯示「丙等」，
  - 其餘顯示「輸入錯誤」。

```
1   score = eval(input("請輸入成績："))
2
3   if 90 <= score <= 100:
4       print("優等")
5   elif 80 <= score < 90:
6       print("甲等")
7   elif 60 <= score < 80:
8       print("乙等")
9   elif 0 <= score < 60:
10      print("丙等")
11  else:
12      print("輸入錯誤")
```

Q：「< 90」可拿掉嗎？

主題

- 程式流程控制
- 單向條件式(`if ...`)
- 雙向條件式(`if ... else ...`)
- 多向條件式(`if ... elif ... else ...`)
- 巢狀條件式
- 結構模式比對(`match ... case ...`)

巢狀(nested)

- 在 `if` 的條件式內有條件式，稱為巢狀條件式。
  - Python 並未規定巢狀判斷式的層數，要加多少層條件式都可以。
  - 層數太多會降低可讀性，而且維護困難。

```
1    score = eval(input("請輸入成績："))
2
3    if 50 <= score:
4        if score < 60:
5            score += 10
```

```
8    if (50 <= score) and (score < 60):
9        score += 10
```

```
10   if 50 <= score < 60:
11       score += 10
```

哪個比較容易閱讀？

```
2   score = eval(input("請輸入成績："))
3
4   if 0 <= score < 60:
5       print("不及格！")
6   else:
7       if 60 <= score <=100:
8           print("及格")
9       else:
10          print("輸入錯誤！")
```

```
1   score = eval(input("請輸入成績："))
2
3   if 0 <= score < 60:
4       print("不及格！")
5   elif 60 <= score <=100:
6       print("及格")
7   else:
8       print("輸入錯誤！")
```

練習：成績評等

- 輸入：微積分成績、計算機程式成績（滿分100）
- 輸出：
  - 如果微積分 90 分(含)以上，則
    - 如果程式 90 分(含)以上 ➔ 顯示「超優等」
    - 如果程式 89 分以下 ➔ 顯示「優等」
  - 如果微積分 90 分以下一律顯示「甘罷嗲」

```
微積分成績：91
計算機程式成績：95
超優等
```

```
微積分成績：90
計算機程式成績：80
優等
```

```
微積分成績：80
計算機程式成績：95
甘罷嗲
```

主題

- 程式流程控制
- 單向條件式(`if ...`)
- 雙向條件式(`if ... else ...`)
- 多向條件式(`if ... elif ... else ...`)
- 巢狀條件式
- 結構模式比對(`match ... case ...`)

switch 敘述

- 許多程式語言都有 switch 敘述針對單一條件的不同狀況來執行程式區塊
  有些時候會比多向條件式更直覺些。
  - if…elif…else… 敘述能設定任意條件

```c
#include <stdio.h>

int main(){
    int score = 0;

    printf("請輸入成績：");
    scanf("%d", &score);

    switch (score / 10){
        case 10: case 9:
            printf("優等");
            break;
        case 8:
            printf("甲等");
            break;
        case 7:
            printf("乙等");
            break;
        case 6: case 5: case 4: case 3:
        case 2: case 1: case 0:
            printf("丙等");
            break;
        default:
            printf("輸入錯誤");
            break;
    }

    return 0;
}
```

Python 沒有類似 `switch` 的語法

- 在 Python 社群中確實曾有提案與討論，而且早在 2001 與 2006 年就提出，可是最終都被否決了。
  - 基本上，否決原因在於各種提案中，沒有一個實現可適切地融入 Python 既有的語法與風格。
- 在 Python 官方的「Design and History FAQ」直接列出條目來回答「為何沒有 `switch`？」
  - 總之，結論是使用 `if…elif…else…` 或 `dict`（詞典，之後會介紹）完全可以應付需求。

結構模式比對(match ... case ...)

- 直到 2021 年的 Python 3.10，委員會通過《PEP634、PEP635、PEP636》加入一個新的 match…case… 語法，稱之為結構模式比對(structural pattern matching)。

```python
lang = input("請輸入想學的程式語言：")

match lang:
    case "Python":
        print("想做資料科學家")
    case "JavaScript":
        print("想從事網頁開發")
    case "Java":
        print("想從事APP開發")
    case _:
        print("抱歉，我不知道這個程式語言！")
```

```
請輸入想學的程式語言：Python
想做資料科學家
```

```python
score = eval(input("請輸入成績："))

level = score // 10
match level:
    case 10 | 9 as s:
        print("優等")
        if s == 10:    print("太棒了，滿分呢！")
    case 8:
        print("甲等")
    case 7:
        print("乙等")
    case 6 | 5 | 4 | 3 | 2 | 1| 0:
        print("丙等")
    case _:
        print("輸入錯誤")
```

C 語言

```c
#include <stdio.h>

int main(){
    int score = 0;

    printf("請輸入成績：");
    scanf("%d", &score);

    switch (score / 10){
        case 10: case 9:
            printf("優等");
            break;
        case 8:
            printf("甲等");
            break;
        case 7:
            printf("乙等");
            break;
        case 6: case 5: case 4: case 3:
        case 2: case 1: case 0:
            printf("丙等");
            break;
        default:
            printf("輸入錯誤");
            break;
    }

    return 0;
}
```

注意

- 由於 `match…case…` 語法是 Python 3.10 才有且 CodeJudger 目前用的是 Python 3.8.8，所以

match…case… 語法在 CJ 平台不能使用 ☹

本章結束

Py02

選擇敘述與迴圈 (2)

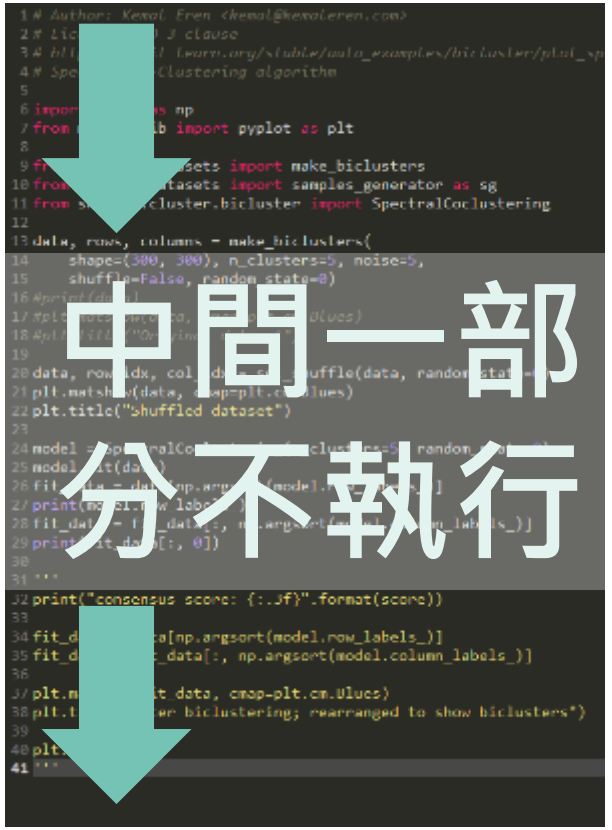
王頌文

主題

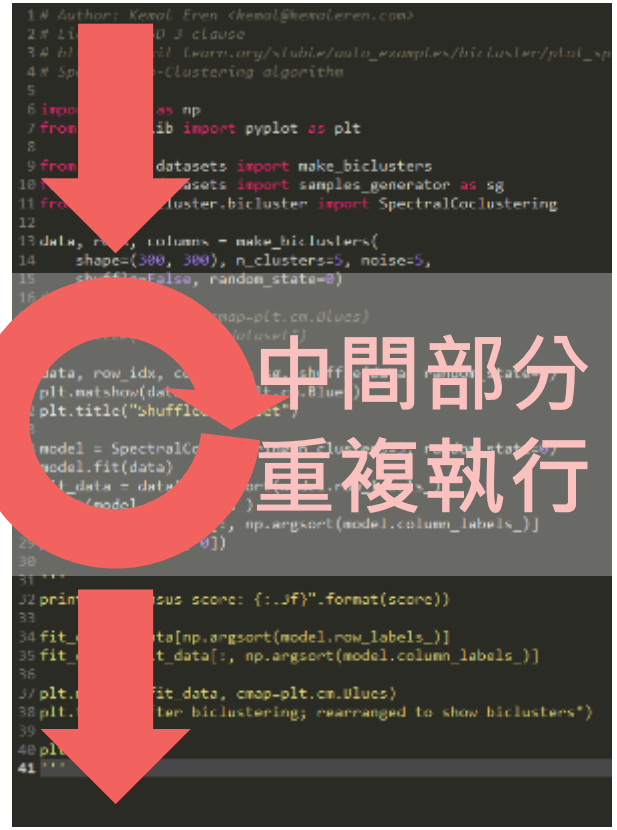
- while 迴圈
- for 迴圈
  - range()
  - break 與 continue
  - while … else … 與 for … else …
- 巢狀迴圈

結構化程式設計(structured programming)

循序(sequence)



選擇(selection)



重複(repetition)

迴圈(loop)

- 重複執行同樣動作是電腦最擅長的事情，而這個「重複執行」正是迴圈最核心的概念。
  - 也是結構化程式的基礎流程之一。
  - 日常生活中也到處可見類似不斷重複的行為，比方說我們每天的刷牙、走路、每月固定的小額投資等。
  - 把無聊且重複的動作交給電腦自動執行，不僅能節省人力資源並高效率產出，也能有一定保證品質。

Python 的迴圈

- 在許多程式語言裡常見的迴圈敘述有三種：
  - `for …`
  - `while …`
  - `do … while …`

Python 的迴圈只有 `for` 與 `while`

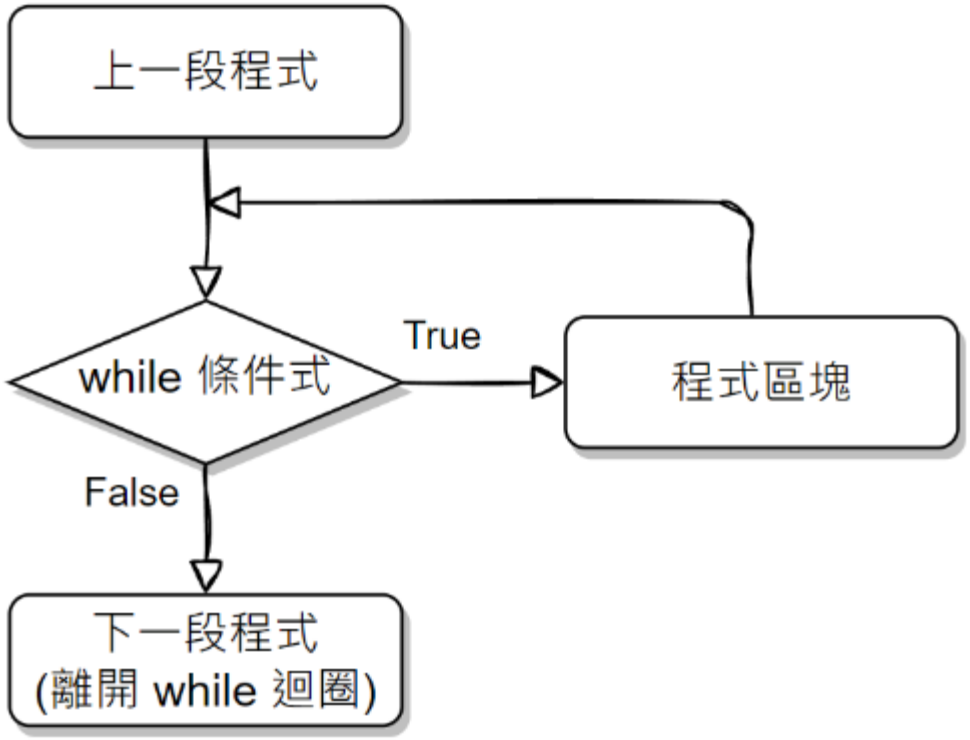
while 迴圈

- 這個敘述用來表達「若某條件成立，則一直重複執行既定動作，直到該條件不再成立為止」的語意，運作流程如下：

- 語法為

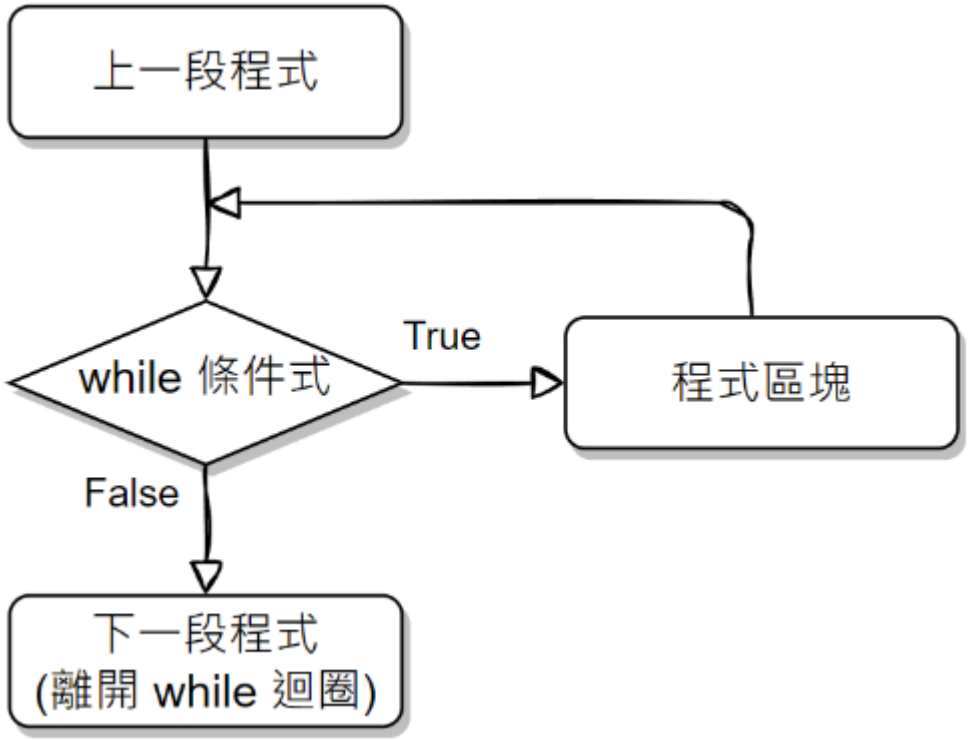
```
while 條件式:
    程式區塊
```

是否與 if 的運作流程很像？



無窮迴圈(infinite loop)

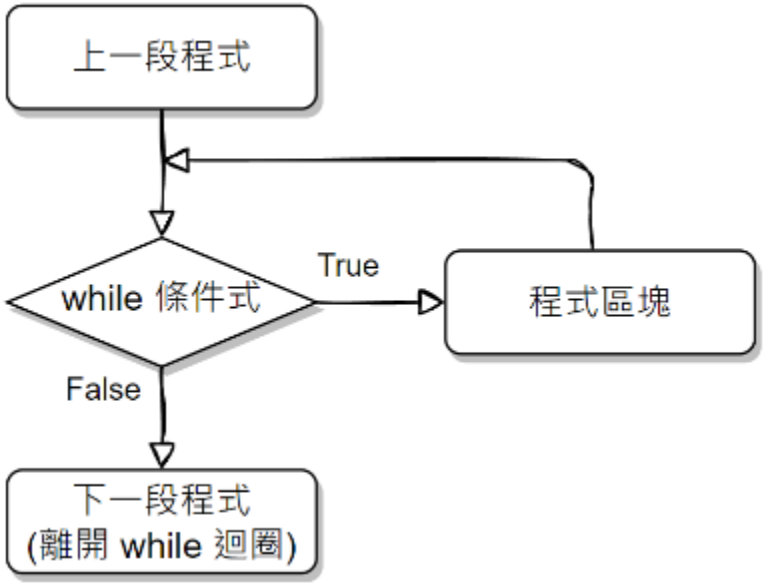
- while 迴圈的運作流程有時候對初學者來說是個頗大的雷區，只要一不小心沒把條件式或程式區塊寫好就可能踩雷。
  - 可能是沒有設定迴圈的終止條件
  - 可能是結束迴圈的條件不可能成立
  - 這些都會導致迴圈永無止盡地重複執行，即所謂無窮迴圈。



無窮迴圈會用掉所有可用的處理器時間，是造成系統「假死機」（指應用程式沒有回應，一種與當機類似的現象）的原因之一，不可謂不慎。

範例

```
1   num = eval(input("請輸入一個數："))
2
3   i = 1
4   while i <= num:
5       print(f"i = {i}")
6       i += 1
7
8   print(f"離開while迴圈, i = {i}")
```




```
請輸入一個數：3
i = 1
i = 2
i = 3
離開while迴圈, i = 4
```

執行這程式會發生何事？

```
1    num = eval(input("請輸入一個數："))
2
3    i = 1
4    while i >= num:
5        print(f"i = {i}")
6        i += 1
7
8    print(f"離開while迴圈, i = {i}")
```

《Ctrl + c》可中止執行程式

浮點數的數值誤差

- 0.01 + 0.02 + … + 0.99 + 1 = ?

正確答案是 50.5

```
1    sum_ = 0
2
3    i = 0.01
4    while i <= 1:
5        sum_ += i
6        i += 0.01
7
8    print(f"總和是 {sum_:.1f}")
```

```
總和是 49.5
```

練習：計算成績

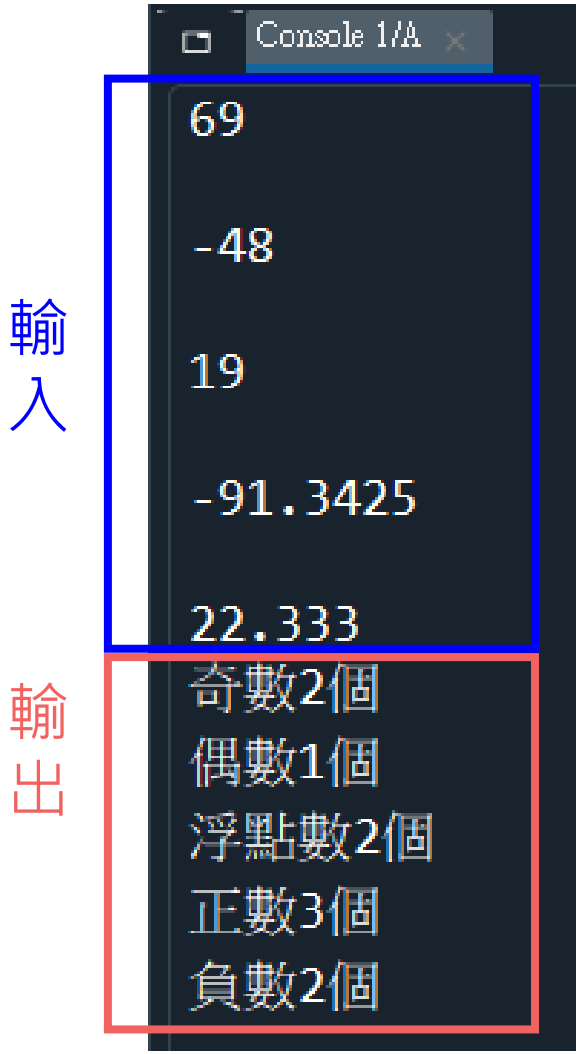
- 請以 `while` 迴圈設計一個輸入成績的程式，要求如下：
  - 輸入：成績或「-1」
  - 輸出：
    - 「-1」表示成績輸入結束
    - 結束後顯示班上成績加總及平均成績

```
請輸入分數：90
請輸入分數：80
請輸入分數：70
請輸入分數：-1
總分為 240，平均成績為 80.00
```

練習：數值計數

- 請撰寫一程式，讓使用者輸入五個數值，計算並輸出下列數量：
  - 偶數、奇數、正數、負數、浮點數

  - 提示：
    ① 0 為偶數
    ② 0 既不是正數也不是負數
    ③ 奇偶數僅適用於整數



主題

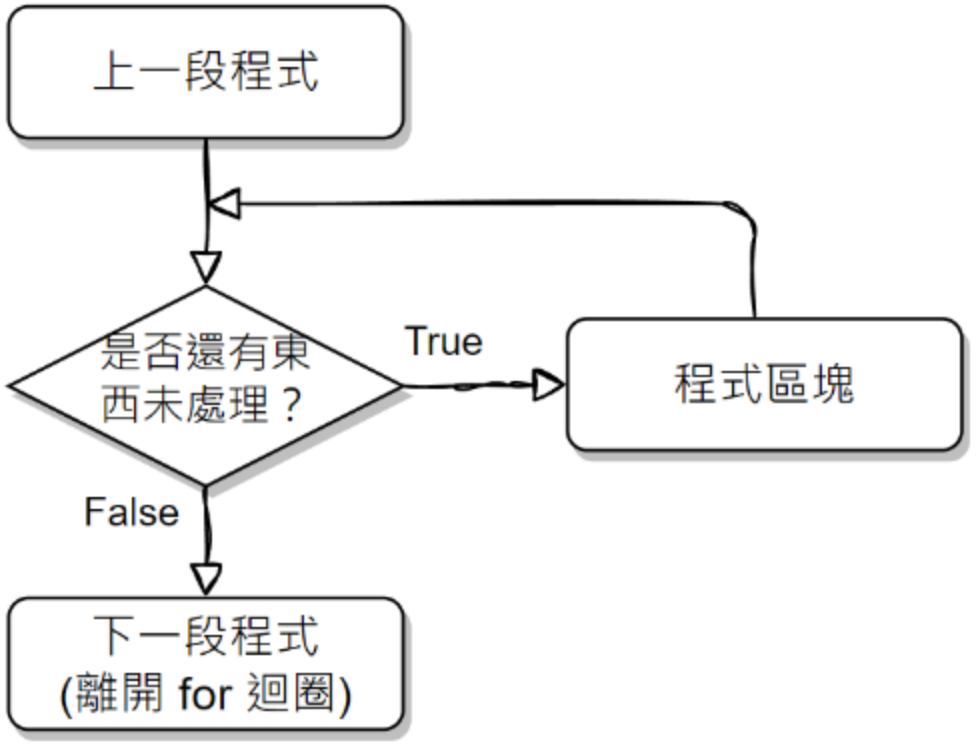
- while 迴圈
- for 迴圈
  - range()
  - break 與 continue
  - while … else … 與 for … else …
- 巢狀迴圈

for 迴圈(1/2)

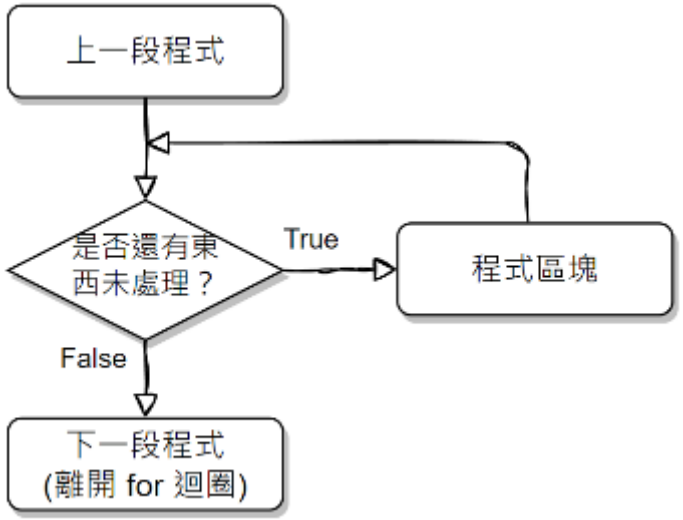
- 用來表達「對目標內的每一筆資料執行既定動作，直到每筆資料都執行完畢為止」的語意。

- 語法為
  - `for 變數 in 可迭代物件:`
    `程式區塊`

是否與 `while` 的運作流程很像？



for 迴圈(2/2)



- `for 變數 in 可迭代物件:`
  `程式區塊`

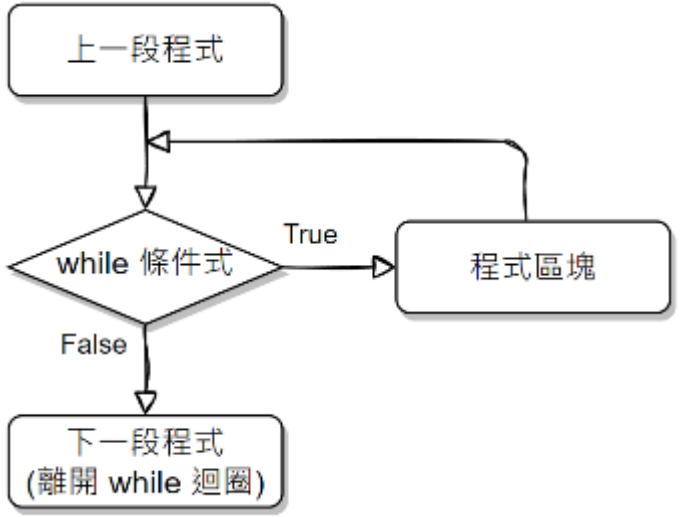
- 可迭代物件(iterable)可簡單理解為「一個裝著很多東西的容器，且裡面的東西能一個個按照順序拿出來。」
  - e.g., 串列(list)、數組(tuple)、字典(dictionary)、集合(set)與 `range()`

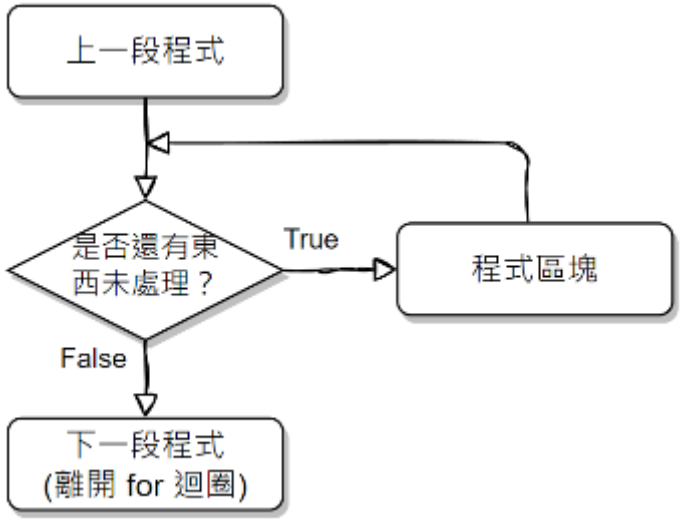
```
1    poke = "Pikachu"
2    for c in poke:
3        print(c)
```

比一比

	相同處	不同處
while	前測試型迴圈	● 不需要先指定重複次數 ● 看條件式是否成立
for		● 通常用於執行固定次數的重複動作 ● 看目標內的所有東西是否都處理完畢

先測試條件式是否成立，倘若成立才會執行迴圈內的程式區塊。





主題

- while 迴圈
- for 迴圈
  - range()
  - break 與 continue
  - while … else … 與 for … else …
- 巢狀迴圈

range() (1/5)

- 為何要這個？
  - C 語言的 for 迴圈

```c
for(int i = 0; i < 5; i++) {
    printf("%d ", i);
}
```

變數 i → 0, 1, 2, 3, 4

迴圈中常需要整數的等差數列 (arithmetic sequence)

  - Python 的 for 迴圈

```python
for i in range(5):
    print(i, end=' ')
```

```
0 1 2 3 4
```

range() (2/5)

- 迴圈中常使用整數數列(e.g., 1, 2, 3, …)，因此 Python 有個內建(built-in)函式 range() 用來建立整數數列。

- range() 的語法有三種，以參數的個數來區分。

① 1 個參數的語法為：

- range(終止值)
- range(5)

0, 1, 2, 3, 4

range() (3/5)

- range() 的語法有三種，以參數的個數來區分。

② 2 個參數的語法為：

- range(起始值, 終止值)

- range(3, 7) → 3, 4, 5, 6
- range(-6, -3) → -6, -5, -4
- range(-2, -6)

range() (4/5)

- range() 的語法有三種，以參數的個數來區分。

③ 3 個參數的語法為：

- range(起始值, 終止值, 間隔值)

- range(3, 8, 1) = range(3, 8)

3, 4, 5, 6, 7

- range(3, 8, 2)

3, 5, 7

- range(8, 3, -1)

8, 7, 6, 5, 4

range() (5/5)

- 想要直接輸出 range() 產生的等差數列有點麻煩，因為其本質畢竟還是一個封裝過的物件。
  - 常見作法則是在輸出前就利用內建函式 list()，強制將其轉換為串列（之後介紹）。

```
1   print(range(5))
2
3   print(list(range(5)))
4
5   print(list(range(1, 5)))
6
7   print(list(range(1, 5, 2)))
8
9   print(list(range(0, -6, -2)))
```

```
range(0, 5)
[0, 1, 2, 3, 4]
[1, 2, 3, 4]
[1, 3]
[0, -2, -4]
```

練習：計算連加與連乘

- 讓使用者輸入一個正整數 n，程式會計算由 1 ~ n 的總和與乘積。
  - 輸入：一個正整數 n
  - 輸出：1 + 2 + … + n
    1 × 2 × … × n

練習：判斷質數

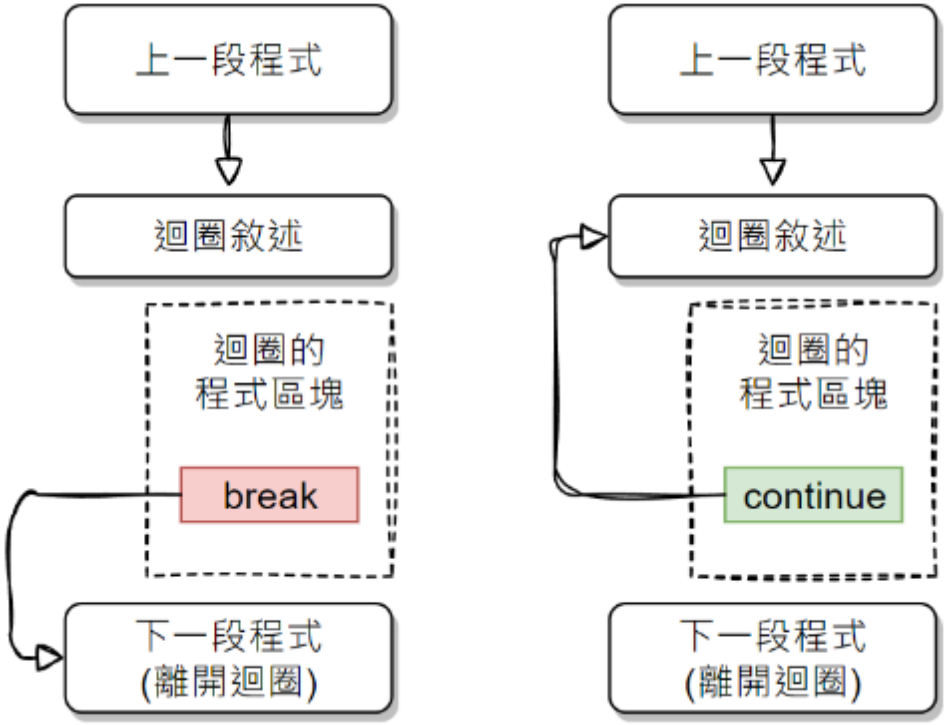
- 使用者輸入一個正整數 num，判斷 num 是否為質數。
  - 輸入：一個正整數 num
  - 輸出：
    - num 為質數 ➔ 輸出 "num 是質數"
    - num 不為質數 ➔ 輸出 "num 不是質數"

主題

- while 迴圈
- for 迴圈
  - range()
  - break 與 continue
  - while … else … 與 for … else …
- 巢狀迴圈

進階流程控制

- 一般來說，迴圈不斷地重複執行既定動作，可是在某些情境下，我們需要在迴圈執行過程中強制改變其執行流程。



兩者通常會搭配邏輯判斷一同使用

break 敘述

- 當我們希望迴圈在執行程式區塊的過程中，直接中斷離開這個迴圈，並繼續執行迴圈後的下一段程式，就能使用 break 敘述。

```
1    for x in range(5):
2        if x == 3:    break
3        print(x)
```

```
0
1
2
```

範例

```
請輸入分數：70
請輸入分數：90
請輸入分數：80
請輸入分數：-1
總分為 240，平均成績為 80.00
```

```python
sum_ = count = 0

while True:
    score = eval(input("請輸入分數："))
    if score == -1:     break

    sum_ += score       # 累加輸入的分數
    count += 1          # 計算輸入筆數

print(f"總分為 {sum_}，平均成績為 {sum_/count:.2f}")
```

利用 break 來除錯

- 原本目標是計算連加「1 + 0.9 + … + 0.1」的值

```python
num, sum_ = 1, 0

while num != 0:
    sum_ += num
    num -= 0.1

    print(num)
    if num <= -0.2:     break

print(f"總和 = {sum_}")
```

```
0.9
0.8
0.7000000000000001
0.6000000000000001
0.5000000000000001
0.40000000000000013
0.30000000000000016
0.20000000000000015
0.10000000000000014
1.3877787807814457e-16
-0.09999999999999987
-0.19999999999999987
-0.29999999999999999
總和 = 5.200000000000001
```

continue 敘述

- 當我們想要迴圈執行完部分程式區塊後，即略過剩餘部分直接跳到下一個元素時，就可使用 continue 敘述。

```
1    for x in range(5):
2        if x == 3:    continue
3        print(x)
```

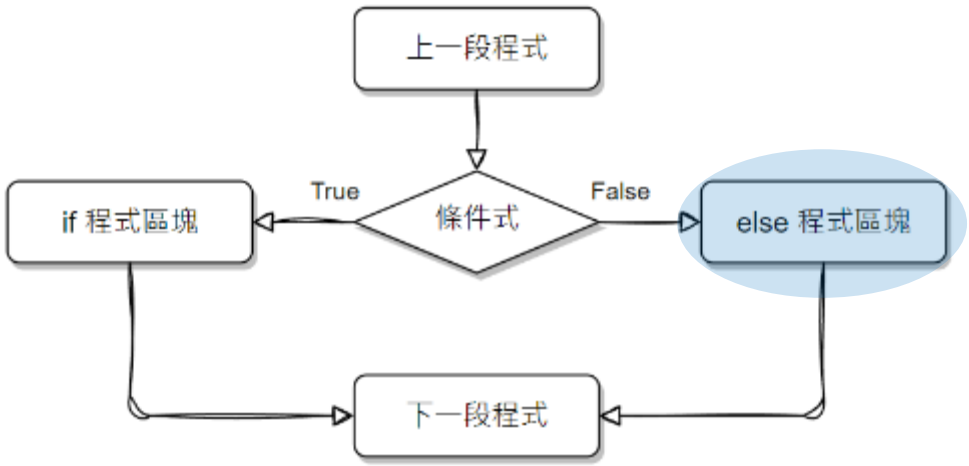
```
0
1
2
4
```

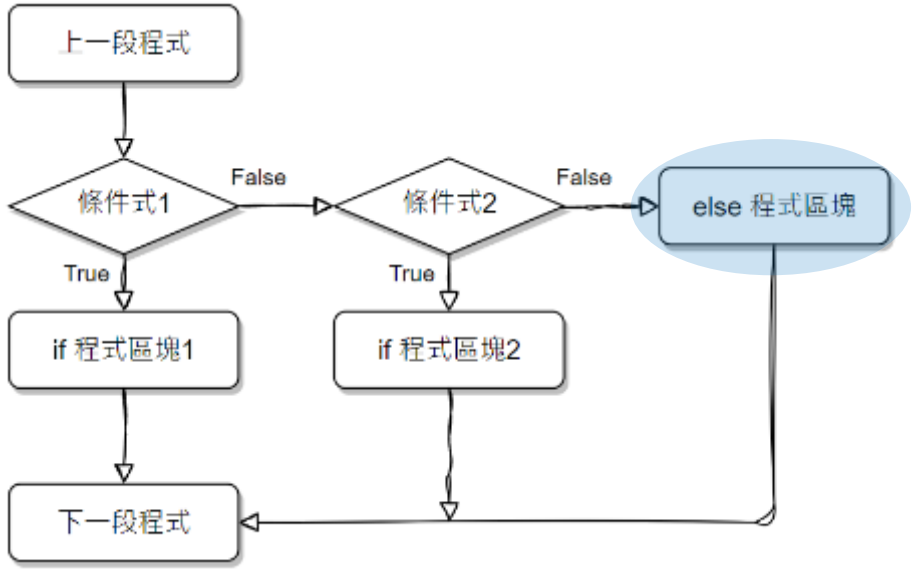
主題

- while 迴圈
- for 迴圈
  - range()
  - break 與 continue
  - while … else … 與 for … else …
- 巢狀迴圈

else 敘述(1/2)

- 大多數程式語言都有 if 與 else 搭配使用，但 else 在 Python 中除了能與 if 配合外，也能與 for、while 迴圈配對。
  - 用來表達「如果迴圈正常結束，就執行既定動作」的語意





else 敘述(2/2)

- 所謂正常結束，
  - while 迴圈 → 當條件式不成立的時候
  - for 迴圈 → 所有元素都被處理完畢

- 簡言之，當迴圈遇到 break 或 return 等敘述而中斷執行並離開時，就不會執行 else 的部分。

範例(1/2)

- 找出能被 17 整除且小於給定數值的最大正整數

旗標(flag)變數

```
1   upper = eval(input("請輸入一個數："))
2
3   flag_find = False
4   for x in range(upper-1, 0, -1):
5       if x%17 == 0:
6           print(f"{x} 符合題目要求")
7           flag_find = True
8           break
9
10  if not flag_find:
11      print("沒找到符合題意要求的正整數")
```

```
請輸入一個數：9487
9486 符合題目要求
```

範例(2/2)

```python
upper = eval(input("請輸入一個數："))

for x in range(upper-1, 0, -1):
    if x%17 == 0:
        print(f"{x} 符合題目要求")
        break
else:
    print("沒找到符合題意要求的正整數")
```

```python
upper = eval(input("請輸入一個數："))

flag_find = False
for x in range(upper-1, 0, -1):
    if x%17 == 0:
        print(f"{x} 符合題目要求")
        flag_find = True
        break

if not flag_find:
    print("沒找到符合題意要求的正整數")
```

練習：判斷質數

- 利用 `for … else …` 迴圈來判斷質數
  - 輸入：一個正整數 num
  - 輸出：
    - num 為質數 ➔ 輸出 "num 是質數"
    - num 不為質數 ➔ 輸出 "num 不是質數"

主題

- while 迴圈
- for 迴圈
  - range()
  - break 與 continue
  - while … else … 與 for … else …
- 巢狀迴圈

巢狀迴圈

- 在一個迴圈的程式區塊內包含更多層迴圈時，就稱這個結構為巢狀迴圈(nested loop)。

```
1    for i in range(2):
2        for j in range(3):
3            print(f"i={i}, j={j}")
```

```
i=0, j=0
i=0, j=1
i=0, j=2
i=1, j=0
i=1, j=1
i=1, j=2
```

添加 break

```
1    for i in range(2):
2        for j in range(3):
3            print(f"i={i}, j={j}")
```

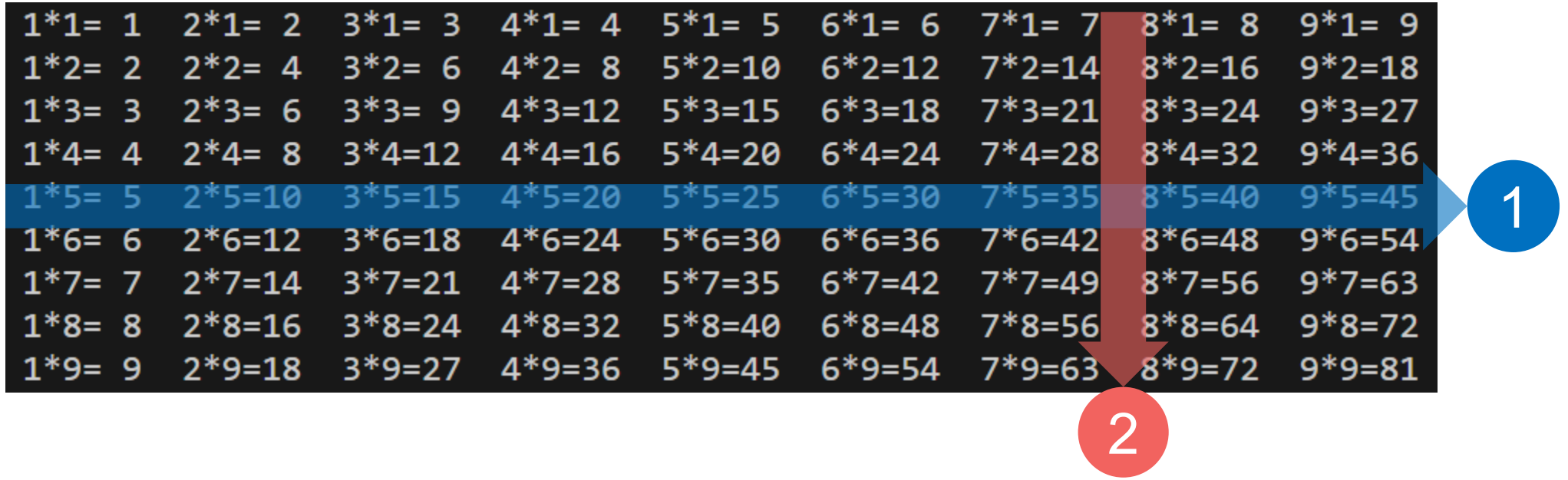
```
i=0, j=0
i=0, j=1
i=0, j=2
i=1, j=0
i=1, j=1
i=1, j=2
```

```
1    for i in range(2):
2        for j in range(3):
3            print(f"i={i}, j={j}")
4            break
```

```
i=0, j=0
i=1, j=0
```

九九乘法表

```
1*1= 1  2*1= 2  3*1= 3  4*1= 4  5*1= 5  6*1= 6  7*1= 7  8*1= 8  9*1= 9
1*2= 2  2*2= 4  3*2= 6  4*2= 8  5*2=10  6*2=12  7*2=14  8*2=16  9*2=18
1*3= 3  2*3= 6  3*3= 9  4*3=12  5*3=15  6*3=18  7*3=21  8*3=24  9*3=27
1*4= 4  2*4= 8  3*4=12  4*4=16  5*4=20  6*4=24  7*4=28  8*4=32  9*4=36
1*5= 5  2*5=10  3*5=15  4*5=20  5*5=25  6*5=30  7*5=35  8*5=40  9*5=45
1*6= 6  2*6=12  3*6=18  4*6=24  5*6=30  6*6=36  7*6=42  8*6=48  9*6=54
1*7= 7  2*7=14  3*7=21  4*7=28  5*7=35  6*7=42  7*7=49  8*7=56  9*7=63
1*8= 8  2*8=16  3*8=24  4*8=32  5*8=40  6*8=48  7*8=56  8*8=64  9*8=72
1*9= 9  2*9=18  3*9=27  4*9=36  5*9=45  6*9=54  7*9=63  8*9=72  9*9=81
```




```python
for i in range(1, 10):
    for j in range(1, 10):
        print(f"{j}*{i}={i*j:-2d}", end="\t")
    print()
```

等腰三角形

- 輸入一個正整數 n，輸出以 n 為高的等腰三角形（or 金字塔）。

```
1   n = eval(input("請輸入一個正整數："))
2
3   for i in range(1, n+1):
4       for j in range(n-i):
5           print(" ", end="")
6       for j in range(2*i-1):
7           print("*", end="")
8       print()
```

```
請輸入一個正整數：7
      *
     ***
    *****
   *******
  *********
 ***********
*************
```

產生排列組合

- 用 1~4 共四個數字組成無重複數字的三位數，共有多少種組合？
  - 可利用巢狀迴圈產生排列組合數目

```
number = range(1, 5)
數目 = 0

for i in number:
    for j in number:
        for k in number:
            if (i != j) and (i != k) and (j != k):
                print(i*100 + j*10 + k)
                數目 += 1

print('排列組合數量=', 數目)
```

```
123
124
132
134
142
143
213
214
231
234
241
243
312
314
321
324
341
342
412
413
421
423
431
432
排列組合數量= 24
```

練習：數字的排列組合

- 假設有 1~5 個數字，欲組成無重複數字的三位數，且滿足底下條件，共有多少種組合？
  - 百位數字 < 十位數字 < 個位數字

本章結束

Py03

函式與陣列(1)

王頌文

主題

- 陣列與串列
  - 一維串列的基礎操作
  - 一維串列的進階操作
  - 串列綜合運算
  - 多維串列的操作
- 函式

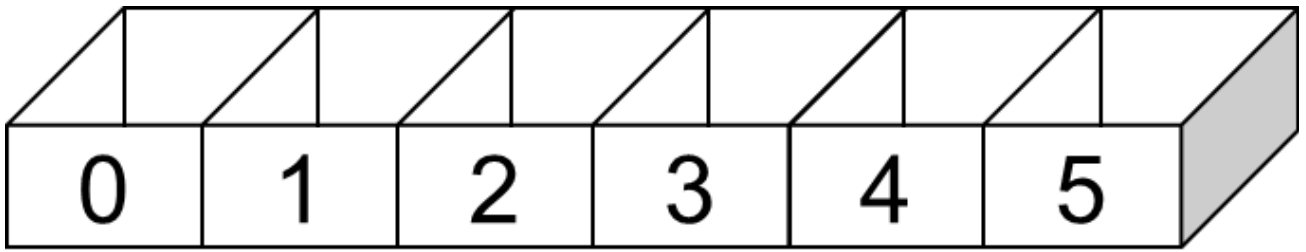
儲存大量資料

- 程式中的資料以變數儲存，如果有大量資料需要儲存，單靠變數來儲存這些資料變得相當繁瑣，使用上也很沒效率。
  - e.g., 儲存「寶可夢」的名字、血量、攻擊力、防禦力等各項資料
    - `pikachu_name = "皮卡丘"`
    - `pikachu_HP = 95`
    - `pikachu_ATK = 90`
    - `pikachu_DEF = 99`

陣列(array)是更有效率的儲存方式

陣列(Array)

- 陣列是一種常見的資料結構(data structure)，它就像是連續格子一樣，每個格子有編號，一般稱為索引值(index)：
  - 用於儲存相同類型的資料
  - 可透過索引值來存放、取用、修改與刪除資料
  - 搭配前面介紹的迴圈，能更有效率地計算或處理大量資料。



不同維度的陣列

$$\vec{a} = (1, 2, 3)$$

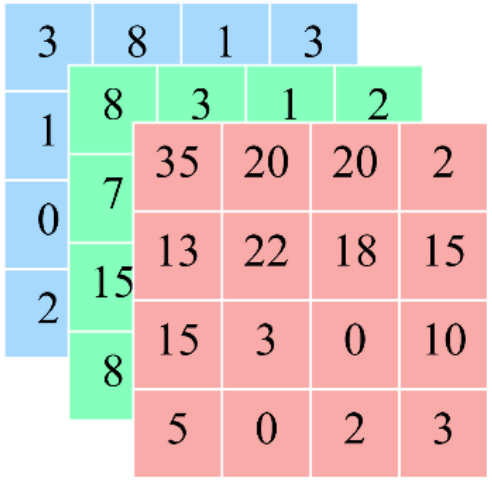
向量（vector）

一維陣列

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

矩陣（matrix）

二維陣列

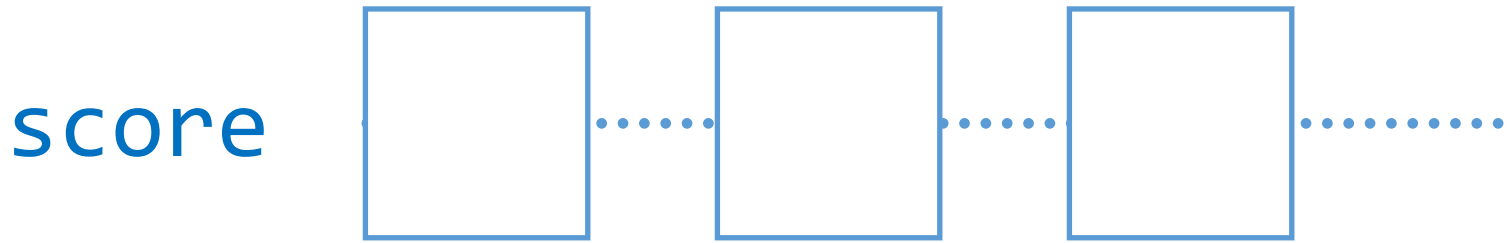


圖像（image）

三維陣列

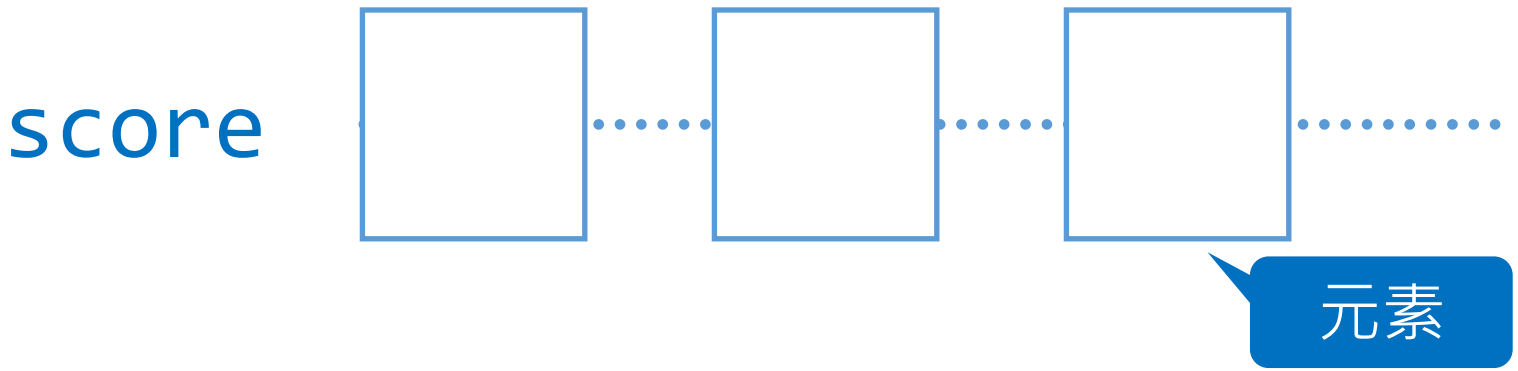
串列(List)

- 事實上，Python 沒有名稱為陣列的結構，而是採用名為串列的結構來實現陣列。
  - 根據翻譯的不同，串列有時也稱為列表或清單。
  - 實際使用時，串列與一般變數類似，都是提供儲存資料的記憶體空間，且都有用來識別的名稱。

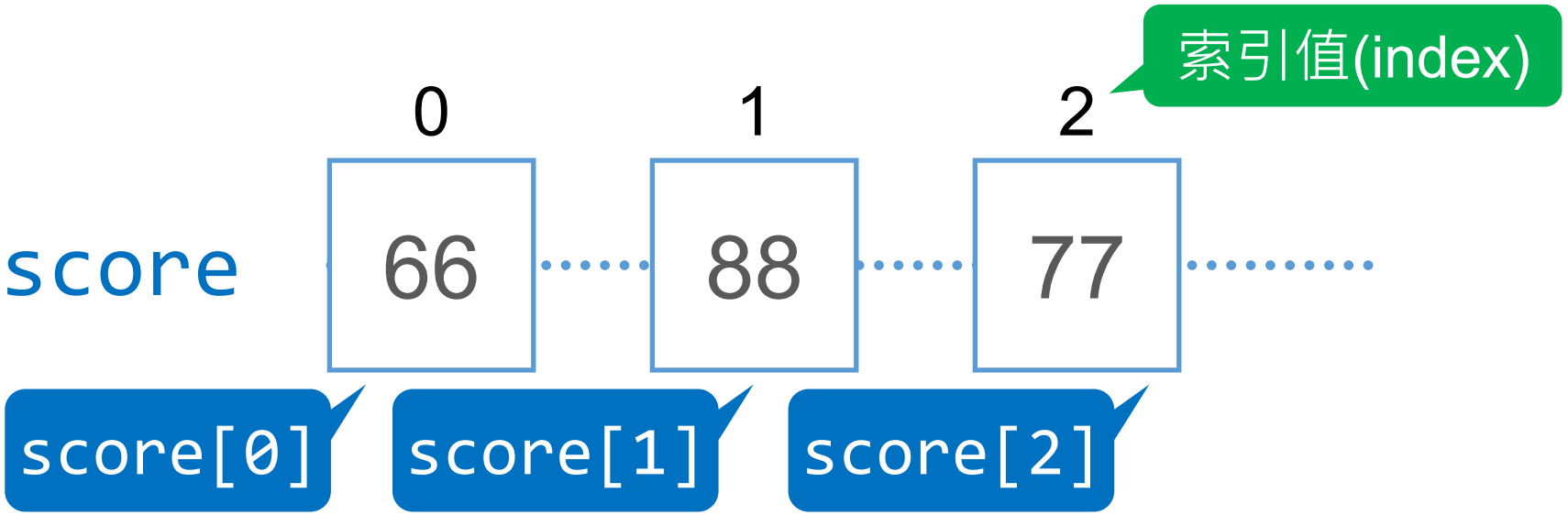


元素(Element)

- 串列內存放的資料統稱為元素或項目(item)，而每個元素相當於一個變數，如此就能輕易地儲存大量不同型別的資料。
  - 這些元素的存放有順序性，可透過索引值存取和改變元素內容。



使用串列

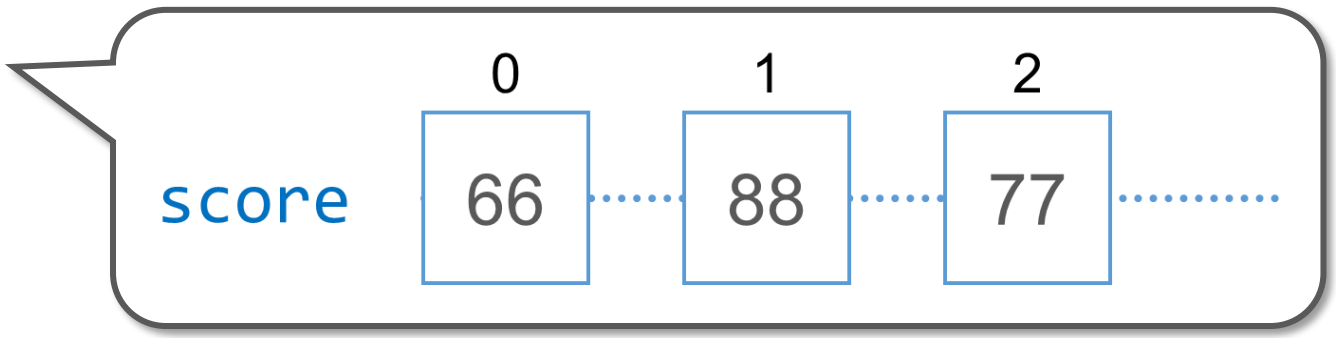


主題

- 陣列與串列
  - 一維串列的基礎操作
  - 一維串列的進階操作
  - 串列綜合運算
  - 多維串列的操作
- 函式

串列的語法

- 串列的使用方式是將元素置於中括號([ ])內，元素間以逗號分隔。語法為：
  - 串列名稱 = [元素1, 元素2, 元素3, …]
  - e.g., score = [66, 88, 77]



範例

- 元素的資料型別可以不同

```
1    lst1 = [66, 88, 77]
2    lst2 = ["皮卡丘", "小火龍", "傑尼龜"]
3    lst3 = [66, "皮卡丘", 88.77, "123", True]
4
5    print(lst3)
```

```
[66, '皮卡丘', 88.77, '123', True]
```

存取串列元素(1/3)

index → 0 1 2 3 4

```
1   lst = ["皮卡丘", "小火龍", "傑尼龜", "妙蛙種子", "請假王"]
2
3   print(lst[1])
4   print(lst[4])
5   print(lst[-2])
6   print(lst[-7])
```

IndexError: list index out of range

存取串列元素(2/3)

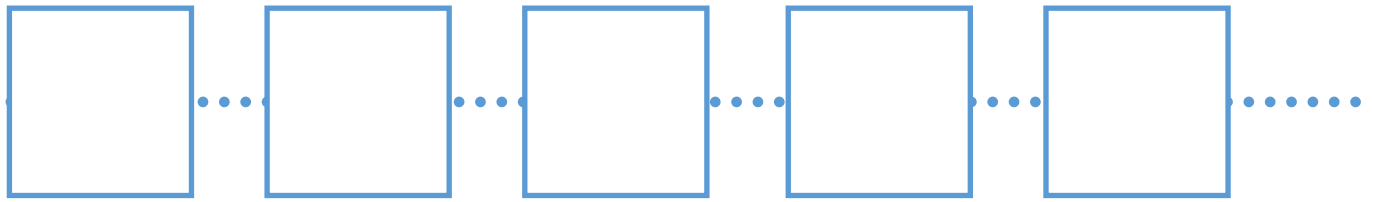
- 存取串列元素**可不用透過索引值**

```
1    lst = [66, "皮卡丘", 8.77, True]
2
3    for i in range(4):
4        print(lst[i], end=' ')
5    print()
6    for x in lst:
7        print(x, end=' ')
```

```
66 皮卡丘 8.77 True
66 皮卡丘 8.77 True
```

存取串列元素(3/3)

- 除了上述兩種方式能存取串列元素外，Python 還提供一個相當簡便的作法，稱為串列分割(slice)，可同時取出數個元素，而這些元素
  - 可以佔據連續位置，
  - 也可以是有一定規律分隔開來，
  - 感覺上就像是將串列分割一段段再取出來。



串列分割的語法

- 串列分割的使用方式與 range() 類似，都由三個參數來區分。
  - 變數 = lst[起始索引值 : 終止索引值 : 間隔值]
    - 起始索引值(start)：欲取出元素片段的起始位置，負值代表倒數的位置。若省略不給，則預設值為 0。
    - 終止索引值(stop)：欲取出元素片段的終止位置，但不包含這個位置，這點與 range() 的用法相同，要特別注意。同樣地，負值代表倒數的位置，而若省略不給，則預設為串列的長度。
    - 間隔值(step)：欲取出元素之間的間隔，若省略不給，則預設為 1，此時為串列的連續片段。

範例(1/2)

- 串列分割的使用方式與 range() 類似，都由三個參數來區分。
  - 變數 = lst[起始索引值 : 終止索引值 : 間隔值]

  - e.g.,
  - lst[m:n] → 從第 m 個元素開始取，取到第 n-1 個元素為止。
  - lst[m:] → 從第 m 個元素開始取到最後
  - lst[:n] → 從開頭取到第 n-1 個元素
  - lst[:] → 全取
  - lst[m:n:t] → 從第 m 個到第 n-1 個元素，每隔 t 個取一次。

範例(2/2)

```python
lst = ["皮卡丘", "小火龍", "傑尼龜", "妙蛙種子", "請假王"]

print(lst[1:4])       # 取出索引值 1~3 的元素
print(lst[-4:-1])     # 取出倒數第4 ~ 倒數第2的元素
print(lst[:4])        # 取出索引值 0~3 的元素
print(lst[1:])        # 取出索引值 1~最後端 的元素
print(lst[:])         # 取出所有元素

print(lst[1:4:2])     # 取出索引值 1~3，且間隔為 2 的元素
```

```
['小火龍', '傑尼龜', '妙蛙種子']
['小火龍', '傑尼龜', '妙蛙種子']
['皮卡丘', '小火龍', '傑尼龜', '妙蛙種子']
['小火龍', '傑尼龜', '妙蛙種子', '請假王']
['皮卡丘', '小火龍', '傑尼龜', '妙蛙種子', '請假王']
['小火龍', '妙蛙種子']
```

想想看

- 如何將串列以反轉的順序輸出？
  - lst=["皮卡丘", "小火龍", "傑尼龜", "妙蛙種子", "請假王"]

lst[::-1]

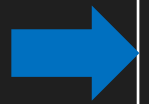
- 如何將使用者輸入的字串以反轉的順序輸出？

```
1    inp = input("請輸入：")
2    print(inp[::-1])
```

```
請輸入：Pokemon
nomekoP
```

簡化

```python
# 民國1年是鼠年，
# 寫程式判斷今年是什麼年(鼠、牛 …)

year = int(input())

if year%12 == 1:
    print("今年是鼠年")
elif year%12 == 2:
    print("今年是牛年")
elif year%12 == 3:
    print("今年是虎年")
# …
# 省略部分程式碼
# …
elif year%12 == 11:
    print("今年是狗年")
```



```python
year = int(input())

生肖 = ["豬","鼠","牛","虎","兔","龍","蛇","馬","羊","猴","雞","狗"]
print("今年是" + 生肖[year%12] + "年")
```

輸出串列內容(1/4)

- 想輸出底下串列的內容，該怎麼做？
  - `print(lst)`

```
['皮卡丘', '小火龍', '傑尼龜', '妙蛙種子', '請假王']
```

```
1   lst = ["皮卡丘", "小火龍", "傑尼龜", "妙蛙種子", "請假王"]
```

輸出串列內容(2/4)

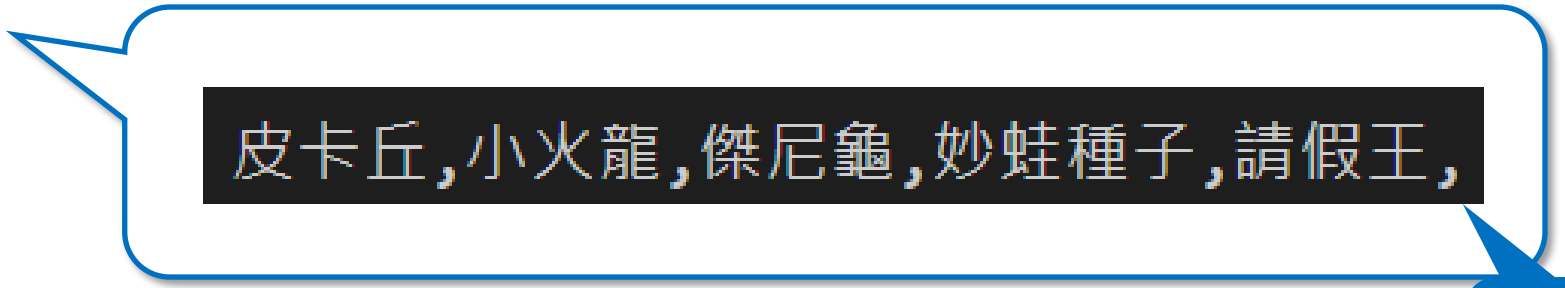
- 想輸出底下串列的內容，該怎麼做？
  - `print(lst[0], end=',')`
  - `print(lst[1], end=',')`
  - `print(lst[2], end=',')`
  - …

皮卡丘,小火龍,傑尼龜,妙蛙種子,請假王

```
1   lst = ["皮卡丘", "小火龍", "傑尼龜", "妙蛙種子", "請假王"]
```

輸出串列內容(3/4)

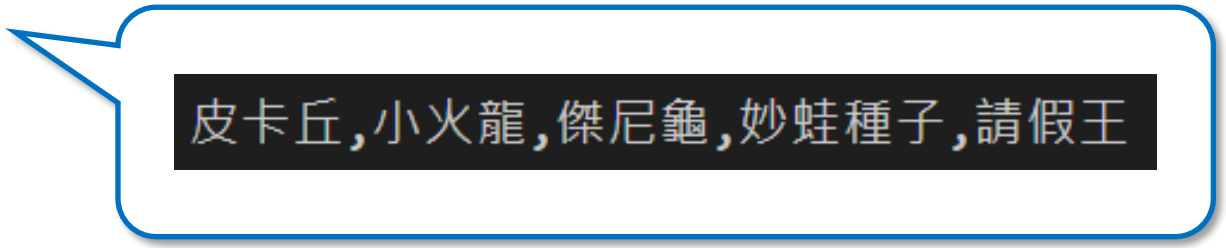
- 想輸出底下串列的內容，該怎麼做？
  - `for x in lst:`
  - `    print(x, end=',')`




```
1   lst = ["皮卡丘", "小火龍", "傑尼龜", "妙蛙種子", "請假王"]
```

輸出串列內容(4/4)

- 想輸出底下串列的內容，該怎麼做？
  - `for i in range(4):`
  - `    print(lst[i], end=',’)`
  - `print(lst[i+1])`




```
1   lst = ["皮卡丘", "小火龍", "傑尼龜", "妙蛙種子", "請假王"]
```

加入元素到串列(1/2)

- 我們知道如何存取串列元素，也知道在一開始使用串列時可以一次性指派大量元素，但如何在程式執行過程中把元素逐一放到串列內呢？

使用 append()

```
1   lst = [66, "皮卡丘", 8.77, True]
2
3   for i in range(4):
4       print(lst[i], end=' ')
5   print()
6   for x in lst:
7       print(x, end=' ')
```

加入元素到串列(2/2)

```
1    lst = []
2
3    for x in range(1, 7, 2):
4        lst.append(x)
5
6    print(lst)
```

```
[1, 3, 5]
```

猜猜看 append() 如何把資料放到串列內？

姑且將 append() 當成是串列的特有方法，但與內建函式不同，不能單獨使用。

串列的宣告(declaration)

- 串列在使用時要指派儲存的資料
  - e.g., `lst = [66, 88, 77]`

- 否則就得先宣告，後續才能直接使用，否則會發生「NameError」。
  - `lst = []`
  - `lst = list()`

內建函式，可單獨使用。

```
1    lst = []
2
3    for x in range(1, 7, 2):
4        lst.append(x)
5
6    print(lst)
```

+ 與 *

- 之前看過的算術運算子 + 與 *，也可用在串列。

```
1    lst1 = [66, 88]
2    lst2 = ["皮卡丘", "小火龍"]
3
4    lst = lst1 + lst2
5    print(lst)
6    print(lst1*3)
```

```
[66, 88, '皮卡丘', '小火龍']
[66, 88, 66, 88, 66, 88]
```

猜猜看它們的用途

將算術運算子運用到串列上的做法，是 OOP 三大特性之一的多型 (polymorphism)，其概念白話一點說就是「一樣的事情，有不同做法」。

成員運算子

- is / not in 可檢查某元素是否在串列中

```
1    lst = ["皮卡丘", "小火龍", "傑尼龜", "妙蛙種子"]
2
3    print("皮卡丘" in lst)
4    if "卡比獸" not in lst:
5        print("尚未收服卡比獸！")
```

```
True
尚未收服卡比獸！
```

練習：計算成績

- 請設計一個輸入成績的程式，要求如下：
  - 輸入：成績或「-1」
  - 輸出：
    - 「-1」表示成績輸入結束
    - 結束後顯示 (1).所有成績 (2).總成績 (3).平均成績

主題

- 陣列與串列
  - 一維串列的基礎操作
  - 一維串列的進階操作
  - 串列綜合運算
  - 多維串列的操作
- 函式

串列進階操作(1/3)

- `lst = [66, 88, 77]`

方法	意義	範例	變數 n 的值
len(lst)	取得串列元素數目（即串列長度）	n = len(lst)	3
min(lst)	取得串列元素最小值	n = min(lst)	66
max(lst)	取得串列元素最大值	n = max(lst)	88
sum(lst)	計算串列元素總和	n = sum(lst)	231
sorted(lst)	由小到大排序串列元素	n = sorted(lst)	[66, 77, 88]

內建函式

範例

```python
lst = [66, 88, 77, 22]

for i in range(len(lst)):
    print(lst[i], end=' ')

print(f"\n串列最大值: {max(lst)}")
print(f"串列排序結果: {sorted(lst)}")
```

```
66 88 77 22
串列最大值: 88
串列排序結果: [22, 66, 77, 88]
```

串列進階操作(2/3)

- `lst = [66, 88, 77]`

這些方法類似前面提到的 append()，要接在一個串列的後面，完成後會回傳處理結果。

	方法	意義	範例	變數 n 的值
★	index(x)	元素 x 第一次出現的索引值	n = lst.index(77)	2
★	count(x)	元素 x 的出現次數	n = lst.count(77)	1
	pop()	取出串列尾端的元素 並從串列中移除	n = lst.pop()	77
	copy()	複製串列	n = lst.copy()	[66, 88, 77]

index() 只取出指定元素第一次出現的索引值，當串列裡沒有指定元素時，會引發「ValueError」的錯誤，所以經常與前面提過的 in 運算子搭配使用。

範例

```python
lst = [66, 88, 77, 22, 88, 77]

if 77 in lst:
    print(lst.index(22))

print(lst.count(88))

print("--- pop() ---")
print(lst.pop())
print(lst)
print(lst.pop(1))
print(lst)
```

```
3
2
--- pop() ---
77
[66, 88, 77, 22, 88]
88
[66, 77, 22, 88]
```

複製串列？

內建函式 id() 是取得物件的識別碼，這是一個整數，代表該物件在記憶體中的位置。

```
1    lst = [66, 88, 77, 22, 88, 77]
2    print(id(lst))
3
4    new = lst
5    print(id(new))
6    lst[3:] = []
7    print(f"原串列: {lst}")
8    print(f"複製的串列: {new}")
```

```
2161937751040
2161937751040
原串列: [66, 88, 77]
複製的串列: [66, 88, 77]
```

複製串列？

```python
lst = [66, 88, 77, 22, 88, 77]
print(id(lst))

lst_copy = lst.copy()
print(id(lst_copy))

lst[3:] = []
print(f"原串列: {lst}")
print(f"複製的串列: {lst_copy}")
```

```
1773114716160
1773164503488
原串列: [66, 88, 77]
複製的串列: [66, 88, 77, 22, 88, 77]
```

「==」 vs 「is」

```
1    a, b = 1234, 1234
2    print(a == b)
3    print(a is b)
4
5    lst1 = [66, 88, 77]
6    lst2 = lst1
7    lst_new = [66, 88, 77]
8    print(lst1 is lst2)
9    print(lst1 is lst_new)
10   print(lst1 == lst_new)
```

```
True
True
True
False
True
```

串列進階操作(3/3)

- `lst = [66, 88, 77]`

方法	意義	範例	lst 的值
remove()	移除第 1 次出現的元素 x	lst.remove(88)	[66, 77]
clear()	移除串列所有元素	lst.clear()	[]
reverse()	反轉串列元素的順序	lst.reverse()	[77, 88, 66]
★ sort()	由小到大排序串列元素	lst.sort()	[66, 77, 88]
☆ append(x)	新增元素 x 到串列最後端	lst.append(1)	[66, 88, 77, 1]
insert(n, x)	在位置 n 插入元素 x	lst.insert(1, 1)	[66, 1, 88, 77]
entend(L)	將串列 L 內的元素逐一新增到原串列最後端	lst.extend([1, 2])	[66, 88, 77, 1, 2]

範例

```python
lst = [66, 88, 77, 22, 88, 77]

lst.remove(77)
print(lst)

lst.reverse()       # 作用等同於 lst = lst[::-1]
print(lst)

# 預設由小排到大，加入參數 reverse=True 可降序排序
lst.sort(reverse=True)
print(lst)

lst.clear()         # 作用等同於 lst = []
print(lst)
```

```
[66, 88, 22, 88, 77]
[77, 88, 22, 88, 66]
[88, 88, 77, 66, 22]
[]
```

增加串列元素

- `lst = [66, 88, 77]`

方法	意義	範例	lst 的值
remove()	移除第 1 次出現的元素 x	lst.remove(88)	[66, 77]
clear()	移除串列所有元素	lst.clear()	[]
reverse()	反轉串列元素的順序	lst.reverse()	[77, 88, 66]
sort()	由小到大排序串列元素	lst.sort()	[66, 77, 88]
append(x)	新增元素 x 到串列最後端	lst.append(1)	[66, 88, 77, 1]
insert(n, x)	在位置 n 插入元素 x	lst.insert(1, 1)	[66, 1, 88, 77]
entend(L)	將串列 L 內的元素逐一新增到原串列最後端	lst.extend([1, 2])	[66, 88, 77, 1, 2]

append() vs insert()

- insert() 方法是將元素加在串列的指定位置
  - 位置透過索引值來指定
  - 若索引值超過串列範圍，會直接將元素新增在串列尾端。

- append() 方法是將元素加在串列最後面

append() vs extend()

- 這兩個方法都是將資料加在串列尾端，不同處在於
  - append()可以新增的元素，也能新增串列。
    - 如果是串列，會將整個串列當成一個元素加入。
  - extend()只能新增如串列、range() 般的可迭代物件。
    - 將串列中的元素做為個別元素逐一加入串列

範例

```python
lst = [66, 88]
lst.append(77)          # 結果等同於 lst += [77]
print(lst)

lst.insert(1, 11)
print(lst)

lst = [66, 88]
X = ["皮卡丘", "小火龍"]
lst.append(X)           # 結果等同於 lst += [X]
print(lst)

lst = [66, 88]
lst.extend(X)           # 結果等同於 lst += X
print(lst)
```

```
[66, 88, 77]
[66, 11, 88, 77]
[66, 88, ['皮卡丘', '小火龍']]
[66, 88, '皮卡丘', '小火龍']
```

整理

- 底下以功能為主，整理前面提到的各個串列操作方法：
  - 宣告串列：中括號([])、list()
  - 存取元素：中括號、分割運算子([:])
  - 基本操作：in/not in運算子、len()、min()、max()、sum()
  - 增加元素：+/*運算子、append()、insert()、extend()
  - 刪除元素：del敘述、pop()、remove()、clear()
  - 改變順序：sorted()、sort()、reverse()
  - 複製元素：分割運算子、copy()

練習：尋找完全數

- 一個大於 1 的數如果恰好等於它的因數（自己除外）總和，這個數就稱為完全數(perfect number)，例如：
  - 6 = 1 + 2 + 3 → 6 是完全數
  - 8 != 1 + 2 + 4 → 8 不是完全數

- 請撰寫一程式，讓使用者輸入一個正整數 num，計算並輸出所有小於 num 的完全數。

主題

- 陣列與串列
  - 一維串列的基礎操作
  - 一維串列的進階操作
  - 串列綜合運算
  - 多維串列的操作
- 函式

串列綜合運算(List Comprehension)

- 串列綜合運算提供一個簡潔方法建立串列
  - 由中括號裡面包含運算式、for 與零個或多個 if 敘述所組成
  - 串列綜合運算後將產生一個新串列

```
squares = []
for x in range(10):
    squares.append(x**2)
print(squares)

print("-- 串列綜合運算產生 --")
lst = [x**2 for x in range(10)]
print(lst)
```

```
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

範例

- 串列綜合運算的運算式也能使用內建函式：

```
 3    nums = list(range(0, 100, 10))
 4    lst = []
 5    for x in nums:
 6        lst.append(max(nums)-x)
 7    print(lst)
 8
 9    print("-- 串列綜合運算產生 --")
10    lst = [max(nums)-x for x in nums]
11    print(lst)
```

```
[90, 80, 70, 60, 50, 40, 30, 20, 10, 0]
-- 串列綜合運算產生 --
[90, 80, 70, 60, 50, 40, 30, 20, 10, 0]
```

範例

- 透過巢狀迴圈產生的串列，同樣也能用串列綜合運算輕鬆產生。

```
['a1', 'a2', 'a3', 'b1', 'b2', 'b3', 'c1', 'c2', 'c3']
-- 串列綜合運算產生 --
['a1', 'a2', 'a3', 'b1', 'b2', 'b3', 'c1', 'c2', 'c3']
```

```
3   lst = []
4   for i in "abc":
5       for j in range(1, 4):
6           lst.append(i + str(j))
7   print(lst)
8
9   print("-- 串列綜合運算產生 --")
10  lst = [i + str(j) for i in "abc" for j in range(1, 4)]
11  print(lst)
```

範例

- 串列綜合運算不僅能用 `for` 迴圈，也能搭配 `if` 條件式篩選串列內容。

```
lst = [x for x in range(5)]
```

```
new = [a for a in lst if a%2]
```

範例

- 以前學過的三元運算子，也可以與串列綜合運算結合。

```
3   lst = []
4   for i in range(1, 10):
5       if i%2:
6           lst.append(i)
7       else:
8           lst.append(100)
9   print(lst)
10
11  print("-- 串列綜合運算產生 --")
12  lst = [x if x%2 else 100 for x in range(1, 10)]
13  print(lst)
```

```
[1, 100, 3, 100, 5, 100, 7, 100, 9]
-- 串列綜合運算產生 --
[1, 100, 3, 100, 5, 100, 7, 100, 9]
```

練習：計算出現次數

- 找出文句內出現次數最多的中文字，並輸出該字與其出現次數
  - 輸入：底下的中文字句
  - 輸出：出現次數最多的中文字、出現次數

```
text = \
"昔汝雖拒吾於千里之外，傷吾心，然吾怪之罪何有於汝乎？\
反，吾思與日俱增，吾情與時俱進。於汝，情痴一片，思濃于海，\
山不可動，海不可淹，火不可滅。愛慕之情，時時加之，\
豈有他人可吾及乎？"
```

直接複製這個字串到程式碼裡

HINT: 利用 `list(字串變數)` 可分離出每個中文字，並回傳一個字元的串列。

主題

- 陣列與串列
  - 一維串列的基礎操作
  - 一維串列的進階操作
  - 串列綜合運算
  - 多維串列的操作
- 函式

多維串列(multi-dimensional list)

- 前一節在介紹新增元素到串列的用法時，若藉由 append() 新增一個串列 L 到另一個串列內，則 L 會被視為一個元素添加到串列的尾端。
  - 此時，因為串列裡還有串列，所以也可稱為巢狀串列(nested list)，不過一個更常見的名稱是多維串列(multi-dimensional list)。

```
1    lst = [2, 3]
2    lst.append(['皮卡丘', '小火龍'])
3    print(lst)
```

```
[2, 3, ['皮卡丘', '小火龍']]
```

二維串列(two-dimensional list)

- 二維串列經常用來儲存矩陣或表格
  - e.g., 儲存寶可夢四項數值的表格

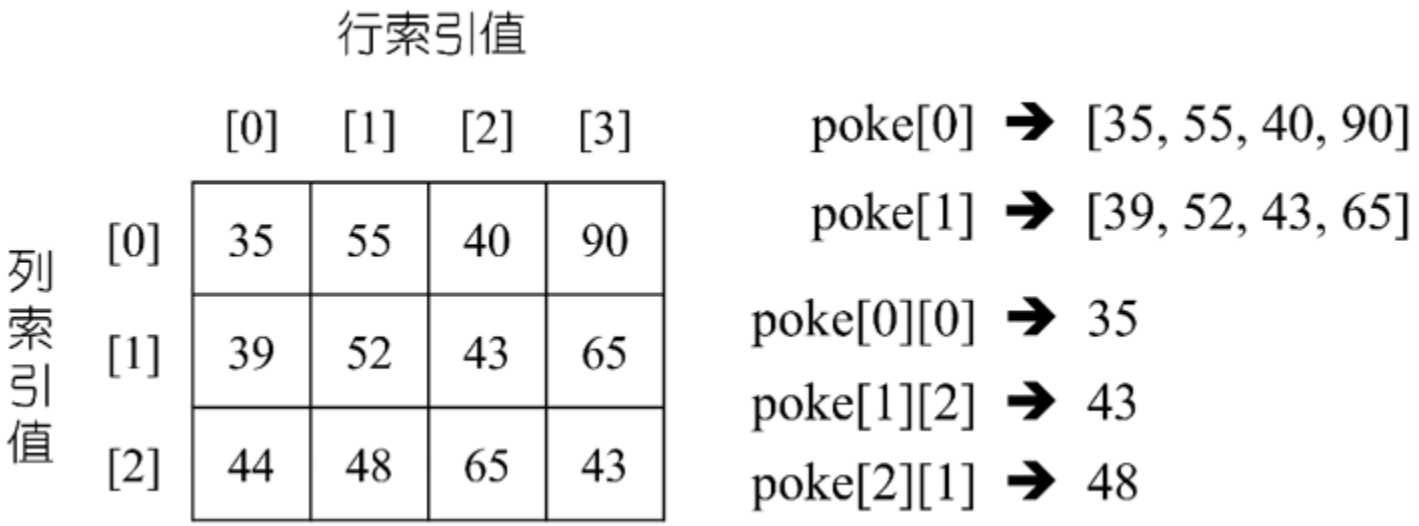
	血量	攻擊力	防禦力	速度
皮卡丘	35	55	40	90
小火龍	39	52	43	65
傑尼龜	44	48	65	43

```
poke = [ [35, 55, 40, 90],
         [39, 52, 43, 65],
         [44, 48, 65, 43] ]
```

存取二維串列

```
poke = [ [35, 55, 40, 90],
         [39, 52, 43, 65],
         [44, 48, 65, 43] ]
```

- 存取二維串列的元素仍然要依靠索引值，只是因為有兩個維度，所以又分為列索引值(raw index)與行索引值(column index)。



宣告二維串列

- 宣告時可以沒有儲存值，也可以先存放空字串或是 None。

```python
lst = []
for i in range(2):      # 宣告一個二維串列
    lst.append([])
print(lst)

lst = []
for i in range(2):      # 宣告一個2*3的二維串列
    lst.append([])
    for j in range(3):
        lst[i].append("")
print(lst)

# 利用串列綜合運算宣告一個2*3的二維串列
lst = [["" for j in range(3)] for i in range(2)]
print(lst)

lst = [[""]*3 for i in range(2)]
print(lst)
```

```
[[], []]
[['', '', ''], ['', '', '']]
[['', '', ''], ['', '', '']]
[['', '', ''], ['', '', '']]
```

加總每一列

- 要輸出二維串列的內容一般會透過兩層巢狀迴圈來達成，而要計算一些基本性質也是類似的作法，但要注意迴圈針對的操作對象。
  - e.g., 加總二維串列每一列以及所有元素的總和

```python
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
total = 0

for i in range(len(matrix)):
    sum_ = sum(matrix[i])
    total += sum_
    print(f"第{i}列元素總和為 {sum_}")

print(f"所有元素的總和為 {total}")
```

1	2	3
4	5	6
7	8	9

```
第0列元素總和為 6
第1列元素總和為 15
第2列元素總和為 24
所有元素的總和為 45
```

加總每一行

- 若是要計算二維串列每一行的元素加總，得注意迴圈變數要針對的對象。

```python
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]

for col_i in range(len(matrix[0])):
    total = 0
    for row_i in range(len(matrix)):
        total += matrix[row_i][col_i]

    print(f"第{col_i}行元素總和為 {total}")
```

1	2	3
4	5	6
7	8	9

```
第0行元素總和為 12
第1行元素總和為 15
第2行元素總和為 18
```

串列比大小

- 兩個串列比大小的方式與數學上的向量比大小相仿，都是先比第一個元素，若相同再比第二個元素，依此類推。

```
1   lst = [[3, 2], [3, 5], [1, 8], [2, 4], [1, 2], [2, 9]]
2   lst.sort()
3   print(lst)
```

```
[[1, 2], [1, 8], [2, 4], [2, 9], [3, 2], [3, 5]]
```

練習：尋找列的最大總和

- 請撰寫程式讓使用者輸入三列資料，每列由兩個數值且數值間以逗點隔開，計算並輸出列總和最大值及其列索引值。
  - 輸入

```
請輸入兩個數值，並以逗點隔開： 35,55
請輸入兩個數值，並以逗點隔開： 39,52
請輸入兩個數值，並以逗點隔開： 44,48
```

  - 輸出

```
第 2 列有列總和最大值為 92
```

練習：找出最近的兩點

- 撰寫程式讓使用者輸入二維平面的點座標
  - 每個點座標輸入時有兩個數值，並以逗點隔開
  - 輸入0,0則結束
- 計算這些輸入的點集合中，彼此歐式距離最接近的兩點，並輸出點座標與距離值。

```
每個點座標有兩個數值，輸入時以逗點隔開(例如 2,3)
第 1 個點座標(輸入0,0則結束): -3,-2
第 2 個點座標(輸入0,0則結束): 2,3
第 3 個點座標(輸入0,0則結束): 1,1
第 4 個點座標(輸入0,0則結束): 0,0
```

```
座標點個數 = 3
最近兩點為： [2, 3] [1, 1]
距離 = 2.23606797749979
```

```
每個點座標有兩個數值，輸入時以逗點隔開(例如 2,3)
第 1 個點座標(輸入0,0則結束): -1,3
第 2 個點座標(輸入0,0則結束): -1,-1
第 3 個點座標(輸入0,0則結束): 1,1
第 4 個點座標(輸入0,0則結束): 2,0.5
第 5 個點座標(輸入0,0則結束): 2,-1
第 6 個點座標(輸入0,0則結束): 3,3
第 7 個點座標(輸入0,0則結束): 4,2
第 8 個點座標(輸入0,0則結束): 4,-0.5
第 9 個點座標(輸入0,0則結束): 0,0
```

```
座標點個數 = 8
最近兩點為： [1, 1] [2, 0.5]
距離 = 1.118033988749895
```

本章結束

Py03

函式與陣列(2)

王頌文

主題

- 陣列與串列
- 函式
  - 自訂函式
  - 傳遞參數與回傳值
  - 變數的有效範圍
  - 常見的內建函式

函式(Function)

- 經過前面的介紹與練習，我們已經能夠寫出架構完整的 Python 程式，而隨著程式越寫越長，也越趨複雜，這時我們會發現部分程式碼經常被重複撰寫。

- 將這些重複或有特別定義的程式，拆分成容易管理的小程式，即為函式。
  - 換言之，函式是一種有名稱且獨立的程式片段，可以接收參數，且執行完成後也可以回傳計算結果。

模組化(Modular)

- 模組化開發的精神，就是把大而冗長的程式碼拆解成不同區塊的小程式來個別運作，讓大功能變成個別的小模組，並確保拆解完的模組可以各自獨立運作。
  - 所以模組化的特性就是「獨立性」與「重複使用性」。

- 同時，這個概念也是由上而下(top-down)的方式來拆解問題，將大問題逐步分解為能處理的小問題。
  - 函式將程式碼組織為一個較小且獨立的運作單元，並可在程式內重複多次執行，所以函式是實現模組化開發的方式之一。

使用函式的優點

- 一般來說，使用函式的程式設計方式有以下優點：
  - 程式可讀性高，容易除錯及維護。
  - 可縮短程式長度，程式碼也可重複使用。
  - 將程式分割後由多人撰寫，有利於團隊分工及縮短開發時間。

使用函式

- 對程式設計師來說，函式是由程式片段組成，一旦明確定義後即可隨時呼叫來執行。

- 如果把函式比喻為動畫中的召喚獸，整個函式的運作過程就像是法師的召喚程序。
  - 首先，法師要先與召喚獸訂定契約（定義函式），
  - 接著視當下條件及需求繪製法陣或結手印進行召喚（呼叫函式），
  - 召喚獸出現後會執行既定任務（執行函式），
  - 而任務結束後也可能會回傳執行結果（函式回傳）。

函式如何執行任務？

- 至於函式如何執行任務，可視為如黑盒子般隱藏起來。
  - 試想我們之前使用內建函式（如 `print()`、`eval()` 等）時，也不清楚函式內的程式碼為何。
  - 只要熟悉其語法與一些注意事項就能運用自如，讓我們的程式能具備更多功能。

- `print(項目1 [, 項目2, …, sep=分隔字元, end=結束字元])`
  - 項目1, 項目2, …：可以一次輸出多個項目，以逗號(,)分隔。
  - sep：分隔項目的字元，預設值為一個空白。
  - end：輸出完後自動加入的字元，預設值為換行字元(\n)。

Python 的函式類型大致有三種(1/2)

① 內建函式(built-in function)：Python 本身已內建許多函式，如之前看過的 `len()`、`range()`、`type()` 等，皆可直接呼叫使用。

② 標準函式庫(standard library)：亦可稱為第三方套件(third-party packages)，所謂套件可簡單想成是特定功能函式的組合，豐富的第三方套件是 Python 為人津津樂道的特色之一。

- 在 PyPI 網站(https://pypi.org/)上有許多高手上傳的第三方套件，供人免費下載使用，已累積有近五十萬個專案。
- 使用這類函式，必須先匯入該函式所屬套件。

Python 的函式類型大致有三種(2/2)

③ 自訂函式(user-defined function)：要先具體定義好函式，然後才能呼叫使用。

- 這種函式是依自己需求量身訂製，除了自己使用外，當然也能給他人使用，甚至是晉升成第三方套件共享給全球開發人員。

- 我們從這個開始介紹

數學上的函數(Function)

- 要注意的是，數學上的函數用來描述兩個集合的對應關係，通常是將定義域(domain)裡的每個元素對應到值域(codomain)裡的一個元素。
  - e.g., 函數 $y = f(x) = \sin(x) + x^2$，其定義域與值域皆為實數。
  - 雖然英文同樣都是 function，但數學的函數與程式語言的函式是完全不同的東西。

主題

- 陣列與串列
- 函式
  - 自訂函式
  - 傳遞參數與回傳值
  - 變數的有效範圍
  - 常見的內建函式

自訂函式(1/2)

```
2 def getName(n):
3     poke = ["皮卡丘", "小火龍", "傑尼龜",
4             "妙蛙種子", "請假王"]
5     return poke[n]
6
7 print(getName(3))
```

- 建立函式的語法為：
  - def 函式名稱([參數序列]):
  - 程式區塊
  - [return 回傳值序列]
    - 關鍵字 def 用來定義函式，其後空一格接自行命名的函式名稱，此命名仍需遵守 Python 的規範。
    - 參數(parameter)序列：可省略也可包含多個參數，且參數間以逗點(,)隔開，也稱為形式參數(formal parameter)。

當函式被呼叫時，透過參數傳遞值給函式。

自訂函式(2/2)

```
2 def getName(n):
3     poke = ["皮卡丘", "小火龍", "傑尼龜",
4             "妙蛙種子", "請假王"]
5     return poke[n]
6
7 print(getName(3))
```

- 建立函式的語法為：
  - def 函式名稱([參數序列]):
  - 程式區塊
  - [return 回傳值序列]
    - 回傳值(return value)序列：一樣可省略也可有多個回傳值，且回傳值間也用逗點(,)隔開。

當函式執行完畢後，可透過回傳值反饋執行結果。

呼叫函式

- 函式建立後並不會執行，必須在主程式中呼叫函式(call / invoke)，才會執行函式，呼叫函式的語法為：
  - [變數=] 函式名稱([參數串列])

```
2 def getName(n):
3     poke = ["皮卡丘", "小火龍", "傑尼龜",
4             "妙蛙種子", "請假王"]
5     return poke[n]
6
7 print(getName(3))
```

注意

- 先定義函式後才能呼叫該函式來執行，若在定義函式之前就進行呼叫，執行時會引發「not defined」的錯誤。

```
1   def hello():
2       print("人生苦短, 我用Python")
3
4   hello()
```

```
人生苦短, 我用Python
```

```
1   hello()
2
3   def hello():
4       print("人生苦短, 我用Python")
```

```
NameError: name 'hello' is not defined.
```

主題

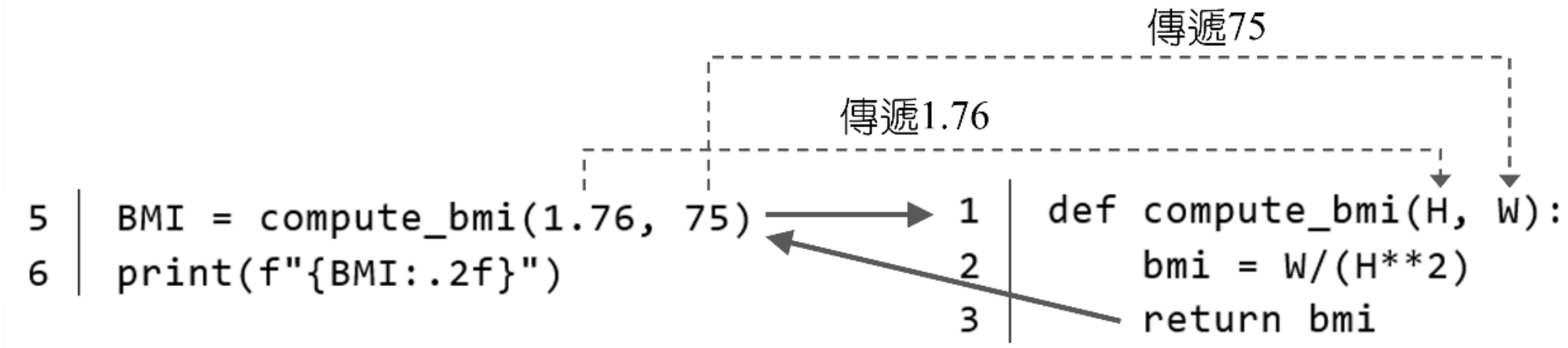
- 陣列與串列
- 函式
  - 自訂函式
  - 傳遞參數與回傳值
  - 變數的有效範圍
  - 常見的內建函式

傳遞參數

- 如果函式所執行的工作需要特定資料才能運作，可以在呼叫時將資料傳遞到函式的參數，而這些資料又被稱為引數(argument)或實際參數(actual parameter)。
  - 與函式的形式參數相對應

- 同時，若函式執行完畢後要反饋訊息，可以用關鍵字 `return` 回傳給呼叫它的地方。

範例




```python
def compute_bmi(H, W):
    bmi = W/(H**2)
    return bmi

BMI = compute_bmi(1.76, 75)
print(f"{BMI:.2f}")
```

```
24.21
```

參數的數量太多

- 雖然我們能傳遞參數給函式，可是如果參數的數量太多，不僅在設定上有些繁瑣，也容易因為弄錯參數順序而出包。
  - 因此，在呼叫參數時可以直接指定參數名稱，這個作法不用考慮參數的順序，可減少犯錯的機會。
  - 例如底下三種呼叫函式方式會有相同的結果：
    - `compute_bmi(1.76, 75)`
    - `compute_bmi(H=1.76, W=75)`
    - `compute_bmi(W=75, H=1.76)`

```
1    def compute_bmi(H, W):
2        bmi = W/(H**2)
3        return bmi
4
5    BMI = compute_bmi(1.76, 75)
6    print(f"{BMI:.2f}")
```

參數預設值

- 建立函式時可以為參數設定預設值，呼叫函式時，如果沒有傳入該參數時，就會使用預設值。

```python
def compute_bmi(H, W, show=True):
    bmi = W/(H**2)
    if show:
        if 18.5 <= bmi <24:
            print("健康體位")
        elif 24 <= bmi:
            print("體位異常")

    return bmi

BMI = compute_bmi(1.76, 75)
print(f"{BMI:.2f}")

BMI = compute_bmi(1.68, 50, show=False)
print(f"{BMI:.2f}")
```

```
體位異常
24.21
17.72
```

關鍵字 return

- return 可讓函式回傳程式運算後的結果
  - 反過來說，函式一遇到 return 敘述就會中止執行，並依設定回傳，所以也常用來控制函式的執行流程。

```
1    def compute_bmi(H, W):
2        bmi = W/(H**2)
3        if bmi < 18.5:
4            return "體重過輕"
5        elif 18.5 <= bmi <24:
6            return "健康體位"
7        else:
8            return "體位異常"
9
10   print(compute_bmi(1.70, 50))
```

```
體重過輕
```

回傳多個值

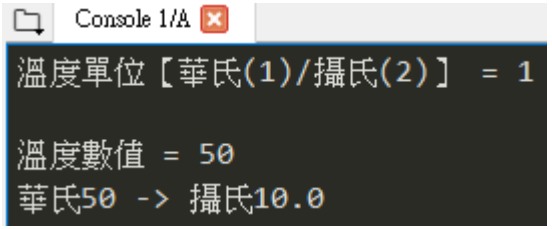
```python
def compute_bmi(H, W):
    bmi = W/(H**2)

    if bmi < 18.5:
        return [bmi, "體重過輕"]
    elif 18.5 <= bmi <24:
        return [bmi, "健康體位"]
    else:
        return [bmi, "體位異常"]

lst = compute_bmi(1.76, 75)
print(lst)

BMI, 判定 = compute_bmi(1.68, 50)
print(BMI, 判定)
```

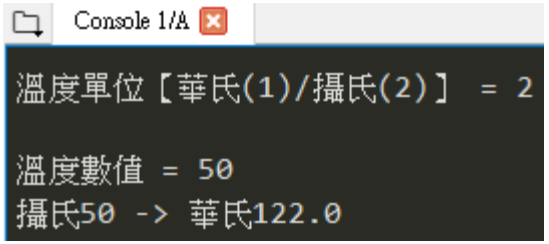
```
[24.212293388429753, '體位異常']
17.71541950113379 體重過輕
```

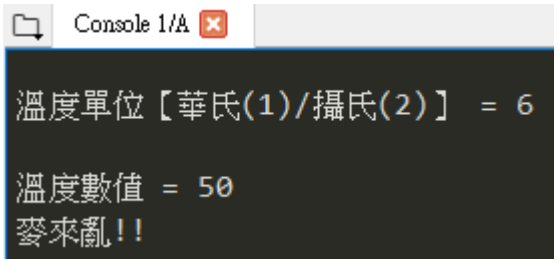
練習：華氏/攝氏溫度轉換

- 輸入華氏/攝氏溫度 temp，利用函式呼叫傳遞兩個參數並進行轉換，最後輸出溫度轉換結果。

- 撰寫底下函式：
  - 溫度轉換(unit, temp)
    - unit：1→華氏；2→攝氏
    - temp：溫度數值







主題

- 陣列與串列
- 函式
  - 自訂函式
  - 傳遞參數與回傳值
  - 變數的有效範圍
  - 常見的內建函式

變數的有效範圍(Scope)

- 變數的有效範圍指的是變數在程式中可被參考的範圍，白話一點講就是在程式中哪些地方能合法地存取這個變數。

- Python 的變數有效範圍分為兩種：
  - 全域變數(global variable)
  - 區域變數(local variable)

變數

全域

定義在函式外的變數，其有效範圍是整個 Python 檔案

區域

定義在一個函式中的變數，有效範圍是在該函式內

全域 vs 區域變數

- 若把函式比喻為寶可夢訓練家，野外和許多訓練家身邊可能都有皮卡丘：
  - 訓練家小智只能驅使自己的皮卡丘，相當於函式只能存取自己的區域變數。
  - 所有訓練家皆能捕捉野外的皮卡丘，相當於存取全域變數。

- 若全域與區域變數撞名，則在函式內以區域變數優先，可是在函式外看不到區域變數，因此只能使用全域變數。

範例

```
1    def scope():
2        var1 = 1
3        var3 = var2 + 50
4        return var1, var3
5
6    var1, var2 = 10, 50
7    x, y = scope()
8
9    print(x, y)
10   print(var1, var2)
```

```
1 100
10 50
```

注意(1/2)

- 在函式內可存取全域變數，但不能變更其值。
  - 比方說在 scope() 內可以存取全域變數 var2 的值，但如果想要直接變更其值（例如直接寫「var2 += 50」）會引發「UnboundLocalError」的錯誤訊息。

```
1   def scope():
2       var1 = 1
3       var3 = var2 + 50
4       return var1, var3
5
6   var1, var2 = 10, 50
7   x, y = scope()
8
9   print(x, y)
10  print(var1, var2)
```

注意(2/2)

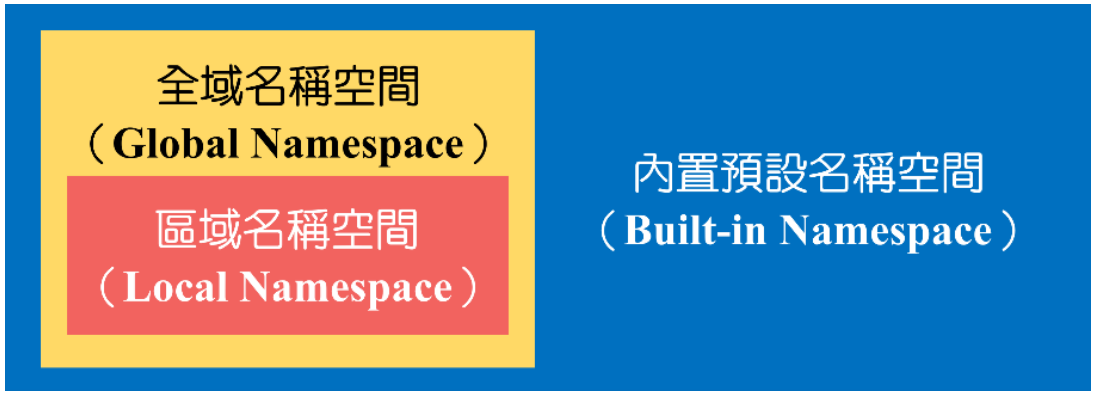
- 在函式外看不到區域變數
  - 譬如在 scope() 之外（即第6行之後）存取不到區域變數 var3，
  - 若直接存取會產生錯誤訊息「NameError」

- 函式名稱也屬於變數名稱，若與其它變數重名，可能會覆蓋變數內容，產生意料之外的錯誤。

```
1   def scope():
2       var1 = 1
3       var3 = var2 + 50
4       return var1, var3
5
6   var1, var2 = 10, 50
7   x, y = scope()
8
9   print(x, y)
10  print(var1, var2)
```

名稱空間(Name Space)

- 其實在 Python 的主程式與每個函式，都有各自的名稱空間(namespace)，其中主程式定義「全域」而各別函式定義「區域」的名稱空間。
  - 每個名稱空間內的變數名稱都是唯一，而不同名稱空間內的變數名稱則可以相同。



Python 提供三個名稱空間，當使用變數時，會由最內層（區域名稱空間）逐步往外層搜尋，直到找到對應的名稱為止，若找不到就會引發錯誤。

關鍵字 global

- 如果要在函式內變更全域變數的值，可以在函式內使用關鍵字 global 宣告該變數。

```
1   def scope():
2       var1 = 1
3       var3 = var2 + 50
4       return var1, var3
5
6   var1, var2 = 10, 50
7   x, y = scope()
8
9   print(x, y)
10  print(var1, var2)
```

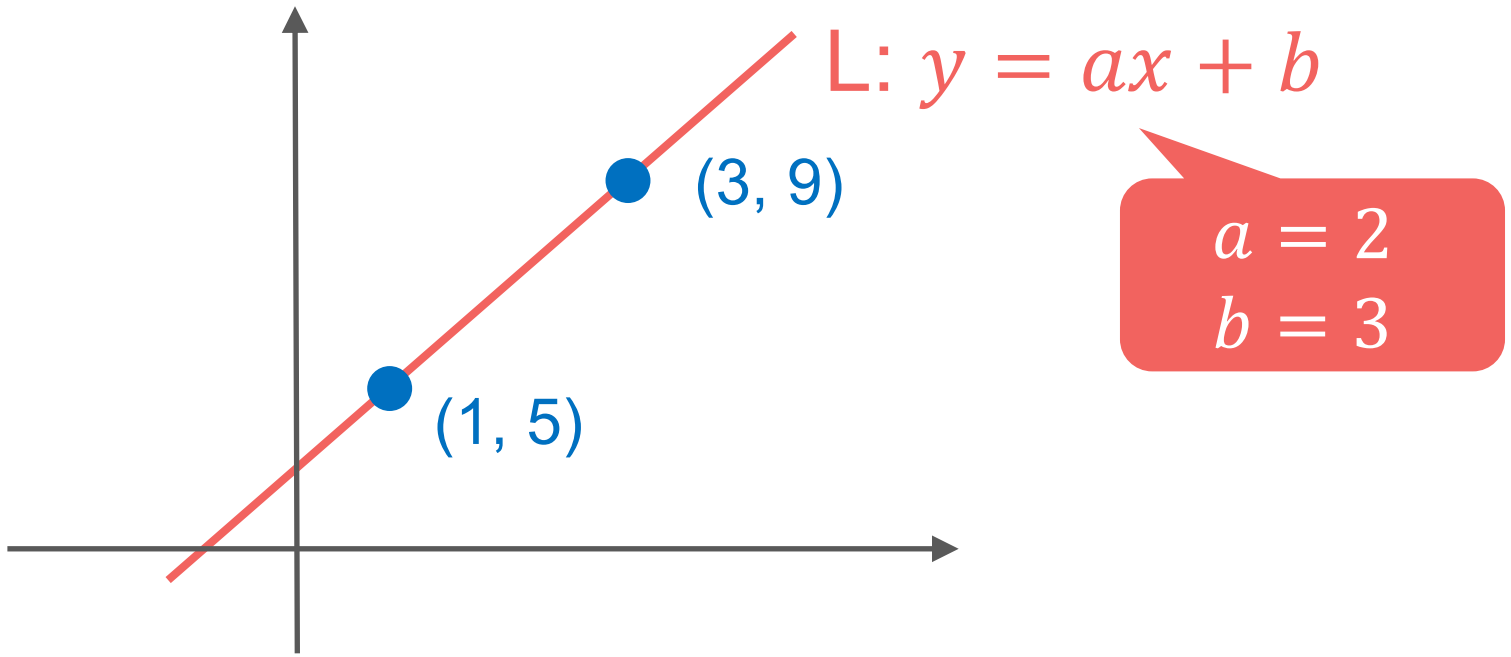
```
1 100
10 50
```

```
1   def scope():
2       global var2
3       var1 = 1
4       var2 += 50
5       return var1, var2
6
7   var1, var2 = 10, 50
8   x, y = scope()
9
10  print(x, y)
11  print(var1, var2)
```

```
1 100
10 100
```

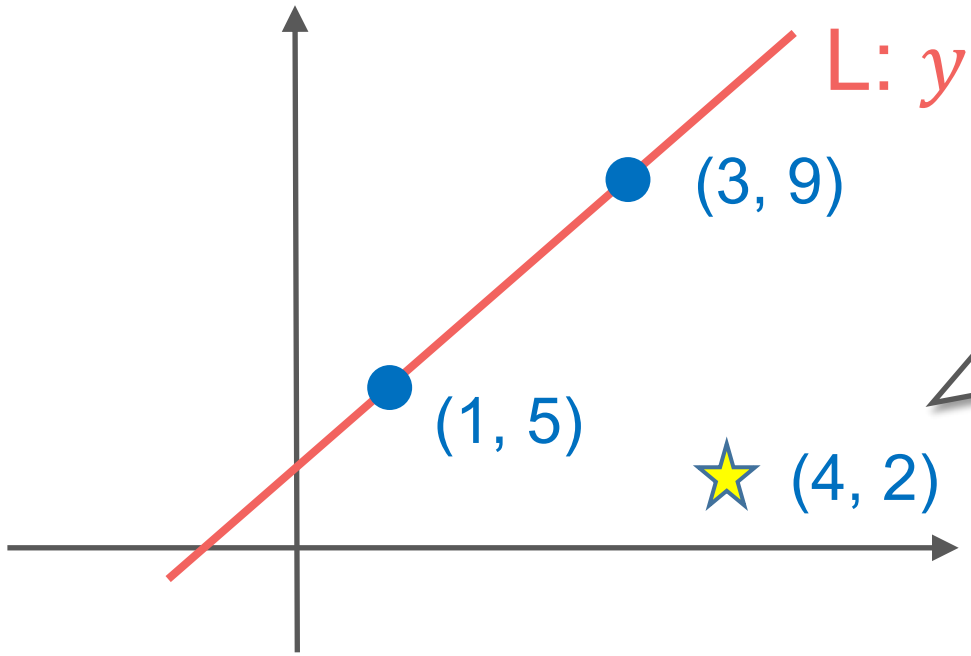
插播(1/2)

- 給定平面上兩點 p0, p1，如何計算通過兩點的直線 L？



插播(2/2)

- 給定平面上三點 p0, p1, q，直線 L 通過 p0, p1 兩點，如何知道 q 點位於 L 的上面、下面或線上？



- $q_x = 4$ 代入 L
  - $2 \times 4 + 3 = 11$
- $q_y = 2 < 11$
  - 位於 L 下面

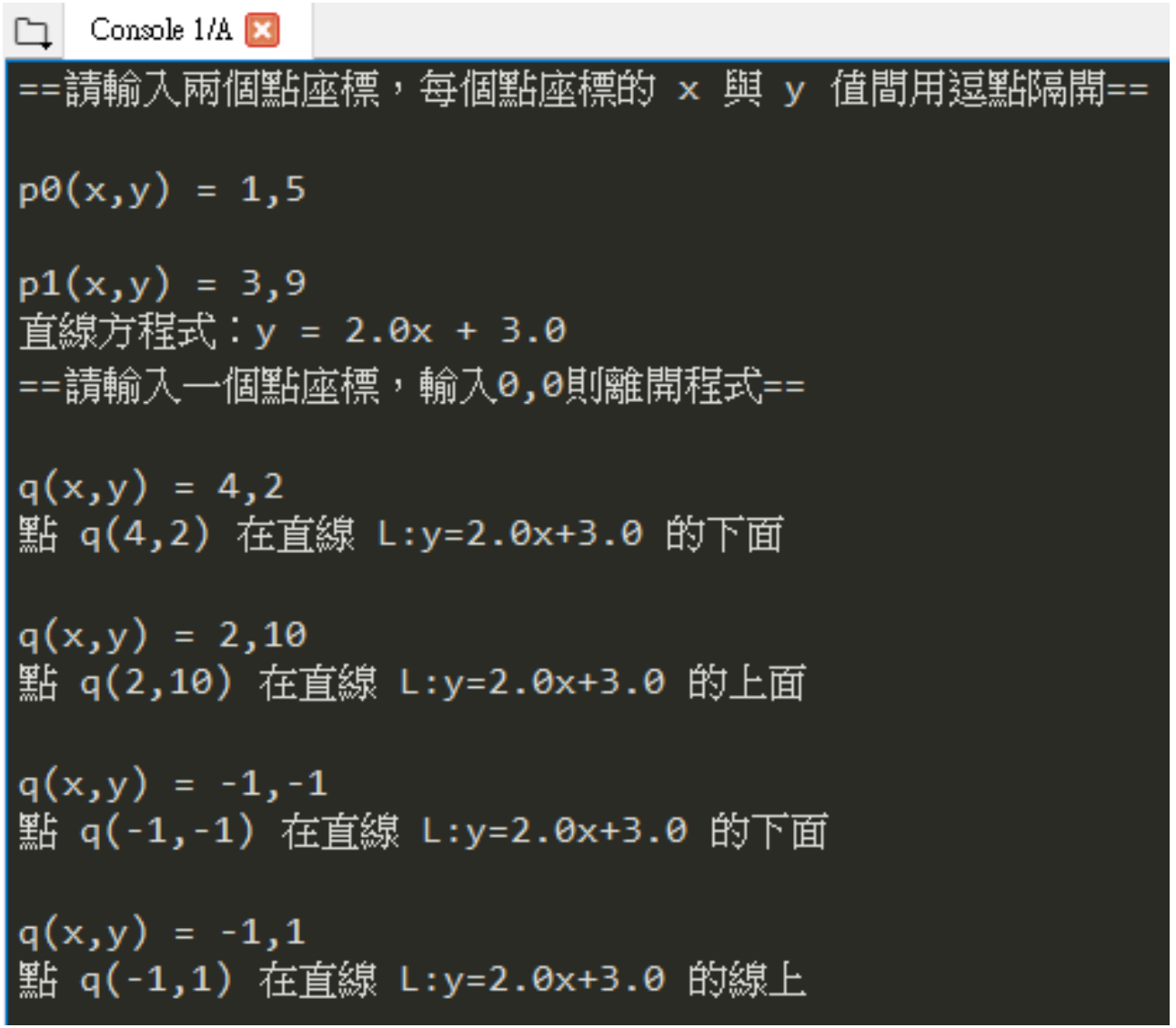
練習(1/2)

- 撰寫程式讓使用者輸入三個點座標 p0, p1, q，判斷 q 點位於 p0 與 p1 所成直線 L 的上面、下面或線上。
- 撰寫下列函式：
  - lineEquation(x0, y0, x1, y1)
    - p0:(x0, y0), p1:(x1, y1) 為平面上的點座標
  - 回傳一個係數串列 coef_lst 代表 L 的係數，其中
    - y = coef_lst[1] * x + coef_lst[0]
    - y = a*x + b

練習(2/2)

- 撰寫下列函式：
  - position(lst, q_x, q_y)
    - lst：直線方程式 L 的係數串列
    - q:(q_x, q_y) 為平面上的點座標
  - 判斷點 q 與直線 L 的相對位置，並輸出
    - “點 q(q_x, q_y) 在直線 L:y=lst[1]x+lst[0] 的上面”
    - “點 q(q_x, q_y) 在直線 L:y=lst[1]x+lst[0] 的下面”
    - “點 q(q_x, q_y) 在直線 L:y=lst[1]x+lst[0] 的線上”

輸出結果參考


Console 1/A
==請輸入兩個點座標，每個點座標的 x 與 y 值間用逗點隔開==
p0(x,y) = 1,5
p1(x,y) = 3,9
直線方程式：y = 2.0x + 3.0
==請輸入一個點座標，輸入0,0則離開程式==
q(x,y) = 4,2
點 q(4,2) 在直線 L:y=2.0x+3.0 的下面
q(x,y) = 2,10
點 q(2,10) 在直線 L:y=2.0x+3.0 的上面
q(x,y) = -1,-1
點 q(-1,-1) 在直線 L:y=2.0x+3.0 的下面
q(x,y) = -1,1
點 q(-1,1) 在直線 L:y=2.0x+3.0 的線上

主題

- 陣列與串列
- 函式
  - 自訂函式
  - 傳遞參數與回傳值
  - 變數的有效範圍
  - 常見的內建函式

內建函式(Built-in Function)

- 將經常需要用到或特定功能的程式寫成函式，需要時只要呼叫來執行即可，既方便又容易維護。
  - 但如果每一項功能都要由開發者自行撰寫，不僅工程浩大且艱鉅，需要注意的大大小小細節也容易讓一般人望而卻步。

- Python 擁有許多能直接使用的內建函式，如同軍火庫內擺滿各式各樣的槍械般，能依個人喜好與實務條件來使用，輕易設計出符合需求的應用程式。

內建函式(1/3)

	方法	意義	範例	範例結果
串列操作：list()、len()、min()、msx()、sum()、sorted()、reversed()				
資料型別：int()、float()、bool()、str()、type()				
記憶體相關：id()				
字元轉換	chr(x)	取得整數 x 對應的 ASCII 編碼的字元	chr(65)	A
	ord(x)	取得字元 x 對應的 unicode 編碼	ord("皮")	30382
進位轉換	bin(x)	將 x 轉成二進位數字	bin(7)	"0b111"
	oct(x)	將 x 轉成八進位數字	oct(45)	"0o55"
	hex(x)	將 x 轉成十六進位數字	hex(45)	"0x2d"

內建函式(2/3)

	方法	意義	範例	範例結果
數學運算	range(x)	產生等差數列	range(2)	0, 1, 2
	eval(x)	執行運算式 x	eval("2 + 3")	5
	abs(x)	取得 x 的絕對值	abs(-7)	7
	divmod(x,y)	取得 x 除 y 的商和餘數	divmod(45, 7)	(6, 3)
	pow(x,y[,z])	取得 x 的 y 次方除以 z 的餘數，z 的預設值為 None，可省略	pow(2, 3)	8
			pow(2, 3, 3)	2
	round(x [,y])	取得 x 四捨五入到小數點第 y 位，y 的預設值為 None，可省略	round(3.14)	3
			round(3.141, 2)	3.14

pow() 範例

```
1    print(pow(2, 3))
2    print(2**3)
3    print("-- 3個參數的範例 --")
4    print(pow(2, 3, 3))
5    print((2**3)%3)
```

```
8
8
-- 3個參數的範例 --
2
2
```

pow() 的作用可透過次方與取餘數運算子的結合來達成，但 pow() 有進行優化過，對於較大數字的運算能節省執行時間。

round() 範例

- round() 也相當直覺，將給定數值四捨五入到指定位數，但仍要小心浮點數精確度問題，時不時有出人意表的錯誤。

```
1    PI = 3.1415926
2    print(round(PI))
3    print(round(PI, 2))
4    print(round(PI, 3))
5
6    print(round(3.15, 1))
```

```
3
3.14
3.142
3.1
```

內建函式(3/3)

	方法	意義	範例	範例結果
邏輯判斷	all(x)	判斷可迭代物件 x 內的元素是否全為 True	all([1, 2, 3])	True
			all([1, 2, ""])	False
	any(x)	判斷可迭代物件 x 內是否有一元素為 True	any([1, 2, ""])	True
			any([0,None,""])	False
函式迭代	map(func, x)	使用指定函式 func，依序處理可迭代物件 x 的每個元素，並回傳處理結果（範例在底下）		
	filter(func, x)	使用指定函式 func，依序判斷可迭代物件 x 的每個元素，並取出判斷為 True 的元素（範例在底下）		

map() vs filter()

- map() 與 filter() 兩個內建函式是將指定函式逐一套用到可迭代物件內的每個元素。
  - 都需要兩個參數，一個是指定函式，另一個是可迭代物件。
  - map() 的指定函式要能接受至少一個參數，並回傳一個運算式。
  - filter() 的指定函式也是至少一個參數，並回傳能產生布林值的條件式。

範例

```python
scores = [x for x in range(5, 100, 10)]
print(scores)

def 調分(x):
    return int(10*x**0.5)

print("調整分數:", list(map(調分, scores)))

def 及格(x):
    return x >= 60

print("及格分數:", list(filter(及格, scores)))
```

```
[5, 15, 25, 35, 45, 55, 65, 75, 85, 95]
調整分數: [22, 38, 50, 59, 67, 74, 80, 86, 92, 97]
及格分數: [65, 75, 85, 95]
```

map() 與 filter() 的套用結果無法直接看到，要以 list() 轉換成串列型態才能看到結果。

更多用法

- 更多內建函式及其用法，可參考 Python 官網的說明 (https://docs.python.org/zh-tw/3/library/functions.html)

內建函式

A	E	L	R
abs()	enumerate()	len()	range()
aiter()	eval()	list()	repr()
all()	exec()	locals()	reversed()
anext()			round()
any()	F	M	
ascii()	filter()	map()	S
	float()	max()	set()
B	format()	memoryview()	setattr()
bin()	frozenset()	min()	slice()
bool()			sorted()
breakpoint()	G	N	staticmethod()
bytearray()	getattr()	next()	str()
bytes()	globals()		sum()
		O	super()
C	H	object()	
callable()	hasattr()	oct()	T
chr()	hash()	open()	tuple()
classmethod()	help()	ord()	type()
compile()	hex()		
complex()		P	V
	I	pow()	vars()
D	id()	print()	
delattr()	input()	property()	Z
dict()	int()		zip()
dir()	isinstance()		
divmod()	issubclass()		_
	iter()		__import__()

本章結束

Py04
字串與檔案處理 (1)

王頌文

主題

- 字串
  - 字元與字串
  - 字串基本操作
  - 更多操作函式
- 檔案處理

字串(String)

- 我們已經知道 Python 的基本資料型別(data type)，除了數值、布林值外，還有字串型別。
  - 事實上，在程式開發過程中，不乏與文字字串的處理打交道，其頻率也許還比處理數字要高。
  - 外部資料進到電腦內的預設型別
- 字串是由一連串字元(character)所組成且有順序的序列，而所謂的字元就像是字母、數字系統或標點符號等，之前介紹過的控制字元（如換行、水平移位等）也都是。

字元與字串(1/2)

- Python 對於字串有提供 `str` 型別，但卻沒有字元型別。

- 若要表示一個字元，可以使用長度為 1 的字串，例如底下這些變數都是 Python 的字串型別：
  - `letter = 'X'`
  - `num_char = '7'`
  - `pokemon = "皮卡丘"`

字元與字串(2/2)

- 字元是字串的基本單元，一連串有順序的字元即稱為字串，因此，想了解字串還得從字元著手。
  - 由於電腦內部使用二進位數字（即 0、1），所以電腦內的資料會被編碼成一連串的位元樣式(bit pattern)。
    - 如 00100101、11001100 等，方便儲存，也容易在網路上傳輸。
  - 大量位元樣式組合在一起可表示文字、音訊、圖像或視訊等內容，但確切的意義得視其應用而定。

字元編碼(Character Encoding)

- 字元在電腦內也是以一串 0、1 的序列儲存，而將字元對應到其二進位表示的過程稱為字元編碼。

- ASCII 與 UTF-8 是兩個常見的文字編碼方式，其中前者是 Python 2.x（現在已沒有更新）的預設選項，而 Python 3.x 預設則是後者，以下簡介這幾種編碼方式。

美國標準資訊交換碼(ASCII)

- 基於拉丁字母的電腦編碼系統，使用 7 個位元來表示十進位 0 ~ 127 共 128 (2^7)個不同字元，其中
  - 33 個控制字元 + 95 可顯示字元

ASCII可顯示字元（共95個）

二進位	十進位	十六進位	圖形	二進位	十進位	十六進位	圖形	二進位	十進位	十六進位	圖形
0010 0000	32	20	(space)	0100 0000	64	40	@	0110 0000	96	60	`
0010 0001	33	21	!	0100 0001	65	41	A	0110 0001	97	61	a
0010 0010	34	22	"	0100 0010	66	42	B	0110 0010	98	62	b
0010 0011	35	23	#	0100 0011	67	43	C	0110 0011	99	63	c
0010 0100	36	24	$	0100 0100	68	44	D	0110 0100	100	64	d
0010 0101	37	25	%	0100 0101	69	45	E	0110 0101	101	65	e
0010 0110	38	26	&	0100 0110	70	46	F	0110 0110	102	66	f

Source: Wikipedia

ASCII 的缺點

- 缺點在於其編碼的字元不夠多，雖然有 EASCII (Extended ASCII) 將 7 個位元擴充至 8 位，解決了部分西歐語言的顯示問題，但對更多其他語言依然無能為力。

- 因此，現在大多採用 Unicode，特別是與 ASCII 向下相容的 UTF-8。

統一碼(Unicode)

- 又稱為萬國碼，整理與編碼世界上大部分文字系統，讓電腦能以統一的字元集來處理及顯示文字。
  - 改善在不同編碼系統間轉換的困擾，更是跨平台跨域的解決方案。

- Unicode 使用 16 位元的編碼空間，即每個字元占用 2 個位元組 (bytes)，理論上最多可表示 65,536 (2^{16})個字元，涵蓋各種語言。
  - Unicode 至今仍在不斷增修，每個新版本都納入更多新字元，至 2023 年底的最新版本是 2022.9 公布的 15.0.0，已經收錄超過 14 萬個字元。

Unicode 轉換格式(UTF)

- 這是 Unicode 的不同實現方式，主要是避免浪費。
  - 例如：每個 7 位元的 ASCII 字元都使用 16 位元的 Unicode 傳輸，導致最前面的 8 個位元始終都設定為 0，編碼效率較差。
  - 因此，出現一種針對 Unicode 的可變長度字元編碼 UTF-8 (8-bit UTF)，其編碼的第一個位元組與 ASCII 相同。
    - 原來處理 ASCII 字元的軟體稍加修改，即可繼續使用。
  - 目前，UTF-8 逐漸成為全球資訊網最主要的編碼形式。

範例

- 想要知道字元對應的 ASCII（或者說是 UTF-8）編碼，或者反過來把編碼結果轉換為字元，可透過內建函式 ord() 與 chr()。

```
1   print(ord('A'))
2   print(ord("皮"))      # 輸出"皮"的十進位 Unicode
3   print("\u76AE")       # 輸出十六進位 Unicode 對應的字元
4
5   print(chr(65))
6   print(chr(30382))
```

```
65
30382
皮
A
皮
```

字串

- 字串是經常會用到的資料型態，也是「外部資料」進到電腦內的預設型別。
  - "海賊王"
  - '草帽海賊團'
  - '魯夫是"草帽海賊團"的船長'

主題

- 字串
  - 字元與字串
  - 字串基本操作
  - 更多操作函式
- 檔案處理

基本操作

- 之前介紹字串型別時，我們已經看過一些建立字串與其基本操作。
  - 例如格式化輸出、藉由 `str()` 可強制轉換為字串型別、透過跳脫字元的設定可輸出一些無法在螢幕上顯示的控制字元等。

```
1    str1 = "寶可夢"
2    str2 = "皮卡丘"
3    print(str1 + ' ' + str2)
4
5    print("="*20)
6    print("OK"*3)
```

```
寶可夢 皮卡丘
====================
OKOKOK
```

字串比大小

- 字元的比較則是根據其 Unicode 編碼的大小
  - 數字 < 大寫字母 < 小寫字母 < 中文字

```
1    print('A' < '皮')
2    print('A' < '1')
3    print('A' < 'a')
4    print("="*10)
5    print("小火龍" <= "皮卡丘")
```

```
True
False
True
==========
True
```

成員運算子 in/not in

- 檢查某個字串是否在另一個字串內

```
1    print("電系" in "皮卡丘是電系寶可夢")
2    print("火系" in "皮卡丘是電系寶可夢")
3    print('python' in "Life is Short, Use Python")
4
5    print("He" not in "Hello")
```

```
True
False
False
False
```

原始字串(Raw String)

- 如果在字串的前方加上「r」，表示這個字串為原始字串，此時跳脫字元 (\ ) 不起作用，直接輸出字串內容即可。

```
1    print("abc\ndef")
2    print(r"abc\ndef")
```

```
abc
def
abc\ndef
```

迭代一字串

- 之前在介紹 for 迴圈時提到可迭代物件，其實字串也可迭代 (iterable)，意味著也能使用 for 迴圈逐一拜訪字串中的所有字元。

```
1    for ch in "Pikachu":
2        print(ch, end='-')
```

```
P-i-k-a-c-h-u-
```

字串處理 vs 串列操作

- Python 的字串處理大都承襲串列的概念與操作方式，譬如：
  - 用 `len()` 取出字串長度，而用 `min()` 與 `max()` 則分別能取出字串內Unicode 編碼最小與最大的字元。
  - 也能以索引值的方式，依既定順序遍訪字串的所有字元。

```
1    str_ = "Pikachu"
2    for i in range(len(str_)):
3        print(str_[i], end='-')
4
5    print()
6    for i in range(0, len(str_), 2):
7        print(str_[i], end='-')
```

```
P-i-k-a-c-h-u-
P-k-c-u-
```

子字串(Substring)

- 類似串列分割的作法也能套用到字串上，很容易就能取出一個字串的任意子字串。
  - 注意因為 Python 3.x 採用 UTF-8 編碼，所以一個中文字和一個英文字同樣都是一個字元。

```
1    str_ = "人生苦短, 我用Python"
2    print(str_[2:4])
3    print(str_[5:-7])
4    print(str_[5:])
5    print(str_[:4])
6    print(str_[:-3:2])
```

```
苦短
 我
 我用Python
人生苦短
人苦,我Pt
```

主題

- 字串
  - 字元與字串
  - 字串基本操作
  - 更多操作函式
- 檔案處理

可變 vs 不可變物件(1/2)

- Python 內的所有東西都是物件(object)，其資料型別有兩種：

  ① 可變物件(mutable object)：

  - 該變數所指向記憶體中的值可以被改變，過程中沒有複製動作，也沒有建立新的記憶體位址，白話點講就是「原地改變」。

  - e.g., 前面提過的串列(list)，以及後面會介紹的字典(dict)、集合(set)等。

可變 vs 不可變物件(2/2)

- Python 內的所有東西都是物件(object)，其資料型別有兩種：
  - ② 不可變物件(immutable object)：
    - 該變數所指向記憶體中的值不可以被改變，所以當變更某個變數時，相當於將原來的值複製一份後存在一個新位址，接著變數再指向這個新位址。
    - e.g., 已經學過的整數(int)、浮點數(float)，還有字串(string)等都屬於不可變物件。

可變物件：串列

- 前面也看過當變數儲存的是串列時，若將該變數指派給另一個變數，因為都指向記憶體的同一個位址（可透過內建函式 id() 來查看），所以兩個變數會同時更動。

```
1    lst = [3]
2    new = lst
3    print(id(lst), id(new))
4
5    lst.append(100)
6    print(lst)
7    print(new)
```

```
2051141588992 2051141588992
[3, 100]
[3, 100]
```

不可變物件：字串(1/2)

```
1    str1 = "皮卡丘"
2    str2 = "皮卡丘"
3    print(id(str1), id(str2), id("皮卡丘"))
4
5    str1 += "電系寶可夢"
6    print("str1變更後的id:", id(str1))
```

為了節省儲存空間，若字串的內容相同，則 Python 使用同一物件，所以這裡會輸出三個同樣的識別碼。

因為不可變更的特性，改變字串內容後，會儲存在新的記憶體位址，也就輸出不同的識別碼。

```
2183158944496 2183158944496 2183158944496
str1變更後的id: 2183159353328
```

不可變物件：字串(2/2)

- 若直接變更字串的內容，會引發錯誤。
  - 譬如在範例中加入「`str1[1] = ''`」，企圖直接變更字串變數的第 2 個字元，執行時會得到「`TypeError`」的錯誤訊息。

```
1    str1 = "皮卡丘"
2    str2 = "皮卡丘"
3    print(id(str1), id(str2), id("皮卡丘"))
4
5    str1 += "電系寶可夢"
6    print("str1變更後的id:", id(str1))
```

修改字串內容(1/5)

- 既然不能直接變更字串內容，只好轉個彎間接修改。

- 目標：想移除底下字串內，絕招名稱中的「必殺」字眼。
  - "喬巴超人的必殺絕招：『必殺可愛電波』"
  - 絕招名稱用引號(『』)標明

修改字串內容(2/5)

- 最簡單的作法大概是利用字串處理提供的 replace() 方法，將「必殺」字眼直接取代為空白。

```
1   data = "喬巴超人的必殺絕招為『必殺可愛電波』。"
2   new = data.replace("必殺", "")
3   print(new)
4
5   print(id(data))
6   print(id(new))
```

有甚麼缺點？

```
喬巴超人的絕招為『可愛電波』。
1671045072912
1671045079744
```

修改字串內容(3/5)

- replace() 方法雖然簡單，但卻無法滿足需求，接著我們嘗試以串列分割來進行。

```
1    data = "喬巴超人的必殺絕招為『必殺可愛電波』。"
2    new = data[:11] + data[13:]
3    print(new)
```

有甚麼缺點？

```
喬巴超人的必殺絕招為『可愛電波』。
```

修改字串內容(4/5)

- 雖然上述作法沒能解決問題，但是可以把問題轉化成尋找兩個引號的位置，接著再用串列分割或 replace() 方法進行。

```
1   data = "喬巴超人的必殺絕招為『必殺可愛電波』。"
2
3   for i in range(len(data)):
4       if data[i] == "『":
5           start = i
6       elif data[i] == "』":
7           end = i
8           break
9
10  new = data[:start] + \
11        data[start:end].replace("必殺", "") + \
12        data[end:]
13  print(new)
```

有甚麼缺點？

```
喬巴超人的必殺絕招為『可愛電波』。
```

搜尋子字串

- 輸出結果的確有滿足題目的要求，但這段 Python 程式寫得點長。
  - 其實這題麻煩的地方在於找出兩個引號位置，而 Python 有提供一系列搜尋子字串的方法。

	方法	意義（s 是字串）	範例	執行結果
★	count(s)	計算 s 的出現次數	stn.count('o')	2
	startswith(s)	字串是否以 s 開頭	stn.startswith('P')	True
	endswith(s)	字串是否以 s 結尾	stn.endswith('P')	False
★	find(s)	計算 s 出現的最小索引值	stn.find('o')	1
★	index(s)		stn.index('o')	
	rfind(s)	計算 s 出現的最大索引值	stn.rfind('o')	5
	rindex(s)		stn.rindex('o')	

修改字串內容(5/5)

- 利用 find() 與 index() 方法皆可搜尋給定子字串的索引值，也就是位置，而且預設是由字串開頭（即左側）往尾端（即右側）。

```
1   data = "喬巴超人的必殺絕招為『必殺可愛電波』。"
2
3   start = data.find("『")
4   end = data.index("』")
5   new = data[:start] + \
6         data[start:end].replace("必殺", "") + \
7         data[end:]
8   print(new)
```

```
喬巴超人的必殺絕招為『可愛電波』。
```

字串裁切與結合(1/2)

- strip() 與 split()，經常用在讀取檔案資料後的處理動作。

方法	意義 （ch 是字串、lst 是串列）	範例 執行結果
★ strip([ch])	從字串兩側刪除 ch 指定的字元，預設為空白字元。	"Pokemon".strip("Pon")
		kem
lstrip([ch])	從字串左側刪除 ch 指定的字元，預設為空白字元。	"Pokemon".lstrip("Pon")
		kemon
rstrip([ch])	從字串右側刪除 ch 指定的字元，預設為空白字元。	"Pokemon".rstrip("Pon")
		Pokem
★ split([ch])	以 ch 指定的字元（預設為空白字元）切割字串，並回傳一個串列。	"2019/11/23".split("/")
		['2019', '11', '23']

strip() 裁切字串兩側

- strip() 作用在字串上是在字串兩側重複去除指定字元，直到遇到沒有指定的字元為止。
  - 倘若沒有指定字元，則預設為空白字元(whitespace character) 。
  - 乍看之下，strip() 的使用頻率似乎不高，實則不然，試想一下所謂的空白字元其實也包含 \t、\r、\n 等控制字元。
  - 因此，經常作為程式接收到一個字串後的第一個處理動作，尤其是之後介紹讀取檔案時更是如此。

範例

- strip() 作用在字串上是在字串兩側重複去除指定字元，直到遇到沒有指定的字元為止。
  - 倘若沒有指定字元，則預設為空白字元(whitespace character) 。

```
1    data = "\t皮卡丘\n"
2
3    print(1, data.strip())
4    print(2, data.lstrip())
5    print(3, data.rstrip())
```

```
1 皮卡丘
2 皮卡丘

3         皮卡丘
```

split() 拆分字串

- 使用 split(ch) 可以根據指定字元 ch 來拆分字串，並回傳串列。
  - 非常適合用在字串有標準格式時的拆分，例如年、月、日皆以斜線(/)分隔開來。

```
1   data = "2019/11/23"
2   lst = data.split('/')
3   print(f"這是西元 {lst[0]} 年")
4
5   data = "10,20,30,40,50"
6   lst = [eval(x) for x in data.split(',')]
7   print(f"總和為 {sum(lst)}")
8
9   data = "Life is Short, Use Python"
10  lst = data.split(' ')
11  print(f"共有 {len(lst)} 個單字")
```

```
這是西元 2019 年
總和為 150
共有 5 個單字
```

練習：計算單字數量

- 輸入一句英文，判斷句子內有多少個單字？
  - e.g.,
  - 輸入：Success is simple. Do what’s right, the right way, at the right time.
  - 輸出：13

練習：算術運算

- 輸入數學運算式（包含數字、加號、減號），輸出運算結果。
  - e.g.,
  - 輸入：3+5.6-7.1+8
  - 輸出：9.5

字串裁切與結合(2/2)

- 用來結合串列裡字串元素的 join() 也相當實用，雖然直接寫程式實現這個功能也不難，但 join() 能讓程式碼簡潔許多。

方法	意義 （ch 是字串、lst 是串列）	範例 執行結果
partition(ch)	以 ch 指定的字元切割字串為三個部分。	"2019/11/23".partition('11') ('2019/', '11', '/23')
★ ch.join(lst)	利用 ch 指定的字元當膠水，結合串列 lst 內的字串元素。	",".join(['2019', '11', '23']) 2019,11,23

範例

```python
lst = ["皮卡丘", "小火龍", "傑尼龜"]

data = ""
for x in lst:
    data += x
    data += '-' if x != lst[-1] else ''
print(data)

print("="*20)
print('-'.join(lst))
```

```
皮卡丘-小火龍-傑尼龜
====================
皮卡丘-小火龍-傑尼龜
```

字串測試(1/2)

- 很多時候需要測試字串內的字元是否全為數字、英文字母等，且測試結果只有 True 與 False 兩種。

方法	意義	範例	執行結果
★ isalpha()	測試字串所有字元是否全為英文字母、中文字，且不為空字串。	"Python".isalpha()	True
		"Python3".isalpha()	False
isdigit()	測試字串所有字元是否全為數字，且不為空字串；而絕大多數時候，兩個方法的結果一樣。	"666".isdigit()	True
isdecimal()		"666P".isdecimal()	False
★ isalnum()	測試字串所有字元是否全為數字、英文字母、中文字，且不為空字串。	"Python3".isalnum()	True
		"3.14".isalnum()	False

範例

原本 isalpha() 用來檢測字串內是否全為英文字母組成，但因為編碼的緣故，實際使用時會發現也能允許出現中文字。

```
1    data = "人生苦短,我用Python"
2    print(data.isalpha())
3
4    data = "人生苦短我用Python"
5    print(data.isalpha())
6    print(data.encode("UTF-8").isalpha())
```

```
False
True
False
```

倘若只想測試英文字母，可將字串先用 UTF-8 編碼過後再交給 isalpha() 測試，即可得到正確的測試結果。此外，isalnum() 也有類似的情況。

測試中文字

- Python 沒有提供測試字元全都為中文字的方法，但只要知道語言對應 Unicode 的編碼範圍，不難自行撰寫檢測程式。
  - 常見語言的 Unicode 十六進位編碼範圍如下，其餘可參考 Unicode 編碼表(https://jicheng.tw/hanzi/unicode)。

  - 中文（含簡體與繁體）：0x4E00 ~ 0x9FA5
  - 韓文：0xAC00 ~ 0xD7FF
  - 日文（含平假名與片假名）：0x3040 ~ 0x309F、0x30A0 ~ 0x30FF

範例

- 中文（含簡體與繁體）的 Unicode 編碼範圍：0x4E00 ~ 0x9FA5

```python
data = "喬巴超人X的必殺絕招1為『可愛電波』。"

start = int('0x4E00', 16)     # 轉換為十進位
end = int('0x9FA5', 16)

for x in data:
    if not (start <= ord(x) <= end):
        print('不在範圍', x)
```

```
不在範圍 X
不在範圍 1
不在範圍 『
不在範圍 』
不在範圍 。
```

練習：字串檢查

- 輸入一字串 string，檢查這個字串裡是否包含英文字母、數字以外的字元。
  - 使用 Python 測試字串的函式來做
  - 不用 Python 測試字串的函式來做

字串測試(2/2)

- 其它還有檢查空白、識別字及字母大小寫的函式。

方法	意義	範例	執行結果
isspace()	測試字串所有字元是否全為空白，且不為空字串。	" ".isspace()	True
		" 1".isspace()	False
isidentifier()	測試字串所有字元是否合法的識別字（含關鍵字），且不為空字串。	"if". isidentifier()	True
		"3if". isidentifier()	False
★ istitle()	測試字串內每個單字的開頭字母是否全為大寫，其餘皆小寫。	"Oh Ya".istitle()	True
		"OH YA".istitle()	False
★ isupper()	測試字串所有英文字母是否全為大寫。	"OH_YA".isupper()	True
		"Oh_YA".isupper()	False
★ islower()	測試字串所有英文字母是否全為小寫。	"oh_ya".islower()	True
		"oh_Ya".islower()	False

範例

- 注意 istitle()、isupper()、islower() 只針對英文字母，不管字串裡的其它字元。
  - 尤其是 istitle() 會把連續的英文字母視為一個單字，而這並非是實際存在的英文單字。

```
1   data = "Life Is Short, Use Python"
2   print(data.istitle())
3
4   data = "Life_Is_Short_Use_Python!"
5   print(data.istitle())
6
7   data = "人生苦短,我用Python"
8   print(data.istitle())
```

```
True
True
True
```

其餘字串操作方法

- 這些字串轉換函式都會先複製原字串，再進行轉換或取代的動作，因此原字串都保留不變。

	方法	意義（s 是字串）	範例	執行結果
★	lower()	將所有英文字母改為小寫	"Python".lower()	python
★	upper()	將所有英文字母改為大寫	"Python".upper()	PYTHON
	swapcase()	英文字母的大小寫互換	"Python".swapcase()	pYTHON
	capitalize()	將字串中的第一個英文字母大寫，其餘小寫。	"oh ya".capitalize()	Oh ya
	title()	將每個詞的第一個英文字母大寫，其餘小寫。	"oh ya".title()	Oh Ya
★	replace(old, new)	將子字串 old 取代為 new	"oh ya".replace(' ', '')	ohya

範例

```
1    data = "Life is short, use Python"
2
3    print(1, data.lower())
4    print(2, data.upper())
5    print(3, data.swapcase())
6    print(4, data.capitalize())
7    print(5, data.title())
8    print(6, data.replace("s", "X"))
```

```
1 life is short, use python
2 LIFE IS SHORT, USE PYTHON
3 lIFE IS SHORT, USE pYTHON
4 Life is short, use python
5 Life Is Short, Use Python
6 Life iX Xhort, uXe Python
```

練習：字串轉換

- 輸入一句英文，依下列規則轉換每個單字內字母的大小寫：
  - 單字字首大寫→小寫，小寫→大寫。
  - 除字首外，其餘皆為小寫。

- e.g.,
  - 輸入字串："ChoppEr is sO CUTE"
  - 輸出結果："chopper Is So cute"

格式化字串

方法	說明
`center(width)`	在給予寬度欄位下回傳向中靠齊的字串
`ljust(width)`	在給予寬度欄位下回傳向左靠齊的字串
`rjust(width)`	在給予寬度欄位下回傳向右靠齊的字串
☆ `format(items)`	字串格式化

範例

```
1    s = "Chopper"
2
3    print(s.center(11))
4    print(s.ljust(11))
5    print(s.rjust(11))
```

```
  Chopper
Chopper
      Chopper
```

本章結束

Py04

字串與檔案處理 (2)

王頌文

主題

- 字串
- 檔案處理
  - 檔案的存取路徑
  - 檔案的運作流程
  - 讀取與寫入資料

檔案(File)

- 變數用來儲存單筆資料，而串列可以儲存多筆資料，這些儲存的資料都存放在記憶體內，只存在於程式執行期間，一旦程式執行結束或電腦關機就無法再進行存取。

- 若想日後還能繼續存取這些資料，得儲存在檔案並置於硬碟內。
  - 可把檔案當成是一個帶有名稱的資料容器，在電腦內保存與組織資訊，包含各種類型的資料，如文字、圖像、影片、程式碼等。
  - 檔案也是作業系統(operating system)的一個重要組成，負責長久儲存、資料交換以及組織文件等任務。

字串 vs 檔案

- 不管檔案裡原本儲存的是甚麼類型的資料，當程式讀取檔案內的資料時，一律以字串的形式進行。
  - 因此，字串操作對檔案處理而言，可謂重中之重。
  - 熟悉字串操作後，再來看檔案處理的概念，屆時進行讀寫檔案等檔案操作自然能水到渠成。

檔案處理(File Handling)

- 對於少量資料的處理，之前學的輸入與輸出勉強還能應付，可是要對付稍微大點的資料量就有些左支右絀。
  - 輸入值太多，或者要處理的是一些已經存在的資料，還是透過鍵盤來輸入就相當沒有效率。
  - 僅靠螢幕輸出結果，不僅難以將好不容易得到的結果長久地儲存下來，也不便於傳送給其他人。

- 這時就需要以某種特定格式將資料長期保存，並撰寫對應程式來處理。因此，本節介紹 Python 的檔案處理方法。

主題

- 字串
- 檔案處理
  - 檔案的存取路徑
  - 檔案的運作流程
  - 讀取與寫入資料

路徑(Path)

- 存取檔案之前，得先找到檔案所在的地方，這就是所謂的路徑。
  - 它是一種電腦檔案或目錄名稱的通用表現形式，會指向檔案系統(file system)的唯一位置。
  - e.g.,「C:\Users\Ian\Documnets\皮卡丘.jpg」
    - 路徑通常以字串表示的目錄結構，最開頭的「C」表示檔案系統位置，也稱為根目錄(root directory)。
    - 隨後以分隔字元分開的各部分路徑表示各級目錄，最後就是檔案名稱。
    - 最常使用的分隔字元有斜線(/)、反斜線(\)以及冒號(:)，注意不同作業系統與環境可能採用的不同。

樹狀結構(Tree Structure)

- 電腦的目錄(directory)或資料夾(folder)內儲存著檔案或更多目錄，包含在一個目錄中的其它目錄被稱為它的子目錄(child directory)。
  - e.g.,「C:\Users\Ian\Documnets\皮卡丘.jpg」
  - 這些目錄形成有層次(hierarchy)的結構，一般稱為樹狀結構(tree structure)。

樹狀結構

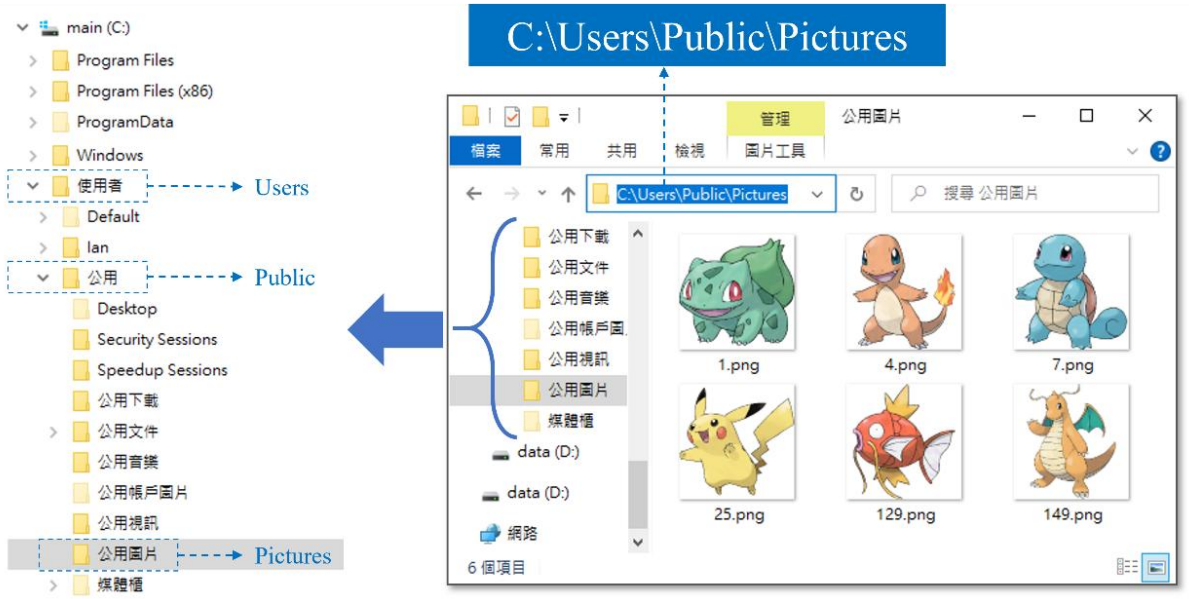
C:\Users\Public\Pictures

C:\Users\Public\Pictures\25.png

描述路徑有兩種方式(1/2)

① 絕對路徑(absolute path)：這是檔案的完整路徑(full path)，描述在本機端或是網路上的絕對位置，包含有根目錄與到該檔案會經過的所有子目錄。

- e.g.,「C:\Users\Public\Pictures\25.png」



描述路徑有兩種方式(2/2)

② 相對路徑(relative path)：相對於目前執行程式的工作目錄(current working path)與所指定檔案的相對位置，以兩者間的子目錄來描述。

- 比方說工作目錄為「C:\Users\Public\」，想存取檔案「25.png」，可用相對路徑為「.\ Pictures\25.png」，其中「.」代表當前目錄，在 Windows 底下可省略不寫。

- 若想存取「C:\Users\Ian\iris.png」，除了使用絕對路徑外，也可用相對路徑為「..\Ian\iris.png」，其中「..」表示上一層目錄。

副檔名(Filename Extension)

- 順帶一提，在檔案名稱「iris.png」的句號(.)後面的部分稱為副檔名，也稱為延伸檔名或後綴名，作用是讓系統決定當使用者想打開這個檔案的時候，要用哪種軟體執行。
  - 例如在 Windows 系統中的「.exe」是可執行檔，而「.png」則是圖檔，會用影像瀏覽或編輯的軟體打開。
  - 要注意直接修改副檔名並不會將檔案轉換成另一種格式（例如將 .png 手動改為 .exe），如此只會讓檔案無法使用而已，如有需要還是得找合適的工具軟體進行轉換。

主題

- 字串
- 檔案處理
  - 檔案的存取路徑
  - 檔案的運作流程
  - 讀取與寫入資料

檔案運作三部曲

- 能利用路徑定位檔案的所在地之後，接著就是經過一系列的檔案的處理流程，一般有三部曲，即
  - 開啟(open)檔案
  - 讀取(read)或寫入(write)資料
  - 關閉(close)檔案

先看開啟與關閉檔案，後面再看讀取與寫入資料。

所謂開啟檔案，並非是指我們將滑鼠移動到檔案圖示上點擊兩次的打開動作，而是撰寫程式碼且在程式執行時主動打開檔案，並進行讀取或寫入等操作。

開啟檔案

- Python 的內建函式 open() 可以開啟檔案，語法如下：
  - 變數 = open(路徑與檔名 [, 模式] [, 編碼])

  - 路徑與檔名
    - 欲操作的檔案路徑與名稱，屬於字串型別，可以是相對或絕對路徑。
    - 若沒有設定路徑，會預設為目前程式檔的路徑。

開啟檔案：路徑與檔名

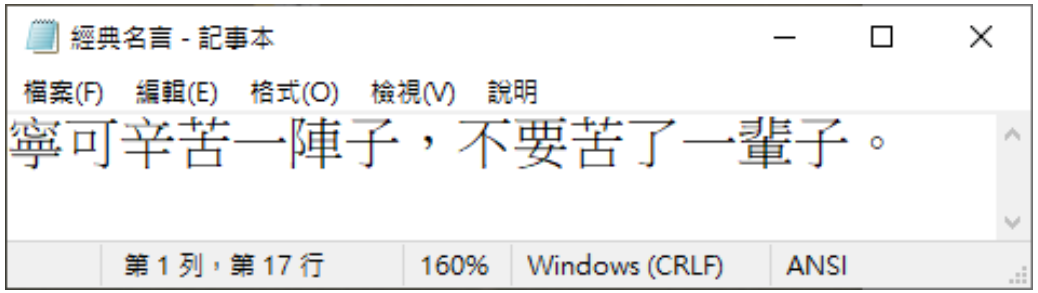
- Python 的內建函式 open() 可以開啟檔案，語法如下：
  - 變數 = open(路徑與檔名 [, 模式] [, 編碼])
  - 注意分隔字元，當路徑為「C:\Users\Public\Pictures\25.png」時，要使用跳脫字元(\)或是加上 r 以表示原始字串(raw string)，例如：
    - `file = open("C:\\Users\\Public\\Pictures\\25.png")`
    - `file = open(r"C:\Users\Public\Pictures\25.png")`
  - 這是在 Windows 下的路徑寫法，若換成 Linux 系統要改成：
    - `file = open("/home/Pictures/25.png")`

開啟檔案：模式(1/3)

- Python 的內建函式 open() 可以開啟檔案，語法如下：
  - 變數 = open(路徑與檔名 [, 模式] [, 編碼])
  - 若省略模式參數不寫，則預設為讀取模式(r)，其餘常用的模式如右表。

模式	說明	模式	說明
r	以讀取模式開啟檔案，檔案指標指向檔頭。若檔案不存在，會引發錯誤訊息「FileNotFoundError」。	r+	以讀寫模式開啟檔案，檔案指標指向檔頭，寫入的資料會覆蓋原檔案內容。若檔案不存在，會引發錯誤訊息。
w	以寫入模式開啟檔案並先清除檔案內容，檔案指標指向檔頭。若檔案不存在，則會建立新檔案；若檔案存在，則會清空內容再寫入。	w+	與模式「w」類似，不同之處在於「w+」以讀寫模式開啟檔案。
a	以附加寫模式開啟檔案，檔案指標指向檔尾，寫入的資料會附加到原檔案的尾端。若檔案不存在，則會建立檔案。	a+	以附加讀寫模式開啟檔案，檔案指標指向檔尾，寫入的資料會附加到原檔案的尾端。若檔案不存在，則會建立檔案。

範例




```
1   file = open("經典名言", 'r')     # 這個檔案沒有副檔名
2   data = file.read()              # 下節會介紹更多讀取方法
3   print(data)
4   file.close()                    # 關閉檔案
```

```
寧可辛苦一陣子，不要苦了一輩子。
```

關閉檔案的同時也會將留在記憶體的資料寫到磁碟上，並且不能再進行該檔案的讀寫等操作。當不再使用某些檔案時，最好能將它們關閉。

開啟檔案：模式(2/3)

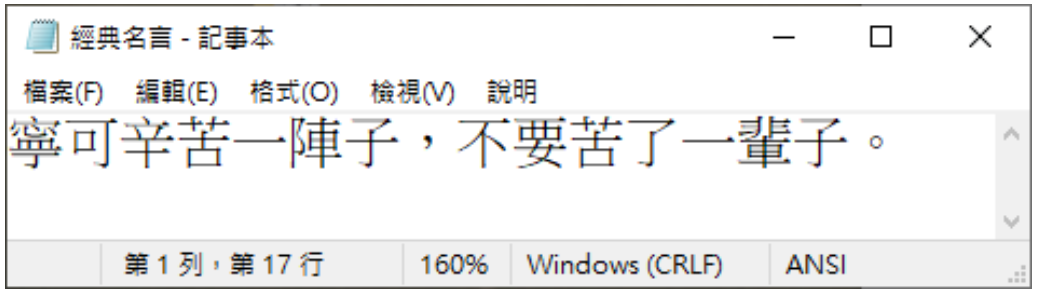
- Python 的內建函式 open() 可以開啟檔案，語法如下：
  - 變數 = open(路徑與檔名 [, 模式] [, 編碼])
  - 讀取模式(r)：若檔案不存在會產生「FileNotFoundError」
  - 寫入模式(w)：若檔案不存在會建立新檔，可要是檔案存在，會直接清空檔案內容再進行寫入，須謹慎使用。
  - 附加模式(a)：將新增資料會附加到檔案尾端(End of File、EOF)

開啟檔案：模式(3/3)

- Python 的內建函式 open() 可以開啟檔案，語法如下：
  - 變數 = open(路徑與檔名 [, 模式] [, 編碼])
  - 「r+、w+、a+」可視為「r、w、a」的升級，使其同時具備讀與寫的功能。
  - 要注意的是「r+」會將檔案指標移動到檔案開頭處，而「w+、a+」則移動檔案指標到檔案的尾端，因此寫入資料的位置有所不同。

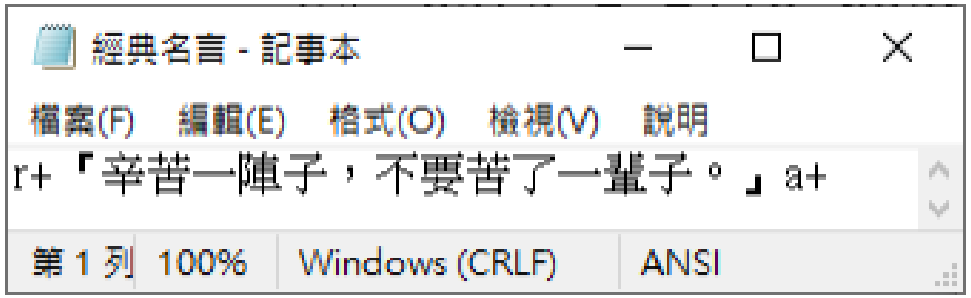
檔案指標(file pointer)是個特殊的標記，記錄著讀取或寫入到檔案的哪個位置，換言之，檔案操作始終發生在檔案指標所指之處。

範例




```
1   file = open("經典名言", 'r+')
2   data = file.write("r+『")       # 寫入檔案
3   file.close()
4
5   file = open("經典名言", 'a+')
6   data = file.write("』a+")
7   file.close()
```

執行後



預設存取路徑

- 兩個範例中都沒有指定路徑，這是因為程式檔與要存取的檔案位於同一個目錄下，因此可省略不寫。
  - 如若不然，要加上描述檔案所在位置的絕對或相對路徑，避免引發「FileNotFoundError」錯誤。

```
1   file = open("經典名言", 'r')
2   data = file.read()
3   print(data)
4   file.close()
```

```
1   file = open("經典名言", 'r+')
2   data = file.write("r+『")       # 寫入檔案
3   file.close()
4
5   file = open("經典名言", 'a+')
6   data = file.write("』a+")
7   file.close()
```

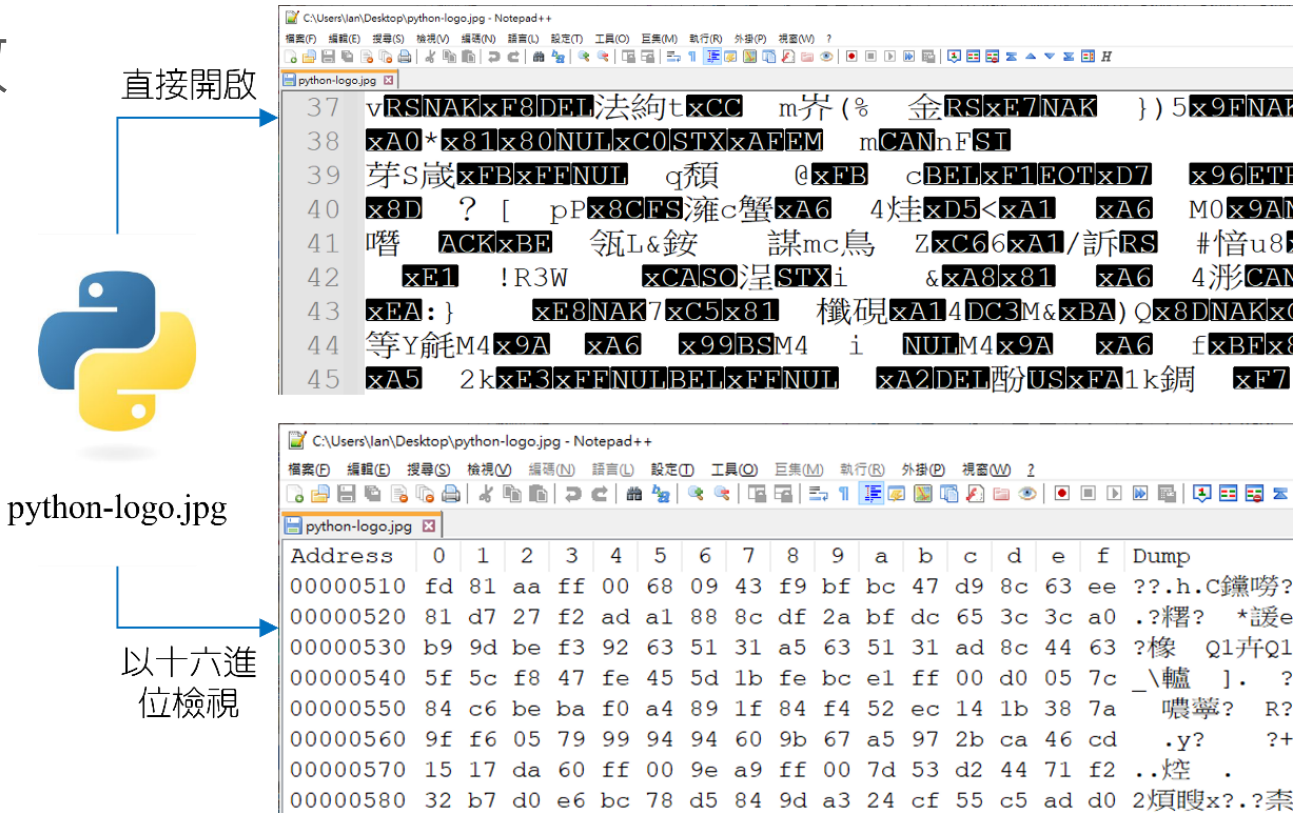
文字與二進位檔案(1/2)

- 檔案按照資料的組織形式可分為文字檔案與二進位檔案兩種，我們之前處理的都是文字檔案，直接用純文字編輯器打開即可看到內容。

- 二進位檔案則包含圖片、音訊、影片、可執行檔等，用純文字編輯器直接打開會看到亂碼，只有按照預先設定的規則讀取，才能明白這些數字的具體涵義。

文字與二進位檔案(2/2)

- 用文字編輯器 NotePad++ 開啟 JPG 圖檔後看到的亂碼，而底下是以十六進位檢視圖檔。
  - 相較於文字檔案，二進位檔案有節省儲存空間、讀寫速度快的優勢，且也有一定的保密作用。
  - 也有不易閱讀與編輯的缺陷。



二進位模式(Binary Mode)

- Python 也有提供二進位模式的開啟檔案方式，只要在前面的常用模式裡加上「b」即可，譬如 rb、wb+、ab 等。
  - 在二進位模式下，資料將被直接讀取或寫入，不進行任何特殊的文字編碼與解碼，且此時面對的是位元組(bytes)型別，而不是之前的字串型別。
  - 同時，因為不進行任何編碼或換行符號轉換，所以在跨平台使用時要特別小心。

關閉檔案

- 每次在結束存取檔案後，要使用 close() 來關閉檔案，著實有點麻煩，也容易忘記。
  - 因此，Python 提供 with 這個獨特敘述，將存取檔案的動作限制在其程式區塊內，並在離開程式區塊時自動關閉檔案。

```
1   with open("經典名言") as file:
2       data = file.read()
3       print(data)
```

```
寧可辛苦一陣子，不要苦了一輩子。
```

with 敘述

- with 敘述的用途是建立資源管理器(context manager)。
  - 資源的管理在程式設計上是很常見的問題，比如管理開啟的檔案、網路 socket 等。
    - 管理上最主要的問題是確保這些開啟的資源在使用完之後，有確實被關閉（或釋放），否則容易導致程式執行的效能問題，甚至出現錯誤。
    - 除了關閉之外，有些特殊的資源在使用完畢之後，還必須進行一些後續的清理動作。
  - 透過 with 敘述讓 Python 自動進行資源的建立、清理與回收動作，我們在使用各種資源時就能更加方便。

檔案的文字編碼(1/2)

- 此外，無法開啟檔案除了找不到檔案外，常見還有一個相當惱人的問題，就是文字編碼，這是將文字（如字母、數字、符號等）轉換為位元組序列（即二進位資料）的規範或方法，
  - 常見的有 UTF-8、ASCII、GBK（簡體中文）、Big5（繁體中文）
  - 當指定的編碼與檔案實際編碼不一致時，就可能會引發錯誤。

檔案的文字編碼(2/2)

- 雖然 Python 3 預設採用 UTF-8 編碼，但使用 open( ) 時會根據系統環境自動挑選合適的編碼格式。譬如：
  - 在西歐或北美的 Windows 系統會挑選 cp1251 編碼
  - 在繁體中文的環境則預設為 cp950
    - 這個編碼是對 Big5 的擴展，除了包含 Big5 所有字元外，還加入了一些額外的繁體中文字和符號。

ANSI 編碼(1/3)

- 在 Windows 系統還有一個常見的 ANSI 編碼，這是美國國家標準協會(American National Standards Institute)按照一系列規範所編寫的編碼標準統稱，並非一個特定的編碼方式。
  - 所以在不同的語言或地區環境下，ANSI 可能會對應到不同的編碼方式。
  - 在繁體中文地區會對應到 cp950，而簡體地區則對應到 cp936。

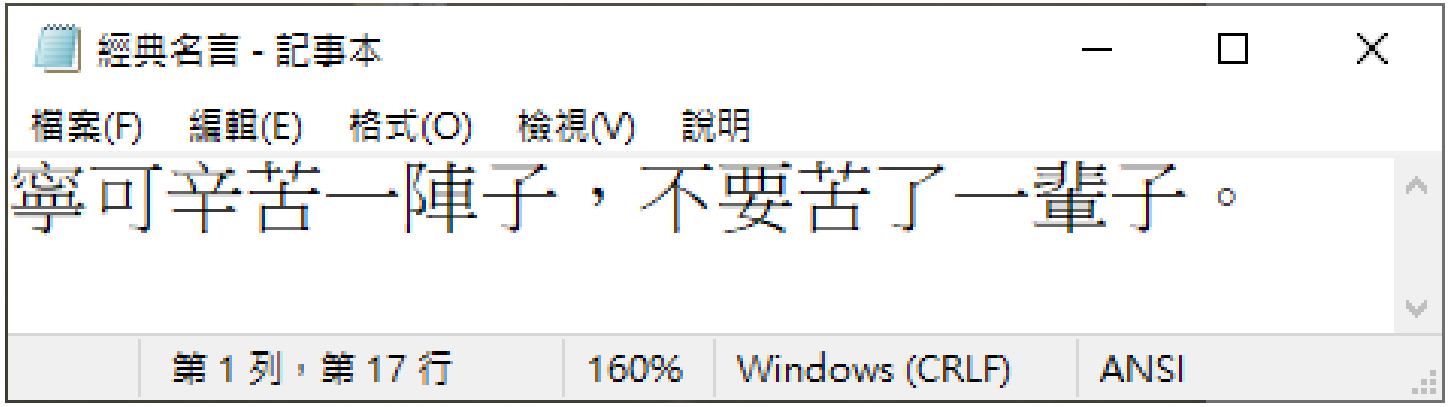
這麼一來，一個相當惱人的問題就出現了

ANSI 編碼(2/3)

- Windows 10 專業版在升級到 1903 版後，把預設編碼 ANSI（繁體中文地區為 cp950）改為 UTF-8，
  - 可是 open() 根據繁體系統環境會挑選 cp950 編碼。
  - 檔案編碼與開啟時指定的編碼不一致，自然無法順利開啟檔案，會得到「UnicodeDecodeError: 'cp950' codec can't decode byte 0xe6 in position 0」的錯誤訊息。

ANSI 編碼(3/3)

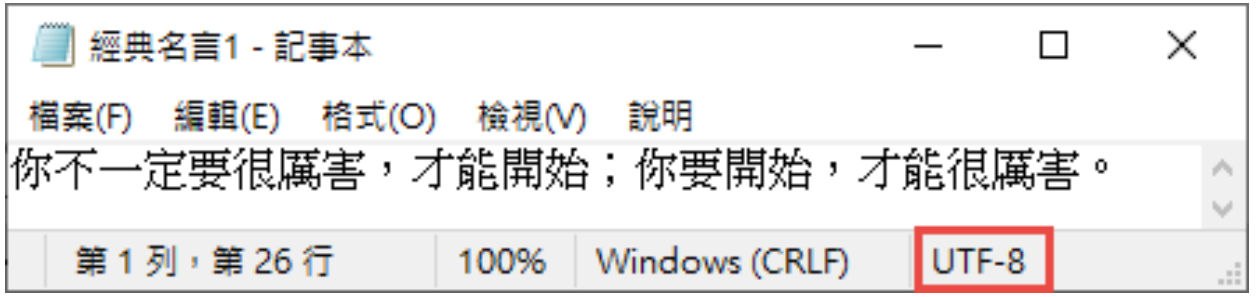
- 那為何前面幾個範例程式都沒指定編碼，採用預設的 cp950，但卻能正常開啟檔案呢？
  - 注意在前面範例程式的檔案內容圖片右下角，顯示的是 ANSI 編碼，與開啟時使用的編碼格式一樣。



開啟檔案：編碼

- Python 的內建函式 open() 可以開啟檔案，語法如下：
  - 變數 = open(路徑與檔名 [, 模式] [, 編碼])
  - 編碼
    - 若能確定檔案的編碼格式，在使用 open() 開啟檔案時可指定 encoding 參數。
    - 通常在繁體中文地區，編碼格式不外乎是 cp950 與 UTF-8 這兩種，輪流使用總會有一種能順利開啟檔案。

範例



UTF-8
utf-8
utf8

```
1    # UTF-8 也可寫成 utf-8、UTF8、utf8
2    with open("經典名言1", encoding='utf8') as file:
3        data = file.read()
4        print(data)
```

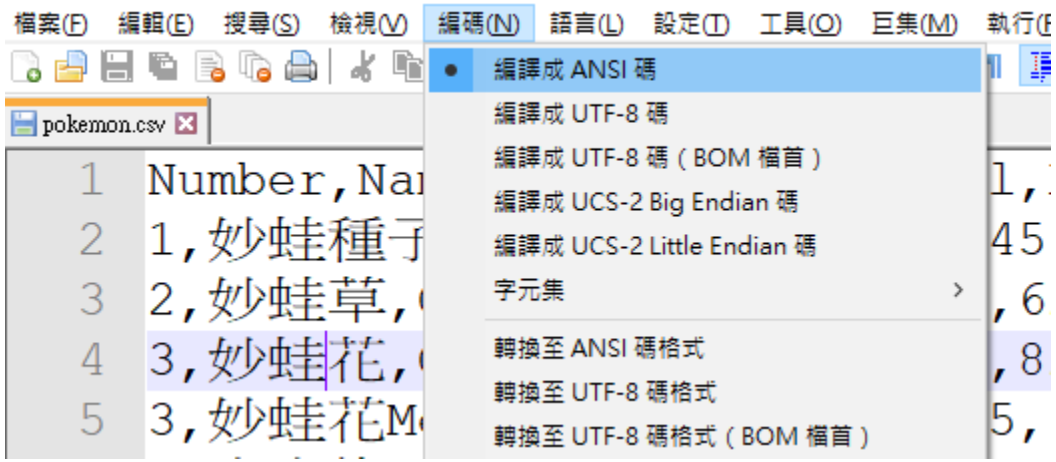
```
你不一定要很厲害，才能開始；你要開始，才能很厲害。
```

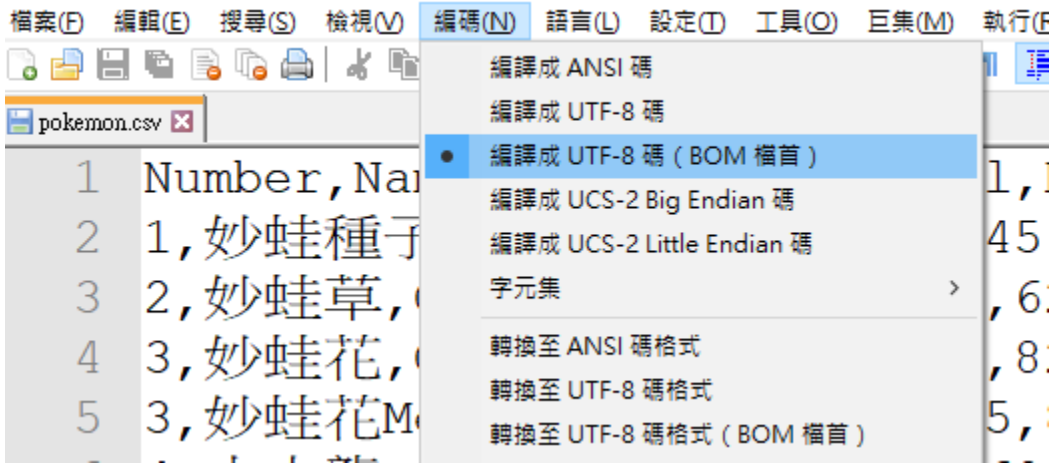
讀取檔案常遇到的問題 ...

```
2 lst = []
3
4 for line in open('pokemon.csv'):
5     data = line.strip().split(',')
6     lst.append(data)
7
8 print(lst[:3])
```

OK

```
UnicodeDecodeError: 'cp950' codec can't decode byte
0x99 in position 95: illegal multibyte sequence
```





主題

- 字串
- 檔案處理
  - 檔案的存取路徑
  - 檔案的運作流程
  - 讀取與寫入資料

檔案操作

- 當我們成功開啟檔案之後，接下來就是對這個檔案進行操作。
  - 最常用的操作是讀取與寫入，其它還有關閉、移動檔案指標等。
  - 除了這些基本操作外，其餘如檔案移動、刪除、檢查是否存在等功能，需要匯入 os 模組才能進行，這部分等後續再來介紹。

檔案操作常用方法

方法	說明
readable()	測試檔案是否可讀取，回傳結果為 True 或 False。
★ read([size])	讀取 size 長度的字元，若省略 size 會直接讀取整個檔案內容。
★ readline([size])	讀取檔案指標所在列的 size 長度字元，若省略 size 會讀取一整列內容，包括換行字元「\n」。
★ readlines([size])	若省略 size 參數，會直接讀取檔案的所有列，且把每列內容置於串列內並回傳。若有設定 size，將只讀取 size 長度的字元到串列內。
★ close()	關閉檔案，將資料寫入磁碟，並釋放佔用的記憶體空間。
flush()	強制將緩衝區的資料寫入檔案中，並清除緩衝區。
writable()	測試檔案是否可寫入，回傳結果為 True 或 False。
★ write(文字字串)	將指定的文字字串寫入檔案中。
tell()	回傳目前檔案指標的位置。
seek(位置編號)	移動檔案指標，位置編號 0（預設值）表示檔案開頭，1 為當前位置，2 為檔案結尾 。
next()	移動到下一列。

測試檔案是否可讀取與可寫入

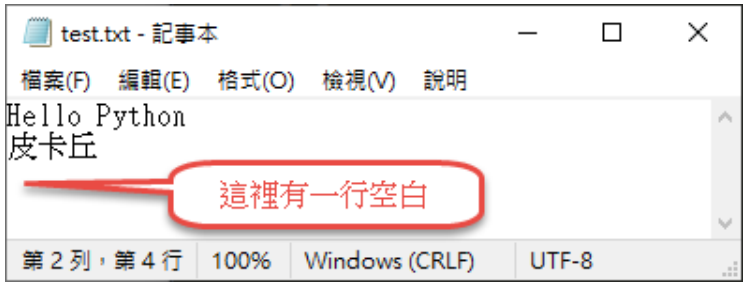
```
1   with open("經典名言", 'r') as file:
2       if file.readable():     print("可讀取")
3       else:                   print("不可讀取")
4
5       if file.writable():     print("可寫入")
6       else:                   print("不可寫入")
```

```
可讀取
不可寫入
```

讀取檔案：read()

- read() 可讀取指定長度的字元，或是整個檔案內容，並以字串型別回傳。

```
1  with open("test.txt", encoding='utf8') as file:
2      data = file.read()
3      print(1, type(data))
4      print(2, data)
5      print('='*10)
6      print(3, file.read(5))
7
8  with open("test.txt", encoding='utf8') as file:
9      print(4, file.read(5))
```

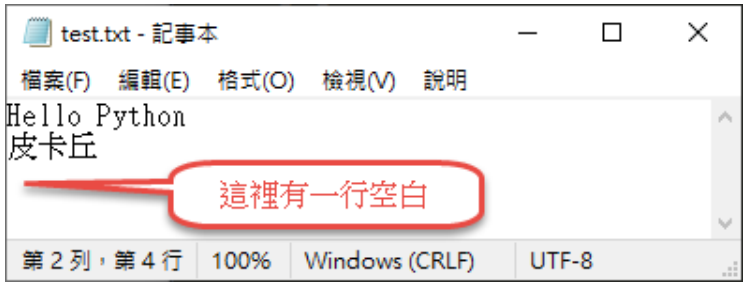



```
1 <class 'str'>
2 Hello Python
皮卡丘

==========
3
4 Hello
```

讀取檔案：readline()

- readline() 用以讀取一整列內容，也是回傳一個字串型別。

```
1   with open("test.txt", encoding='utf8') as file:
2       data = file.readline(3)
3       print(1, type(data))
4       print(2, data)
5       print(3, file.readline())
6       print('='*10)
```

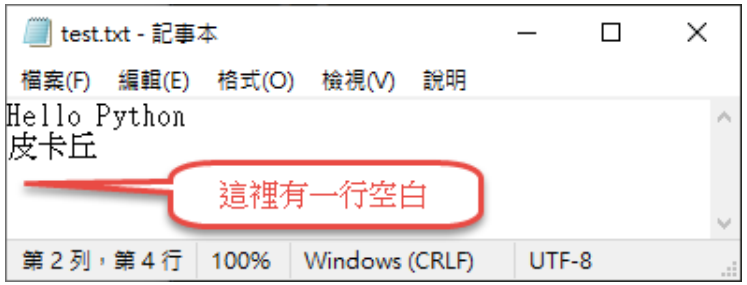



```
1 <class 'str'>
2 Hel
3 lo Python

==========
```

讀取檔案：readlines()

- readlines() 則是讀取每列內容，並回傳一個串列。

```
1   with open("test.txt", encoding='utf8') as file:
2       data = file.readlines()
3       print(1, type(data))
4       print(2, data)
```




```
1 <class 'list'>
2 ['Hello Python\n', '皮卡丘\n']
```

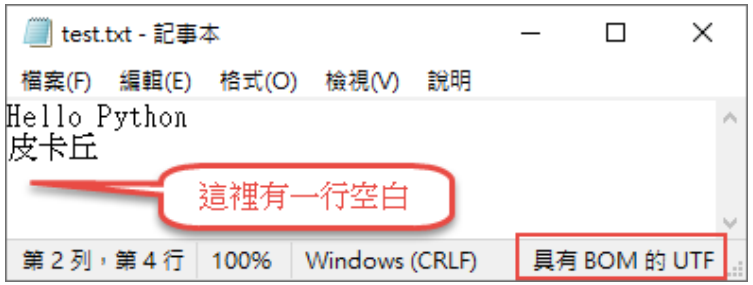
搭配 strip()
去除空白字元

readlines() 沒有讀取
檔案最後一行的空白

BOM 檔首的 UTF-8 編碼(1/2)

- 嘗試讀取 BOM 檔首的 UTF-8 編碼檔案，再嘗試讀取該檔案。
  - 使用 read()、readline() 讀取的檔案內容與之前都一樣，
  - 但透過 readlines() 發現在串列的第一個元素前多了「\ufeff」。

```
1  with open("test.txt", encoding='utf8') as file:
2      data = file.readlines()
3      print(1, type(data))
4      print(2, data)
```

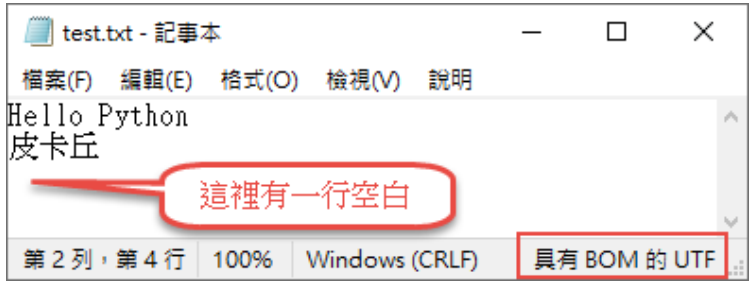



```
1 <class 'list'>
2 ['\ufeffHello Python\n', '皮卡丘\n']
```

BOM 檔首的 UTF-8 編碼(2/2)

- 這就是所謂的 BOM (Byte Order Mark)，用來說明檔案的編碼訊息。
  - 由於 BOM 佔用一個字元，可利用字串處理去除，也可在開啟檔案時指定編碼參數「`encoding = "utf-8-sig"`」。

```
1   with open("test.txt", encoding='utf8') as file:
2       data = file.readlines()
3       print(1, type(data))
4       print(2, data)
```



寫入檔案：write()

```
1   with open("test.txt", 'w') as file:
2       output = "經驗永遠不會過時"
3       file.write(output)
4
5   with open("test.txt") as file:
6       data = file.read()
7       print(data)
```

```
經驗永遠不會過時
```

練習：按下列步驟實作

① 打開一個檔案
- 檔名為「經典名言」，編碼為 utf8。

② 以寫入模式(w)開啟檔案，寫入下列字串。
- "擁有夢想只是一種智力，實現夢想才是一種能力。"

③ 關閉檔案

④ 以附加模式(a)開啟檔案，寫入下列字串後關閉檔案。
- "人生就像騎腳踏車，想保持平衡就得往前走。 - by 愛因斯坦"

練習：出現次數

- 找出檔案內出現次數最多的中文字前 9 名，並輸出該字與其出現次數。
  - 輸入：「三分鐘看懂人工智慧.txt」
  - 輸出：出現次數最多的中文字前 9 名、出現次數。
    - 標點符號、英文、數字不計

本章結束

Py05
綜合應用一

王頌文

主題

- 元組、字典與集合
  - 元組
  - 字典
  - 集合
- 例外處理
  - 例外處理方式
  - 自行拋出例外

元組、字典與集合(1/2)

- 在前面的範例與練習題中，相信已經發現串列好用之處。
  - 除了能容納不同資料型別的元素，也有依靠索引值與迭代的存取方式，更有串列綜合運算以及多個方便的操作函式。
  - 透過串列不僅能輕易撰寫解決問題的程式，而且也能讓程式碼更加簡潔。
  - 儘管串列好用，但其執行速度也比較慢，當處理的資料量一大就很容易感覺到差異。

元組、字典與集合(2/2)

- 同時，在應對某些情況時使用串列需要轉換邏輯思維，並不直覺。

- Python 除了串列，還有三個常見能儲存資料的容器，分別是
  - 元組(tuple)
  - 字典(dictionary)
  - 集合(set)

主題

- 元組、字典與集合
  - 元組
  - 字典
  - 集合
- 例外處理
  - 例外處理方式
  - 自行拋出例外

資料結構(Data Type)

- 串列、元組、字典與集合都是 Python 常用的資料結構，它們各自具有不同的特性和用途。
  - 初學 Python 時大多由串列入手，可是隨著接觸的需求越來越多元，單單只倚賴串列難免會有力不從心的感覺。
  - 畢竟靠一招半式就想闖蕩江湖，有些不切實際了。

元組(Tuple)

- 元組和串列一樣都是有順序的序列結構，但有以下幾點不同：
  - 元組在建立後不能修改元素值
    - 所以元組又稱是「不能修改的串列」
  - 元組使用小括號(())，但串列使用中括號([])。
  - 如果元組內只有一個元素，後方必須加上逗點，多個元素就不用。

元組的用法

- 元組的使用方式是將元素置於小括號中（串列是中括號），元素之間以逗號分隔，語法為：
  - 元組名稱 = (元素1, 元素2, …)

```
1   tup1 = ()          # 建立一個空元組，也可用 tuple()
2   tup2 = ("皮卡丘", "小火龍", "傑尼龜")
3   print(type(tup2))
4   print(tup2[1])
5
6   tup = tup2 + ("可達鴨",)
7   lst = list(tup)  # 利用串列間接修改元組內容
8   lst[0] = "雷丘"
9   tup = tuple(lst)
10  print(tup)
```

```
<class 'tuple'>
小火龍
('雷丘', '小火龍', '傑尼龜', '可達鴨')
```

元組的優點

- 相較於串列，雖然元組在使用上有不少限制，也有些優點。
  - 讀取速度快
  - 資料更安全

  - 此外，Python 提供的內建和第三方函式，其回傳值也大多是元組結構，以確保資料不會被變更。

數學的元組

- 順帶一提，數學上所謂的元組是指有限個元素所組成的有序序列。
  - 若元組有 n 個元素，通常稱為 n 元組(n-tuple)。
  - e.g., (2, 7, 4, 1, 7) 即為5元組。

主題

- 元組、字典與集合
  - 元組
  - 字典
  - 集合
- 例外處理
  - 例外處理方式
  - 自行拋出例外

字典(dict)

- 字典是將元素置於一對大括號({})中，語法為：
  - 字典名稱 = { 鍵1:值1, 鍵2:值2, … }
  - 元素是以「鍵:值」對的方式儲存，可透過以鍵(key)為索引值來存取字典裡對應的值(value)。
  - 「鍵」可以是數字或字串，只要不重複就行。
  - 「值」可以是前面學過的數字、字串、串列，甚至是字典也行。

串列 vs 字典

- 若把串列的索引值看到鍵，那麼串列也可視為一種鍵皆為數字的特殊字典。

- 與串列相比，字典有以下幾點不同：
  - 字典使用大括號，而串列使用中括號。
  - 元素在字典內沒有順序（與設定順序無關），但串列是有序結構。

範例

宣告一個空字典

```python
dct = {}          # 建立一個空字典，也可用 dict()
hp_dct = {"皮卡丘":90, "小火龍":80, "傑尼龜":75}
print(type(hp_dct))
print(hp_dct)
print("小火龍 HP =", hp_dct["小火龍"])

if "傑尼龜" in hp_dct:
    hp_dct["傑尼龜"] = 85     # 修改元素內容
else:
    hp_dct["傑尼龜"] = 85     # 新增元素內容

print("傑尼龜 HP =", hp_dct["傑尼龜"])
```

```
<class 'dict'>
{'皮卡丘': 90, '小火龍': 80, '傑尼龜': 75}
小火龍 HP = 80
傑尼龜 HP = 85
```

難以區分是修改還是新增元素

刪除字典的三種方式

```python
dct = {"皮卡丘":90, "小火龍":80, "傑尼龜":75}

del dct["皮卡丘"]     # 刪除字典內的特定元素
print(dct)

dct.clear()          # 刪除字典內所有元素
print(dct)

del dct              # 刪除字典變數
print(dct)
```

```
{'小火龍': 80, '傑尼龜': 75}
{}
Traceback (most recent call last):
  File "C:\Users\Ian\VS_Code\Untitled-1.py", line 10, in <module>
    print(dct)
NameError: name 'dct' is not defined. Did you mean: 'oct'?
```

取得字典所有鍵(key)

```python
dct = {"皮卡丘":90, "小火龍":80, "傑尼龜":75}

for key in dct:
    print(f"key:{key} -> value:{dct[key]}")
```

```
key:皮卡丘 -> value:90
key:小火龍 -> value:80
key:傑尼龜 -> value:75
```

練習

- 寶可夢訓練家小智、小霞與小剛一同旅行，三人都持有一些寶可夢如下，且每隻寶可夢有一個代表強弱的數值。
- 同一種寶可夢只保留最大數值，請列出保留下來的寶可夢名稱、強弱數值與持有者。
  - 小智={'皮卡丘':90, '小火龍':80, '傑尼龜':75, '菊石獸':35, '小拳石':55 , '太陽珊瑚':25, '可達鴨':60}
  - 小霞={'小火龍':20, '傑尼龜':85, '菊石獸':55, '太陽珊瑚':75 , '可達鴨':65}
  - 小剛={'皮卡丘':60, '小火龍':50, '可達鴨':45, '小拳石':75, '太陽珊瑚':65}

假設同名稱的寶可夢數值皆不相同

字典的操作方法

```
dct = {'A':3, 'B':5}
```

方法	意義	範例	變數 n 的值
len(dct)	取得字典元素個數	n = len(dct)	2
copy()	複製字典	n = dct.copy()	{'A':3, 'B':5}
in / not in	檢查鍵是否在字典內	'X' in dct	False
items()	取出所有元素的「鍵:值」	n = dct.items()	[('A':3), ('B':5)]
keys()	取出所有元素的「鍵」	n = dct.keys()	['A', 'B']
values()	取出所有元素的「值」	n = dct.values()	[3, 5]
pop()	取得「鍵」對應的「值」，並刪除該「鍵:值」。	n = dct.pop('B')	5
get(鍵, 值)	取得「鍵」對應的「值」，若該「鍵」不存在，則回傳參數內的值。	n = dct.get('B', 9)	5
		n = dct.get('X', 9)	9
Setdefault (鍵, 值)	若「鍵」不存在，則以參數的「鍵:值」建立新元素。	dct.setdefault ('A', 9)	{'A':3, 'B':5}
		dct.setdefault ('X', 9)	{'A':3, 'B':5, 'X':9}

範例

```python
dct = {'X':90, 'a':80, 'A':75, 'x':65}

for key, val in dct.items():
    print(f"[{key}]->{val},", end=' ')

print("\n對「鍵」排序:", end=' ')
for key in sorted(dct):
    print(f"[{key}]->{dct[key]},", end=' ')

print("\n對「值」排序:", end=' ')
for key in sorted(dct, key=dct.get):
    print(f"[{key}]->{dct[key]},", end=' ')
```

items() 可在 for 迴圈內逐步取出所有「鍵:值」元素

```
[X]->90, [a]->80, [A]->75, [x]->65,
對「鍵」排序: [A]->75, [X]->90, [a]->80, [x]->65,
對「值」排序: [x]->65, [A]->75, [a]->80, [X]->90,
```

合併多個字典

```python
dct1 = {"皮卡丘":90, "小火龍":80, "傑尼龜":75}
dct2 = {"雷丘":120, "傑尼龜":90}
dct1.update(dct2)
print(dct1)

dct3 = {"皮卡丘":10, "小火龍":20}
# 「**」會將字典拆解為元素，再由大括號組合，需注意相同「鍵」的覆蓋
dct = {**dct1, **dct2, **dct3}
print(dct)
```

```
{'皮卡丘': 90, '小火龍': 80, '傑尼龜': 90, '雷丘': 120}
{'皮卡丘': 10, '小火龍': 20, '傑尼龜': 90, '雷丘': 120}
```

練習

- 找出文句內出現次數最多的中文字，並輸出該字與其出現次數。
  - 輸入：底下的中文字句
  - 輸出：出現次數最多的中文字、出現次數
    - 改用字典撰寫

```
text = \
"昔汝雖拒吾於千里之外，傷吾心，然吾怪之罪何有於汝乎？\
反，吾思與日俱增，吾情與時俱進。於汝，情痴一片，思濃于海，\
山不可動，海不可淹，火不可滅。愛慕之情，時時加之，\
豈有他人可吾及乎？"
```

主題

- 元組、字典與集合
  - 元組
  - 字典
  - 集合
- 例外處理
  - 例外處理方式
  - 自行拋出例外

集合(set)

- 集合是無序、元素不重複的群集，語法如下：
  - 集合名稱 = {元素1，元素2，…}
    - 集合以大括號標示，元素可以是不同資料型別，且以逗點隔開。

  - Python 定義的集合在概念上就比較接近數學的集合，同樣是包含一些沒有順序且不重複的元素，而且也有類似的集合運算，如交集、聯集、差集等。

範例

```
1    st = set()          # 建立一個空集合
2
3    st = {1, 3, 5, 7, 9}
4    print(len(st))
5    print(max(st))
6    print(sum(st))
```

```
5
9
25
```

集合的基本操作方法

```
st = {1, 3, 5}
```

方法	意義	範例	集合 st 的值
add(x)	新增元素 x 到集合內	st.add(9)	{1, 3, 5, 9}
remove(x)	從集合中移除元素 x，若該元素不存在，會引發錯誤。	st.remove(1)	{3, 5}
pop()	由集合內取出一個元素並回傳	n = st.pop()	{3, 5, 9}
copy()	複製集合	st1 = st.copy()	{1, 3, 5}
clear()	移除集合所有元素	st.clear()	set()

集合常見操作

```
S1 = {1, 3, 5}
S2 = {5, 7, 9}
```

方法	意義	範例	運算結果
in / not in	檢查元素是否在集合內	5 in S1	True
issubset(S)	檢查集合是否為 S 的子集合	S1.issubset({1})	False
isssuperset(S)	檢查集合是否為 S 的超集合	S1.issuperset({1})	True
isdisjoint(S)	檢查集合與 S 是否沒有相同元素	S1.isdisjoint({1})	False
union(S)	將集合與 S 進行聯集（也可用運算子 |）	S1.union(S2)	{1, 3, 5, 7, 9}
		S1 | S2	{1, 3, 5, 7, 9}
intersection(S)	將集合與 S 進行交集（也可用運算子 &）	S1.intersection(S2)	{5}
		S1 & S2	{5}
difference(S)	將集合與 S 進行差集（也可用運算子 -）	S1.difference(S2)	{1, 3}
		S1 - S2	{1, 3}
symmetric_ difference(S)	將集合與 S 進行對稱差集（也可用運算子 ^）	S1 ^ S2	{1, 3, 7, 9}

集合應用

- 串列可透過索引值存取元素，但集合不行，可仍然能透過 for 迴圈逐一拜訪集合元素。

- 此外，利用集合內元素不能重複的性質，能協助快速取出不重複資料。

```
1   data = [11, 22, 33, 44, 11, 22, 33, 11, 33, 44]
2
3   st = set(data)
4   for x in st:
5       print(x, end=',')
```

```
33,11,44,22,
```

練習：全字母句

HINT: 可先將所有英文字母放在集合內，再以超集合 `issuperset()` 判斷之。

- 全字母句(pangram)是英文字母表所有的字母都出現至少一次（最好只出現一次）且言之成義的句子。
- 請撰寫一程式，讓使用者輸入一個英文句子，並判斷該句子是否為全字母句。
  - 範例輸入

範例輸入	範例輸出
`請輸入：The quick brown fox jumps over the lazy dog.`	`True`
`請輸入：The jay, pig, fox, zebra and my wolves quack!`	`True`
`請輸入：Sex-charged fop blew my junk TV.`	`False`

主題

- 元組、字典與集合
  - 元組
  - 字典
  - 集合
- 例外處理
  - 例外處理方式
  - 自行拋出例外

例外(Exception)

- 在程式執行的過程中，難免有機會發生錯誤。
  - 原因可能是使用者的輸入不合法、檔案不存在、資料型別不符或甚至是網路斷線等。
  - 常看到的例外訊息有「`TypeError`」、「`IndexError`」、「`FileNotFoundError`」等
  - 在程式執行期間發生錯誤時會引發例外，若沒有規劃好相應的處理機制，程式很可能因此中斷或發生不可預期的情況。

例外處理(Exception Handling)

- 當不小心輸入錯誤時，會希望程式直接中斷，亦或是提醒輸入有誤可再重新輸入呢？
  - 若客戶直接看到系統給的錯誤訊息，對產品品質的信心會大打折扣。

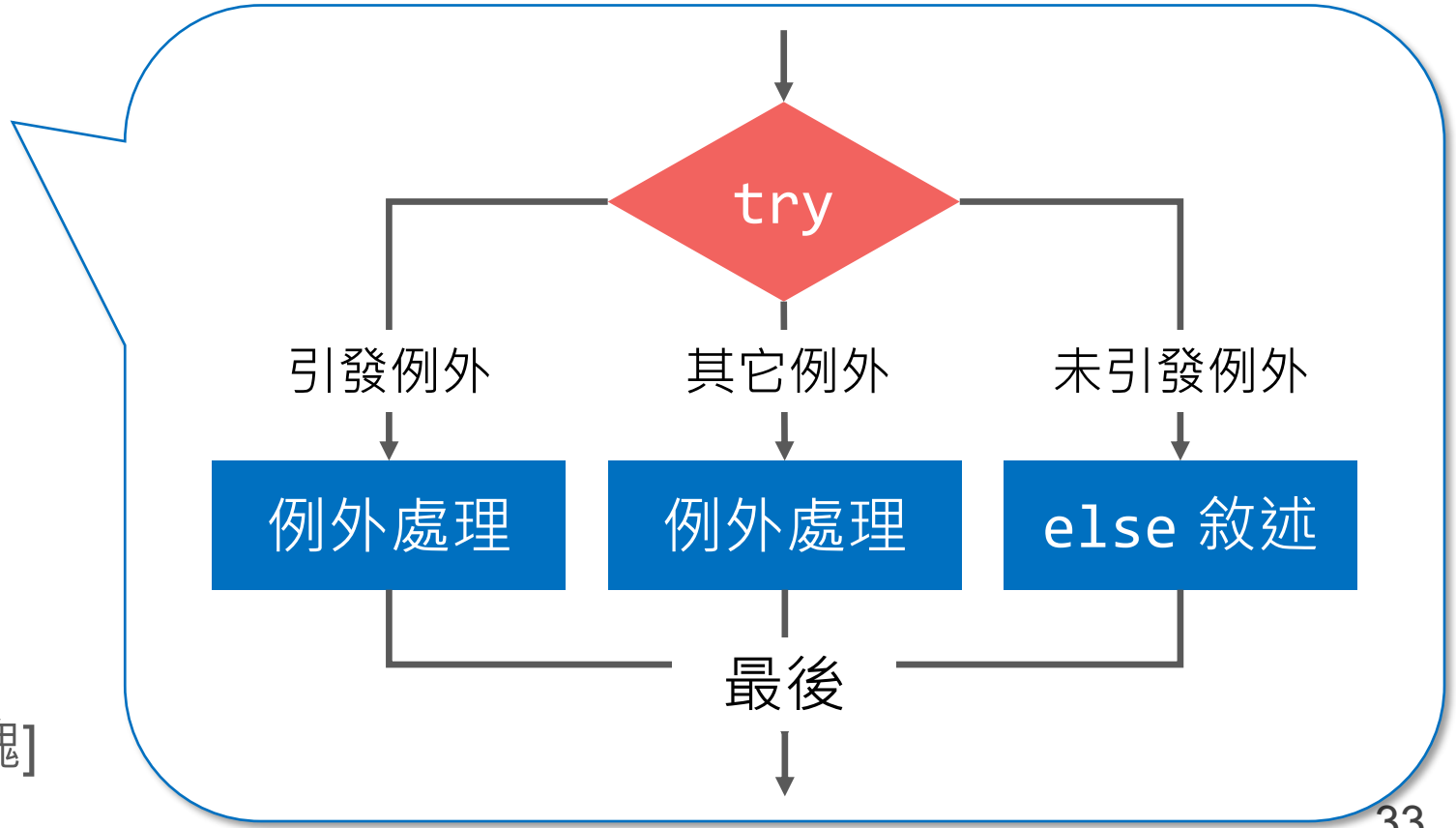
- 為了避免上述窘境，在程式開發過程中就要考慮到當錯誤發生時，程式要給出哪些適當的反應，並據此撰寫例外處理。
  - 除了不讓程式輕易中斷外，也能友善地提醒並引導使用者進行操作。

主題

- 元組、字典與集合
  - 元組
  - 字典
  - 集合
- 例外處理
  - 例外處理方式
  - 自行拋出例外

捕捉 + 處理例外(1/2)

- 使用 try…except 敘述來捕捉與處理例外，語法如下：
  - try:
    - 執行測試的程式區塊
  - [except 例外名稱:
    - 處理發生指定的例外]
  - except:
    - 處理發生其它例外
  - [else:
    - 沒有發生例外]
  - [finally:
    - 一定會執行的程式區塊]



捕捉 + 處理例外(2/2)

- 使用 try…except 敘述來捕捉與處理例外，語法如下：
  - try:
    - 執行測試的程式區塊
  - [except 例外名稱:
    - 處理發生指定的例外]
  - except:
    - 處理發生其它例外
  - [else:
    - 沒有發生例外]
  - [finally:
    - 一定會執行的程式區塊]

try 敘述：將可能發生錯誤的程式碼放在 try 敘述的程式區塊內。

except 敘述：用來捕捉指定的例外，一旦有捕捉到就執行對應的程式區塊。若有指定的例外，需要寫在前面，省略不寫的話代表所有的例外類型都要捕捉。

else 敘述：當 try 敘述的程式區塊沒有引發錯誤，會接著執行 else 敘述部分。

finally 敘述：無論是否有發生例外，都會執行這個敘述的程式區塊，通常是用來清除錯誤或是收尾。

範例

```python
try:
    a = int(input("請輸入整數："))
    b = int(input("請輸入整數："))
    print("總和為", a + b)
except Exception as e:
    print("發生錯誤為", e)
else:
    print("程式執行沒有發生錯誤")
finally:
    print("離開程式 ...")
```

捕捉例外發生時的錯誤訊息

```
請輸入整數：3.14
發生錯誤為 invalid literal for int() with base 10: '3.14'
離開程式 ...
```

例外處理

- 在合適的地方撰寫例外處理
  - 對程式設計者來說可協助除錯，也讓程式不會因為錯誤而中斷執行。
  - 對使用者而言，不會因為程式錯誤而不知所措。
- 常見需要例外處理的狀況是程式需要與外部溝通的時候
  - 比方存取檔案、和資料庫溝通、透過網路傳遞或接受資料等
  - 此時有可能程式本身沒問題，可是外部卻發生意外，例如檔案不存在、網路連線中斷等，導致系統拋出例外而中斷程式，這時就能藉由例外處理來排除障礙或是顯示錯誤訊息。

主題

- 元組、字典與集合
  - 元組
  - 字典
  - 集合
- 例外處理
  - 例外處理方式
  - 自行拋出例外

raise 敘述拋出例外

- 除了系統因程式執行錯誤而拋出例外，我們也能透過 raise 敘述自行拋出指定的例外。
  - 常用在 try 敘述內需要強制中斷執行的時候，例如：

```
try:
    a = int(input("請輸入0 ~ 9的數字:"))
    if (a < 0) or (9 < a):
        raise ValueError("輸入不合法數字")
    print(a)
except ValueError as err:
    print(err)
else:
    print("輸入合法數字")
```

```
請輸入0 ~ 9的數字:111
輸入不合法數字
```

assert 敘述拋出例外

- 還有一個用法與 raise 類似的 assert 敘述，同樣都是執行後就會拋出例外，並傳遞錯誤資訊。
  - 不同之處在於 assert 是在條件測試為假時才引發例外，例如：

```
1   try:
2       lst = []
3       assert len(lst), "空串列"
4       print(lst)
5   except AssertionError as err:
6       print(err)
7   else:
8       print("不是空串列")
```

```
空串列
```

本章結束

Py06
綜合應用二

王頌文

主題

- 模組與套件
  - 匯入模組
  - 使用套件
- 再探函式
  - 參數傳遞方式
  - 匿名函式 lambda

模組(Module)、套件(Package)

- 在我們對程式語言還不嫻熟，開發經驗也還不足的時候，一旦遇到稍微複雜或是大型應用程式的開發需求時，難免手忙腳亂。
  - 好不容易將問題拆解成數個待實現的功能，卻又不知從何做起。

- Python 最為人津津樂道的特色之一就是大量功能覆蓋眾多領域的模組與套件，能滿足各式各樣應用的開發需求。
  - 將這些模組與套件想像成許多積木，可以快速取用、組合以實現各種功能，不僅增加開發效能，也讓程式碼更容易維護。

主題

- 模組與套件
  - 匯入模組
  - 使用套件
- 再探函式
  - 參數傳遞方式
  - 匿名函式 lambda

匯入模組(1/2)

- 事實上，模組也是一個 Python 程式的檔案，裡面定義了一些資料、函式與類別。
  - 要使用模組所提供的功能時，得先匯入(import)該模組。
  - 套件可以想成是一堆模組的集合，能提供更全面的功能，使用前也一樣要先匯入。

匯入模組(2/2)

- 匯入模組與套件的方式一樣，語法為：
  - `import` 模組或套件名稱

```
1   import random          # 匯入亂數模組
2
3   print(random.random())          # 在0~1間隨機取一個浮點數
4   print(random.randint(1, 10))    # 在1~10間隨機取一個整數
5   print(random.uniform(1, 10))    # 在1~10間隨機取一個浮點數
```

```
0.8937309924822843
5
8.402913655123353
```

簡便的匯入方式(1/2)

- 模組內通常有許多函式可用，在匯入之後就能透過內建函式查詢。
  - `dir()` 列出模組內所有函式
  - `help()` 查閱更詳細的使用說明

- 有些模組的名稱很長，每次使用時都要輸入名稱也很麻煩，也有有比較簡便的使用方式，例如：
  - 單獨匯入模組中要使用的函式名稱
  - 或是透過萬用字元「*」直接匯入模組的所有函式

不推薦這種 all in 的匯入方式，因為容易與其他模組同名稱的函式相衝突。

簡便的匯入方式(2/2)

- 這種 `from…import` 的用法雖然免去使用模組函式時要先輸入模組名稱的麻煩，
  - 但也需要對該模組有相當程度的了解，
  - 且使用時因為沒有標註模組名稱，也容易誤會或與自己撰寫的同名稱函式相混在一起。

  - `from random import randint, uniform`
  - `from random import *`

範例

```python
import random as rd

print(rd.random())
print(rd.choice([1, 2, 3, 4, 5]))      # 隨機挑1個號碼
print(rd.sample([1, 2, 3, 4, 5], 3))  # 隨機挑3個不重複號碼
```

```
0.36725747976288462
[1, 5, 3]
```

暴力窮舉(1/2)

- 再舉一個實用的模組為例，有些問題可以透過暴力窮舉所有可能性，再逐一檢驗排查。
  - 例如只使用 1 ~ 4 個數字組合成兩位數，且個位數要大於十位數。
  - 這類排列組合問題可透過兩層巢狀迴圈（因為是兩位數）列舉所有可能性並檢驗之，可是若題目再複雜些，使用多層巢狀迴圈容易降低可讀性與執行效率。

暴力窮舉(2/2)

- Python 有一個內建模組 `itertools` 可輕鬆產生所有排列組合

```python
import itertools as it

for x in it.permutations(range(1,5), 2):  # 窮舉所有排列
    if x[0] < x[1]:
        print(x[0]*10 + x[1], end=' ')

print("\n === 底下是所有組合 ===")
print(list(it.combinations(range(1,5), 2)))
```

```
12 13 14 23 24 34
 === 底下是所有組合 ===
[(1, 2), (1, 3), (1, 4), (2, 3), (2, 4), (3, 4)]
```

自己建立模組

- 此外，既然模組也是 Python 檔案，除了使用內建與第三方模組外，也能自己建立模組打包共用的程式，再由其他程式匯入使用。
  - 此時，匯入模組使用的目錄底下會多一個名為「__pycache__」的資料夾，內有「.pyc」檔案。
  - 這是 Python 透過解譯器(interpreter)對程式碼逐行翻譯成位元組碼(byte code)的結果，有助於提供更靈活架構與跨平台能力，也能提升載入模組的速度。

練習：找出五位數

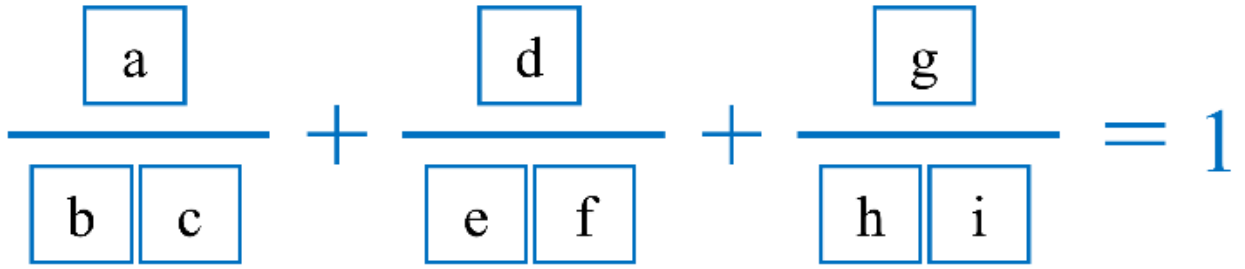
- 有一個五位數「ABCDE」，其限制條件如下，請撰寫一程式找出符合要求的所有五位數。
  - A、B、C、D、E分別代表五個不同數字
  - A > E，B > D
  - 「ABCDE」+「EDCBA」 = 163535

  - 範例輸出

```
98746
```

練習：填入九個數字

- 請撰寫一程式，在底下 a ~ i 標示的九個空格填入不重複數字 1 ~ 9，使得三個分數相加的和為 1。



- 範例輸出

```
(5, 3, 4, 7, 6, 8, 9, 1, 2)
(5, 3, 4, 9, 1, 2, 7, 6, 8)
(7, 6, 8, 5, 3, 4, 9, 1, 2)
(7, 6, 8, 9, 1, 2, 5, 3, 4)
(9, 1, 2, 5, 3, 4, 7, 6, 8)
(9, 1, 2, 7, 6, 8, 5, 3, 4)
```

主題

- 模組與套件
  - 匯入模組
  - 使用套件
- 再探函式
  - 參數傳遞方式
  - 匿名函式 lambda

套件(Package)

- 若把模組比喻成工具，套件就像工具箱一樣，裡面擺放數個模組。
  - 原則上，只要該目錄底下有包含「__init__.py」檔案，就會被當作 Python 的套件。
  - 相較於內建模組與套件會在安裝 Python 時也一併安裝，第三方套件(third-party package)則需要手動安裝後才能匯入使用。

安裝套件

- 常見是在命令提示字元的視窗內，透過 pip 指令進行安裝，例如：
  - pip list　　# 列出目前安裝的套件與版本
  - pip show 套件名稱　　# 查詢已經安裝的套件資訊
  - pip install 套件名稱　　# 安裝指定套件
  - pip uninstall 套件名稱　# 移除指定套件

  - 當透過 pip 指令來安裝套件時，預設是從 PyPI 網站下載套件。
    - PyPI(the Python Package Index)是官方的第三方套件儲存庫，類似 Google Play 和 App Store 般。

常見套件

- 迄今為止，PyPI 已收錄了近五十萬個專案，涵蓋眾多應用領域，在開發過程中所需要的功能，幾乎都能在這裡找到適合的套件。
- 以下列出幾個常見的第三方套件與其功能：
  - NumPy：提供陣列與平行處理能力，許多重量級套件的基礎。
  - SciPy：基於numpy開發而成，主要用於科學計算。
  - Pandas：進行數據處理，能大幅簡化複雜的數據操作過程。
  - PyGame：多媒體與遊戲開發
  - Statsmodels：統計分析
  - Scikit-learn：機器學習
  - TensorFlows、PyTorch：兩個主流的深度學習(deep learning)框架。

NumPy

- NumPy 提供陣列與大量操作陣列的函式，同時也能處理傅立葉轉換、線性代數、多項式等科學計算。
  - 陣列(array)是一種能存放多筆資料的結構，與 Python 串列類似的地方是存放在陣列裡的資料也稱為元素(element)，且一樣是透過索引值(index)來存取元素。
  - 然而，最明顯的差異在於陣列只能存放同型別的元素，Python 的串列則沒有這個限制。

ndarray

- NumPy 提供的陣列型別稱為 ndarray (n-dimensional array)，其建立方式通常是使用 `array()` 方法轉換串列而來。
- ndarray 三個最重要的屬性如下：
  - ndarray.ndim：陣列的維度，例如 2 代表是二維陣列。
  - ndarray.shape：陣列的形狀，例如 (3, 2) 意謂第一維度有 3 個元素，而第二維度有 2 個元素。
  - ndarray.dtype：陣列的元素型別，可以是 Python 內建的 `int`、`bool`、`str` 等型別，也可以是 NumPy 提供的 `int32`、`float64` 等型別。

範例

```python
import numpy as np

arr = np.array([1, 2, 3, 4, 5])
print(arr)
print("ndim =", arr.ndim)
print("shape =", arr.shape)
print("dtype =", arr.dtype)

print(np.arange(0, 2, 0.3))
print(np.linspace(0, 2, 5))
```

```
[1 2 3 4 5]
ndim = 1
shape = (5,)
dtype = int32
[0.  0.3 0.6 0.9 1.2 1.5 1.8]
[0.  0.5 1.  1.5 2. ]
```

用法類似 range()

在範圍內均勻產生的數量點

遮罩(Mask)

- 陣列可搭配條件式建立所謂的遮罩，可用來選取陣列內的目標元素。

```python
import numpy as np

score = np.array([60, 11, 33, 70, 22, 80])
print("原始分數", score)
print("遮罩", score < 60)

score[score < 60] += 10     # 只選出低於60分的來加分
print("修改分數", score)
```

```
原始分數 [60 11 33 70 22 80]
遮罩 [False  True  True False  True False]
修改分數 [60 21 43 70 32 80]
```

二維陣列(Two-Dimensional Array)

- 二維陣列的操作方式與二維串列相仿
  - 一樣是透過列索引與行索引來存取元素
  - 建立方式可由二維串列轉換而來
  - 也可用 reshape() 方法改變一維陣列的形狀而成

```
1    import numpy as np
2
3    mat = np.array([[1, 2, 3], [4, 5, 6]])
4    print(mat)
5    print('='*10)
6    mat = np.array(range(1, 7)).reshape(2, 3)
7    print(mat)
```

```
[[1 2 3]
 [4 5 6]]
==========
[[1 2 3]
 [4 5 6]]
```

矩陣(Matrix)

- 矩陣以二維陣列來表示後，就能進行一些矩陣基本運算。

```python
import numpy as np

mat_A = np.array(range(1, 7)).reshape(2, 3)
mat_add = mat_A + mat_A     # 矩陣對應項相加
print(mat_add)
mat_mul = mat_A * mat_A     # 矩陣對應項相乘
print(mat_mul)
```

```
[[ 2  4  6]
 [ 8 10 12]]
[[ 1  4  9]
 [16 25 36]]
```

矩陣相乘

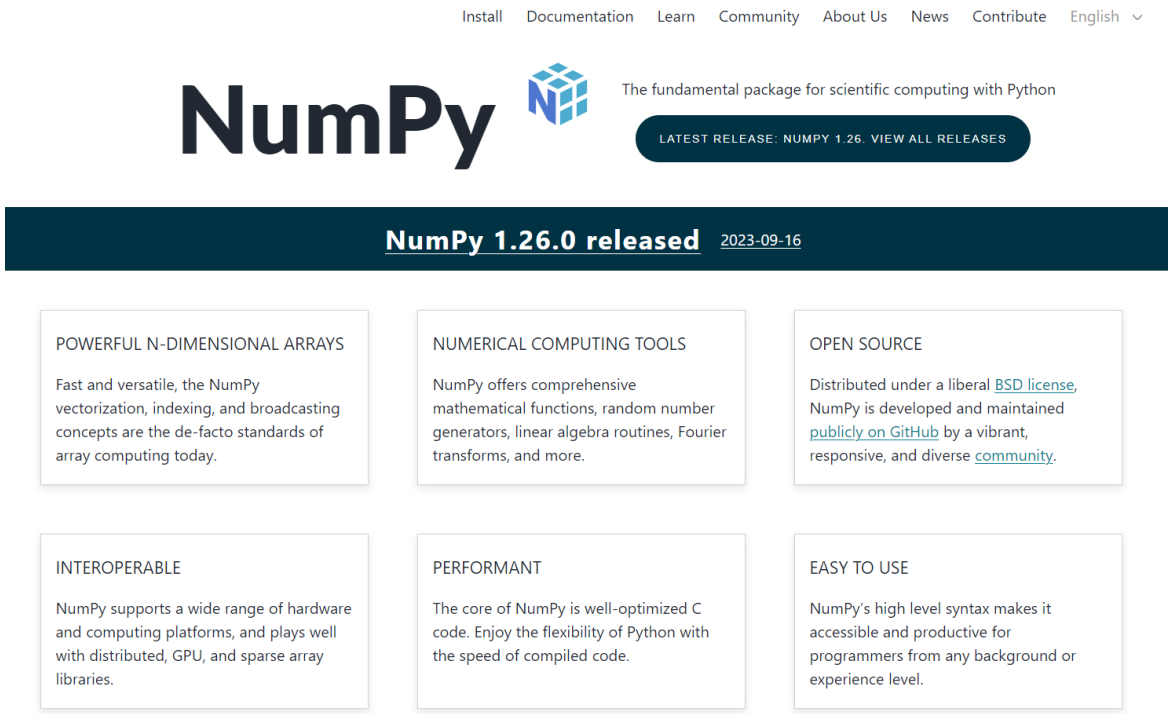
- 若是想得到線性代數裡的矩陣相乘結果，可使用 dot() 方法。
  - 若因為矩陣形狀而無法相乘，會得到「ValueError」錯誤訊息。
  - 因此，執行矩陣相乘的程式碼也常被放在 try 敘述內，以便於在引發錯誤時能捕捉到系統拋出的例外。

```
1   import numpy as np
2
3   try:
4       mat_A = np.array(range(1, 7)).reshape(2, 3)
5       mat_B = np.array(range(1, 7)).reshape(3, 2)
6       print(np.dot(mat_A, mat_B))
7       print(np.dot(mat_A, mat_A))
8   except:
9       print("矩陣無法相乘")
```

```
[[22 28]
 [49 64]]
矩陣無法相乘
```

NumPy

- 做為許多重量級應用（如數據分析、機器學習、深度學習等）的基礎套件，NumPy 不僅支援高維度的陣列與矩陣運算，也具備大量的數學與統計函式庫以及平行處理的能力。
  - 可到 NumPy 官網 (https://numpy.org/)查閱更多功能。



主題

- 模組與套件
  - 匯入模組
  - 使用套件
- 再探函式
  - 參數傳遞方式
  - 匿名函式 lambda

函式

- 函式能協助將重複或有特別定義的程式片段，拆開成容易管理且獨立的程式片段。
  - 有提高程式可讀性、容易除錯與維護等優點。
  - 採用這種模組化開發也有利於團隊分工

- 因此，底下繼續介紹 Python 函式的參數傳遞模式與匿名函式。

函式的參數傳遞方式

- 已經學過如何自訂函式、使用內建函式與第三方套件等技巧；而在呼叫函式執行任務時，能將資料傳遞到函式的參數，這個傳遞方式一般有兩種：
  - 傳值(pass-by-value)與傳址(pass-by-reference)
  - Python 採用所謂的 pass-by-assignment，這相當於傳值與傳址的一種綜合。
  - 此外，Python 不允許程式設計者自行選擇參數傳遞方式，而是利用所傳遞的引數是屬於可變或不可變物件來判斷。

傳遞不可變物件

- 傳值方式在呼叫函式時，會將引數的值逐一複製給函式的參數。
  - 因此，在函式內變更參數值不會影響原來的引數。

- 當要傳遞的引數屬於不可改變內容的物件(immutable object)時，如數值、字串與元組，Python 會採取傳值呼叫。

範例

```
1  def swap(x, y):
2      print(f"函式內交換前: x={x}, y={y}")
3      x, y = y, x
4      print(f"函式內交換後: x={x}, y={y}")
5
6  a, b = 1, 99
7  print(f"函式外交換前: a={a}, b={b}")
8  swap(a, b)
9  print(f"函式外交換後: a={a}, b={b}")
```

```
函式外交換前: a=1, b=99
函式內交換前: x=1, y=99
函式內交換後: x=99, y=1
函式外交換後: a=1, b=99
```

在函式內修改參數值（第 3 行），不會變更原來的引數值（第 9 行）。

傳遞可變物件

- 若要傳遞的引數是可改變內容的物件(mutable object)時，如串列、字典與集合，此時 Python 會採用傳址呼叫。
  - 意謂著在呼叫函式時，系統沒有額外分配記憶體空間給函式的參數，而是直接將引數的位址傳遞給對應參數。
  - 這麼一來，因為佔用同一個記憶體位址的緣故，在函式內修改參數值會連帶影響原來的引數。

範例

```
1  def swap(num):
2      print(f"函式內交換前：{num}")
3      num[0], num[1] = num[1], num[0]
4      print(f"函式內交換後：{num}")
5
6  num = [1, 99]
7  print(f"函式外交換前：{num}")
8  swap(num)
9  print(f"函式外交換後：{num}")
```

```
函式外交換前：[1, 99]
函式內交換前：[1, 99]
函式內交換後：[99, 1]
函式外交換後：[99, 1]
```

在函式內修改參數值（第 3 行），第 9 行的輸出可以看到串列的內容也跟著改變。

範例

- 雖然傳遞的是可變內容物件，但若是在函式內進行重新指派的動作，系統將會指派新的記憶體位址，也就不會影響到原來的引數。

```
1    def swap(num):
2        print(f"函式內交換前：{num}")
3        num = [100, 200]
4        print(f"函式內交換後：{num}")
5
6    num = [1, 99]
7    print(f"函式外交換前：{num}")
8    swap(num)
9    print(f"函式外交換後：{num}")
```

```
函式外交換前：[1, 99]
函式內交換前：[1, 99]
函式內交換後：[100, 200]
函式外交換後：[1, 99]
```

Python 的參數傳遞方式

- 由於 Python 不允許我們自行選擇參數傳遞方式，而是以傳遞的引數是屬於可變或不可變物件來判斷。

- 因此，在使用函式時要多加小心，以避免在函式內的動作改變了原始引數值卻不自知。

主題

- 模組與套件
  - 匯入模組
  - 使用套件
- 再探函式
  - 參數傳遞方式
  - 匿名函式 `lambda`

匿名函式

- lambda 是只有一行的函式，用來取代一些小型函式。
  - 主要目的是簡化程式，不用為了一小段程式碼而建立具名的函式。

- 因此，lambda 也稱為匿名函式(anonymous function)，與一般函式相比有以下特性：
  - 匿名函式不用定義名稱，但一般函式需要。
  - 匿名函式只有一行運算式，而一般函式可以有多行。
  - 匿名函式執行完後自動回傳結果，但一般函式透過關鍵字 `return` 才能回傳。

lambda 的語法

- lambda 參數序列: 運算式

```
1    def hello(word):
2        print(word)
3
4    hello("Hello! Python")
5
6    (lambda word: print(word))("Hello! Python")
```

```
Hello! Python
Hello! Python
```

lambda 函式：多個參數

- lambda 函式可有多個參數，只要參數間同樣以逗點(,)隔開即可。
  - 計算完畢後就直接回傳結果，不用額外撰寫 return 敘述。

```
1   def compute(x, y):
2       return x + y
3
4   a = compute(2, 3)
5   print(a)
6
7   b = (lambda x, y: x + y)(2, 3)
8   print(b)
```

```
5
5
```

lambda 函式：指派給變數

- 有趣的是，我們也可將 lambda 函式指派給一個變數作為其名稱，使用時就像呼叫函式一樣。
  - 只是 lambda 函式僅能撰寫一行，所以只適用於實現小功能。

```
1    def compute(x, y):
2        return x + y
3
4    print(compute(2, 3))
5
6    my = lambda x, y: x + y
7    print(my(2, 3))
```

```
5
5
```

lambda 函式：搭配條件式與迴圈

- lambda 函式也能搭配 if 條件式與 for 迴圈，產生更多作用。

```python
def compute(n):
    if n < 10:
        return list(range(5))
    else:
        return 100

print(compute(5))

my = lambda n: list(range(5)) if n<10 else 100
print(my(20))
```

```
[0, 1, 2, 3, 4]
100
```

lambda 函式：比大小

- 還記得在介紹串列時提到的排序方法 sorted()
  - 在排序二維串列的時候，會依序比較每個元素串列的第一個元素，若相同再比較下一個，依此類推。
  - 底下範例(第2行)經過這個既定的排序方式，最終輸出排序結果。

```
1   lst = [[3, 2], [3, 5], [1, 8], [2, 4], [1, 2], [2, 9]]
2   print(sorted(lst))
3
4   print("--底下先比較第2個元素 --")
5   print(sorted(lst, key = lambda x:x[1]))
```

想先比第二個元素，若相同再比較第一個元素，可用 lambda 實現。

```
[[1, 2], [1, 8], [2, 4], [2, 9], [3, 2], [3, 5]]
--底下先比較第2個元素 --
[[3, 2], [1, 2], [2, 4], [3, 5], [1, 8], [2, 9]]
```

lambda 函式：搭配內建函式

- 搭配內建函式 filter() 與 map() 也能讓 lambda 達成更多任務，而且程式碼也相當簡潔。

```
1   scores = [x for x in range(5, 100, 10)]
2   print(scores)
3
4   scores_adj = list(map(lambda x:int(10*x**0.5), scores))
5   print("調整分數：", scores_adj)
6
7   pass_scores = list(filter(lambda x: x>=60, scores_adj))
8   print("及格分數：", pass_scores)
```

```
[5, 15, 25, 35, 45, 55, 65, 75, 85, 95]
調整分數： [22, 38, 50, 59, 67, 74, 80, 86, 92, 97]
及格分數： [67, 74, 80, 86, 92, 97]
```

map()、filter() 函式在作用完畢後會回傳一個序列(sequence)，若想查看其內容可利用 list() 先轉換成串列型態。

兩個函式均將各自的 lambda 套用在給定串列的每一個元素，以此達到調整分數與挑出及格分數的結果。

本章結束

Py07
綜合應用三

王頌文

主題

- 目錄管理
  - 檔案搜尋：glob 模組
  - 檔案操作：os 模組
- 堆疊與佇列結構
  - 堆疊
  - 佇列

處理多個檔案

- 資料儲存在檔案內，比較能夠長久地保存。
  - 我們也知道如何進行開啟、讀取、寫入以及關閉檔案等操作。
  - 這樣已經能妥善地處理單一檔案，可是對於處理一個目錄下的多個檔案，目前除了手動一個個利用程式處理外，似乎也沒有比較方便的做法。
  - 然而，許多實務應用的第一步大多是從眾多檔案裡讀取資料，這裡涉及目錄管理與檔案操作的各種技巧，能將整個操作過程自動化。

Python 管理目錄

- 所幸 Python 有一些內建模組能協助管理目錄，且配合各種篩選方式能輕易取得符合條件的檔案列表，接著方能逐一處理目標檔案。

- 底下介紹 Python 提供的 `glob` 與 `os` 兩個實用模組，以便於操作檔案與目錄。

主題

- 目錄管理
  - 檔案搜尋：glob 模組
  - 檔案操作：os 模組
- 堆疊與佇列結構
  - 堆疊
  - 佇列

glob 模組

- 若想快速地取得指定條件的檔案名稱，可使用 glob 模組來進行。
  - 這是一個簡單的檔案搜尋模組，除了能明確指定檔案名稱外，也可以搭配正規表示法(regular expression)來描述指定條件。
  - 比方說用萬用字元「*」代表所有可能性
    - photo.* → 代表所有檔案名稱為 photo 的檔案，不管副檔名

範例

資料夾內容

名稱	修改日期	類型
1.csv	2023/11/8 下午 03:41	Microsoft Excel ...
2.csv	2023/11/8 下午 03:41	Microsoft Excel ...
3.csv	2023/11/8 下午 03:41	Microsoft Excel ...
FBAFC0001-04.png	2023/11/8 下午 03:41	PNG 檔案
FBAFC0002-05.png	2023/11/8 下午 03:41	PNG 檔案
FBAFC0003-05.jpg	2023/11/8 下午 03:41	JPG 檔案
FBAFC0005-11.jpg	2023/11/8 下午 03:41	JPG 檔案
glob模組.py	2023/11/8 下午 03:47	Python 來源檔案

```python
from glob import glob

files = glob('*.csv') + glob('*05.*')
print(type(files))

for f in files:
    print(f)
```

所有副檔名為 csv 的檔案

檔名結尾為 05 的所有檔案

執行結果

```
<class 'list'>
1.csv
2.csv
3.csv
FBAFC0002-05.png
FBAFC0003-05.jpg
```

主題

- 目錄管理
  - 檔案搜尋：glob 模組
  - 檔案操作：os 模組
- 堆疊與佇列結構
  - 堆疊
  - 佇列

os 模組

- 如果要處理的目標檔案不是單純放置在單一資料夾內，而分散在許多資料夾，甚至是涉及數個不同目錄，可以考慮使用 os 模組。
  - 這個模組提供取得工作目錄、目錄與檔案操作、執行作業系統命令等，功能包山包海，但使用起來也較繁瑣。

範例

```python
import os

print(os.getcwd())          # 取得目前的工作目錄

file = "myFile.txt"
if os.path.exists(file):    # 檢查檔案是否存在
    os.remove(file)         # 移除指定檔案
else:
    print(file + "檔案不存在")

os.system("cls")            # 執行作業系統命令，清除螢幕
os.system("mkdir myDir")    # 建立 myDir 目錄
```

執行結果

```
C:\Users\Ian\VS_Code
```

os.path 模組的其他方法

	os.path 的方法	說明
檢查（True/False）	★ exists()	檔案或路徑是否存在
	isabs()	是否為絕對路徑
	isfile()	指定路徑是否為檔案
	isdir()	指定路徑是否為目錄
處理檔案路徑	abspath()	傳回絕對路徑
	basename()	傳回指定字串最後的檔案或路徑名稱
	★ dirname()	傳回指定檔案的絕對路徑
	★ split()	分割路徑成 dirname 和 basename
	splitdrive()	分割路徑成磁碟機與路徑名稱
	★ join()	合併路徑與檔名為完整路徑
處理檔案	getsize()	傳回檔案大小
	getctime()	傳回檔案建立時間
	getatime()	傳回檔案最後的訪問時間

範例

```python
import os.path as op

file = op.abspath("os_path.py")
print("檔案絕對路徑：", file)

if op.exists(file):
    f_path, f_name = op.split(file)
    print("檔案路徑：", f_path)
    print("檔案名稱：", f_name)
    print("檔案大小：", op.getsize(file), "bytes")

print("組合路徑：", op.join(f_path, f_name))
```

執行結果

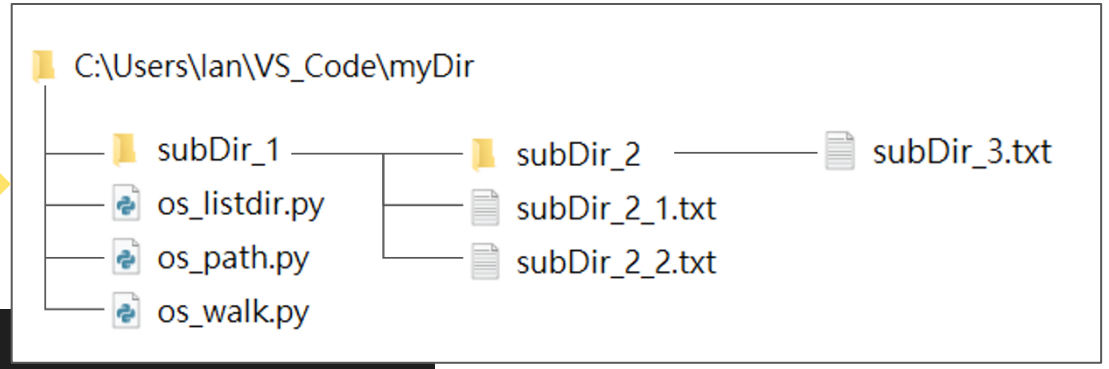
```
檔案絕對路徑： C:\Users\Ian\VS_Code\myDir\os_path.py
檔案路徑： C:\Users\Ian\VS_Code\myDir
檔案名稱： os_path.py
檔案大小： 341 bytes
組合路徑： C:\Users\Ian\VS_Code\myDir\os_path.py
```

os.lisrdir 與 os.walk

- 在使用 Python 開發處理檔案的程式時，時常會需要列舉目錄內的所有檔案名稱，然後再透過迴圈逐一處理之。此時，
  - 可使用 os.lisrdir 模組取得檔案列表，
  - 而 os.walk 模組更是能以遞迴(recursive)方式列出特定路徑下的所有子目錄與檔案。

範例

目錄內容




```python
import os.path as op
from os import listdir

my_path = r"C:\Users\Ian\VS_Code\myDir"
files = listdir(my_path)        # 取得所有檔案與目錄名稱

for f in files:
    full_path = op.join(my_path, f)

    if op.isfile(full_path):
        print("檔案：", f)
    elif op.isdir(full_path):
        print("目錄：", f)
```

執行結果

```
檔案： os_listdir.py
檔案： os_path.py
檔案： os_walk.py
目錄： subDir_1
```

os.walk()

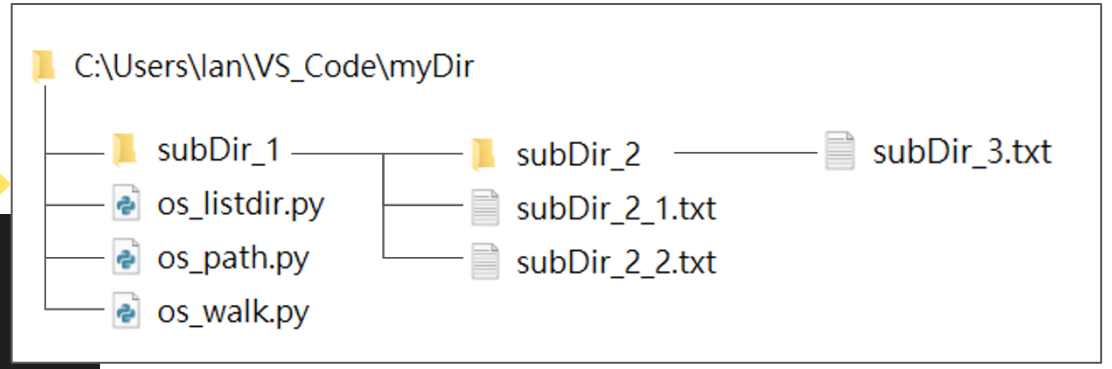
- os.walk() 方法會回傳一個包含三個元素的元組，分別是
  - 目前路徑
  - 資料夾串列
  - 檔案串列

- 且由於是採用遞迴方式處理，功能強大可是程式也不太好理解。

範例

```python
import os.path as op
from os import walk

my_path = r"C:\Users\Ian\VS_Code\myDir"

# 遞迴列出所有目錄與檔案
for root, dirs, files in walk(my_path):
    print("路徑：", root)
    print("└目錄：", dirs)
    print("└檔案：", files)
```

目錄內容

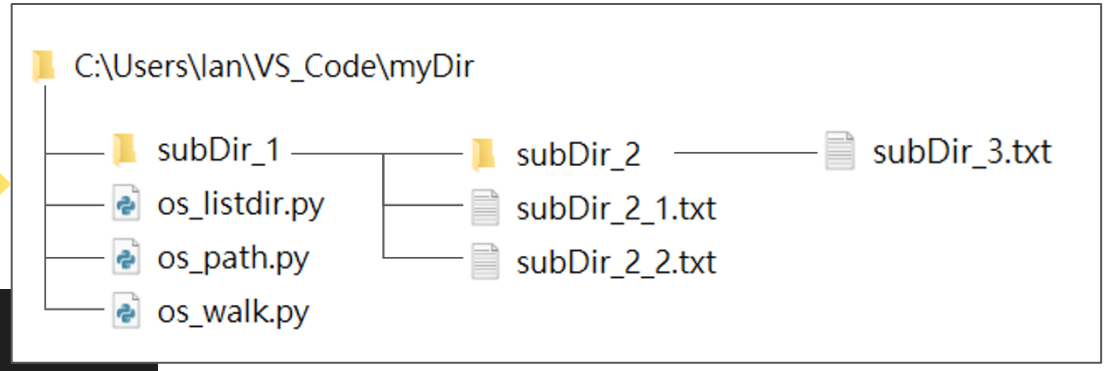


執行結果

```
路徑： C:\Users\Ian\VS_Code\myDir
└目錄： ['subDir_1']
└檔案： ['os_listdir.py', 'os_path.py', 'os_walk.py']
路徑： C:\Users\Ian\VS_Code\myDir\subDir_1
└目錄： ['subDir_2']
└檔案： ['subDir_2_1.txt', 'subDir_2_2.txt']
路徑： C:\Users\Ian\VS_Code\myDir\subDir_1\subDir_2
└目錄： []
└檔案： ['subDir_3.txt']
```

範例

目錄內容




```python
import os.path as op
from os import walk

my_path = r"C:\Users\Ian\VS_Code\myDir"

for root, dirs, files in walk(my_path):
    for f in files:
        full_path = op.join(root, f)
        print(full_path)
```

目的：取得目錄下所有檔案的絕對路徑

執行結果

```
C:\Users\Ian\VS_Code\myDir\os_listdir.py
C:\Users\Ian\VS_Code\myDir\os_path.py
C:\Users\Ian\VS_Code\myDir\os_walk.py
C:\Users\Ian\VS_Code\myDir\subDir_1\subDir_2_1.txt
C:\Users\Ian\VS_Code\myDir\subDir_1\subDir_2_2.txt
C:\Users\Ian\VS_Code\myDir\subDir_1\subDir_2\subDir_3.txt
```

練習：計算空氣品質指標(AQI)

- 計算各地空氣品質指標(AQI)的平均值。
  - 資料來源：https://data.epa.gov.tw/dataset/detail/AQX_P_488
  - 年份：2021.1 ~ 2021.3
  - 每天一個 csv 檔案

aqx_p_488_2021-01-01.csv

```
SiteName,County,AQI,Pollutant,Status,SO2,CO,O3
基隆,基隆市,38,,良好,1.7,0.21,34.9,34,26,8,2.7
汐止,新北市,50,,良好,0.5,0.27,23.5,25,22,15,9.
萬里,新北市,56,懸浮微粒,普通,1.2,0.2,,,69,16,2
新店,新北市,42,,良好,1.6,0.23,29.5,24,21,9,3.1
土城,新北市,46,,良好,1.1,0.21,28.3,25,19,12,3.
板橋,新北市,48,,良好,1.2,0.31,21.4,17,19,13,10
新莊,新北市,55,細懸浮微粒,普通,1.1,0.34,22,26,
菜寮,新北市,48,,良好,1.8,0.22,32,28,25,12,4.4,
林口,新北市,53,細懸浮微粒,普通,0,0.2,33.1,33,2
淡水,新北市,43,,良好,1.5,0.21,33.8,32,24,13,2.
```

主題

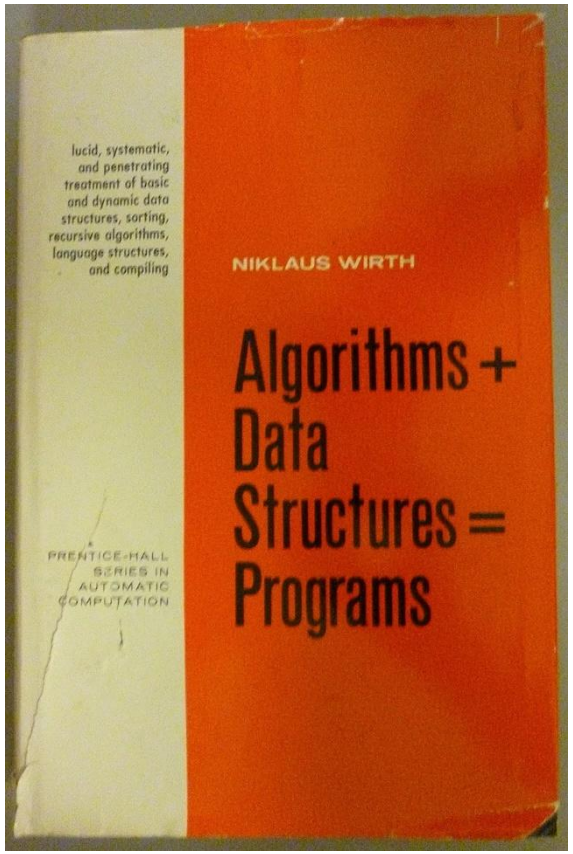
- 目錄管理
  - 檔案搜尋：glob 模組
  - 檔案操作：os 模組
- 堆疊與佇列結構
  - 堆疊
  - 佇列

資料結構(Data Structure)

- 當我們要透過電腦解決問題前，必須以電腦能理解的模式來描述問題，而資料結構就是描述資料的方法，也是電腦內儲存資料的基本架構。
  - 資料結構不僅涉及儲存的資料與儲存方法，同時也考慮到存取資料的方式，能讓程式的執行速度加快，並降低佔用的計算資源（如CPU、記憶體等）。

尼克勞斯·維爾特(Niklaus Wirth)

- 著名的瑞士計算機科學家**尼克勞斯·維爾特**曾在 1984 年因發展數個程式語言而獲圖靈獎(Turing Award)
  - 他寫過一本書的書名是『Algorithms + Data Structures = Programs』（演算法 + 資料結構 = 程式），這已經是計算機科學的名句。



Source: Wiki

資料結構 vs 演算法

- 一個程式應該包括對「資料」與「操作」的描述，其中
  - 資料是指在程式中要指定資料的類型和資料的組織形式，亦即前面提到的資料結構；
  - 操作則意指操作步驟，也就是演算法。
- 以圖書館的藏書來做比喻，

資料結構就好比書籍擺放的方式，不僅是要依書籍特性制定合適的索書號，也要依書架與動線來設計同類型書籍的擺放位置。

演算法就像是找書的方法，可能是先從系統查詢到書籍對應的索書號，再據此找到目標書籍。

合適的資料結構

- 程式設計師必須依照問題或實作的特性，選擇合適的資料結構來進行資料的新增、修改、刪除、儲存等動作，方能讓演算法發揮最大效能。
  - 傳統上，常討論的基礎資料結構有陣列、堆疊、佇列、鏈結串列(linked list)、樹狀結構(tree)、圖形(graph)、雜湊表(hash table)等。
  - 我們在之前的章節已經認識了陣列，也知道利用 Python 提供的串列來實現陣列的操作，可以更加有彈性且更好使用。
  - 接著介紹堆疊與佇列的概念，並同樣利用 Python 的串列來實現。

主題

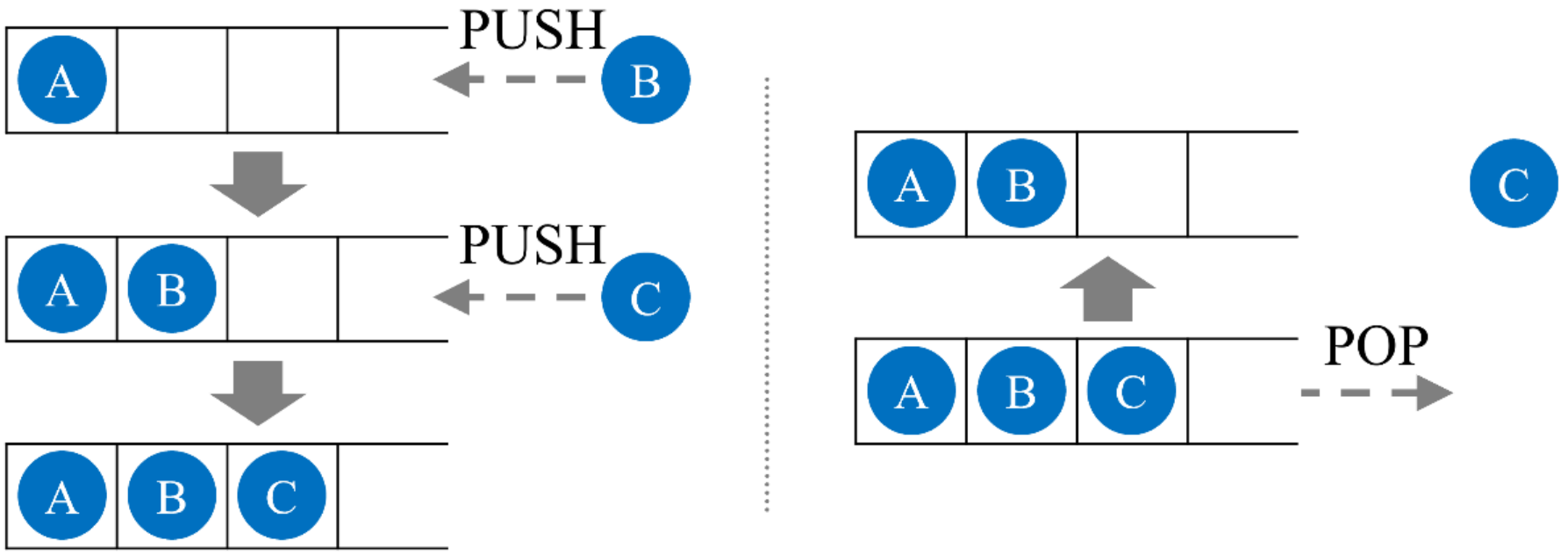
- 目錄管理
  - 檔案搜尋：glob 模組
  - 檔案操作：os 模組
- 堆疊與佇列結構
  - 堆疊
  - 佇列

堆疊(Stack)

- 堆疊是一個有序串列(ordered list)，或者稱為線性串列(linear list)。
  - 其放入(push)與取出(pop)元素的操作都發生在同一端，通常稱為頂端(top)，
  - 並具有後進先出(Last in First out、LIFO)的特性。

  - 所謂 LIFO 的概念，可以想像一個乒乓球筒，只能從一端取出或放入乒乓球的作法。

堆疊的 PUSH 與 POP 操作方式



範例

```python
def create():               # 建立一個空堆疊
    return []

def isEmpty(stk):           # 檢查堆疊是否為空
    if len(stk):    return False
    else:           return True

def isFull(stk):            # 檢查堆疊是否為滿
    return False

def PUSH(stk, data):        # 放入資料
    if not isFull(stk):
        stk.append(data)

def POP(stk):               # 取出資料
    if not isEmpty(stk):
        return stk.pop()

stack = create()
PUSH(stack, 'A'); PUSH(stack, 'B'); PUSH(stack, 'C')
print("堆疊內容：", stack)
print("取出資料：", POP(stack))
print("堆疊內容：", stack)
```

```
堆疊內容： ['A', 'B', 'C']
取出資料： C
堆疊內容： ['A', 'B']
```

主題

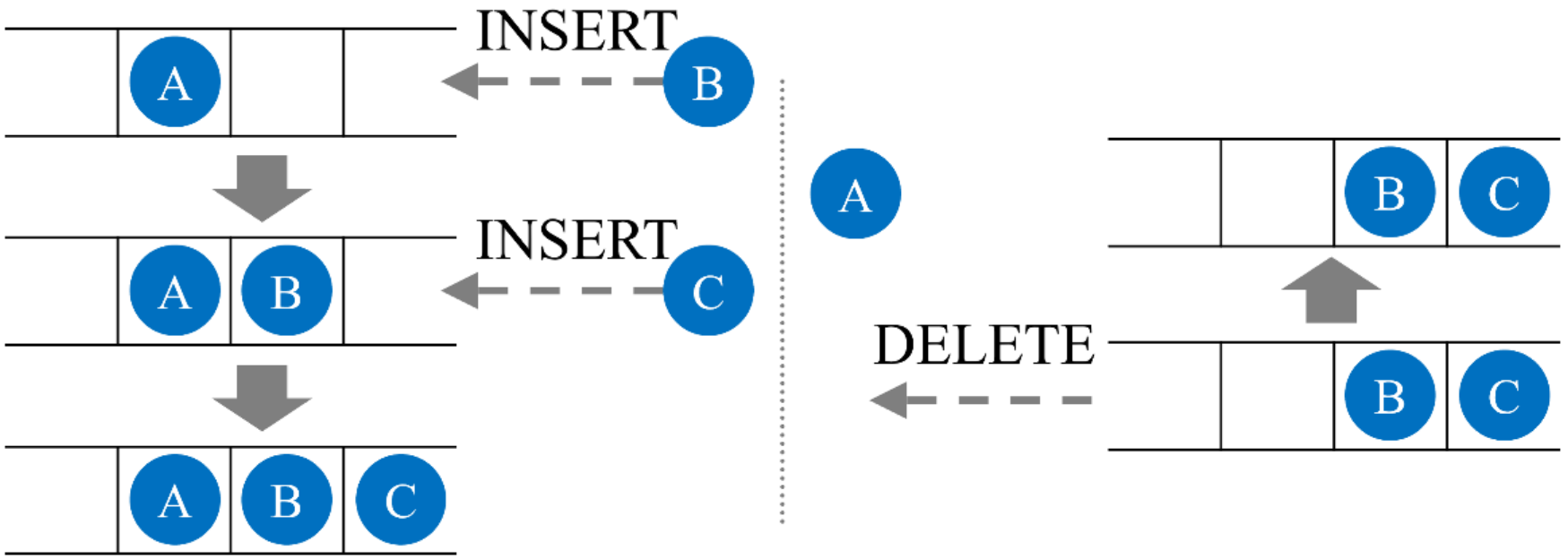
- 目錄管理
  - 檔案搜尋：glob 模組
  - 檔案操作：os 模組
- 堆疊與佇列結構
  - 堆疊
  - 佇列

佇列(Queue)

- 佇列也稱為對列，與堆疊一樣是個有序串列。
  - 其兩個操作插入(insert)與刪除(delete)元素分別發生在不同的兩端，
  - 並具有先進先出(Fast in First out、FIFO)的特性。

  - 所謂 FIFO 可類比成羽球筒的取和放球，在看的到羽球頭的那端便於取出，而看到羽毛的那端則用於放入羽球。

佇列的 INSERT 與 DELETE 操作方式



範例

```python
def create():               # 建立一個空佇列
    return []

def isEmpty(que):           # 檢查佇列是否為空
    return False if len(que) else True

def INSERT(que, data):      # 插入資料
    que.append(data)

def DELETE(que):            # 刪除資料
    return que.pop(0) if not isEmpty(que) else None

queue = create()
INSERT(queue, 'A'); INSERT(queue, 'B'); INSERT(queue, 'C')
print("佇列內容：", queue)
print("刪除資料：", DELETE(queue))
print("佇列內容：", queue)
```

```
佇列內容： ['A', 'B', 'C']
刪除資料： A
佇列內容： ['B', 'C']
```

練習：檢查小括號個數

HINT: 本題可嘗試使用堆疊結構撰寫

- 請撰寫一程式，讓使用者輸入包含左/右小括號之四則運算式，並判斷左/右小括號的個數是否匹配，同時顯示成對或不成對的小括號數量。

範例輸入 請輸入含小括號之四則運算式：((1+2)-3)*(4/5)

輸出 成對小括號數量 = 3

範例輸入 請輸入含小括號之四則運算式：(((1+2+3)

輸出 不成對小括號數量 = 2

範例輸入 請輸入含小括號之四則運算式：((1+2)*(3+4)*(5+6))/(7+8)

輸出 成對小括號數量 = 5

本章結束